

Spectral Based Information Retrieval

Laurence A. F. Park

Submitted in total fulfillment of the requirements of the degree of Doctor of Philosophy

December 17, 2003

Department of Electrical and Electronic Engineering
The University of Melbourne

ABSTRACT

THE information found on the Internet is growing at such a rapid rate that soon methods of searching through text using terms frequencies will not be enough. At the moment, many Web search engines are showing signs of imprecision because they are based on these term counting methods which do not examine the relationships between the document terms. These methods begin to fail as the number of indexed documents increases past an allowable limit.

Natural language processing has been performed in the past and we have found that it is only useful within its own domain. For example, if we use a natural language system to extract documents from a sporting database, we will find that the same tool will not be very effective for medical articles.

Spatial methods have been developed to tackle the problem of the ever growing World Wide Web. Many have failed but a few have risen to the level of the frequency based methods mentioned above. Due to the extra document analysis performed, the spatial methods are slower than the frequency based methods and require more storage.

This thesis presents a novel method of information retrieval entitled “Spectral Information Retrieval”. This method achieves the speed of the vector space methods with the benefits of the proximity methods to provide an overall high quality information retrieval system. Rather than using the spatial locality information (used in proximity searches), a spectral information retrieval method utilises the query terms’ spectral locality information (found with the aid of either the Fourier transform, Cosine transform, Gaussian transform or Wavelet transform). By combining the query term spectra, we are able to make fast proximity calculations and also make use of the many varieties of vector space method weighting schemes. This method provides superior results to existing text based information retrieval systems.

It is shown that spectral information retrieval methods provide high precision results at query times comparable to the widely used vector space methods, when using an index of comparable size to a vector space method index. This was possible using compression techniques such as spectral component cropping and quantisation, and speed

up techniques such as early termination. When querying with a small set of terms, we saw that the spectral document retrieval methods using a certain vector space method weighting scheme, always improved the precision of the vector space method by a significant margin (at least 10%).

It is also shown that the spectral information retrieval system can be further enhanced when working in conjunction with a relevance feedback system.

DECLARATION

This is to certify that

1. the thesis comprises only my original work towards the PhD
2. due acknowledgment has been made in the text to all other material used,
3. the thesis is less than 100,000 words in length, exclusive of tables, maps, bibliographies and appendices.

Laurence A. F. Park

ACKNOWLEDGMENTS

First I would like to thank my supervisors, Associate Professor Marimuthu Palaniswami and Professor Kotagiri Ramamohanarao. Without their guidance, this research would not be possible. Included in the set of toughest challenges faced, was trying to spell their names correctly.

I would like to thank the ARC Special Research Centre for Ultra-Broadband Information Networks for their support and funding of this research. They also provided an excellent research environment filled with experts of many different fields. The experience with them has helped to broaden my mind.

I would also like to thank Alistair Moffat and his research team for use of their resources and help. Especially Tim A. H. Bell who has always been able to fix any computer problems on ‘the other side’.

My gratitude goes toward the usual suspects at CUBIN, namely Malcolm Peh, Tao Peng, Raj “the king” Parthiban, Jolyon White (my previous mentee), Marija Bohocki, Rami Mukhtar, Bartek Wydrowski, Chia-min Lee, Nam Nguen (my mentee), Andrew Liu (my grandmentee), John Papandriopoulos and Dr. Brian Krongold. With these people around, there was never a dull moment. I’d like to give a special mention to Dr. Lachlan Andrew, if anyone has the answer, it would be him.

Last, but not least, I want to thank my family. Mum, dad, Lizzy, James, Gena, Jon and Michael have always been there for me and supported me. The more I go through life, the more I find I have a little piece of each of them which makes up the whole of me. Also, the newest member of my family (and my soul mate), Mariquita, for being so patient and for always bringing a smile to my face.

NOTATION

Symbol	Definition
d_n	The n th document vector
t_n	The n th term vector
$f_{d,t}$	The frequency of term t in document d
$f_{q,t}$	The frequency of term t in query q
f_t	The number of documents which contain term t
T	The set of query terms
$\hat{T}_{d,b}$	The set of query terms which are non-zero in bin b of document d
N	The number of documents in the set
n_k	The number of documents containing term k
$\#(T)$	The cardinality of set T
$S_x(d_n, q)$	The score of document d_n from query q using method x
w_t	The weight of term t
f_t	The number of documents containing term t
f^m	The greatest f_t for all t
$r_{d,t}$	Relative document term frequency
$r_{q,t}$	Relative query term frequency
$w_{d,t}$	The weight of term t in document d
$w_{q,t}$	The weight of term t in query q
W_d	The norm of document vector d
W_q	The norm of query vector q
Prec. x	The precision after x documents have been retrieved
$\tilde{f}_{d,t}$	Term signal t of document d
$f_{d,t,b}$	Count of term t in bin b from document d
$\tilde{\omega}_{d,t}$	Weighted term signal t of document d
$\omega_{d,t,b}$	Weighted component b of term t in document d
$\Omega_d()$	Document weight function for term signals
$\Omega_q()$	Query weight function for term signals

Symbol	Definition
$\Phi_{d,b}$	Phase precision of bin b in document d
$\Phi_{d,b}^z$	Zero phase precision of bin b in document d
$\Phi_{d,b}^n$	Non-zero phase precision of bin b in document d
$\Phi_{d,b}^1$	Phase precision of bin b in document $d = 1$
$H_{d,b}^m$	Magnitude of bin b in document d using sum magnitudes
$H_{d,b}^v$	Magnitude of bin b in document d using sum vectors
$s_{d,b}$	Score component of bin b in document d
$\tilde{v}_{d,t}$	Term signal spectrum of term t in document d after the Fourier transform
$\nu_{d,t,b}$	Spectral component of bin b in term t from document d after the Fourier transform
$\tilde{\eta}_{d,t}$	Term signal spectrum of term t in document d after the Cosine transform
$\eta_{d,t,b}$	Spectral component of bin b in term t from document d after the Cosine transform
B	The number of bins
$H_{d,t,b}$	The magnitude of the b th spectral component in term t of document d .
$\phi_{d,t,b}$	The phase of the b th spectral component in term t of document d .
$q_u()$	Uniform quantisation function
$q_{lg}()$	Left geometric quantisation function
$\mathcal{A}$	An accumulator
$\mathcal{A}_i$	The i th element of the accumulator
I_t	The inverted list for term t
$\emptyset$	The empty set
$\mathcal{G}_\sigma$	A Gaussian function of standard deviation σ
$\mathcal{F}\{X\}$	The Fourier transform of X
$\check{\mathcal{G}}_\sigma$	The Fourier transform of a Gaussian function of standard deviation σ
$\star$	The convolution operator
$ x $	The absolute value of x
S	The Gaussian spreading constant
$\boxtimes$	The element-wise multiplication operator
$\psi_{u,s}$	A wavelet function
$Wf(u,s)$	A wavelet transform of $f(x)$
$\tilde{\zeta}_{d,t,b}$	Spectral component of the b th bin from the t th term in the d th document after the wavelet transform

Symbol	Definition
$\tilde{S}_{d,t}$	Term signal spectrum of the t th term in the d th document after the wavelet transform
$\ x\ _p$	the l_p norm of x

CONTENTS

1	Introduction	1
1.1	Looking Forward	1
1.2	Focus of this Thesis	3
1.3	Organisation of the Thesis	4
1.4	Contributions of this Thesis	6
1.5	Publications by the Author related to this Thesis	7
2	Document Retrieval	9
2.1	Introduction	9
2.2	Document Retrieval Process	11
2.3	Storing Information	11
2.3.1	File Structures	12
2.3.2	Coding	15
2.3.3	Lexical Preprocessing	18
2.4	Retrieving Information	19
2.4.1	Retrieval Methods	19
2.4.2	Query Process	25
2.5	Advanced Retrieval	30
2.5.1	Relevance Feedback	30
2.5.2	Natural Language Processing	31
2.5.3	The Google Phenomenon	32
2.6	Retrieval Experimentation	32
2.6.1	Document Collections	32
2.6.2	Retrieval Evaluation	34
2.7	Evaluation of Retrieval Systems	36
3	Spectral Document Ranking	37
3.1	Introduction	37
3.2	Fourier Domain Scoring	39

3.2.1	Collect words into spatial bins	40
3.2.2	Create spatial inverted index	40
3.2.3	Weighting	41
3.2.4	Perform Fourier transform	42
3.2.5	Combining word spectra	43
3.2.6	Combining spectral components	46
3.3	Comparison to Vector Space Method	48
3.4	Performance with Large Queries	49
3.5	Proximity Analysis	50
3.6	Computational Complexity	54
3.7	Experiments	55
3.7.1	Document and query set	55
3.7.2	Methods Performed	58
3.8	Discussion and Summary	58
4	Efficient Implementation	65
4.1	Introduction	65
4.2	Spectral Document Ranking	66
4.2.1	Fourier Domain Scoring	67
4.2.2	The Discrete Cosine Transform	68
4.3	Cosine Domain Scoring	70
4.3.1	Prewriteighting	72
4.3.2	Combination of word spectrums	73
4.3.3	Combination of spectral components	74
4.4	Compressed Spectral Inverted Index	75
4.4.1	Quantisation	75
4.4.2	Component Cropping	81
4.4.3	Compression achieved	82
4.5	Fast Document Ranking for Spectral Indices	83
4.5.1	Sorting the Spectral data	84
4.5.2	The Spectral Accumulator	85
4.5.3	Fast Query Processing	85
4.5.4	Fast Query Processing with Spectral Accumulators	86
4.5.5	Effect on Precision	88
4.6	Discussion and Summary	88

5	Gaussian Spreading	91
5.1	Introduction	91
5.2	Term Signal Dispersion	92
5.2.1	Gaussian Function	92
5.2.2	Convolution	93
5.2.3	Signal Energy	94
5.3	Choosing the Gaussian parameters	96
5.4	Sampling Artifacts	99
5.4.1	Aliasing Effects	100
5.4.2	Wrap around effects	103
5.5	Methods	106
5.6	Analysis	106
5.6.1	Occurrence Analysis	108
5.6.2	Proximity Analysis	108
5.7	Experiments	109
5.7.1	Application to Fourier Spectral Domain	111
5.7.2	Gaussian Convolution	116
5.8	Discussion and Summary	117
6	Multiresolution Document Retrieval	119
6.1	Introduction	119
6.2	Multiresolution Analysis	120
6.2.1	Time-Frequency Analysis	121
6.2.2	Short Time Fourier Transform	121
6.2.3	Wavelet Transform	122
6.2.4	Discrete Wavelet Transform	125
6.3	Choosing a wavelet	126
6.3.1	Wavelet Properties	127
6.3.2	Selected Wavelets	129
6.4	Document Retrieval	133
6.4.1	Create term signals	133
6.4.2	Apply preweighting	134
6.4.3	Apply Wavelet transform	134
6.4.4	Combine query term spectra	134
6.4.5	Combine score components	136
6.5	Computational Complexity and Storage	136

6.5.1	Computational Complexity	136
6.5.2	Storage Size	137
6.6	Analysis	138
6.6.1	Occurrence Analysis	138
6.6.2	Proximity Analysis	139
6.7	Experiments and Results	141
6.7.1	Sample document set	141
6.7.2	Application to TREC document set	143
6.8	Extension	145
6.9	Discussion and Summary	148
7	Anaphora Resolution and Relevance Feedback	151
7.1	Introduction	151
7.2	Anaphora Resolution	152
7.2.1	Anaphoric Resolution in English	152
7.2.2	Speech Tagging	153
7.2.3	Types of Anaphora	153
7.2.4	Pronoun Resolution	156
7.2.5	Application to Information retrieval	157
7.2.6	Experiments	157
7.3	Relevance Feedback	160
7.3.1	Rocchio Relevance Feedback	161
7.3.2	Automatic Feedback	162
7.3.3	Interactive Feedback	163
7.3.4	Query Expansion	164
7.3.5	Term Re-weighting	165
7.3.6	Iterations	165
7.3.7	Local Context Analysis	166
7.3.8	Feedback in the Spectral Domain	166
7.3.9	Experiments	167
7.4	Discussion and Summary	169
8	Conclusions	171
8.1	Summary of Work	171
8.1.1	Spectral Document Retrieval	172
8.1.2	Efficient Implementation	172
8.1.3	Gaussian Spreading	172

8.1.4	Multiresolution Document Retrieval	172
8.1.5	Anaphora Resolution and Relevance Feedback	173
8.2	Summary of Precision Results	173
8.3	Future Directions	175
A	Method Naming Scheme	189
B	Query Set	193

LIST OF FIGURES

1.1	The results page after searching for “rabbit ears” in the Google search engine. Portions of the document content are presented to the searcher to assist in the manual document filtering.	3
2.1	The Assyrian King Ashurbanipal shown carrying a basket to rebuild a temple in Babylon (http://web.utk.edu/~giles/).	10
2.2	The information retrieval process [30]. The shaded portions of the chart are the post-retrieval processes.	12
2.3	A PAT tree for the text “101110001...”. The numbers in the tree leaves represent the position of the chosen sequence. Eg. the binary sequence 110 begins at the 5th bit in the binary text sequence.	15
2.4	Documents (D1, D2 and D3) are represented by vectors in the vector space method.	21
2.5	Pivoted normalisation adjusts the document score to give higher relevance to longer documents and lower relevance to shorter documents	28
2.6	Recall vs. precision plot for two sample methods. We can see that method 1 has a higher precision than method 2 for every level of recall, therefore method 1 is superior to method 2.	35
3.1	A visual example of how the word signals are obtained. The top two lines, labeled “travel” and “wales” show the positions of the words travel and wales in some document (the position is signified by the vertical stroke through the line). The bottom half shows the binned positions of the words.	40
3.2	An example of before and after the Fourier transform. We can see the binned positions of word 1 and word 2 from a certain document in the top plot. The bottom two plots show the information obtained after computing the Fourier transform on each of the binned word signals. Notice the reflection of the spectral information about component 4 (as stated in the Nyquist-Shannon sampling theorem).	44

3.3	The effect of the displacement between two terms ($\Delta_1 = b_n - b_m $) on the document score.	52
3.4	The effect of the displacement between a third term and the centre of the first two terms ($\Delta_2 = \frac{b_n+b_m}{2} - b_a$) on the document score. Δ_1 is the displacement between the first and second terms.	53
3.5	A typical document from the TREC document set	56
3.6	A sample query from the TREC collection	57
4.1	Fitting the sampled weighted term signal covariance matrix to the Markov-1 covariance matrix. We can see that the best fit is at $\rho = 0.79$	70
4.2	A comparison of the basis functions generated by the KLT (solid line) and DCT (dashed line) of the weighted term signals. The bracketed number indicates which spectral component the function represents.	71
4.3	Example of the Karhunen-Loève transform. The top plot displays 100 randomly generated points. The bottom plot shows the same points after performing the KLT.	71
4.4	Distributions of the two quantisation methods. We can see that the left geometric distribution gives finer quantisation at the lower end of the scale.	77
4.5	Once stored in the index, each term from the document set has an inverted list and an associated maximum and minimum magnitude value used to quantise the inverted list.	78
4.6	Distribution of the magnitude values of the word 'radio' from the data set AP2. Most of the magnitude values appear at the lower end of the scale (close to zero).	79
4.7	A comparison of the CDS method incorporating quantisation and cropping. Each point represents a different magnitude bits, components cropped combination. The cross represents the BD-ACI-BCA method precision and index size.	82
4.8	A close up of the trials with the greatest precision and smallest spectral index size from figure 4.7. The points are labeled with their (Magnitude bits, components cropped) setting. The cross represents the BD-ACI-BCA method precision and index size.	83
4.9	A comparison of the CDS method incorporating quantisation, cropping and fast document ranking. Each point represents a different magnitude bits, components cropped combination.	88

4.10	A close up of the trials which have the fastest query time from figure 4.9. Each point is accompanied with a (quantised magnitude bits, components cropped) label.	89
4.11	A comparison of the CDS method incorporating quantisation, cropping and fast document ranking. We can see the results are almost identical to those with out fast document ranking.	90
5.1	The normalised Gaussian function	93
5.2	A filter diagram. In our case, $H(x) = \mathcal{G}(x)$, $X(t)$ is the term signal, and $Y(t)$ is the dispersed term signal.	93
5.3	Gaussian function applied to two terms from different documents. The longer document of length l_1 has a wider spread than the shorter document l_2 if the scale parameter only depends on the term weight.	97
5.4	Fourier transform of a Gaussian function with $\sigma = 0.1$ and $\sigma = 3.5$	98
5.5	Histogram showing the distribution of w_t for all terms in the document set ap2wsj2 using SMART Lnu.ltu weighting.	99
5.6	The effects of sampling in the frequency domain. The spectrum of the Gaussian function is centered on the positions Nf_s . If the spectrum is too wide, aliasing occurs.	101
5.7	The error induced by including only the four closest Gaussian functions in the frequency domain. We can see that the error increases for small σ due to the wide spread of each Gaussian in the frequency domain.	103
5.8	The error induced by including only the four closest Gaussian functions in the frequency domain with a cutoff value of $\sigma = 0.26$. We can see that the error for small σ has been reduced significantly due to the cutoff being added.	104
5.9	If the wrapped Gaussian is clipped, the total signal energy is reduced. . .	105
5.10	By adding padding, we increase the distance between the first and last bins.	106
5.11	The FDS + Gaussian method scores derived from a document containing two query terms separated by term bin displacement. Different plots are given for different values of σ to show how the proximity is affected. . . .	110
5.12	The Gaussian Convolution Method scores derived from a document containing two query terms separated by term bin displacement. Different plots are given for different values of σ to show how the proximity is affected.	111

5.13	Plot of precision after 5,10,15 and 20 documents for $S = 5$. The dashed lines represent phase precision included in the calculation, and cross markers indicate padding was added, the closed markers (circle and triangle) imply no padding. Note how the trials with phase precision included provide much better results for $S = 5$	112
5.14	Plot of precision after 5,10,15 and 20 documents for $S = 0.1, 0.5, 1, 5, 10$ and 20. The legend is the same as in figure 5.13. Note that the clustering of trials with phase precision and without still exists.	115
6.1	An example of the enclosing scaling function ($\phi_{u,s}(t) \in V_n$) spaces as ovals and the wavelet function spaces ($\psi_{u,s}(t) \in W_n$) as annuli.	126
6.2	Recursive filtering process. H is a high pass filter (using the wavelet coefficients) and G is a low pass filter (using the scaling function coefficients). The low pass data gets split and decimated repeatedly to obtain the wavelet transform. An example of this is shown in equation 6.4.	127
6.3	Functions exhibit compact support if they have no infinite interval of non-zero values.	129
6.4	The Haar scaling function ($\phi(t)$) and wavelet ($\psi(t)$)	131
6.5	The Daubechies-4 scaling function ($\phi(t)$) and wavelet ($\psi(t)$)	132
6.6	a) Score derived from the Haar wavelet transform using a) sum, b) sum of squares, of a document containing two query terms of equal weight. The first is found in bin zero, the second is found in bin 'term bin displacement'. 140	
6.7	a) Score derived from the Daubechies-4 wavelet transform using a) sum, b) sum of squares, of a document containing two query terms of equal weight. The first is found in bin zero, the second is found in bin 'term bin displacement'.	140
6.8	Examples of queries taken from TREC queries 51-200 titles.	144
6.9	Precision-recall plot for recall of 5,10,15 and 20 documents for the Haar and Daubechies-4 wavelet methods, and the AB-AFD-BAA vector space method.	148
7.1	http://www.austmus.gov.au/factsheets/huntsman_spiders.htm	153
7.2	The output of LTPOS when given figure 7.1. Each term is tagged with a Penn Treebank tag.	155

LIST OF TABLES

2.1	An example of a simple index to be used with the vector space methods. .	13
2.2	An example of an inverted file to be used with the vector space methods. Only the terms related to the query need to be extracted.	13
2.3	Example of signatures associated to the terms.	14
2.4	Description of weighting method symbols used by Salton and Buckley. . .	26
2.5	Formulation of the Lnu.ltu weighting method.	26
2.6	Description of weighting method symbols used by Zobel and Moffat. . . .	27
2.7	Formulation of the BD-ACI-BCA weighting method, where $W'_d = \sqrt{\sum_{t \in T} w_{d,t}^2}$	27
2.8	Document collections and associated query sets.	33
2.9	Categories found in the Reuters-21578 collection.	34
3.1	Calculation of the different Magnitude and Phase precision values of the terms <i>mariquita</i> , <i>travels</i> and <i>wales</i>	46
3.2	Method names are of the form fds-x-y-z.bn where the values of x,y,z,n associate to the description in this table	59
3.3	Best 20 document retrieval methods ordered by precision after a) 5 and b) 10 documents were retrieved for titles of queries 51-200 on AP2WSJ2. Vector space methods not in the best 20 are shown below the bar.	60
3.4	Best 20 document retrieval methods ordered by precision after a) 15 and b) 20 documents were retrieved for titles of queries 51-200 on AP2WSJ2. Vector space methods not in the best 20 are shown below the bar.	61
3.5	Best 20 document retrieval methods ordered by precision after a) 5 and b) 10 documents were retrieved for titles of queries 51-200 on FR2ZIFF2. Vector space methods not in the best 20 are shown below the bar.	62
3.6	Best 20 document retrieval methods ordered by precision after a) 15 and b) 20 documents were retrieved for titles of queries 51-200 on FR2ZIFF2. Vector space methods not in the best 20 are shown below the bar.	63

3.7	Best 20 document retrieval methods ordered by precision after 10 documents retrieved for long queries 51-200 on AP2WSJ2. Vector space methods not in the best 20 are shown below the bar.	64
4.1	Index types for Spectral based document retrieval systems.	66
4.2	Short queries applied to the AP2 document set using the reduced dimension cds-5-4-1 method. The column D_x refers to the cds-5-4-1 method using the first x components and VS refers to the BD-ACI-BCA vector space method.	76
4.3	Results from running queries 51-200 on AP2 with varying magnitude quantisation bit size. Prec.10 and Prec.20 are the precision after 10 and 20 documents respectively.	80
4.4	Experiments using document set AP2, AP2% is the size relative to the AP2 document set size. a) Size of inverted indexes relative to the document collection indexed. b) Size of inverted spectral indexes relative to the document collection indexed where the magnitude is quantised to 4 bits. . . .	81
4.5	The order in which vector space and spectral data is encoded after performing quantisation	84
4.6	Example inverted index sorted by w_t	86
5.1	Sum of the error between the Gaussian estimate and the actual Gaussian function for σ values of 0 to 2.	104
5.2	Results for the method fds-9-4-6 with and without padding and the top 20 results of the same methods involving different forms of Gaussian spreading a) Precision after the first 5 documents. b) Precision after the first 10 documents.	113
5.3	Results for the methods fds-9-4-6 with and without padding and the top 20 results of the same methods involving different forms of Gaussian spreading a) Precision after the first 15 documents. b) Precision after the first 20 documents.	114
5.4	Results for the methods fds-9-4-6 with and without padding and the results of gauss-9-4-6. a) Precision after the first 5 documents. b) Precision after the first 10 documents.	117
5.5	Results for the methods fds-9-4-6 with and without padding and the results of gauss-9-4-6 a) Precision after the first 15 documents. b) Precision after the first 20 documents.	118

6.1	The many resolutions of decomposition after the wavelet transform of $\tilde{f}_{d,t}$ is applied in equation 6.3.	123
6.2	Haar wavelet decomposition of signal with impulse at term bin position. .	131
6.3	A sample set of terms with their term signals within three documents. . .	141
6.4	The wavelet transforms of the term signals found in table 6.3.	142
6.5	Method names are of the form wavelet-x-y-z.ln where the values of x,y,z,n associate to the description in this table	145
6.6	Top 20 methods using data set AP2WSJ2 with queries 51 to 200 sorted by precision after a) 5 and b) 10 documents. The methods under the lower bar are the vector space and proximity methods which did not make the top 20.	146
6.7	Top 20 methods using data set AP2WSJ2 with queries 51 to 200 sorted by precision after a) 15 and b) 20 documents. The methods under the lower bar are the vector space and proximity methods which did not make the top 20.	147
7.1	A non-exhaustive list of tags inserted after using a part-of-speech (POS) tagger. Taken from the Penn Treebank Tag-Set and used with LTPOS [51].	154
7.2	The list of noun types and their POS tags.	158
7.3	The list of pronoun types and their POS tags.	158
7.4	List of noun types and the possible personal and possessive pronouns that can reference them	159
7.5	Results for the method fds-9-4-1 using AP2 and the same method involving our simple pronoun resolution algorithm. We can see that the added pronoun resolution increases precision only at the 20 document point (d) when summing 3 and 4 bins.	160
7.6	Results for the method fds-9-7-1 using AP2WSJ2 and the same method involving various methods of relevance feedback.	168
8.1	Top 20 spectral document retrieval methods developed using BD-ACI-BCA weighting in terms of precision after a) 5 documents and b) 10 documents. The BD-ACI-BCA vector space method precision is shown below the bar.	174
8.2	Top 20 spectral document retrieval methods developed using BD-ACI-BCA weighting in terms of precision after a) 15 documents and b) 20 documents. The BD-ACI-BCA vector space method precision is shown below the bar.	175

A.1 Transforms performed on term signals. 190

A.2 Pre-transform calculations performed on term signals. 190

A.3 Post-transform calculations performed on term signals. 190

A.4 Component combination used. 191

A.5 Constants and extensions used in generating results. 191

LIST OF ALGORITHMS

3.1	Fourier Domain Scoring (FDS) algorithm	39
4.1	Cosine Domain Scoring algorithm	72
4.2	Quit- n for CDS using K accumulators and $\hat{\beta}$ components.	87
5.1	Fourier Domain Scoring (FDS) algorithm with Gaussian term spreading. Note that $\boxtimes$ is an element-wise multiplication.	107
5.2	Gaussian convolution method.	107
6.1	Wavelet Domain Scoring	135

CHAPTER ONE

INTRODUCTION

1.1 Looking Forward

THE advancements in computer networking, mobile telecommunications and storage technology in the past ten years have caused a massive explosion of digitally stored information. The public have been taken by this communications trend by connecting to the Internet, organising themselves with the help of a PDA (personal digital assistant) or even using a mobile phone with WAP. Each of these would be useless if it were not for the information that is passed back, stored and forward from them. For example, the interest in the World Wide Web is due to the fact that it is available to everyone for reading information and for publishing their own information. If more people use the Web it, becomes more resourceful and informative and results in attracting more users. Without the information, the Web would be a collection of links going nowhere. The answers to many questions can be found in large collections of data (such as the World Wide Web) due to the abundance of data stored in it. But just because the answer exists in the Web does not mean that we are able to find it.

The capacity of disks found in personal computers is growing at such a rate that they are approaching the capacity of what used to be a small network. To deal with all of this information, the personal computer user also requires the data organisation found in large text collections. For example, each email can be considered a text document. Millions of emails are sent through the Internet every day. Each email is usually stored on both the sender and receivers computer. Unless these emails are organised into categories by both the sender and receiver, they will find it hard to locate specific emails when needed.

The challenge at hand is to develop a system which is able to store large amounts of textual information and organise it in such a way, that it can be easily retrieved. We would want a generic system which analyses text so that if there were any other features

of the documents (eg. hyper-links), these could be processed over the text.

Locating text documents of a certain topic is not a simple task. The retrieval aspect is more complicated than extracting a record from a database. Each document may cover many topics and some topics may be user dependent (people might disagree on the topic of a document because the categorisation of these documents is subjective).

Many systems have in the past devised structures in the headers of text document, where the author would insert information about the document that would be useful for document retrieval systems (such as the meta-tags used in HTML). This system was used initially by Web search engines, but later found that this procedure failed as it places an element of trust on the author. Web page designers were taking advantage of this and inserting in the meta-tags terms not relevant to the document so that their pages would appear highly ranked in many different queries. This practice is called search engine persuasion (SEP) and is the reason why Web search engines turned to analysing the content of the documents rather than trusting tags.

There are also many huge document collections that already exist in textual form (such as patient records, log files, emails). To manually label each of these documents with a topic set would be infeasible. In systems where the documents are continually being created this process could take for ever! Therefore machines are needed to approximate the topic of documents

There have been many instances where users have tried to search for specific documents on the Web but have been disappointed by the quality of the results.

At present, the only way to find useful information on the Internet is to use a search engine and manually sort through all of the results returned. The search engines try to help us manually filter out the irrelevant results by displaying a small portion of the document text (an example of this can be seen in figure 1.1).

There has been a huge interest in relevant document retrieval, and several groups have developed methods to assist the user in obtaining relevant information. For example, Spertus [107] uses different types of connectivity and spatial locality to detect relevant pages; Mladenić [81] examines the pages previously visited by the user and uses these examples to learn what to retrieve in the future; Carrière et al. [21] calculates the score of a page based on how many relevant pages point to it through links; Ngu et al. [84] recommend that rather than search engines maintaining information about the entire Internet, each site should produce the information needed to produce better search results; Howe et al. [61] queries the existing range of search engines to obtain the best results from the collection of pages returned. We can see that all of the proposed methods need to be able to identify documents relevant to the query to operate correctly. There-

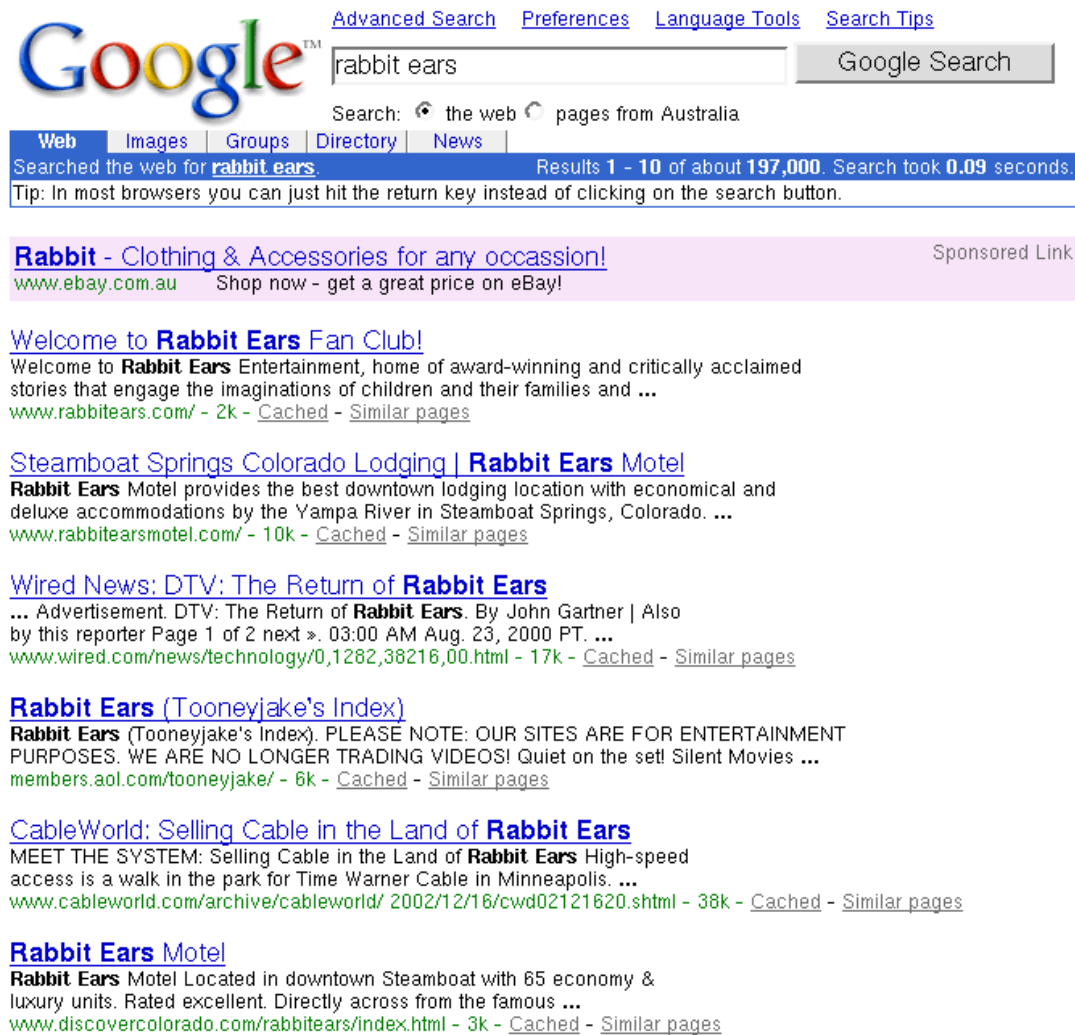


Figure 1.1: The results page after searching for “rabbit ears” in the Google search engine. Portions of the document content are presented to the searcher to assist in the manual document filtering.

fore they rely heavily on the underlying text processing algorithms to classify documents correctly.

1.2 Focus of this Thesis

This thesis focuses on the underlying text processing aspects of information retrieval. We have seen that many retrieval systems do not always provide us with the results we expect. We will assume that this is due to the fact that many methods only consider the

frequency of the terms in the documents, rather than the position as well. Methods that do consider the position of the terms do not take into account the advancements that have been made in using statistical measures, and are usually slow in comparison.

This thesis develops a new paradigm for representing terms throughout a document called a term signal. This term signal allows us to map the document information to the spectral domain by using the Fourier transform.

This thesis investigates a novel method of retrieving information from large collections of text documents. Like other text retrieval methods, the retrieval is performed by locating the appearances of the query terms in each document of the set. But unlike other text retrieval methods, we map this document information to the spectral domain and analyse the document spectrum with the help of the Fourier transform. By using the Fourier transform we are able to combine the current vector space methods with the proximity methods.

This thesis also examines disadvantages of using the Fourier transform method and devises methods to overcome these. Factors such as precision of results, query time and storage required are examined and experimented with. The effect of using the Cosine transform in place of the Fourier transform is also discussed and experiments are performed.

The notion of the proximity of the terms is investigated further through the application of the Gaussian function. This function allows us to control the region of proximity of the terms and also allows us to control the region of proximity of each term

Multi-resolution analysis is applied to the term signals to analyse the effect wavelet have on the document retrieval in terms of precision and especially compression.

Finally, a few extensions to the basic retrieval methods are discussed and examined through experimentation. These are applied to our spectral document retrieval method.

1.3 Organisation of the Thesis

Chapter 2: Document Retrieval provides an overview of where document retrieval has been and where it is currently at, including topics such as document ranking, index file structures, index compression and document and query weighting schemes. It also provides an explanation of the problem found in document retrieval ranking schemes and how we have chosen to solve this problem.

Chapter 3: Fourier Domain Scoring describes our new document ranking scheme. The ranking method compares query term signal spectra to provide a combination of

vector space ranking and proximity ranking into a fast and precise document ranking scheme. Analysis is performed on how the spectral ranking method copes with changes in frequency and proximity of the terms. Experiments are performed and the results are compared to existing vector space and proximity methods.

We find that the spectral method out performs the vector space and proximity methods when considering precision, but the query time and storage needed are greater than those of the vector space method.

Chapter 4: Efficient Implementation presents a method of performing spectral document ranking which has query time and index storage comparable to those of the vector space method, but also makes a small sacrifice in terms of precision. This method uses the Cosine transform which is an approximation to the Karhunen-Loève transform when using term signals. To perform the index compression, several techniques of component cropping and quantisation are enlisted. We achieve a faster query time by employing the use of an accumulator technique using early termination and slight modifications to deal with term signals.

Experiments are performed and the results show that by using the properties of the cosine transform, we are able to achieve high precision at the speed of the vector space methods, with comparable storage requirements.

Chapter 5: Gaussian Spreading can be used to spread the term signals via convolution, so that the proximity of each term can be found additively, or by furthering the spectral document ranking method by employing the use of Gaussian functions to spread the term signal elements across the term signals. This provides us with more control over the regions which individual terms cover when considering proximity. An investigation into sampling artifact effects such as aliasing and term signal wrap around is carried out.

Experiments were performed using various functions for the Gaussian width. It was found that an increase in precision was found after retrieving 10 documents when applying the Gaussian convolution method to the term signals. Little increase was found through applying the Gaussians after the Fourier transform. This would be due to the separation of the phase from the magnitude of the term signals.

Chapter 6: Multi-resolution Document Retrieval gives insight as to how we can apply various transformations on term signals to extract different types of information from them. The multi-resolution analysis done in this chapter is performed using the wavelet transform and various suited wavelets. By utilising the wavelet prop-

erties, we are able to adjust the resolution of the retrieval system so that we could have ranking based on the count of the terms in the document, or we could focus on each term signal component.

Experiments were performed and the results indicated that the use of wavelets increases the precision of the spectral ranking method and also reduces the compressed index size.

Chapter 7: Anaphora Resolution and Relevance Feedback examines and explores the techniques that can be combined with the basic document retrieval method to help improve the precision of the results. It explores the areas of anaphora resolution (which is a natural language processing technique that assists the document retrieval system in locating terms that refer to the query terms) and relevance feedback (which is a method of expanding the users query term set by examining the documents considered relevant for common terms).

Experiments were performed and the results provided the insight that relevance feedback helped improved the precision at the cost of extending the query time, and the anaphora resolution did not increase the precision of the results. The degrading effect of the anaphora resolution would have been due to the simple resolver, and we would expect an improvement if a more complex system was used.

1.4 Contributions of this Thesis

Below is a list of the major contributions of the thesis. The original contributions are sorted in order of appearance and are accompanied by the appropriate section number and citation.

1. Development of the term signal, which is essential for performing any spectral analysis of terms in documents (Section 3.2.1 [87, 91]).
2. Development of the Fourier Domain Scoring method to perform spectral document ranking (Section 3.2 [87, 91]).
3. Development of magnitude and phase precision used to calculate spectral document rank (Section 3.2.5 [87, 91]).
4. Investigation and verification that the Cosine transform will provide high precision results when using a cropped term signal (Section 4.3.3 [88]).
5. Investigation of term signal spectra quantisation error (Section 4.4.1 [90]).

6. Analysis of the use of query early termination with a spectral document ranking system (Section 4.5 [90]).
7. Implementation of a spectral document ranking system using the Cosine transform with quantisation and cropping of term signal spectra and verification that its query time is comparable to those of vector space measures (Section 4.5.4 [90]).
8. Analysis of discrete Gaussian functions in respect to spectral document ranking (Section 5.2.1).
9. Application of Gaussian spreading to spectral document ranking and experimental comparisons (Section 5.7).
10. Analysis of the use of the Haar and Daubechies-4 wavelets for spectral document retrieval (Section 6.3.2 [92]).
11. Verification by experimentation that spectral document retrieval using the Haar and Daubechies-4 wavelets provide similar precision with a more compact index, than that with Fourier transform (Section 6.7 [92]).
12. Application of a simple pronoun resolution algorithm to assist the spectral document retrieval system. (Section 7.2).
13. Development and verification through experimentation of a document feedback process for the spectral document retrieval system (Section 7.3).

1.5 Publications by the Author related to this Thesis

- [87] Laurence A. F. Park, Marimuthu Palaniswami, and Ramamohanarao Kotagiri. Internet document filtering using fourier domain scoring. In Luc de Raedt and Arno Siebes, editors, *Principles of Data Mining and Knowledge Discovery*, number 2168 in Lecture Notes in Artificial Intelligence, pages 362–373. Springer-Verlag, September 2001.
- [88] Laurence A. F. Park, Marimuthu Palaniswami, and Kotagiri Ramamohanarao. A novel web text mining method using the discrete cosine transform. In T. Elomaa, H. Mannila, and H. Toivonen, editors, *6th European Conference on Principles of Data Mining and Knowledge Discovery*, number 2431 in Lecture Notes in Artificial Intelligence, pages 385–396, Berlin, August 2002. Springer-Verlag.

-
- [91] Laurence A. F. Park, Kotagiri Ramamohanarao, and Marimuthu Palaniswami. Fourier domain scoring : A novel document ranking method. *IEEE Transactions on Knowledge and Data Engineering*, Accepted May 2003.
 - [90] Laurence A. F. Park, Kotagiri Ramamohanarao, and Marimuthu Palaniswami. A new implementation technique for fast spectral based document retrieval systems. In Vipin Kumar and Shusaku Tsumoto, editors, *2002 IEEE International Conference on Data Mining*, pages 346–353. IEEE Computer Society, December 2002.
 - [89] Laurence A. F. Park, Marimuthu Palaniswami, and Kotagiri Ramamohanarao. A novel document ranking method using the discrete cosine transform. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Submitted April 2003.
 - [92] Laurence A. F. Park, Kotagiri Ramamohanarao, and Marimuthu Palaniswami. A novel document retrieval method using the discrete wavelet transform. *ACM Transactions on Information Systems*, Submitted June 2003.

CHAPTER TWO

DOCUMENT RETRIEVAL

2.1 Introduction

EVER since the Assyrian King Ashurbanipal (~650 B.C.) [78], we have not only known the importance of storing information but also the importance of storing it in some order which is easy to retrieve. Ashurbanipal (shown in figure 2.1) was an Assyrian king who initiated the earliest known library of modern style. To build this library, he sent scribes to many parts of the world to carefully copy important texts onto clay tablets to be stored in the library. More than 5,000 texts were stored, so just as in our libraries today, a system was needed to retrieve these texts. Imagine going into a library to find a book, only to find that there is no particular order in the library. The only way to find the book in question would be to go through each book one by one.

The information retrieval system we use in libraries today was designed by Melville Dewey in 1872 [34]. and is called the Dewey Decimal Classification (DDC) system. This revolutionary organisation system is still practiced in most libraries today. To find a book on a certain topic, we would have to know the topic number (found in a list) and scan through the books in that cluster. To help the library user to find specific books, 3×5 inch index cards were introduced. Each card contained information (such as the name, author, publisher and subjects) about one book. The index cards would be stored in an order to assist the user in finding the book. For example, if we were to look for a specific author, we would look in the author sorted index.

The birth of the computer in the mid 1940's [40] allowed many fields of science to look towards automation, including information retrieval. Up until the late 1950's, much of the work involved in building automatic information retrieval systems was performed manually. Joyce and Needham [65] describe the approaches taken as manually recording information about the documents that would help with retrieval and supplying the computer with this information. Requests (queries) would be given in a similar fash-



Figure 2.1: The Assyrian King Ashurbanipal shown carrying a basket to rebuild a temple in Babylon (<http://web.utk.edu/~giles/>).

ion. The documents with information that matches the query are considered relevant. Experiments were first performed using thesauruses to increase the results returned.

The first fully automatic text retrieval system, called Key Word in Context (KWIC), was built by Hans Peter Luhn [71] (the father of information retrieval). Information was supplied to the computer on punch cards and stored on tape. The computer also calculated which terms were common in the text collection and removed these (stop word removal).

The field of automatic text retrieval was brought to life by Gerard Salton in the 1960's with his creation 'The SMART retrieval system' (known as Salton's Magical Automatic Retriever of Text) [112]. The SMART retrieval system has become the foundation for many researchers in the field of information retrieval. Many commercial systems use the ideas and methods developed with SMART as the main tool for text retrieval.

The birth of the World Wide Web in the early 1990's brought the field of automatic text retrieval to the people. The Web consisted of a collection of documents containing text and hyper-textual information to allow links from pages to pages. The main content on the page was text (which contained the information the users were searching for). Many methods for finding information on the Web have been tried and failed, leaving

only a few who have succeeded.

In this chapter, we will examine the basic concepts of information retrieval and explore the many methods which have been used. This will include a discussion on their downfalls and suggestions on how we can rectify these. In section 2.2 we will provide an overview of the components of an information retrieval system, section 2.3 discusses the many ways which data can be stored and retrieved in a compact manner, section 2.4 examines the different methods of document retrieval. Section 2.5 presents some methods to enhance retrieval. In section 2.6 we observe the methods of experimentation used, and in section 2.7 we will discuss the problems and solutions of modern document retrieval.

2.2 Document Retrieval Process

The information retrieval process (figure 2.2) begins with the query formulation. Before a user can interact with a system, it must be able to provide it with some information that the system will understand. In this case it is taking our information need and translating it into a query. Once this is done, the system will scan the data collection and extract data which best matches the query. This answer set is then presented to the user. The shaded boxes in figure 2.2 extend the information retrieval process with user feedback. After the user reads the results, he will choose the documents which are relevant to the *Information need*. If the need is satisfied, the process finishes. If the need is not satisfied, then the user will formulate a new query based on the new information obtained in the previous results. This new query is then fed back into the retrieval system. This process continues until the user is satisfied.

Properties which we would look for in a well designed information retrieval system are a compact data store which is fast to access and a fast retrieval system which provides precise results from basic queries. We will begin our tour of information retrieval aspects by first examining how we can store data in a compact manner which is fast to retrieve, and second analysing the methods by which we can retrieve the data to provide fast accurate results.

2.3 Storing Information

To design a successful information retrieval system, we must be able to store the data we obtain in a compact manner. This section examines different methods of doing this.

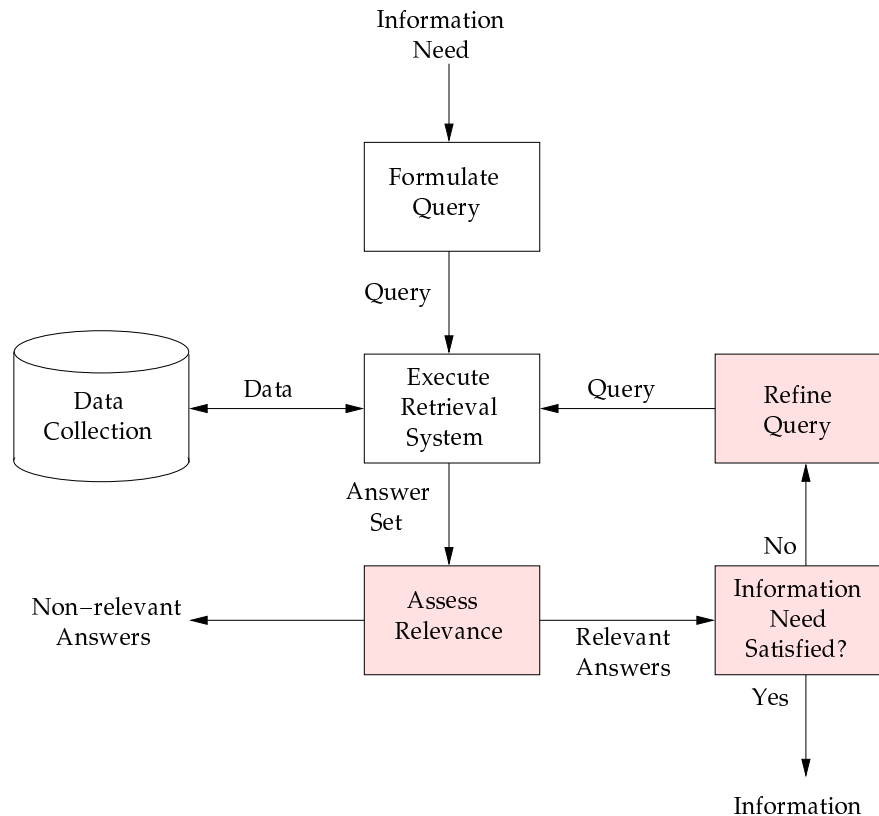


Figure 2.2: The information retrieval process [30]. The shaded portions of the chart are the post-retrieval processes.

2.3.1 File Structures

To create a successful document retrieval system, we must be able to provide accurate results and be able to provide them at a fast rate. For every query that is processed, calculations must be performed on every document in the document set. Therefore, we must spend as little time as possible on each document to obtain a fast query time. So far we have examined different methods of extracting the estimate of the set of relevant documents from the document set. In this section we will discuss several methods of storing the data, so that we can retrieve it with ease and perform the desired calculations.

Inverted Files

The general assumption of most retrieval systems is that it is enough to know how many times the query terms appear in a document to judge whether it is relevant or not. Therefore most storage structures are based upon storing only the count of each term in each

	t_0	t_1	$\dots$	t_M
d_0	0	1	$\dots$	0
d_1	2	0	$\dots$	0
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
d_N	0	3	$\dots$	1

Table 2.1: An example of a simple index to be used with the vector space methods.

	d_0	d_1	$\dots$	d_N
t_0	0	2	$\dots$	0
t_1	1	0	$\dots$	3
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
t_M	0	0	$\dots$	1

Table 2.2: An example of an inverted file to be used with the vector space methods. Only the terms related to the query need to be extracted.

document. If we gather these values for each document, we obtain a document vector. Each document vector d_n consists of a sequence of integers which represent the count of a term in the document. The first integer is the count of term t_0 , the second integer is the count of term t_1 and so on for all terms $t_m \in T$ (where T is the set of all unique terms in the document collection). Using this method, the entire document collection can be stored in a table, where the rows represent each document d_n and the columns represent each term t_m (as in table 2.1).

To obtain how relevant a document is to a query, we will only need to extract the information about each of the query terms for each of the documents. When an index is stored on a disk, it will be stored row by row. If we were to extract a row, we would simply grab the chunk of data which represents the row, but if we were to extract a column, we would need to find each column element in each row and extract them one by one (which is a much longer process). Therefore it makes sense to transpose our index, so that the terms become rows and the documents become columns (as in table 2.2).

Signature Files

Rather than taking the approach of scanning through all of the information for every query, we can use a signature file [42] to reduce the set of documents to scan.

Term	Signature
t_0	0000 0110 0000 0001
t_1	0001 0010 0010 0000
$\vdots$	$\vdots$
t_M	0010 0000 0001 1000

Table 2.3: Example of signatures associated to the terms.

The idea behind using a signature file, is to assign each unique term in the document set with a binary signature (where the number of 1s per signature is sparse). The signature is calculated using several hash functions to calculate the positions of the 1s. The remaining positions will be zeros. An example of the term signatures is shown in table 2.3. By combining the signatures of the terms which exist in document d_n (using the OR operation), we obtain the signature for the document. For example, if the document contained only the terms t_0 , t_1 and t_M from table 2.3 then the document signature would be 0011 0110 0011 1001. When we receive a query, we can compare each query term signature with the document signature to find if the term does not exist in the document. If the query term signature AND the document signature is not equal to the query term signature, then the term does not appear in the document. Otherwise, we must examine the document for the query term.

To make the operation faster, we can use the same idea as used with the inverted file, by inverting the document signatures to obtain a bit sliced signature file. When a query is given, we need only extract the bit slices that correspond to the 1 bits in the query signature.

Bitmaps

If we create a signature containing as many bits as we have unique terms, and we use an injective hash function, we have what is known as a bitmap. Bitmaps are also related to inverted files where we only use boolean values rather than integers. Bitmaps can be used to perform boolean queries at a fast rate due to their structure, but because we must store all of the bits, the size of a bitmap can become very large as the size of the document collection increases.

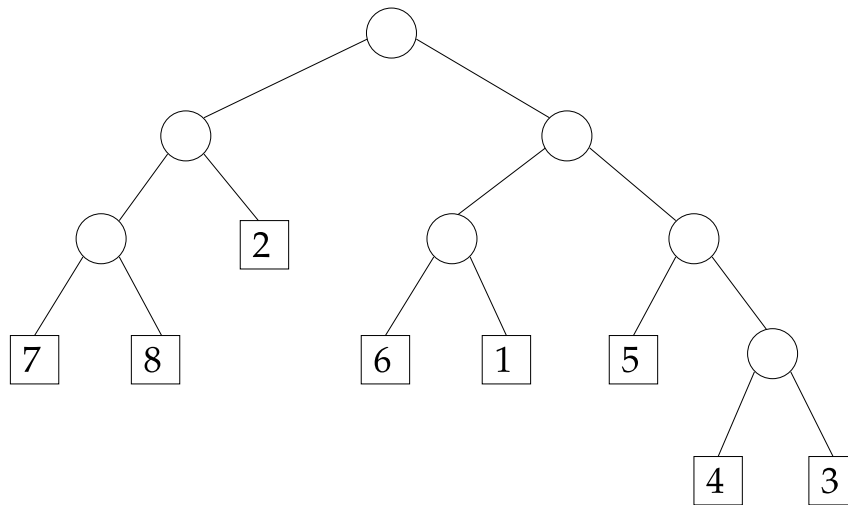


Figure 2.3: A PAT tree for the text “101110001...”. The numbers in the tree leaves represent the position of the chosen sequence. Eg. the binary sequence 110 begins at the 5th bit in the binary text sequence.

PAT Trees

A PAT tree [49] (based on the Patricia tree) is unlike any of the indexing methods we have reviewed. Patricia trees are binary search trees where data is only stored at the leaves of the tree. A PAT tree is a Patricia tree that stores semi-infinite strings at its leaves. A semi-infinite string is a string which has an origin, but no end (just as in a semi-infinite line). The PAT tree stores the location of the origin of the semi-infinite string at the leaf, so not only can we query the index by frequency of the query terms, but also location relative to each other by comparing the positions. An example of a simple PAT tree is found in figure 2.3.

To query the index, we take the query term and proceed down the PAT tree until we reach a leaf, or we can proceed no more due to a lack of characters in the query term. The subtree (beginning at the point which we finished traversing the tree) contains all of the occurrences of the query term. To find the occurrences, we must traverse the entire subtree.

2.3.2 Coding

To perform fast retrieval with a acceptable sized index, we need to have a fast indexing system and a compact method of storing the data. In all of the methods mentioned, we have used the positive set of integers, therefore, this will be considered as our set of

symbols.

Two of the best known coding methods for finite, countable sets are Huffman coding [62] (named after its inventor David Huffman) and arithmetic coding [122]. Huffman coding uses statistical information about the symbols which it has to code to achieve high level of compression. The symbols which occur the most use shorter codes while the more rare symbols use longer codes. Arithmetic coding, rather than coding each character at a time, encodes the message into an interval based on the probabilities of the symbols. Since both of these methods require knowledge about the entire data set before compressing, it would take a long time to encode a large set.

In this section we will present a few methods which balance the tradeoff between time and space. All of these methods assume that the data which we will want to encode are small integers (close to zero). If we consider the inverted file, for each term, we need to encode:

$$\langle f_t, \langle d_1, f_{d_1,t} \rangle, \langle d_2, f_{d_2,t} \rangle, \dots, \langle d_{f_t}, f_{d_{f_t},t} \rangle \rangle \quad (2.1)$$

where f_t is the number of documents containing the term t , d_n is the ordered document numbers and $f_{d,t}$ is the occurrences of term t in document d . Instead of coding this data directly, we can make use of the order of the data and encode the differences rather than the data itself. The document numbers are in order, therefore the encoded differences will all be small positive integers (as long as the term represented appears in many documents). Most frequency counts ($f_{d,t}$) will be small. The term numbers are in order, but all of the terms that are given exist in the document set. Therefore we do not need to encode this value, all we need is a pointer to the beginning of the term entry (inverted list). This implies that we need coding schemes which can code small integers efficiently and use more bits for the rare large integer values.

Unary Coding

This method of coding [39] is a simple scheme that uses ones to define the actual value and zeros to represent the end of the symbol. To encode the integer x , we write $x - 1$ ones followed by a zero. So the number 5 would be encoded 11110, therefore it takes n bits to encode the value n . This would be fine for data set that do not contain any large values, but would not be useful for encoding the frequency values of common terms.

γ Coding

The γ code [39] uses slightly more bits to encode small values, but less values to encode greater values. This code uses the unary code to encode $q + 1 = 1 + \lfloor \log_2 x \rfloor$ of the value

x , followed by q bits containing the remainder $r = x - 2^{\lfloor \log_2 x \rfloor}$ encoded in binary. This is decoded by extracting the unary portion into y , then decoding the next $y - 1$ bits in binary. For example, to encode 5, we would encode $\lfloor \log_2 5 \rfloor + 1 = 3$ in unary, and the remainder (1, due to the round off error) in binary. This gives 110 01. This implies that the symbol will be $1 + 2\lfloor \log_2 n \rfloor$ bits long to encode the integer n .

δ Coding

This form of coding [39] is similar to the γ code, in the way which it encodes a $\log_2$ approximation to x followed by the remainder. The difference is that the *delta* code uses the *gamma* code to encode the divider, rather than the unary code. For example, if we encode the integer 5, we would encode $q + 1 = 1 + \lfloor \log 5 \rfloor = 3$ using the γ code (the γ coding of 3 is 10 1), and encode the remainder (r) using binary. The number of bits needed for remainder (r) is one less than the γ encoded value (q). Therefore, the result of encoding 5 is 101 01. This implies that the number of bits required to encode the integer n would be $1 + 2\lfloor \log_2 1 + \lfloor \log_2 n \rfloor \rfloor + \lfloor \log n \rfloor$

Golomb Coding

The three coding models we have just shown all make tradeoffs between the number of bits needed for to encode small integers and the number of bits needed to encode large integers (eg. unary coding compresses small integers well, but not large integers). Golomb coding [47] was developed to allow easy selection of how much of this tradeoff we desire. The Golomb coding method uses the quotient remainder method, but a parameter is given so that the level of coding can be set. The code quotient portion is given as $q + 1 = \lfloor (x - 1)/b \rfloor + 1$ which is encoded in unary, and the remainder is given as $r = x - qb - 1$ which is encoded in binary. For example, if we want to encode $x = 5$ and $b = 4$, then $q + 1 = \lfloor (5 - 1)/4 \rfloor + 1 = 2$ and $r = 5 - 4 - 1 = 0$. Therefore, the encoded value is 10 00. Witten *et al.* [121] suggest that the parameter b should be chosen by the following equation:

$$b \approx \frac{\log_e 2}{p} \approx 0.69 \cdot \frac{N \cdot n}{f} \quad (2.2)$$

where p is the average frequency count of a term in a document, f is the total number of terms in the collection, N is the number of documents and n is the number of unique terms in the collection. Note that if $b = 1$, then the Golomb code becomes the unary code.

2.3.3 Lexical Preprocessing

While building an index, we are able to construct a database containing all of the terms found in the document set and statistics about each term. This set of terms is called a lexicon. Once we obtain the lexicon of the document set we are using, it is customary to perform preprocessing on this set while building the index. This is to reduce the load on the information retrieval system at the query time. Methods of preprocessing include case folding, stop word removal and stemming.

Case Folding

The operation of case folding involves converting the case of every letter in the lexicon to a common case (usually lower case since there would be less upper case letters to convert). This operation is aided by the form of the ASCII character coding where conversion of case involves a simple shift by a constant hexadecimal value.

The characters which we will case fold must be chosen wisely. Acronyms appear in many texts and most will try to resemble a common term so that the term will remain in the readers memory. For example, if the acronym of 'Words Indicating Tame Hermitians' (being WITH) was used, this would be folded to 'with'. Any queries involving the term 'WITH' would return any document containing the term 'with' as well, this means that case folding this term has rendered it useless as a key term.

Stop Words

Many terms found in the English language (such as a, and, the) are not useful as query terms because they are so common. If they were to be included in the index, their term weight would be very low. If they were to be used in a query, nearly all of the documents in the set would be returned since nearly every document would contain these words.

Since these terms are very not useful, we create a list (called a stop list) of these terms (called stop words) which we know of, and make sure that the indexer ignores these terms when the parsing is performed. By doing this we reduce the number of terms contained in the document set lexicon, and therefore we reduce the amount of processing to be done by the indexer. An example of a stop list can be found in [44].

An argument is given in [121] about whether or not stop words should be included in an index. An example of a reason of why to include them is that important information could be removed with them. If we use the previous example from the case folding section, the term 'with' is a common term which would be removed. Therefore any reference to the acronym WITH would also be removed.

Stemming

When we use an information retrieval system, we would like our query to find all of the relevant document in the document set. If we were to perform a search on 'eating', we would like the search engine to find document containing the term 'eating' and variants of the term (eg. 'eat', 'eats', 'eaten'). To perform such an operation, we need to employ the use of a word stemmer. A word stemmer converts each term to its stem. For example, each of the terms mentioned in this section would be reduced to the stem 'eat'. The two most widely used automatic stemming algorithms are Porters stemmer [93] and the Lovins stemmer [69].

2.4 Retrieving Information

Once we have effectively stored the document information, we need to be able to extract and use it to calculate which documents best match a query.

2.4.1 Retrieval Methods

In this section we will list several methods of performing the document relevance calculations.

Boolean Retrieval

The simplest way to retrieve a document set is to examine it for the query terms given. If a document contains the query terms then the document is returned as being relevant. Queries which are given to perform this task are called *conjunctive queries*. To perform this task, the retrieval system will work term by term. The first query term q_1 is selected and all of the documents which contain query term q_1 are put into the relevant set R . Then all of the documents which do not satisfy query term q_2 are removed from R . The same is done for the rest of the query terms. Once we are finished, we are left with R containing all of the documents which contain all of the query terms given. This set is then returned to the user.

This method will be adequate for small document sets, but as the number of documents grows so do the number of documents returned. The only way to reduce the number of documents returned in a boolean retrieval system is to refine the query and make it more specific.

When giving a boolean query, we must make use of the boolean operators construct the query. The operators come in the form of AND, OR, NOT, and XOR and parenthesis

must be used to group the operations. For example, if we wanted to find all of the documents which were about 'funk music' or 'jazz music' but we did not want any documents about the 'muppets' playing music, we would give the following query:

((funk OR jazz) AND music) NOT muppets)

To perform this type of query, the retrieval system must first find the document sets which are wanted (from the inner most set of parentheses) and then add or remove document according to the query.

The problems associated with method of retrieval and some ways of overcoming them are described by Cooper [26].

Ranked Retrieval

The boolean retrieval method returns a set of documents which are all considered as relevant as each other. This implies that the user should examine each document individually to find the information he wants. To make this process quicker, we could ask the retrieval system to order the documents in what it finds to be the most to least relevant to the topic.

If we consider the inverted file (section 2.3.1), we can see that each column is a document vector. The elements of this vector represent the count of a unique term in the document set. The inverted file is made up of a set of N document vectors $\{d_0, d_1, \dots, d_{N-1}\}$ existing in a T dimensional space ($d_n \in \mathbb{R}^T$), where T is the number of unique terms existing in the document set. If we examine figure 2.4, we can see that document D1 contains two occurrences of term 1 and one occurrence of term 2, D2 contains one occurrence of term 1 and three occurrences of term 2, and D3 contains no occurrences of term 1 and two occurrences of term 2.

If we perform the same mapping to the query, we will obtain a query vector existing in the same space as the document vectors. We want a measure which tells us which documents have the same terms as the query. Each term is an orthogonal basis vector, therefore we want the documents which are pointing in the same direction as the query vector. A measure of directional similarity is the dot product. If the dot product of vectors α_1 and α_2 is greater than the dot product of α_1 and α_3 , then the direction of vector α_2 is more similar to α_1 than α_3 . By using the dot product, we are able to give each document a score based on its directional similarity to the query vector. We can use the score to rank the documents in order of relevance to the query. Therefore, the score function will be:

$$S_{dot\ product}(d_n, q) = d_n \cdot q = \sum_{t \in T} f_{d,t} f_{q,t} \quad (2.3)$$

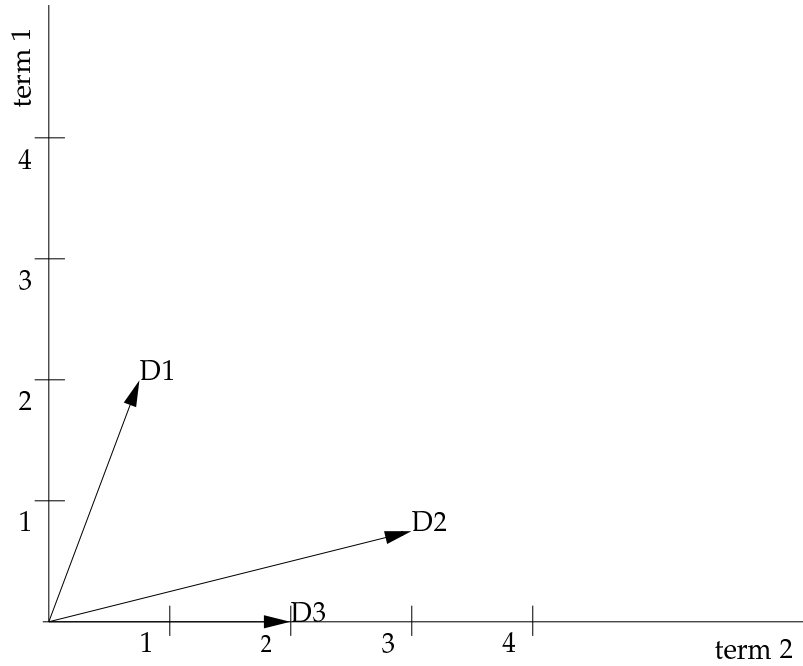


Figure 2.4: Documents (D1, D2 and D3) are represented by vectors in the vector space method.

where $f_{d,t} \in d_n$ and $f_{q,t} \in q$.

This method of ranking has the bonus of displaying the documents to the user in an ordered form based on the occurrence of query terms in each document, but the results may be deceiving. The dot product method is biased towards longer documents (which will probably have more occurrences of the query terms than the shorter documents) and the dot product does not take into account the rarity of the terms. For example, if we were searching for the terms “jazz muppets”, and the term “muppets” appeared in only one document, but “jazz” appeared in many documents, the term “muppets” is more rare than the term “jazz” and should have more impact. This would make the document which contains “muppets” more relevant.

Due to the factors mentioned, we need to use weighted frequency counts of the terms based on the statistics of the document set, rather than just using the frequency count. There have been many weighting schemes derived from many different sources. Salton and Lesk [99] observed this problem and used the cosine measure to aid in one aspect of it. By using the cosine measure instead of the dot product, we are normalising each of the document and query vectors so that their l_2 norm is 1.

$$S_{\cos}(d_n, q) = \frac{d_n \cdot q}{\|d_n\|_2 \|q\|_2} \quad (2.4)$$

where $\|x\|_2 = \sqrt{x \cdot x}$. This measure gives the cosine of the angle between the document and query vector. Therefore, if the vectors are aligned, the angle is 0 and the score will be 1. The greater the angle, the lower the score. We will discuss further weighting schemes in section 2.4.2.

Probabilistic Models

The key terms that identify a document are all in the eye of the beholder. One person might identify a set of key terms while another will identify a different set (usually with some overlap). If we cannot classify a document ourselves, how are we meant to program a machine to do it for us? This was the approach taken by Maron and Kuhns [75] in their paper on probabilistic information retrieval, where they introduce the concept of semantic noise. Semantic noise occurs when relating a set of documents to a few key terms. In the previous models mentioned, we have assigned documents with topic terms (either manually or automatically), so if a search is performed and the query terms do not exist in the set of document topic terms, the document is considered irrelevant. But what if the terms only partially represent a document. We should be able to weight terms, so that a term which is considered a little relevant to the document can be assigned 0.1 rather than 0, and terms which do not totally represent the document can be assigned 0.9 rather than 1. This term weighting technique is called Probabilistic Indexing. This type of probabilistic weighting can be calculated from a document set when given relevance information about the documents. Examples of probabilistic models can be found in [20, 97, 95, 111].

Croft and Harper [27] discuss how to construct a probabilistic model without using any information about document relevance. They show that if this is done, the model collapses to the vector space model. This is due to their use of treating documents as vectors for their initial calculations.

Cognitive Models

Belkin *et al.* [8] states that the current retrieval methods use a ‘best match’ principle, which can be a problem depending on the query to be posed. They write that the problems are:

- the query terms specified as *being significant*, actually *are* significant
- the query must be expressed in the same style as the documents are written

If these are not satisfied, then the information retrieval system will not be able to match the query to the most relevant documents.

They also state that the reason why the information retrieval system will be used is because the searcher has a lack of knowledge on the subject being searched for. Therefore, how could the user provide a query on a topic which he is ignorant of? The concept of the cognitive retrieval system is interaction between the machine and the searcher. With their combined state of knowledge, they could both learn from each other and eventually produce documents of interest to the searcher. Another way of thinking of this model is as if it is a librarian. We don't give key terms to librarians, we ask questions and they try to answer or ask more questions.

Latent Semantic Indexing

If we again examine the inverted file (section 2.3.1) and treat the documents as vectors, we are assuming that every term is independent of every other term (this is due to each term being orthogonal to one another). An idea from Deerwester *et al.* [32, 37, 9] states that every document set has its own hidden semantic rules, which must be uncovered before we can successfully perform document retrieval on it. This is called Latent Semantic Indexing (LSI) and the method suggested to perform this was singular value decomposition (SVD) to extract the left and right singular vector sets. Singular value decomposition is similar to an eigenvalue decomposition in that it will reposition the basis vectors of the space so that the basis vectors are ordered in terms of importance and the first n dimensions will be the least squares estimate in n dimensions of the entire set. SVD differs to the eigenvalue decomposition in the way that it decomposes the data matrix into a left and right singular matrix and a set of singular values. The Eigenvalue decomposition splits the data matrix into a set of Eigenvectors and a set of Eigenvalues. The SVD is as follows:

$$A = U\Sigma V \quad (2.5)$$

where A is the original data matrix, U is the left singular matrix, Σ is the diagonal matrix of singular values, and V is the right singular matrix.

If A is the index we are using which has each row being a document and each column being a term, then the left singular matrix will be a set of document vectors (one for each row) and the right singular matrix will be a set of term vectors (one for each column). Each of the singular values correspond to an element of the document and term vectors. The magnitude of the singular value represents how much effect the corresponding component has on the space. Therefore, after the SVD has been performed, we can start

removing components which have singular values lower than a certain threshold.

By reducing the dimensionality of the term space, we no longer have orthogonal terms. This implies that terms which appear in the same documents will be clustered together. By reducing the dimensionality of the document space, the documents are no longer orthogonal either, therefore documents which contain similar terms will be clustered together.

When performing a query, we must first map the query vector into the reduced dimension document space. Then we can perform our usual vector space method ranking.

A problem with using the SVD is that it takes a long time to perform. The time complexity to decompose an $m \times n$ matrix is $O(mn^2)$ [48] and if any new data is added or old data removed, we can use one of the incremental SVD methods as in [22] (which becomes very inaccurate as the amount of data added grows) or we must compute the SVD of the whole matrix again. Methods have also been devised to calculate the SVD of sparse matrices [10] (the document index table is very sparse because most documents contain only a fraction of the terms found in the document collection lexicon). But time must be spent uncompressing and compressing the values in the compressed index.

LSI has also been performed with the ULV and URV decompositions [11] and the semi-discrete matrix decomposition (SDD) [67].

If we ignore the problems of time complexity, LSI using SVD becomes a huge competitor to the vector space methods.

Document Clustering

Document clustering has existed in our library system for many years. Books in a library cataloged according to the Dewey system are given unique identification numbers according to their classified topic. This enables us to find the books we want and also obtain an idea of any other literature that exists on the topic.

One of the first mentions of automatic document clustering was by Doyle [35]. The idea given was to examine documents for common co-occurring terms. Documents containing these terms would be put into the relevant cluster, which would be linked to other clusters according to their co-occurring terms.

Many document clustering methods are drawn from the machine learning field and can be classed in to either supervised or unsupervised learning [60]. Unsupervised methods allow the clusters to form naturally. The clusters are usually identified by a dense collection of vectors within some area based on some chosen metric. Supervised methods begin with identified cluster centers and the documents situated about these centers belong to the cluster.

When designing these systems, there are many choices to make. One being how close is close enough? What make a vector belong to a cluster and not another?

Clustering documents is a hard task because documents do not normally focus on one topic. Therefore, there would be many cases where we would be unsure as to which topic it should cluster to.

2.4.2 Query Process

Once a query is given to a document retrieval system, we must use this query to extract relevant documents from the document set and return them to the user. Most retrieval systems do this by comparing the similarity of the query to each document and attaching a score to each document based on this similarity. The documents are presented in order of relevance to the user, who then chooses the desired documents from the set. In this section we will discuss some of the methods that are used to calculate this similarity.

Weighting Schemes

When scoring documents, we want the score to be independent of the document length and we want common terms to have a smaller effect than less common terms.

When comparing terms to documents, it is usually the case that a document will be relevant if it contains many occurrence of the query terms. But if a document contains many occurrences of query term 1 and another document contains many occurrences of query term 2, which is more relevant? Term importance can be calculated by gathering statistics about the document set. Term frequency (tf) is regarded as the number of occurrences of that term in the document in question. Inverse document frequency (idf) is the inverse of the number of documents the term appears in. Salton and Buckley [101] suggest that to obtain a reasonable measure of term importance, we should calculate its term frequency $\times$ inverse document frequency (tf $\times$ idf).

Another factor that affects the ranking process is the length of the documents. If we have a short document that is totally focused on the topic, and we have a huge document that is partially focused on the topic, and they both have the same number of occurrences of the query terms, should they be ranked equally? The general view is that the shorter should be ranked higher because it is more focused (hence more relevant). To take into account the length of the document when calculating the document score we should normalise the score according to document length. If we treat each document as a vector, where each element is the count of a certain term, normalisation of the documents would reduce all of the document vectors to unit length.

Position	Description
a	Document term frequency component
b	Term inverse document frequency
c	Document normalisation
x	Query term frequency component
y	Term inverse query frequency
z	Query normalisation

Table 2.4: Description of weighting method symbols used by Salton and Buckley.

Symbol	Description
L	$(1 + \log f_{d,t}) / (1 + \log (\text{average } f_{d,t}))$
n	1
u	$(1.0 - \text{slope}) \times \text{pivot} + \text{slope} \times \text{\#of unique terms in document}$
l	$1 + \log f_{q,t}$
t	$\log(N / f_t)$
u	$(1.0 - \text{slope}) \times \text{pivot} + \text{slope} \times \text{\#of unique terms in query}$

Table 2.5: Formulation of the Lnu.ltu weighting method.

Salton and Buckley [101] give a method of representing the weighting scheme they are using as $abc.xyz$ where abc is the document term weight ($w_{d,t}$) and xyz is the query term weight ($w_{q,t}$). A description of the symbols can be found in table 2.4. The document score was calculated by multiplying the two and summing over all query terms.

$$S_{q,d} = \sum_{t \in T} w_{d,t} \cdot w_{q,t} \quad (2.6)$$

An example of a method using this weighting formulation is the Lnu.ltu weighting method from SMART [17]. This method consists of the components shown in table 2.5

Zobel and Moffat [126] provided a more general structure of the form AB-CDE-FGH, to allow for the many different scoring functions that had appeared. The description of each symbol can be found in table 2.6. Their study compared 720 different weighting schemes based on the TREC document set. Their results showed that the BD-ACI-BCA method was the overall best performer. The components of the BD-ACI-BCA method can be seen in table 2.7.

Position	Description
A	Combining function ($S_{q,d}$)
B	Term weight (w_t)
C	Document term weight ($w_{d,t}$)
D	Relative document term frequency ($r_{d,t}$)
E	Document length (W_d)
F	Query term weight ($w_{q,t}$)
G	Relative query term frequency ($r_{q,t}$)
H	Query length (W_q)

Table 2.6: Description of weighting method symbols used by Zobel and Moffat.

Symbol	Description
B	$S_{q,d} = \sum_{t \in T} (w_{d,t} \cdot w_{q,t}) / (W_d \cdot W_q)$
D	$w_t = \log_e (1 + \frac{f_t^m}{f_t})$
A	$w_{d,t} = r_{d,t}$
C	$r_{d,t} = 1 + \log f_{d,t}$
I	$W_d = (1 - s) + s \cdot W'_d / \text{average} W'_d$
B	$w_{q,t} = r_{q,t} \cdot w_t$
C	$r_{q,t} = 1 + \log f_{q,t}$
A	$W_q = 1$

Table 2.7: Formulation of the BD-ACI-BCA weighting method, where $W'_d = \sqrt{\sum_{t \in T} w_{d,t}^2}$

Pivoted Document Normalisation

The concept of the cosine measure to score documents was wide spread because it provides high precision and it is easy to understand and implement. It was assumed that normalisation relative to document length could be performed by dividing with the norm of the vector, but experiments produced by Singhal *et al.* [106] have shown otherwise. A study was performed to examine the relationship between the probability of relevance of a document versus the size of the document. When the actual relevance was compared to the relevance given by the cosine measure, it was found that this document normalisation favoured documents of shorter length

The cosine measure requires that the weighted query and document vectors are normalised by dividing by their Euclidean norm (l_2). This norm is directly related to the

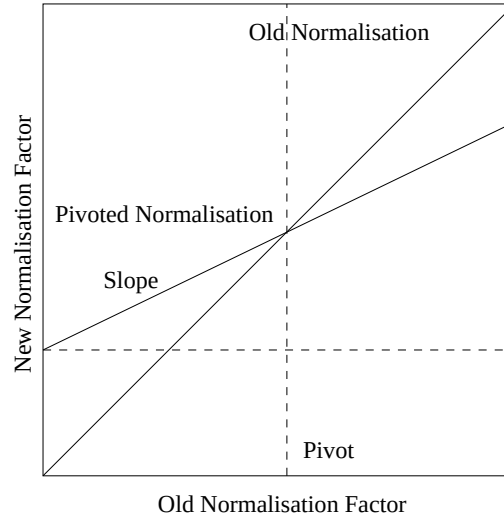


Figure 2.5: Pivoted normalisation adjusts the document score to give higher relevance to longer documents and lower relevance to shorter documents

document length, therefore if we want to remove the bias towards shorter documents, we must adjust the document normalisation.

If we examine the difference between the cosine measure and true relevance plots, we can see that the error can be corrected if we pivot the cosine measure about the point where the two intersect (the pivot point, shown in figure 2.5). The new document normalisation becomes:

$$\begin{aligned} \text{new normalisation} &= \text{old normalisation} \times \text{slope} + \text{pivot} - \text{pivot} \times \text{slope} \\ &= \text{old normalisation} \times \text{slope} + \text{pivot} \times (1 - \text{slope}) \end{aligned} \quad (2.7)$$

Pivoted document normalisation is used in the previously mentioned Lnu.ltu and BD-ACI-BCA weighting schemes.

Proximity Methods

Proximity document retrieval methods delve deeper into each document. Rather than just scratching the surface of each document by counting the terms, they use the term positional information to calculate the document score. But like many data mining methods, it is not easy to find an algorithms to combine the extra information we obtain from each document into a single document score.

An algorithm based on term position suggested by Keen [66] requires the positions of every query term to be extracted from the document in question. Experiments were

performed using several algorithms. Some of the scoring methods implemented were: to use the term distance between pairs of query terms, to use the term distance between the set of query terms, to select only the best sentence in the document, to select all sentences in the document, to base the score on the number of query terms contained in a chunk of text, or to base the score on the number of terms separating the query terms. The document score is then calculated by finding some permutation of the average term distance

Clarke and Cormack [24] present a thorough analysis in their paper on query term proximity searching. A boolean information retrieval model is given, in which the document scores are based on the shortest sub-string of text in the document which matches the query. This is done by creating a data structure called a ‘Generalised Concordance List’ (GC-list). These GCL’s contain the span of a given term throughout the document, in such a way that the spans are minimum and unique. An example of a GC-list for the terms “hot dog” is:

$$\mathcal{I}(\text{“hot dog”}) = \{(3,4), (19,34), (40,41), (92,98)\}$$

In this example we can see that the extents are very close ((3,4) and (40,41)) because “hot” and “dog” are often found together. An efficient implementation is also discussed. The implementation requires an index to be built, where the term positions are stored with every term. By doing this, the terms can easily be extracted at query time and the corresponding Concordance list can be built to generate a document score. The document score calculation was based on two assumptions; that the shorter the extent, the more likely that the section of text is relevant, and the more extents found within a document, the more likely the document is relevant. The score for each extent arrived at was:

$$I(p, q) = \begin{cases} \left(\frac{\mathcal{K}}{q-p+1} \right)^\alpha & \text{if } q - p + 1 \geq \mathcal{K} \\ 1 & \text{if } q - p + 1 < \mathcal{K} \end{cases} \quad (2.8)$$

The top parameter was chosen to give monotonically increasing effect as the extent parameters drew closer together. The constant $\mathcal{K}$ is chosen as a closeness parameter. If the extent parameters are within this value from each other, it signifies that the extent parameters are close enough to be given a score of 1. For example, we might consider all terms within 1,2 or 3 terms to be just as close. In this case we would set $\mathcal{K} = 3$. The other constant (α) controls the fall off of the score.

Once all of the extent scores have been calculated, we simply sum them together to obtain the document score.

$$S_{d,q} = \sum_{i=1}^N I(p_i, q_i) \quad (2.9)$$

If we take a closer look at this method, we will see that the method is limited when we use more than two query terms. For example, if we have the query terms “hot dog vendor”, the following two text strings will have the same score.

“hot dogs taste terrible when sold from vendors”

“hot collars can be bought from dog vendors”

This is due to the span of the query terms being the same, even though they are of different topics.

Experiments were performed and compared against a vector space method with Okapi weighting [97]. The results showed that the proximity method outperformed the Okapi method when using boolean queries manipulated from the TREC query set and applied to the TREC document set.

Hawking and Thistlewaite [57] devised a similar scheme using query term span lengths. In their method, they allow overlapping query term spans, as long as they each have a unique starting point in the text. If two have the same starting point, the shorter of the two is chosen. Once the spans have been found, the document score is given by the equation:

$$S_{d,q} = \sum_{t=1}^k (I_p \times r_{(p,d)}) \quad r_{(p,d)} = \sum_{i=1}^j \frac{1}{s_i - 1} \quad (2.10)$$

where k is the number of proximity relationships in the query, I_p is the importance of proximity relationship p , $r_{(p,d)}$ is the relevance of document d according to proximity relationship p , and S_i is the length of the i th proximity span.

2.5 Advanced Retrieval

If we move on from the basic retrieval where we examine the documents for the query terms, we will discover that there are extensions to the basic method which could help improve our retrieval system. Such extensions are relevance feedback and natural language processing techniques.

2.5.1 Relevance Feedback

When a user supplies a list of key terms to a document retrieval system, the list is always subjective. Different users will supply different sets of key terms and expect the same results. A user will also want to put in as few key terms as possible (no one wants to write an essay as a query) and expect the query to be focused on the wanted topic. We

cannot expect the retrieval system to read our mind, it can only provide us with results that match the query or it could prompt us for more information.

The notion of relevance feedback [98] is a process where the retrieval system supplies the user with a small set of documents which match the users query. The user then selects the documents that best match the desired topic. Based on this new information, the retrieval system can perform a more refined document search and provide more precise results.

Experiments in automatic relevance feedback have been performed [15]. In this case the retrieval assumes that the documents which were calculated to be most relevant are relevant. These are used in the feedback process. This makes the retrieval process quicker because there is no human interaction during the feedback, but it is less precise because we are not guaranteed that the feedback documents are relevant.

2.5.2 Natural Language Processing

For a person to be able to communicate effectively with a machine, he must first learn the machines language. The same goes for the document retrieval process. If the user understands how the document retrieval system works, he will be able to give it the appropriate queries and get the most out of the machine. Most people will not want to learn about the machine, they just want the results. Also, teaching every user the principles of information retrieval will be a long tedious task. So rather than us learning the machines language, why don't we get the machine to understand our language?

Participants in the field of natural language processing (NLP) have been developing algorithms for many years which try to extract information out of written English and other languages. One of the applications of NLP to information retrieval is to extract query terms from a given question. For example, the user will write "How can I eat fried fish without getting my fingers greasy?". The NLP parser will extract key terms from the question (like fish, eat, greasy, fingers) and maybe include other terms related to these to assist in the searching. Once this is done the document retrieval system can perform its usual task. This type of retrieval is performed by the search engine called "Ask Jeeves" [64]. But it seems that most Web users prefer key term searches. Sullivan [109] provided results on searches per day for a collection of search engines. Ask Jeeves came sixth out of eight.

2.5.3 The Google Phenomenon

In the middle of the year 2000 a new Web search engine was brought to the attention of many researchers which was fast and precise. There was no advertising when it was released. The new search engine called “Google” [14] was brought to the attention of the Web community by word of mouth due to its superior performance over the existing search engines.

To compute the rank of a specific Web page, Google uses the information about the query terms in the text and also utilises the HTML found in the page to weight the terms. For example if a query term was found in the heading, it would be considered more important. A major part of HTML are the hyperlinks. Hyperlink information is also gathered and used to calculate the PageRank [86] of the page, which is measure of the page importance. The PageRank of page *A* is generated by finding a linear combination of the PageRanks of each page linked to page *A*. It can be thought of as a voting scheme. If many page are linked to page *A*, it is similar to a vote of approval.

Using PageRank, Google boosts the pages that have the approval of others (popular pages). Due to this, it is expected that new information would be hard to find, unless it became popular. But to become popular it must be found. So we would expect the Google search engine to provide results from pages which are well known.

Without the underlying text processing, Google would not be able to retrieve any relevant documents. So it is important to have a precise and fast underlying text retrieval system in any information retrieval system, just as we present in this thesis.

2.6 Retrieval Experimentation

To compare different retrieval methods, we must have data to run the experiments (a document set), set queries with answers, and a method of grading the system after we have generated results.

2.6.1 Document Collections

Many document sets have been used to test various information retrieval methods. The early days of information retrieval saw many small document collections. These were used to show that a certain retrieval method was the best, but since methods were tested on different collections there was no real way to compare results.

A survey by Harman [56] presents a list of document collections through the years. The first collection built for experimental purposes was the Cranfield collection. This set

Collection	Documents	Queries
Cranfield	1,398	225
ADI	82	35
MEDLARS	1,033	30
TIME	423	24
CACM	3,204	64
CISI	1,460	112
NPL	11,429	100
INSPEC	12684	84
OSHUMED	348,566	101
Cystic Fibrosis	1,239	100
FSupp	11,953	44
Fed	410,883	44
TREC disk 1	510,637	150
TREC disk 2	231,219	250
TREC disk 3	336,310	50
TREC disk 4	293,710	200
TREC disk 5	262,367	150
VLC2	18.5 million	100
Reuters-21578	21,578	-

Table 2.8: Document collections and associated query sets.

comprises of 1398 documents along with a set of queries and relevance judgments for each query.

Table 2.8 contains a list of some of the many collections which have been used, along with the number of queries with relevance judgments provided. TREC disks 1 to 5 are used with the Text Retrieval Conference. The work by TREC has provided large sets of experimental data for many information retrievalists. The VLC2 collection is now used by TREC in place of disks 1 to 5. This collection contains a snapshot of a portion of the World Wide Web [116].

The Reuters-21578 collection does not have a set of queries because it is designed for information categorisation experiments. Each of the documents in the Reuters set is labeled with at least one item from the categories shown in table 2.9. The labels describe the main points of the document. Half of the documents are then used to train the categoriser and the other half are used to test how effective the system is.

Category	Number of items
Exchanges	39
Orgs	56
People	267
Places	175
Topics	135

Table 2.9: Categories found in the Reuters-21578 collection.

The experiments performed in this thesis used the TREC-1,2,3 document sets because they contain many documents and have been used in many experiments with other systems in recent times.

2.6.2 Retrieval Evaluation

Many methods of information retrieval have been designed and tested. Each show their own results but how can we establish which method is the best? To evaluate a retrieval system, we must ask how the system is going to be used? To get the best out of an information retrieval system, we would want it to rank every document that is relevant to the query higher than those that are not. But so far, there has not been a method to accomplish this task. Therefore, if a set of results are presented from one method and another set from another method, how can we compare these results. The two main measures of effectiveness are called *Recall* and *Precision* [102]. Recall is a measure of how many documents we have retrieved that are relevant to the query, precision proportion of relevant documents in the set we have retrieved.

$$\text{Recall} = \frac{\text{Number of relevant documents retrieved}}{\text{Number of relevant documents in the document set}} \quad (2.11)$$

$$\text{Precision} = \frac{\text{Number of relevant documents retrieved}}{\text{Number of documents retrieved}} \quad (2.12)$$

Figure 2.6 has an example of a precision vs. recall plot. By viewing this we can compare different methods. A better method would have a higher precision for each value of recall. A perfect method would have precision of 1 for all values of recall. This would mean that if the retrieval system chose x documents from the set (where x is less than the total number of relevant documents) then all of them would be relevant.

There have been other methods which try to grade a retrieval systems performance in terms of one number, but they are hard to compare and less obvious than the precision vs. recall evaluation.

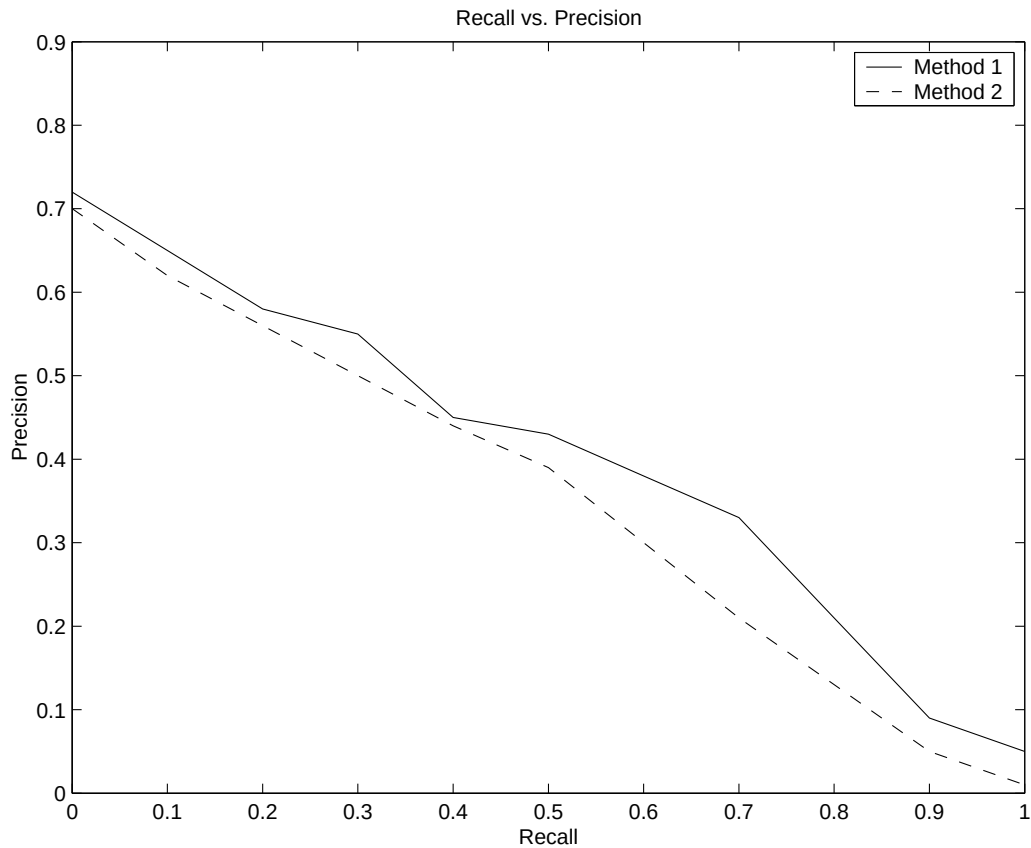


Figure 2.6: Recall vs. precision plot for two sample methods. We can see that method 1 has a higher precision than method 2 for every level of recall, therefore method 1 is superior to method 2.

Most of our experiments observed the figures calculated for precision after 5, 10, 15 and 20 documents retrieved. These measurements allow us to obtain an idea of the number of relevant documents a typical user would observe. If the documents are not found within the top 20 results, we would expect the user to become frustrated and change the query.

Other criteria which should be observed when comparing retrieval systems are the response time when giving a query, the amount of effort the user must put in to generate a query and the amount of disk space that is needed to store the data.

2.7 Evaluation of Retrieval Systems

After reviewing all of the major areas in information retrieval, we can see that most of the methods depend on finding query terms in documents as a judge of relevance of the document.

In this age of the Internet and the World Wide Web, the amount of information being stored in a digital form is growing every day. After using several basic information retrieval systems, we have found that it is not enough just to examine the documents in question for occurrences of query terms. There are too many documents which appear within the top 20 results and are irrelevant to the information wanted.

Proximity methods go as far as comparing the displacement between the query terms within documents, but they do not have any means to use the important statistical information that the vector methods use. Proximity methods tend to be slower and have larger indices than vector space methods leaving them to be undesirable for many uses.

In this thesis we will present a new document retrieval paradigm which allows us to combine the best aspects of the vector space and proximity methods, obtain a compact index for fast retrieval using the coding ideas mentioned, and which responds promptly to any information requests. We perform this task by analysing the term spectrum of each document rather than the position or the count of the terms. By examining the spectrum, we can easily compare positions of terms and take into account how the terms are distributed through the documents.

CHAPTER THREE

SPECTRAL DOCUMENT RANKING

3.1 Introduction

TO identify the topic of a document, the text within must be analyzed. This is why the text analysis method used in any search engine is critical. By obtaining a clear understanding of a page, we are able to find more relevant documents and make it harder for people to cheat the system.

Variants of the vector space model [121] have been the dominant methods used to analyse the text in information retrieval systems for many years. The concept behind the vector space model is to convert each document into a vector, so they can be easily compared to other document vectors (to find similarity amongst documents) and they can also be compared to query vectors (to find relevance to the query). The document vectors exist in a space where each dimension is one unique term from the document set. The size of the document vector along each dimension is a weighted count of that word in the document.

The problem with this technique is that any spatial information contained in the documents is lost. Once the documents are converted into document vectors, the number of times each word appears is represented in the vector, but the positions of the words (the flow of the document) is ignored.

We present a method entitled Fourier Domain Scoring (FDS) which can retain the document spatial information and use it to effectively rank documents. The difference between FDS and vector space similarity measures is that rather than storing only the count of a frequency term per document, FDS stores a term signal. The term signal shows how the term is spread throughout the document. If the spectrum of the query term signals are compared, we are able to observe which documents have a high occurrence of the query terms and which documents have the query terms appearing together. This information is obtained by comparing the magnitude and phase of the spectrum across

different term signals respectively.

FDS is a new text classification method which can be used in information retrieval systems by replacing the existing vector space similarity measures. Just as in information retrieval systems which use vector space similarity measures, other enhancements can be added on top of FDS to fully utilize the document environment and improve the results of the search (as in examining hypertext links on the Web, using a thesaurus,...).

Cases of systems which use the vector space similarity measures which could easily be replaced by FDS can be found everywhere. Some examples are as follows.

Google [14] records the position of every word in the document to give a higher ranking to document which contain query terms closer together. Systems such as PADRE [57] and Okapi [96] also use this style of proximity searching. To calculate the distances between every term would require a lot of calculations at query time, or a large space to store all of the previously calculated distances. FDS could easily be implemented in the Google search engine to store the precalculated phases of each word.

Ebert *et al.* [38] describe a method of visualizing clusters in documents. This system takes sequences of characters from the text to create vectors for the user to explore. It would be very interesting to replace these n -gram vectors with the Fourier transform of each character sequence or word sequence. This might provide a more compact representation and give better results.

The Latent Semantic Indexing method [9] uses singular value decomposition (SVD) to reduce the dimension of the index used by conventional search engines. By reducing the dimension, we are trying to remove noise generated by our use of language and trying to find the true semantic structure of the document. This method uses document vectors containing the count of each word as the elements, therefore any spatial information is lost. If the document vectors were replaced by a document matrix, generated by FDS, then SVD could easily be performed on each of the sets of spatial bins. This would produce a structural latent semantic index. This idea is beyond the scope of the thesis, therefore no results are shown for such a method.

This chapter is organised as follows. Section 3.2 will give an introduction to the Fourier Domain Scoring method and outline the basic steps needed to perform such a task on large document sets. Section 3.3 will show how the vector space methods are a special case of FDS. Section 3.4 will look into the new data extracted from the document and how it copes with large queries. Section 3.7 explains in detail the experiments performed using the TREC data set with short queries and compares the results with existing ranking methods.

3.2 Fourier Domain Scoring

Vector space similarity measures will only give a score based on the count of each word in the document. FDS tries to capture the location of the word through the document. Words can then be compared to find if they are relevant to the document.

FDS compares the positions of the words by comparing their phase and adjusts according to the appearance of the word by observing the magnitude. The basic structure of the FDS procedure (shown in algorithm 3.1) is as follows:

1. Collect words into spatial bins
2. Create inverted index
3. Perform document weighting
4. Perform Fourier transform
5. Calculate Magnitude of query using query weighting
6. Calculate Phase Precision of query
7. Combine Magnitude and Phase Precision to obtain document score

Some initial work on this topic can be found in [87]. We discuss each of these steps in brief as it is essential to understand how FDS works before we can examine any of its properties.

1. For each query term $t \in T$
 - (a) Create query term signals $\{\tilde{f}_{0,t}, \tilde{f}_{1,t}, \dots, \tilde{f}_{D,t}\}$
 - (b) Weight signals $\tilde{\omega}_{d,t} = \Omega_d(\tilde{f}_{d,t})$
 - (c) Transform signals into spectrums $\tilde{v}_{d,t} = \text{DFT}(\tilde{\omega}_{d,t})$
2. For each document d in collection
 - (a) For each spectral component $v_{d,t,b} \in \tilde{v}_{d,t}$
 - i. Calculate component magnitudes $H_{d,b}^*$
 - ii. Calculate component phase precision $\Phi_{d,b}^*$
 - iii. Obtain component score $s_{d,b} = \Phi_{d,b}^* H_{d,b}^*$
 - (b) Combine component scores $s_{d,b}$ to obtain document score s_d

Algorithm 3.1: Fourier Domain Scoring (FDS) algorithm

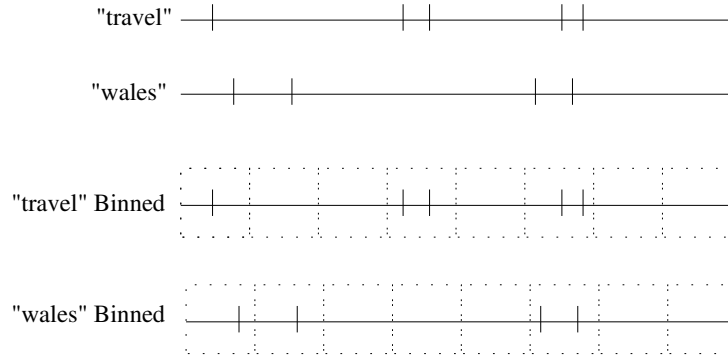


Figure 3.1: A visual example of how the word signals are obtained. The top two lines, labeled “travel” and “wales” show the positions of the words travel and wales in some document (the position is signified by the vertical stroke through the line). The bottom half shows the binned positions of the words.

3.2.1 Collect words into spatial bins

Rather than mapping a document to a vector that contains the count of each word, FDS maps each document into a set of word vectors. These word vectors show the position of the word throughout the document, where the position of the element represents the position of the word in the document. To reduce the word vector size (and hence the calculations needed), the words should be grouped into bins. If the number of bins is set to B , a document containing W words would have W/B words in each bin. Therefore the first element of the word vector would contain the number of times the word appeared in the first W/B words. The second element would be the number of times the word appeared in the second W/B words, and so on. For example, the top half of figure 3.1 gives us the positions of the words “travel” and “wales” throughout a document (signified by the vertical bars). If we choose $B = 8$, we obtain the word signal $[1\ 0\ 0\ 2\ 0\ 2\ 0\ 0]$ for the word “travel” in that document and $[1\ 1\ 0\ 0\ 0\ 2\ 0\ 0]$ for the word “wales” in that document (as shown in the bottom half of figure 3.1).

3.2.2 Create spatial inverted index

Just as in any vector space model, an inverted index can be created to enable quick retrieval of word vectors. In this case, a little more information needs to be stored due to the documents being split into bins. The words in each document were represented as:

$$\langle n \rangle \langle b_1, f_1 \rangle \langle b_2, f_2 \rangle \dots \langle b_n, f_n \rangle \quad (3.1)$$

where n is the number of non-zero bins, b_a is the bin number and f_a is the count of the word in bin b_a . This is known as a spatial inverted index due to its storage of term position information.

3.2.3 Weighting

The cosine similarity measure can be improved considerably by adding weighting to the document vectors before the score is calculated [37]. A few of the better performing schemes are BD-ACI-BCA, BI-ACI-BCA and AB-AFD-BAA in Zobel and Moffat [126] and the Lnu.ltu method contained in SMART [17]. Both BD-ACI-BCA and Lnu.ltu are good all round weighting schemes, while AB-AFD-BAA is better for short queries and BI-ACI-BCA is more precise when using long queries.

Each of these schemes contain document weighting and query weighting. For example, BD-ACI-BCA refers to the following weights being used:

$$\Omega_d(f_{d,t}) = \frac{1 + \log_e f_{d,t}}{(1 - s) + s \cdot W_d / \text{av}_{d \in D} W_d} \quad (3.2)$$

$$\Omega_q(f_{q,t}) = (1 + \log_e f_{q,t}) \log_e \left(1 + \frac{f_t^m}{f_t} \right) \quad (3.3)$$

where Ω_d, Ω_q are the document and query weights respectively, $f_{d,t}, f_{q,t}$ are the count of term t in document d and query q respectively, s is the slope factor (set to 0.7), $W_d, \text{av}_{d \in D} W_d$ are the document vector norm and average document vector norm respectively, f_t is the number of documents having term t and f_t^m is the largest f_t . We can see that BD-ACI-BCA uses pivoted document normalisation.

The document normalisation is used to reduce the effect of repetition of a word in a document and to negate any effect the size of the document has on a query. The query normalisation is used to reduce the effect of repetition of terms in the query and reduces the effect common terms have on the document score. FDS also requires these properties, therefore preweighting (applying weights before performing the Fourier transform) can be performed to increase the accuracy of the relevance scores. Since we will be comparing FDS to the vector space methods mentioned, we will also use these for weighting. To use the weights on the word signals, we simply apply the same weighting scheme to the bin count, rather than the document count. For example, if we were using the BD-ACI-BCA weighting scheme with FDS, the weighted bin values would become:

$$\omega_{d,t,b} = \Omega_d(f_{d,t,b}) = \frac{1 + \log_e f_{d,t,b}}{(1 - s) + s \cdot W_d / \text{av}_{d \in D} W_d} \quad (3.4)$$

where $f_{d,t,b}$ is the count of term t in bin b of document d . The query weighting ($\Omega_d(f_{d,t})$, which is applied later) remains the same as in the vector space methods.

3.2.4 Perform Fourier transform

The Fourier transform has been used in many areas of electrical engineering. It is designed to change the basis of a signal (or function) to linearly independent sinusoidal waves. The discrete form of the transform (DFT) is of the form:

$$v_{d,t,\beta} = \sum_{b=0}^{B-1} \omega_{d,t,b} \exp\left(\frac{-i2\pi\beta b}{B}\right) \quad (3.5)$$

in our case the signal will be the weighted word signal consisting of elements $\omega_{d,t,b}$ where $b \in \{0, 1, \dots, B-1\}$. Since each $v_{d,t,b}$ is the projection of the word signal $\tilde{\omega}_{d,t}$ onto a sinusoidal wave of frequency β , the signal $\tilde{v}_{d,t}$ is the spectrum of the given word signal. The spectral component number β is an element of the set $\{0, 1, \dots, B-1\}$.

Therefore, the Fourier transform maps a signal from the time or spatial domain to the frequency domain. Once the spectrum of a signal can be seen, we are able to identify the major frequency components which give the signal its shape. The Fourier transform produces the following mapping:

$$\{\omega_{d,t,b}\} \rightarrow \{v_{d,t,b}\} = \{H_{d,t,b} \exp(i\phi_{d,t,b})\}$$

where $\omega_{d,t,b}$ is the weight of term t in bin b of document d , $v_{d,t,b}$ is the b th frequency component of term t in document d , $H_{d,t,b}$ and $\phi_{d,t,b}$ are the magnitude and phase of frequency component $v_{d,t,b}$ respectively (and are therefore real values) and i is $\sqrt{-1}$.

The Nyquist-Shannon sampling theorem [120] states that the greatest frequency component to be found in a real signal is equal to half of the sampling rate. This implies that if we choose B bins for the word signal, we will only need to examine frequency components 0 to $\frac{B}{2}$ to analyse all of the information found in the spatial word signal.

By performing the Fourier transform on the spatial word bins, we are able to see how the word flows through the document. Each frequency component contains magnitude and phase information which can be interpreted as the effect and shift of the component respectively. The effect gives us an idea of the shape of the word signal. If a lower frequency component magnitude is large with respect to the other components, then the word should appear clustered in a few places in the document. If a higher frequency component magnitude is large with respect to the other components, then the word clusters would be scattered through the document. The shift represents the position of the word throughout the document and is measured in radians. The shift is useful when comparing word signals. If two signals have the same phase then they are said to be 'in phase', meaning they appear together. But if two signals have opposite phase (different by π radians), then they are 'out of phase'. Therefore, if two or more word signals are in

phase, this implies that they appear together throughout the document. If two or more words are not in phase, it implies that they appear in the document but not together most of the time.

For example, if we consider the word signals for the words “travel” and “wales” at the top of figure 3.2, we can see that they appear together in bins 0 and 5. If we perform the DFT on each signal we can examine the magnitude and phase of the spectral components. If a component c from word 1 has a similar phase to component c from word 2, these two components are considered in phase, which implies that the words 1 and 2 appear together in that region of the spectrum. This is the case for components 0, 3 and 4 when comparing the words “travel” and “wales”. If the magnitude for those components are high (relative to the other components), it implies that these are major frequency components of the word signal. This is also the case with components 0, 3 and 4 when comparing the words “travel” and “wales”. Therefore this document can be considered relevant to the query “travel” and “wales”. An example of the effect of the Fourier transform on a word signal can be seen in figure 3.2.

3.2.5 Combining word spectra

Once we have performed the Fourier transform on the B length vectors, we have a set of $\frac{B}{2} + 1$ independent complex numbers for each query term, known as the word spectra. From intuition, a relevant document should have large magnitudes and the corresponding phases from each word should be similar (in phase), so we should deal with the magnitude and phase separately. We will combine the spectra to obtain the magnitude ($H_{d,b}$) and phase precision ($\Phi_{d,b}$) for each component b in document d .

Magnitude

To calculate the effect of the query terms on the document we can either treat the spectral components as complex values or just examine the magnitude of each.

The first method treats each set of frequency components as though it is a separate index. Each spectral component is multiplied by the weight of the corresponding query term. The resulting complex values are then added and the magnitude is taken. If the complex values are in phase for a particular component, the resulting score component ($s_{d,b}$) will be large. If the spectral components are out of phase, they will cancel each other out and result in a small score vector. The magnitude ($H_{d,b}^v$) of each component b

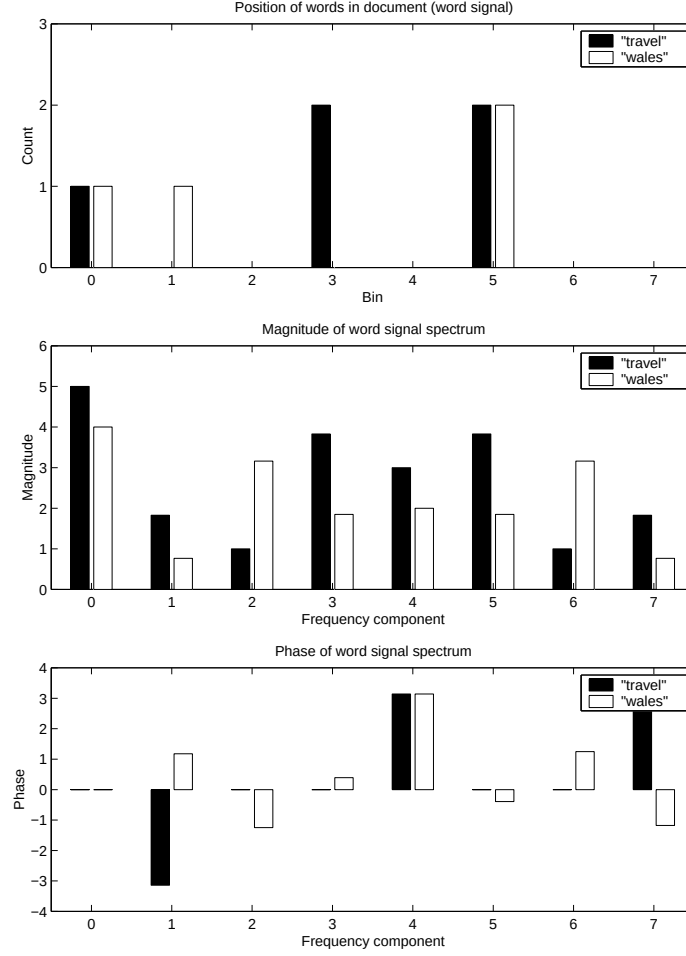


Figure 3.2: An example of before and after the Fourier transform. We can see the binned positions of word 1 and word 2 from a certain document in the top plot. The bottom two plots show the information obtained after computing the Fourier transform on each of the binned word signals. Notice the reflection of the spectral information about component 4 (as stated in the Nyquist-Shannon sampling theorem).

in each document d is:

$$\text{Sum vectors} := H_{d,b}^v = \left| \sum_{t \in T} v_{d,t,b} \cdot \Omega_q(f_{q,t}) \right| \quad (3.6)$$

where $v_{d,t,b}$ is the complex value of frequency component b of word t in document d and $\Omega_q(f_{q,t})$ is the query term t weight.

The motivation behind this idea is to give a high score to the documents which contain all similar word spectrums for all terms in the query. A higher score will also be given to the documents which contain more occurrences of the query terms (which

would lead to higher magnitude of the spectral components).

If we take into account only the magnitude of the complex spectral components (and deal with the phase later) we can add the magnitudes to obtain the occurrence of the word through the document:

$$\text{Sum magnitudes} := H_{d,b}^m = \sum_{t \in T} |\nu_{d,t,b} \cdot \Omega_q(f_{q,t})| \quad (3.7)$$

Phase precision

If we examine only the phase information of the word signal spectrum, we are able to find the spectral components which have similar phase. This can be performed by treating each spectral component as a unit vector with its own phase and finding the average phase which we call phase precision. When finding the phase precision, we are left with the choice of what to do with spectral components of zero magnitude since the phase of these components mean nothing. The choices we have made are to either average over the non-zero components and to average over all components.

Non-zero phase precision is given by the following equation:

$$\begin{aligned} \text{Non - zero phase precision} &:= \Phi_{d,b}^n \\ &= \left| \frac{\sum_{t \in T: H_{d,t,b} \neq 0} \phi_{d,t,b}}{\#(\hat{T}_{d,b})} \right| \end{aligned} \quad (3.8)$$

where

$$\nu_{d,t,b} = H_{d,t,b} \exp(i\theta_{d,t,b}) \quad (3.9)$$

$$\phi_{d,t,b} = \frac{\nu_{d,t,b}}{|\nu_{d,t,b}|} = \exp(i\theta_{d,t,b}) \quad (3.10)$$

and $\theta_{d,t,b}$ is the phase of the b th frequency component of the t th query term in the d th document, $\phi_{d,t,b}$ is the unit phase of the b th component of the t th query term in the d th document, $\#(x)$ is a function which gives the cardinality of the set x , and $\hat{T}_{d,b}$ is the set of query terms which do not have zero magnitude for frequency component b in document d .

Zero phase precision is again similar to non-zero phase precision in the way that it only includes the frequency components with non-zero magnitude but it averages over the total number of query terms. This total averaging is performed so that if any query terms do not have a frequency component b , then the score will be reduced.

$$\begin{aligned} \text{Zero phase precision} &:= \Phi_{d,b}^z \\ &= \left| \frac{\sum_{t \in T: H_{d,t,b} \neq 0} \phi_{d,t,b}}{\#(T)} \right| \end{aligned} \quad (3.11)$$

Term	$\Omega_q(f_{q,t})$	Frequency Component (magnitude, phase)				
		0	1	2	3	4
mariquita	1.8	(3.0, 0)	(1.0, -0.7)	(3.0, -1.5)	(1.0, -2.3)	(3.0, 3.1)
travels	1.0	(1, 0)	(1, -2.3)	(1, 1.5)	(1, -0.7)	(1, 3.1)
wales	1.1	(2.0, 0)	(1.4, 3.1)	(0, 0)	(1.4, 0)	(2.0, 3.1)
$H_{d,b}^v$		8.6	2.2	4.4	2.2	8.6
$H_{d,b}^m$		8.6	4.3	6.4	4.3	8.6
$\Phi_{d,b}^n$		1.0	0.5	0	0.5	1.0
$\Phi_{d,b}^z$		1.0	0.5	0	0.5	1.0
$\Phi_{d,b}^1$		1	1	1	1	1

Table 3.1: Calculation of the different Magnitude and Phase precision values of the terms *mariquita*, *travels* and *wales*.

where T is the set of query terms.

No phase precision ignores any phase information. This is best used when the phase has already been taken into account when creating the magnitude vector.

$$\text{No phase precision} := \Phi_{d,b}^1 = 1 \quad (3.12)$$

Once the magnitude and phase precision have been obtained, we are able to combine them to create the document score vector. Each component of the score vector is calculated by multiplying the corresponding magnitude and phase precision values.

$$s_{d,b} = H_{d,b}^* \Phi_{d,b}^* \quad (3.13)$$

where $H_{d,b}^*$ and $\Phi_{d,b}^*$ represent the selected magnitude and phase precision methods selected respectively.

To assist in understanding the word spectrum combination process we have provided a sample set of word signal spectrums and calculations of each of the magnitude and phase precision methods in table 3.1.

3.2.6 Combining spectral components

Once the word spectrums have been combined into score vectors, we are left with a vector with $B/2 + 1$ elements, containing how relevant each query term was to that frequency component. This section will explain a few methods to help us obtain a single score from this vector and allow us to rank a set of documents in terms of relevance to the query terms. Methods which were experimented with were:

- Sum all components
- Sum largest score vector elements
- Sum largest phase precision components
- Sum largest magnitude components
- Sum threshold phase precision components

Sum all components

Since each element of the score vector represents the content of the query terms in that specific frequency component, a document which contains high values in each element should be more relevant than one that contains lower values. Therefore the most obvious operation to perform is summation.

$$S_d = \sum_{b=1}^{B/2+1} s_{d,b} \quad (3.14)$$

Sum largest score vector elements

It could also be stated that if there are high values contained in any of the elements of the score vector, the document should be considered relevant to the query terms. To implement this, only the components with the two greatest values were added to create the score.

$$S_d = s_{d,b_1} + s_{d,b_2} \quad (3.15)$$

$$s_{d,b_1}, s_{d,b_2} \geq \max_{\forall b \neq b_1, b_2} (s_{d,b})$$

The larger of the two components will be the zeroth component (DC component). The DC component magnitude of a word signal spectrum will be equal to the sum of all bins of the word signal (and hence the value used in vector space measures). This largest component is a real value and therefore has zero phase (leading to a precision of 1 if all words are present).

Sum largest phase precision components

Rather than finding elements of the score vector which have high values, we might want to find the elements which have the largest phase precision component. Components which have high phase precision will identify a sinusoidal pattern of all of the query terms appearing together through the document. Again the component with the largest

phase precision will be the zeroth component. For this reason, the two components with the highest phase precision will be summed to give the score. The score equation will be as in equation 3.15, but the components b_1 and b_2 are chosen according to the following equation:

$$\Phi_{d,b_1}^*, \Phi_{d,b_2}^* \geq \max_{\forall b \neq b_1, b_2} (\Phi_{d,b}^*)$$

where $\Phi_{d,b}^*$ represents whichever of the three phase precision methods we choose.

Sum largest magnitude components

Another score calculation method is to include only the score vector elements with larger magnitude components. The magnitude of the frequency components of the query term vectors gives us some understanding of how many query terms appeared in the pattern of the sinusoidal wave (to the corresponding frequency component). As in the last scoring method, the components with the two greatest magnitudes will be taken to create the score for each document. Again, the component with the greatest magnitude will be the zeroth component.

$$H_{d,b_1}^*, H_{d,b_2}^* \geq \max_{\forall b \neq b_1, b_2} (H_{d,b})$$

The score components added will be s_{b_1} and s_{b_2} .

Sum threshold phase precision components

The final score calculation method is to include only the score vector elements with a phase precision greater than some predefined constant. The idea behind this method is that the components with low phase precision will only disturb the quality of the final score. If the phase precision is low, it must mean that the set of query terms were not in phase for that component, and is therefore considered noise.

$$S_d = \sum_{b \in \{c | \Phi_{d,c}^* > P\}} s_{d,b}$$

where P is the threshold chosen.

3.3 Comparison to Vector Space Method

If we examine the FDS method, we can see that the vector space method is a special case of FDS where $B = 1$. To show this we will step through the FDS method with $B = 1$.

Theorem 3.3.1. *Vector space methods are a special case of FDS where $B = 1$*

Proof. The first step is to gather the document terms into bins. $B = 1$ so the whole document is considered a bin, therefore:

$$f_{d,t,0} = f_{d,t} \quad (3.16)$$

when the document weighting is performed, we apply the weighting to each bin, which is the document in this case:

$$\tilde{\omega}_{d,t} = \Omega_d(\tilde{f}_{d,t}) \quad (3.17)$$

$$= \Omega_d(f_{d,t,0}) \quad (3.18)$$

$$= \Omega_d(f_{d,t}) \quad (3.19)$$

The Fourier transform of a signal of length one is itself. The Magnitude vector ($H_{d,b}^*$) will be the same for both sum vectors and sum magnitudes, since $v_{d,t,0}$ is real and positive:

$$H_{d,0}^* = \sum_{t \in T} v_{d,t,0} \cdot \Omega_q(f_{q,t}) \quad (3.20)$$

$$= \sum_{t \in T} \omega_{d,t,0} \cdot \Omega_q(f_{q,t}) \quad (3.21)$$

$$= \sum_{t \in T} \Omega_q(f_{d,t}) \cdot \Omega_q(f_{q,t}) \quad (3.22)$$

The non-zero phase precision will be 1 because $v_{d,t,0}$ is real and positive, and the zero phase precision will be a fraction of the terms found in the document. Therefore the vector space method is a special case of the FDS method where $B = 1$. $\square$

3.4 Performance with Large Queries

Given that the DC component of a word signal spectrum is the most dominant component, how do the rest of the components affect the score? All of the methods are based on the idea that if the words appear in phase, then the document should be more relevant than a document out of phase. This can be easily seen for a few query terms, but what about when there are many query terms (as in the TREC query shown in figure 3.6).

Since the DC ($b = 0$) component is the largest component and always has zero phase, any other component will also have to have a common phase across the set of query terms in order to be comparable to the DC component. For a component to have common phase across the entire set of query terms, the query terms must occur in the same positional bins. If the number of query terms is greater than the number of words per bin, the query terms have no chance of appearing in the same bins. For each new query term added, the commonality between phases will reduce. Therefore if the number of query

terms is much greater than the number of words per bin, the AC ($b \neq 0$) components will be insignificant when compared with the DC component.

If we look at the basic expression for magnitude $\times$ phase precision (equation 3.13), we can see that the phase precision acts as a weighting term. If we consider component b (not equal to 0), we can see that the phase precision will remain at value 1 as long as all terms in the query are in the same positional bins in the document. Once the number of query terms passes the number of words per bin, it is impossible for all words in the query appear in the same positional bins. Therefore the phase precision will reduce with every new word added to the query term set. Therefore:

$$\#(T) \gg B \Rightarrow \Phi_{d,b} < \epsilon \quad \forall b \in \{1, 2, \dots, B-1\} \quad (3.23)$$

where ϵ is a small and positive real number. This implies that if a large number of query terms are used, the DC component will be the dominant factor in the score.

$$\lim_{\#(T) \rightarrow \infty} S_{\text{FDS}}(d, t) = s_{d,0} \quad (3.24)$$

We have also found in the previous section (section 3.3) the DC component is related to the value used in vector space similarity measures. Based on these two observations, it follows that if the number of query terms is large, FDS will be comparable to the vector space similarity measure using the weighting method chosen. So if we choose the BD-ACI-BCA weighting for FDS, it should be similar to the BD-ACI-BCA vector space measure when large queries are used.

3.5 Proximity Analysis

Before we begin with experimentation, we must try to comprehend how the spectral document retrieval system will deal with surrounding terms

If we analyse the case where we have τ query terms, each appearing exactly once in document d . Each term signal component reduces to:

$$\text{DFT}\{\tilde{f}_{d,t}\}[k] = \sum_{b=0}^{B-1} f_{d,t,b} e^{-2\pi i k b / B} \quad (3.25)$$

$$= e^{-2\pi i k b_n / B} \quad (3.26)$$

The sum of the magnitudes is equal to τ for every component, since every component has a magnitude of one. This does not change with respect to the positions of the query terms in the documents, therefore we will ignore the magnitude result and concentrate on the phase precision.

The phase precision is given as:

$$\Phi_{d,k} = \left| \sum_{t_n \in \tau} \phi_{d,t_n,b_n} \right| = \left| \sum_{t_n \in \tau} e^{-2\pi i k b_n / B} \right| \quad (3.27)$$

If τ is even, we are able to group the sum into pairs

$$\Phi_{d,k} = \left| \sum_{t_n \in \tau} e^{-2\pi i k b_n / B} \right| \quad (3.28)$$

$$= \left| \sum_{t_n, t_m \in \tau} e^{-2\pi i k b_n / B} + e^{-2\pi i k b_m / B} \right| \quad (3.29)$$

Let $u_{n,m} = \frac{b_n + b_m}{2}$, therefore $b_n = u_{n,m} + \frac{b_n - b_m}{2}$ and $b_m = u_{n,m} - \frac{b_n - b_m}{2}$

$$\Phi_{d,k} = \left| \sum_{t_n, t_m \in \tau} e^{-2\pi i k (u_{n,m} + \frac{b_n - b_m}{2}) / B} + e^{-2\pi i k (u_{n,m} - \frac{b_n - b_m}{2}) / B} \right| \quad (3.30)$$

$$= \left| \sum_{t_n, t_m \in \tau} e^{-2\pi i k u_{n,m} / B} \left(e^{-2\pi i k \frac{b_n - b_m}{2B}} + e^{2\pi i k \frac{b_n - b_m}{2B}} \right) \right| \quad (3.31)$$

$$= \left| \sum_{t_n, t_m \in \tau} 2e^{-2\pi i k u_{n,m} / B} \cos \left(2\pi k \frac{b_n - b_m}{2B} \right) \right| \quad (3.32)$$

$$= \left| \sum_{t_n, t_m \in \tau} 2e^{-2\pi i k u_{n,m} / B} \cos \left(\frac{\pi k p_{n,m}}{B} \right) \right| \quad (3.33)$$

where $p_{n,m}$ is the displacement in terms of bins between terms t_n and t_m . If one term of the summation is real, we will want the other terms to be real as well to maximise the phase precision calculation. Therefore we will divide by the last terms coefficient to make it real.

$$\Phi_{d,k} = \left| \sum_{t_n, t_m \in \tau} \left(e^{-2\pi i k u_{a,b} / B} \right) 2e^{-2\pi i k (u_{n,m} - u_{a,b}) / B} \cos \left(\frac{\pi k p_{n,m}}{B} \right) \right| \quad (3.34)$$

$$= \left| \sum_{t_n, t_m \in \tau} 2e^{-2\pi i k (u_{n,m} - u_{a,b}) / B} \cos \left(\frac{\pi k p_{n,m}}{B} \right) \right| \quad (3.35)$$

where $u_{a,b} = \frac{b_a + b_b}{2}$ and b_a, b_b are the bin positions of the final two query terms.

If τ is odd, we can do the same pairing, but there is a left over exponential term we must deal with.

$$\Phi_{d,k} = \left| \sum_{t_n, t_m \in \tau} 2e^{-2\pi i k u_{n,m} / B} \cos \left(\frac{\pi k p_{n,m}}{B} \right) + e^{-2\pi i k b_a / B} \right| \quad (3.36)$$

$$= \left| \left(e^{-2\pi i k b_a / B} \right) \left(\sum_{t_n, t_m \in \tau} 2e^{-2\pi i k (u_{n,m} - b_a) / B} \cos \left(\frac{\pi k p_{n,m}}{B} \right) + 1 \right) \right| \quad (3.37)$$

$$= \left| \sum_{t_n, t_m \in \tau} 2e^{-2\pi i k (u_{n,m} - b_a) / B} \cos \left(\frac{\pi k p_{n,m}}{B} \right) + 1 \right| \quad (3.38)$$

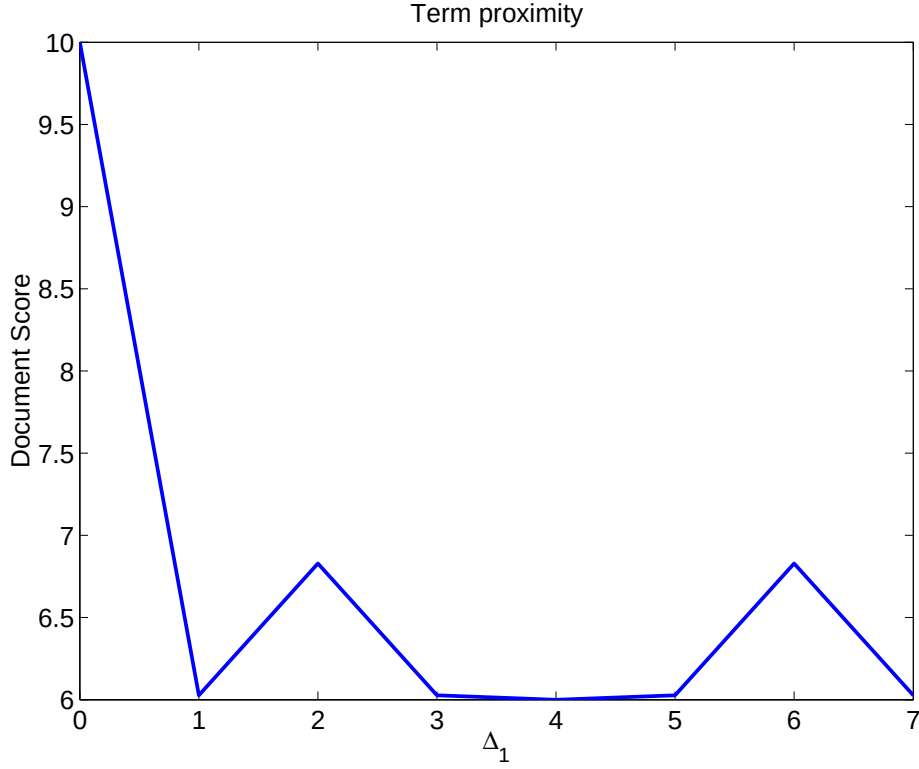


Figure 3.3: The effect of the displacement between two terms ($\Delta_1 = |b_n - b_m|$) on the document score.

where b_a is the bin position of the final query term.

Now that we have adjusted both even and odd forms of the phase precision equation to have a final real term, we can deduce that the maximum phase precision is provided when all of the terms are real. We can see that the maximum phase precision is given when the exponential is real and the cosine is 1. Therefore, $\max_{\forall b_n} \Phi_{d,k} = \tau$. If we set every b_n constant except for the last term (b_a), we can examine how the phase precision reacts to changes in term position.

To analyse the effect of the proximity of the query terms on the document score, we will choose to observe the cases of $\tau = 1, 2, 3$ and the effect their proximity has on the document score. If we choose $\tau = 1$ (only one query term was supplied), the phase precision equation reduces to:

$$S_d \propto \sum_{k=0}^{B-1} \left| e^{-2\pi i k b_0 / B} \right| = B \quad (3.39)$$

The phase precision reduces to a constant (B) when there is only one query term. Therefore it has no effect, which implies that there is no proximity of terms to be taken into

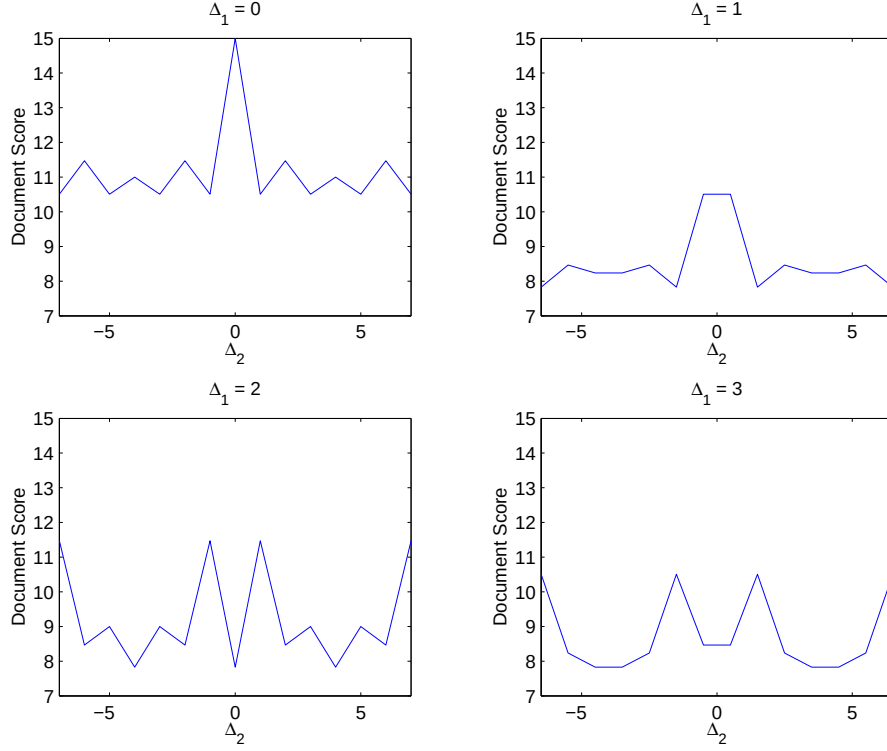


Figure 3.4: The effect of the displacement between a third term and the centre of the first two terms ($\Delta_2 = \frac{b_n + b_m}{2} - b_a$) on the document score. Δ_1 is the displacement between the first and second terms.

account.

For $\tau = 2$, we must observe the even form of the phase precision found earlier (equation 3.35). Since there are only two terms and $u_{a,b}$ represents the midpoint between the last two terms, $u_{n,m} = u_{a,b}$.

$$S_d \propto \sum_{k=0}^{B-1} \left| 2e^{-2\pi i k(u_{n,m} - u_{a,b})/B} \cos\left(\frac{\pi k p_{n,m}}{B}\right) \right| = \sum_{k=0}^{B-1} \left| 2 \cos\left(\frac{\pi k p_{n,m}}{B}\right) \right| \quad (3.40)$$

We can see from this simplification that the document score depends on the displacement between the two terms ($p_{n,m}$). Figure 3.3 contains a plot of the document score related to various displacements between the two query terms. We can see that the document score decreases monotonically up to a displacement of 4 bins if we ignore the displacement at 2 bins. We can also see that the plot is reflected about the line $\Delta_1 = p_{n,m} = 4$. This is due to the reflective properties of the Fourier transform for real data sets.

Finally we will examine the case where $\tau = 3$. To do so, we take the odd form of the

phase precision equation (equation 3.38):

$$S_d \propto \sum_{k=0}^{B-1} \left| 2e^{-2\pi i k(u_{n,m}-b_a)/B} \cos\left(\frac{\pi k p_{n,m}}{B}\right) + 1 \right| \quad (3.41)$$

The document score is dependent on the displacement between the first two terms ($p_{n,m} = b_n - b_m = \Delta_1$) and the displacement between the third term and the average position of the first and second terms ($u_{n,m} - b_a = \frac{b_n+b_m}{2} - b_a = \Delta_2$). Figure 3.4 contains 4 plots of Δ_2 against the document score, each plot has a different set value of Δ_1 . The first plot is set to $\Delta_1 = 0$, implying that the first two terms occur in the same bin. If we vary the relative position of the third term (and hence vary Δ_2), we see that the highest score is produced when $\Delta_2 = 0$. Therefore, the highest document score occurs when all three query terms are located in the same bin. For the case when the third query term is not located in the same bin, the document scores will be of similar magnitude. The reflections can be seen at the edges of the plots, again due to the Fourier transform. The other plots also show that the greatest document score occurs when the terms appear together and drops lower as the third term displacement grows. The plots are not monotonically decreasing away from the centre as desired, but this is expected as a trade off of the method due to the way in which we can compare all terms at once (rather than comparing terms individually as in the spatial proximity methods).

3.6 Computational Complexity

Many calculations must be performed in order to establish a list of ranked documents. To obtain an idea of the rate of growth of the time of the FDS method compared to the vector space method, we will examine the order (asymptotic behavior) of the two.

Each vector space method of document retrieval at query time involves:

1. Apply the specific weighting scheme to each query term in each document ($O(N\tau)$).
2. Summing each weighted query term in each document to obtain the document scores for each document ($O(N\tau)$).

Therefore, the overall computational complexity of a vector space method is $O(N\tau)$. The spectral methods require a bit more computation.

1. Apply the specific weighting scheme to each bin of each query term in each document ($O(N\tau B)$).
2. Perform the Fourier transform on each weighted word signal in each document (A single FFT on a signal of length B , is of order $O(B \log B)$ [94]).

3. Calculate the phase precision for each spectral bin across each query term in each document ($O(N\tau B)$).
4. Calculate the magnitude for each spectral bin across each query term in each document ($O(N\tau B)$).
5. Calculate the score components by multiplying the magnitude and phase precision of each bin in each document ($O(NB)$).
6. Find the document score by summing the score components from each document ($O(NB)$).

Therefore, the overall time complexity of FDS is $O(N\tau B \log B)$ if the transform is performed during the query time.

3.7 Experiments

Initial results have shown that FDS improves the accuracy of search results on small document sets (containing about 100 to 1000 documents) [87]. In these experiments we will show that FDS can effectively improve the results of large document sets. By doing so, we will also show that using FDS in a Web search engine would be a great enhancement.

Before the documents were used, preprocessing was performed. This preprocessing consisted of case folding, removing stop words (the stop word list contained about 400 common English words), and stemming was performed using Porter's stemming algorithm [93]. The FDS methods used eight bins to record the approximate position of the words throughout the document. The number of bins chosen gives a trade off between accuracy and speed. For example if only four bins were chosen, the results would be less accurate, but the processing time would be faster. Trials were run with different numbers of bins and eight seemed to give quick and accurate results.

3.7.1 Document and query set

The document sets used are part of the TREC English document collection. The TREC documents come from many sources and are split on to several disks to allow the users to compare specific sets. The documents used in these experiments are from the Associated Press Newswire (1988) disk 2 and the Wall Street Journal disk 2 (AP2WSJ2) and the Federal Register disk 2 (1989) and articles from Ziff-Davis disk 2 (FR2ZIFF2). This information can be found at the TREC web site [108]. Each document in the set is separated

```

<DOC>
<DOCNO> AP880212-0001 </DOCNO>
<FILEID>AP-NR-02-12-88 2344EST</FILEID>
<FIRST>u i AM-Vietnam-Amnesty 02-12 0398
</FIRST><SECOND>AM-Vietnam-Amnesty,0411
</SECOND><HEAD>Reports Former Saigon
Officials Released from Re-education Camp
</HEAD><DATELINE>BANGKOK, Thailand (AP)
</DATELINE>
<TEXT>
    More than 150 former officers of the
    overthrown South Vietnamese government
    have been released from a re-education
    camp after 13 years of detention, the
    official Vietnam News Agency reported
    Saturday.

    The report from Hanoi, monitored in
    Bangkok, did not give

        :

Vietnamese capital of Saigon.

    The amnesties apparently are part of
    efforts by Communist Party chief Nguyen
    Van Linh to heal internal divisions and
    improve Vietnam's image abroad.
</TEXT>
</DOC>

```

Figure 3.5: A typical document from the TREC document set

by SGML tags and comes in the form shown in figure 3.5. The AP2WSJ and FR2ZIFF2 collections contain 154,439 and 76,780 news articles respectively.

The queries used were those corresponding to the ad-hoc tasks in the TREC-1 (queries 51 to 100), TREC-2 (queries 101 to 150) and TREC-3 (queries 151 to 200) conferences.

```
<top>
<head> Tipster Topic Description
<num> Number: 059
<dom> Domain: Environment
<title> Topic: Weather Related Fatalities
<desc> Description:
Document will report a type of weather
event which has directly caused at least
one fatality in some location.
<smry> Summary:
Document will report a type of weather
event which has directly caused at least
one fatality in some location.
<narr> Narrative:
A relevant document will include the number
of people killed and injured by the weather
event, as well as reporting the type of
weather event and the location of the
event.
<con> Concept(s):
1. lightning, avalanche, tornado, typhoon,
    :
4. NOT earthquakes, NOT volcanic eruptions
<fac> Factor(s):
<def> Definition(s):
</top>
```

Figure 3.6: A sample query from the TREC collection

These can also be found on the TREC web site [108]. A typical query is shown in figure 3.6. As we can see by the sample query shown, a typical TREC query is not equivalent to a typical Web search engine query. We have also shown in section 3.4 that the FDS method is better suited for short Web like queries since the only score component which

will contribute for large queries is the zeroth. To extract Web like queries from the query set, we chose to use only the words which appeared in the title section of the query. The titles only consist of a few terms which are very similar to those which a typical web user would issue when searching for articles on the same topic.

To evaluate the methods performed on each document set, we examined the precision¹ at 10 and 20 documents retrieved. This gives a good idea as to which methods would display the correct results on the first and second pages of results if used as a Web search engine.

3.7.2 Methods Performed

The FDS methods performed were combinations of the document weighting, magnitude and phase precision, and summing components shown in table 3.2. These were compared to the vector space methods BD-ACI-BCA, AB-AFD-BAA, BI-ACI-BCA, and the SMART Lnu.ltu weighting. Tables 3.3 and 3.4 show the results for the top 20 methods when ranked by precision after 10 documents and 20 documents respectively. Any vector space methods which did not make the top 20 are shown at the bottom of the table.

The results show that using FDS does boost the precision of a document search when using Web like queries. We can see from the results in all of the short query tables that by using FDS we are able to increase the precision of the underlying vector space method chosen. For example *fds-9-4-1.b5* uses SMART's Lnu.ltu weighting, we can see in the tables that it appears above SMART, therefore it has increased the precision. We can also see that most of the methods appearing in the top results use sum magnitudes with zero phase precision (*x.4.z.bn*), and about half of the results use sum components (*x.y.1.bn*) in order where a higher value of *n* usually means a higher precision.

To show the effect of long queries, we have also included a few results using the whole query (not just the title) in table 3.7. We can see from these results that FDS drops down to the level of the vector space methods and therefore is not as effective in this case.

3.8 Discussion and Summary

Internet search engines have become an essential tool for locating resources and information on the Web. Due to the large majority of Web information being textual, a need arises for a superior text search engine. We have presented a novel and simple document

¹Precision is the proportion of relevant documents to total documents retrieved.

Table 3.2: Method names are of the form fds-x-y-z.bn where the values of x,y,z,n associate to the description in this table

Label	Value	Description
x	5	BD-ACI-BCA preweighting
	7	AB-AFD-BAA preweighting
	8	BI-ACI-BCA preweighting
	9	Lnu.ltu preweighting
y	1	Sum vectors with No phase precision
	3	Sum magnitude with Non-zero phase precision
	4	Sum magnitude with Zero phase precision
z	1	Sum components in order
	2	Sum largest phase precision components
	3	Sum largest magnitude components
	4	Sum largest score components
	5	Sum threshold phase precision components
n	1-5	Number of score components added

ranking method called Fourier Domain Scoring (FDS) which provides more precise results than the currently used vector space similarity methods. We also showed that the existing vector space similarity methods are a special case of FDS. The results show that FDS is a superior method because it makes use of the spatial information within a document rather than the count of each query term.

Results were shown for two different large datasets supplied by TREC. The experiments were set up to simulate text retrieval on the Web and compared to existing popular vector space methods. We showed that FDS produced more precise results than the vector space methods for both data sets when using Web like queries but reverted back to vector space method quality when using long queries.

This implies that FDS would enhance the performance of a Web search engine in terms of precision if used in the place of a vector space measure as the text classification method. But if we examine the storage required and query times we will see that the vector space methods are much faster and require less storage capacity. In the next chapter we will examine these issues and provide a method which will bring the spectral methods up to speed with the vector space methods.

Method	Precision 5	Method	Precision 10
fds-7-4-2.b2	0.5027	fds-7-4-4.b2	0.4687
fds-5-4-4.b2	0.5027	fds-7-4-1.b5	0.4673
fds-7-4-4.b2	0.4973	fds-7-4-3.b2	0.4667
fds-5-4-3.b2	0.4960	fds-5-4-1.b5	0.4647
fds-9-4-1.b5	0.4960	fds-5-4-3.b2	0.4627
fds-7-4-1.b5	0.4947	fds-7-4-2.b2	0.4620
fds-7-4-3.b2	0.4933	fds-5-4-2.b2	0.4620
fds-5-4-2.b2	0.4933	fds-9-4-1.b5	0.4613
fds-7-4-1.b4	0.4933	fds-7-4-1.b4	0.4613
fds-9-4-2.b2	0.4920	fds-5-4-4.b2	0.4600
fds-7-4-1.b3	0.4880	fds-9-4-4.b2	0.4587
AB-AFD-BAA	0.4880	fds-9-4-3.b2	0.4587
fds-5-4-1.b5	0.4867	fds-9-4-2.b2	0.4580
fds-9-4-4.b2	0.4867	fds-7-4-1.b3	0.4580
fds-9-4-1.b4	0.4867	fds-9-4-1.b4	0.4567
fds-7-4-1.b2	0.4867	fds-5-4-1.b3	0.4560
fds-9-4-3.b2	0.4853	fds-5-4-1.b4	0.4547
fds-7-4-5.b2	0.4813	fds-7-4-5.b2	0.4547
fds-5-4-1.b4	0.4800	fds-7-4-1.b2	0.4547
fds-9-4-1.b3	0.4747	fds-9-4-1.b3	0.4547
SMART	0.4693	AB-AFD-BAA	0.4493
BD-ACI-BCA	0.4440	SMART	0.4493
BI-ACI-BCA	0.4347	BD-ACI-BCA	0.4247
		BI-ACI-BCA	0.4100

(a)

(b)

Table 3.3: Best 20 document retrieval methods ordered by precision after a) 5 and b) 10 documents were retrieved for titles of queries 51-200 on AP2WSJ2. Vector space methods not in the best 20 are shown below the bar.

Method	Precision 15	Method	Precision 20
fds-7-4-1.b5	0.4493	fds-7-4-4.b2	0.4247
fds-7-4-1.b4	0.4458	fds-9-4-1.b5	0.4227
fds-5-4-1.b5	0.4440	fds-9-4-4.b2	0.4220
fds-9-4-1.b4	0.4440	fds-7-4-2.b2	0.4220
fds-9-4-4.b2	0.4436	fds-7-4-1.b5	0.4220
fds-7-4-4.b2	0.4431	AB-AFD-BAA	0.4217
fds-5-4-1.b4	0.4413	fds-7-4-3.b2	0.4210
AB-AFD-BAA	0.4404	fds-7-4-1.b4	0.4207
fds-7-4-2.b2	0.4396	fds-5-4-1.b4	0.4197
fds-9-4-1.b5	0.4391	fds-5-4-3.b2	0.4193
fds-5-4-4.b2	0.4382	fds-5-4-1.b5	0.4193
fds-9-4-2.b2	0.4378	fds-9-4-1.b4	0.4190
fds-7-4-3.b2	0.4369	SMART	0.4180
fds-9-4-3.b2	0.4369	fds-5-4-4.b2	0.4177
fds-5-4-2.b2	0.4364	fds-5-4-2.b2	0.4167
fds-9-4-1.b3	0.4360	fds-5-4-1.b3	0.4167
SMART	0.4356	fds-9-4-3.b2	0.4147
fds-7-4-1.b2	0.4351	fds-7-4-1.b3	0.4147
fds-7-4-1.b3	0.4320	fds-9-4-2.b2	0.4127
fds-5-4-1.b3	0.4320	fds-9-4-1.b3	0.4117
BD-ACI-BCA	0.4142	BD-ACI-BCA	0.3953
BI-ACI-BCA	0.3862	BI-ACI-BCA	0.3657

(a)

(b)

Table 3.4: Best 20 document retrieval methods ordered by precision after a) 15 and b) 20 documents were retrieved for titles of queries 51-200 on AP2WSJ2. Vector space methods not in the best 20 are shown below the bar.

Method	Precision 5	Method	Precision 10
fds-9-4-1.b4	0.2119	fds-9-4-1.b5	0.1713
fds-9-4-5.b2	0.2079	fds-9-4-1.b3	0.1644
fds-9-4-1.b5	0.2059	fds-9-4-5.b2	0.1644
fds-9-4-1.b3	0.2020	fds-9-4-1.b2	0.1624
SMART	0.2000	fds-9-4-1.b4	0.1624
fds-9-4-3.b2	0.1980	fds-9-4-2.b2	0.1614
fds-9-4-4.b2	0.1980	SMART	0.1614
AB-AFD-BAA	0.1960	fds-9-4-3.b2	0.1614
fds-7-4-1.b4	0.1960	fds-9-4-4.b2	0.1614
fds-5-4-5.b2	0.1960	AB-AFD-BAA	0.1594
fds-7-4-1.b5	0.1960	fds-7-1-1.b4	0.1584
fds-7-4-5.b2	0.1960	fds-5-4-1.b3	0.1584
fds-5-4-1.b4	0.1941	fds-5-4-1.b5	0.1584
fds-9-4-2.b2	0.1901	fds-7-4-1.b4	0.1574
fds-5-4-1.b2	0.1901	fds-5-4-5.b2	0.1574
fds-9-4-1.b2	0.1881	fds-5-4-1.b4	0.1574
fds-5-4-1.b3	0.1861	fds-7-4-5.b2	0.1554
fds-5-4-1.b5	0.1822	fds-5-4-1.b2	0.1554
fds-7-1-1.b5	0.1762	fds-7-4-1.b5	0.1545
BD-ACI-BCA	0.1762	fds-7-1-1.b5	0.1535
BI-ACI-BCA	0.1267	BD-ACI-BCA	0.1376
		BI-ACI-BCA	0.0901

(a)

(b)

Table 3.5: Best 20 document retrieval methods ordered by precision after a) 5 and b) 10 documents were retrieved for titles of queries 51-200 on FR2ZIFF2. Vector space methods not in the best 20 are shown below the bar.

Method	Precision 15	Method	Precision 20
fds-9-4-1.b4	0.1505	fds-9-4-1.b5	0.1366
fds-9-4-1.b5	0.1465	SMART	0.1342
SMART	0.1459	fds-5-4-4.b2	0.1337
fds-9-4-1.b3	0.1452	fds-9-4-1.b4	0.1332
fds-5-4-5.b2	0.1432	fds-5-4-1.b3	0.1327
fds-5-4-1.b1	0.1426	fds-5-4-2.b2	0.1327
fds-9-4-5.b2	0.1426	fds-5-4-1.b2	0.1322
fds-9-4-2.b2	0.1426	fds-5-4-1.b1	0.1312
fds-5-4-2.b2	0.1419	fds-5-4-3.b2	0.1307
fds-5-4-3.b2	0.1419	fds-9-4-1.b3	0.1302
fds-7-1-1.b5	0.1419	fds-5-4-1.b4	0.1302
fds-5-4-1.b5	0.1413	fds-5-4-1.b5	0.1287
fds-9-4-1.b2	0.1413	fds-9-4-5.b2	0.1282
fds-9-4-4.b2	0.1406	fds-5-4-5.b2	0.1277
fds-5-4-4.b2	0.1399	fds-9-1-1.b5	0.1272
fds-5-4-1.b4	0.1399	fds-9-4-4.b2	0.1267
fds-9-1-1.b5	0.1386	fds-7-1-1.b5	0.1257
fds-5-4-1.b2	0.1386	fds-9-4-1.b2	0.1252
fds-5-4-1.b3	0.1380	fds-9-4-2.b2	0.1252
fds-7-1-1.b4	0.1360	fds-7-1-1.b4	0.1252
AB-AFD-BAA	0.1360	AB-AFD-BAA	0.1238
BD-ACI-BCA	0.1201	BD-ACI-BCA	0.1139
BI-ACI-BCA	0.0812	BI-ACI-BCA	0.0748

(a)

(b)

Table 3.6: Best 20 document retrieval methods ordered by precision after a) 15 and b) 20 documents were retrieved for titles of queries 51-200 on FR2ZIFF2. Vector space methods not in the best 20 are shown below the bar.

Table 3.7: Best 20 document retrieval methods ordered by precision after 10 documents retrieved for long queries 51-200 on AP2WSJ2. Vector space methods not in the best 20 are shown below the bar.

Method	5	10	15	20
BD-ACI-BCA	0.6040	0.5740	0.5453	0.5257
SMART	0.5800	0.5587	0.5396	0.5207
fds-5-4-1.b5	0.5987	0.5500	0.5200	0.5027
fds-5-4-1.b4	0.5933	0.5473	0.5182	0.5003
fds-5-4-1.b3	0.5840	0.5427	0.5178	0.4947
fds-5-4-1.b2	0.5733	0.5420	0.5187	0.4917
fds-5-4-1.b1	0.5560	0.5300	0.5049	0.4790
BI-ACI-BCA	0.5613	0.5233	0.5018	0.4783
fds-5-1-1.b1	0.5347	0.5213	0.4956	0.4727
fds-9-4-1.b4	0.5640	0.5173	0.4867	0.4737
fds-9-4-1.b5	0.5640	0.5173	0.4898	0.4690
fds-9-4-1.b3	0.5560	0.5133	0.4853	0.4670
fds-9-4-1.b2	0.5413	0.5073	0.4853	0.4693
fds-8-4-1.b5	0.5427	0.5007	0.4769	0.4483
fds-9-4-1.b1	0.5307	0.4980	0.4729	0.4560
fds-8-4-1.b4	0.5480	0.4967	0.4680	0.4470
fds-8-4-1.b3	0.5333	0.4913	0.4676	0.4403
fds-8-4-1.b2	0.5293	0.4833	0.4631	0.4397
fds-8-4-1.b1	0.5013	0.4733	0.4493	0.4250
fds-7-4-1.b5	0.5013	0.4620	0.4449	0.4223
AB-AFD-BAA	0.4507	0.4147	0.4062	0.3833

CHAPTER FOUR

EFFICIENT IMPLEMENTATION

4.1 Introduction

SPECTRAL based searches observe the frequency spectrum of the words throughout the document, rather than the word position. We have introduced Fourier Domain Scoring, which is a spectral based document ranking method which uses the Fourier transform to map the data from the spatial domain to the frequency domain. This method allows us to easily determine the relevance of each term to a document by observing the magnitude of the spectral components, and we can determine the proximity of the terms by comparing the phase of the spectral components.

A list of possible indexing methods to perform spectral based searches is shown in table 4.1. From this set, using no index and using a spectral index are infeasible because the former is too slow and the latter is too large. Our previous work on spectral based search methods used a spatial index to store the document data. We showed how this method performs well in terms of precision, but because we were using a spatial index¹ the query time was slow. For example, a simple four term query could take from one to three seconds when using the FDS method because the DFT must be performed for every word in every document at query time. Relative to current vector space methods, this method is 100 times slower than when using a spatial inverted index.

To resolve this problem, we can perform the transforms before the query is given, which means we must store the spectral information from the transforms in an index rather than the spatial data. To make this method feasible, we must also compress the data further. In order to achieve a high compression ratio, we must consider cropping spectral components. Unfortunately, the precision suffers when cropping is used with FDS. Therefore we propose a new method of spectral document ranking using the Discrete Cosine Transform and show that it provides high precision when cropping occurs.

¹an index which records the positions of the words in the documents

Index Type used with FDS	Precision	Index Size	Query speed
None	High	Zero	Very slow
Spatial index	High	Average	Slow
Spectral index	High	Very large	Fast
Spectral index with quantisation and cropping	High	Average	Fast

Table 4.1: Index types for Spectral based document retrieval systems.

Our focus in this chapter is to introduce the new Cosine Domain Scoring (CDS) method which uses the fourth configuration in table 4.1. We will investigate the effects of indexing documents relative to their spectral information and incorporate the new ideas on reducing latency and storage space developed for vector space methods to the CDS method. This will give us the high precision achieved in CDS and will further provide the compactness and speed of the vector space methods. This chapter is organised as follows. Section 4.2 provides a brief overview of retrieving documents by using word spectrums, section 4.3 covers the CDS method and discusses problems with the index size. Section 4.4 presents the compression techniques used in creating the spectral index. Section 4.5 shows how we can adapt and apply the fast document ranking methods used in existing systems to the spectral index. We also provide extensive results which show that our method is computationally competitive and at the same time provides high precision.

4.2 Spectral Document Ranking

As we found in the last chapter, before we can construct a document ranking system, we must be able to define what makes a document more relevant to a query than another document. Vector space measures use the notion that a document is more relevant if it contains more query terms than another document. Weighting methods are used to make sure that the terms appearing are not just common terms and generally higher scores are given to documents which contain most of the query terms given. To obtain a document of a certain topic, the user will provide the query terms related to the topic. If the topic cannot be described in one word, then the query will have to consist of more than one term. We state that document A will be more related to a query Q than document B if A contains more query terms than B and the query terms appear together throughout document A . The latter part is to make sure that the query terms are related to each other.

For example if we searched for *Mariquita travels Wales*, we would want the documents that were about ‘Mariquita traveling to Wales’, not ‘Laurence traveling to Wales who later meets Mariquita’ or ‘Mariquita traveling to Finland’ and later ‘we find that her father is from Wales’. We could look for the exact phase, but that would limit us to a smaller set of documents. For example we would miss out on ‘Traveling to Wales with Mariquita’ and ‘Mariquita travels to Finland and then Wales’. That is why we want a higher document score for query terms which are closer. This can be achieved through spectral document ranking. We have examined the increase in precision of the FDS spectral method over the vector space and proximity methods in the previous chapter, but we did not discuss how to implement this system efficiently.

In this section we will revisit the FDS method and examine how the increase in precision comes with the cost of lengthy query times. We will also discuss how we can overcome this by using the cosine transform in the place of the Fourier transform.

4.2.1 Fourier Domain Scoring

We can recall from the last chapter that one of the great features of the Fourier transform is the way that it handles shifts in time or space. Any shifts become a difference in phase in the signal spectrum.

$$\mathcal{F}[f(t)] = F[k] \quad \mathcal{F}[f(t - t_0)] = e^{-2\pi i t_0 k} F[k] \quad (4.1)$$

where $\mathcal{F}$ is the Fourier transform. Therefore, the phase of the Fourier transform of a signal tells us how close the signal frequency components are to each other and the magnitude of the signal spectrum will tell us how much effect the frequency component has on the signal. This can help us score document by appearance and closeness of query terms. If we treat each document as a signal, we are able to extract a word signal for every term that appears in the document. The Fourier transform can then be applied to the word signals related to the query terms and the document score calculated from the word signal spectrums. This was performed in chapter 3 and found to be a superior method to the vector space methods in terms of precision, however, there is a disadvantage in its lengthy query times. In order to conserve disk space, the word signals were stored in an index (not their transform). The Fourier transform was performed for each query term in all documents which lead to query times in the order of seconds (about 100 times slower than the vector space methods). To reduce the query time, we must store the word signal spectrums rather than the word signals. This leads to a very large index because the spectral components are floating point values and because there are many more non-zero components after the transform. To reduce the index size, we could

ignore some of the components. Unfortunately, this causes a large change in precision when using the Fourier Transform. To obtain an index of respectable size, we must investigate another transform which is similar to the Fourier transform but will cause a graceful reduction in precision with a reduction of spectral components used.

4.2.2 The Discrete Cosine Transform

The Discrete Cosine Transform (DCT), like the Fourier Transform, converts a sequence of samples to the frequency domain. But unlike the Fourier transform, the basis is made up of cosine waves. Therefore each basis wave has a set phase.

The DCT is of the form:

$$\tilde{X} = \text{DCT}(\tilde{x}) \quad X_k = \sum_{b=0}^{B-1} x_b \cos \frac{(2b+1)k\pi}{2B} \quad (4.2)$$

where $\tilde{X} = [X_0 \ X_1 \ \dots \ X_{B-1}]$ and $\tilde{x} = [x_0 \ x_1 \ \dots \ x_{B-1}]$. Therefore, a real positive signal (as in a word signal) maps to a real signal after performing the cosine transform. The DCT was originally used to aid in solving the problems of pattern recognition and Wiener filtering [1].

Remark 1. *The basis set of the discrete cosine transform (DCT) provides a close approximation to the Eigenvectors of the class of Toeplitz matrices given as:*

$$\psi = \begin{bmatrix} 1 & \rho & \rho^2 & \dots & \rho^{M-1} \\ \rho & 1 & \rho & \dots & \rho^{M-2} \\ \rho^2 & \rho & \ddots & & \vdots \\ \vdots & \vdots & & \ddots & \\ \rho^{M-1} & \rho^{M-2} & \dots & \rho & 1 \end{bmatrix}, \quad 0 < \rho < 1 \quad (4.3)$$

for values of ρ close to 1 [1, 105].

Remark 2. *The covariance matrix for a Markov-1 process is of the form given in equation 4.3.*

A Markov-1 process consists of a set of M states where the transition to the next state depends only on the current state. The states are also ordered, so that the probability of a transition to a closer state (the closest state being the current state) is more likely than a transition to a further state. The process can be thought of as an evolution where small changes are more likely than a huge jump in state. We can see that the covariance (and hence the correlation) between two states reduces as the displacement between transitions grows.

Features in a data set can be described as attributes that vary across different elements (documents) in the set (document set). If an attribute is the same for all elements (documents), it is not an interesting feature because we are unable to distinguish any elements (documents) of the set (document set) from it. Attributes that vary across a set are interesting because we are able to deduce information about the elements based on this attribute. The Karhunen-Loève transform (KLT) transforms a set of vectors, so that each component of the vector represents the direction of greatest variance in decreasing order. Therefore KLT is optimal for finding important features. An example of the KLT can be seen in figure 4.3. We can see that the points have been mapped to a space where the x-axis is the dimension of greatest variance. The y-axis is the dimension of second greatest variance, which is also orthogonal to the x-axis. In this two dimensional case, the y axis is also the dimension of least variance.

To perform the KLT, we must take the whole data set into consideration. For large data sets, this can be computationally expensive.

Remark 3. *The Karhunen-Loève Transform (KLT) basis functions are found by calculating the Eigenvectors of the covariance matrix of the data set.*

The KLT finds the optimal basis functions for a given data set such that the new basis is the set of principle components. These basis functions are found by calculating the Eigenvectors of the covariance matrix of the data set. This leads to our final remark:

Remark 4. *If the given data behaves as a Markov-1 process, the KLT of this data set can be closely approximated by the DCT.*

The flow of colour in natural images is a Markov-1 process, this is the reason why the DCT is used in the JPEG standard and produces high quality lossy compression. It was shown by Shannon [104] that a string of text in English is a first order stationary Markov process. To find how close our term signals are to a Markov-1 process, we have extracted a sample of the weighted term signals from the inverted index and obtained its covariance matrix. This covariance matrix was fitted to the Markov-1 covariance matrix to obtain the best estimate of ρ for our data set. Values of ρ close to 1 give a better approximation of the DCT to the KLT. We found the best fit to have $\rho = 0.79$ which is reasonable. A plot is presented in figure 4.1 showing the error associated to a range of values of ρ . Figure 4.2 provides us with a comparison of the actual basis functions of the sampled weighted term signals given by the KLT (solid lines) and the DCT basis functions (dashed lines). Similarities can be seen amongst the two for each component. For example, all DCT components follow the same trend as the KLT, but they are not

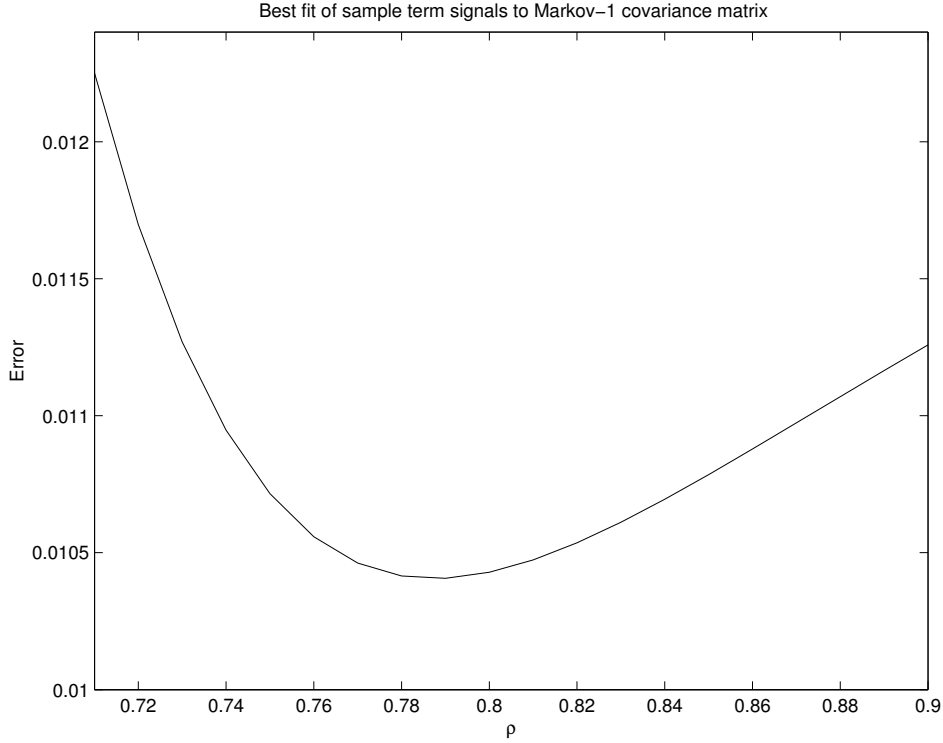


Figure 4.1: Fitting the sampled weighted term signal covariance matrix to the Markov-1 covariance matrix. We can see that the best fit is at $\rho = 0.79$.

exactly alike. This must be expected as the English language is not strict on how texts are formed, but the general form exhibits properties of a Markov-1 process. We will take advantage of the compression qualities of this relationship during our spectral index construction and use the DCT to capture the important features in our data set.

4.3 Cosine Domain Scoring

Cosine domain scoring (CDS) and the closely related Fourier domain scoring (FDS) [87] are unique document ranking methods which analyse the term spectra of each document, rather than just obtain a word count for each document.

CDS extracts word spatial information from documents, rather than just the word count. The spatial information for each document comes in the form of a set of word signals, one for every unique word in the document (as explained in section 3.2.1). Our word signals had a set length (eight bins), each bin contained the count of the word in that fraction of the document.

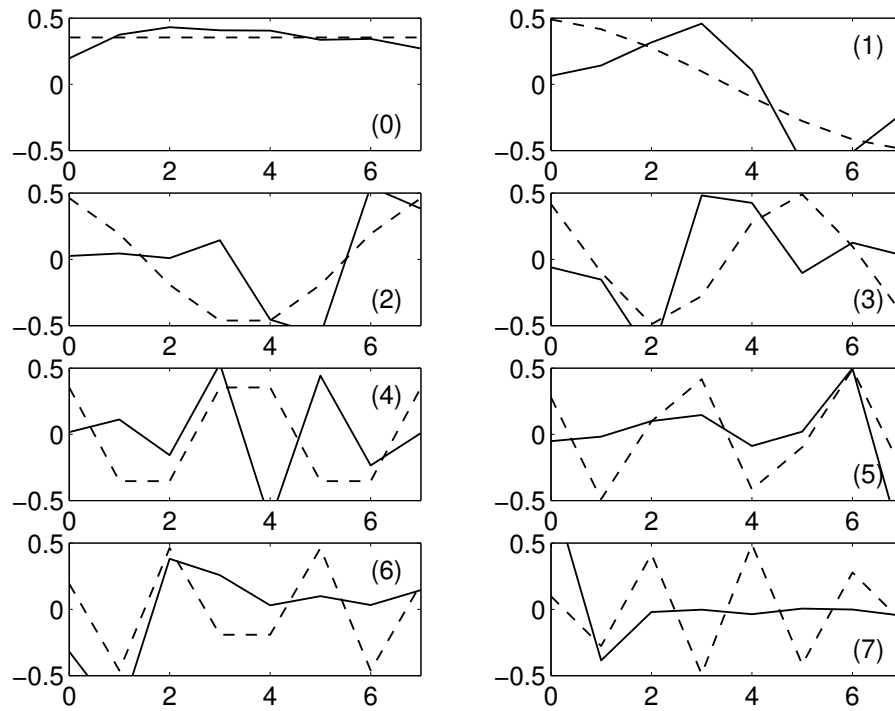


Figure 4.2: A comparison of the basis functions generated by the KLT (solid line) and DCT (dashed line) of the weighted term signals. The bracketed number indicates which spectral component the function represents.

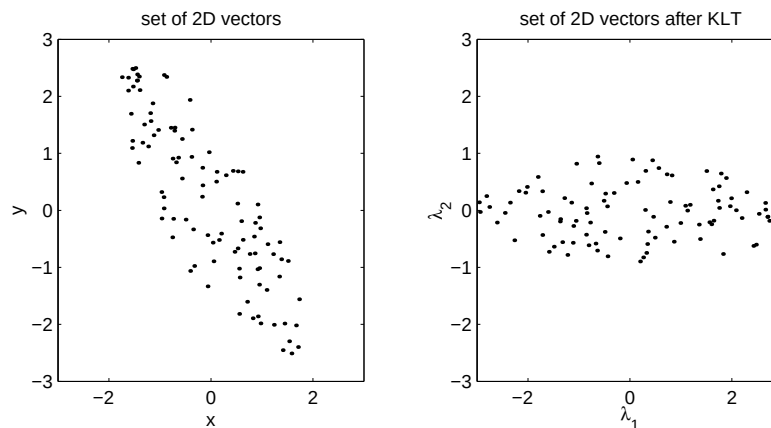


Figure 4.3: Example of the Karhunen-Loève transform. The top plot displays 100 randomly generated points. The bottom plot shows the same points after performing the KLT.

After the word signals have been created, we can use algorithm shown in algorithm 4.1 to rank documents once a query is given.

1. For each query term $t \in T$
 - (a) Extract word signals $\{\tilde{f}_{0,t}, \tilde{f}_{1,t}, \dots, \tilde{f}_{D,t}\}$
 - (b) Weight word signals $\tilde{\omega}_{d,t} = \Omega_d(\tilde{f}_{d,t})$
 - (c) Transform signals into spectrums $\tilde{\eta}_{d,t} = \text{DCT}(\tilde{\omega}_{d,t})$
 - (d) For each spectral component $\eta_{d,t,b} \in \tilde{\eta}_{d,t}$
 - i. Calculate signal component magnitudes $H_{d,t,b} = \Omega_q(|\eta_{d,t,b}|)$
 - ii. Calculate signal component phase $\phi_{d,t,b} = \text{sgn}(\eta_{d,t,b})$
2. For each document d in collection
 - (a) For each component b in word signal spectra
 - i. Calculate the selective phase precision $\bar{\Phi}_{d,b} = \left| \frac{\sum_{t \in T} \phi_{d,t,b}}{\#(T)} \right|$
 - ii. Obtain component score $s_{d,b} = \bar{\Phi}_{d,b} \sum_{t \in T} H_{d,t,b}$
 - (b) Combine component scores to obtain document score $s_d = \sum_{b=0}^{B-1} s_{d,b}$

Algorithm 4.1: Cosine Domain Scoring algorithm

The CDS algorithm can be broken into five steps:

1. Extract query term word signals $\tilde{f}_{d,t}$
2. Perform preweighting ($\tilde{f}_{d,t} \rightarrow \tilde{\omega}_{d,t}$)
3. Perform DCT ($\tilde{\eta}_{d,t} = \text{DCT}(\tilde{\omega}_{d,t})$)
4. Combine word spectrums ($\tilde{\eta}_{d,t} \rightarrow \tilde{s}_d$)
5. Combine word spectrum components ($\tilde{s}_d \rightarrow s_d$)

Each of the non-self explanatory steps are explained in detail below.

4.3.1 Prewriteighting

The CDS method gives a certain degree of freedom when weights are applied. We can see in algorithm 4.1 that the preweighting (Ω_d) and query weighting functions (Ω_q) have not been defined. When compared to the vector space methods, Ω_d is equivalent to the document weighting ($w_{d,t}/W_d$, where $w_{d,t}$ is the document weighting and W_d is the document norm) and Ω_q is equivalent to the query weighting ($w_{q,t}/W_q$, where $w_{q,t}$ is

the query weighting and W_q is the query norm). When querying using a vector space method, weights are always applied to the term counts from documents to emphasise the significance of a term in that document. For example, the BD-ACI-BCA weighting scheme found in Zobel and Moffat [126] is of the form:

$$w_{d,t} = 1 + \log_e f_{d,t} \quad W_d = (1 - s) + s \cdot \frac{W'_d}{\text{av}_{d \in D} W'_d} \quad (4.4)$$

$$w_{q,t} = (1 + \log_e f_{q,t}) \log_e \left(1 + \frac{f^m}{f_t} \right) \quad W_q = 1 \quad (4.5)$$

where $f_{d,t}$ is the count of term t in document d , s is the slope parameter (about 0.7), $W'_d = \sqrt{\sum_{t \in T_d} w_{d,t}^2}$, T is the set of query terms that appear in document d , $f_{q,t}$ is the count of term t in query q , f_t is the number of documents that contain the term t and f^m is the largest f_t of the collection. To use this weighting scheme with word signals, we need to adjust it to fit the number of bins and to incorporate the phase precision. We made appropriate modifications by replacing $f_{d,t}$ with $f_{d,t,b}$ and by multiplying $w_{q,t}$ by the phase precision. The result is then divided by W_d (the norm of all $w_{d,t}$) to normalise the score for each document. Therefore:

$$\Omega_d(f_{d,t,b}) = \frac{1 + \log_e f_{d,t,b}}{W_d} \quad (4.6)$$

and

$$\Omega_q(|\eta_{d,t,b}|) = \frac{|\eta_{d,t,b}| (1 + \log_e f_{q,t}) \log_e \left(1 + \frac{f^m}{f_t} \right)}{W_q} \quad (4.7)$$

where $f_{d,t,b}$ is the count of term t in bin b of document d , $\eta_{d,t,b}$ is the b th spectral component of term t in document d and W_d and W_q are the document and query normalisation factors respectively.

4.3.2 Combination of word spectrums

Once the DCT has been performed, we must combine all of the query word spectrums into one. In these experiments, we used magnitude $\times$ zero phase precision (introduced in section 3.2.5). The combined word spectrum is defined as:

$$\tilde{s}_d = \begin{bmatrix} s_{d,0} & s_{d,1} & \dots & s_{d,B-1} \end{bmatrix} \quad s_{d,b} = \Phi_{d,b} \sum_{t \in T} H_{d,t,b}$$

where B is the number of spatial bins chosen, $H_{d,t,b}$ is the magnitude of the b th frequency component of the t th query term in the d th document and $\Phi_{d,b}$ is the phase precision of the b th frequency component in the d th document. The magnitude and phase precision values are extracted from the frequency components in the following way:

$$\eta_{d,t,b} = H_{d,t,b} e^{i\theta_{d,t,b}} \quad (4.8)$$

where $\eta_{d,t,b}$ is the b th frequency component of the t th query term in the d th document. The phase vector is defined as follows:

$$\phi_{d,t,b} = \frac{\eta_{d,t,b}}{|\eta_{d,t,b}|} = e^{(i\theta_{d,t,b})}$$

The DCT does not produce complex values when applied to a real signal, hence, we treat the sign of the component as the phase. Therefore $\eta_{d,t,b}$ is real and so $\theta_{d,t,b}$ must be of the form πn , where n is an integer. This implies that we will have only $\phi_{d,t,b} \in \{-1, 1\}$. The zero phase precision equation can be simplified to:

$$\begin{aligned} \text{Zero phase precision} := \bar{\Phi}_{d,b} &= \left| \frac{\sum_{t \in T: H_{d,t,b} \neq 0} \phi_{d,t,b}}{\#(T)} \right| \\ &= \left| \frac{\sum_{t \in T} \text{sgn}_0(\eta_{d,t,b})}{\#(T)} \right| \end{aligned} \quad (4.9)$$

where T is the set of query terms, $\#(T)$ is the cardinality of the set T , and

$$\text{sgn}_x(y) = \begin{cases} 1 & \text{if } y \geq 0 \\ x & \text{if } y = 0 \\ -1 & \text{if } y < 0 \end{cases}$$

4.3.3 Combination of spectral components

After combining the word spectrums into one single score spectrum, we are left with B elements to combine in some way to produce a score. If using the Fourier transform, only the first $B/2 + 1$ elements are used since the rest are the complex conjugate of these. If using the DCT, all elements need to be considered, there is no dependence on any of these elements. We will consider the method of summing the first b components, where $0 < b \leq B$. By summing all of the components we will be able to utilise all of the information obtained from the DCT of the word spectrums. The second method (Sum first b components) will be considered due to the closeness of the DCT to the KLT. When the KLT is performed on a signal, we are adjusting the basis of the signals space such that the dimensions are ordered in terms of importance. If we consider only the first b components, we will be making a least squares approximation of the spectral vector for b dimensions. Therefore by performing the DCT and taking the first b components, we should have a close approximation to the B dimensional vector in the b dimensional space.

FDS requires $B+2$ elements to be stored per word signal ($\frac{B}{2} + 1$ for magnitude and $\frac{B}{2} + 1$ for phase). CDS uses the DCT which produces real values, therefore only B elements need to be stored. This is still a large amount of data when we consider that the vector

space methods only require that one element is to be stored. This is where the dimension reduction properties of the DCT are useful.

It is safe to assume that the CDS word signals are first order stationary Markov processes and hence the DCT is a good approximation of the KLT. Therefore, we should expect a reduction of dimension and still obtain results comparable to those without the reduction. By performing the reduction, we do not have to store as much in the index and we do not have to perform as many calculations. Although the reduction may cause a degradation in the quality of results, it should be graceful due to the DCT approximation of the KLT. We performed experiments on the reduced data using a selection of short queries taken from TREC queries 51 to 200 titles. The document set was the TREC Associated Press disk 2 document set [53]. The results can be seen in table 4.2.

In this experiment, we compared CDS with component cropping of D_i (only the first i components are chosen), FDS, and the BD-ACI-BCA vector space method. The titles of eleven queries were chosen from the TREC query set and relevance judgments were created to suit these queries. It can be seen in table 4.2 that CDS has the high precision found in FDS and the precision of each trial reduces slightly when ignoring the latter spectral components due to the DCT's relationship to the KLT.

The CDS method gives a higher precision than the vector space methods, but it uses a very large index which holds the spectral data produced from the cosine transform of the word signals. It is useful to note that the CDS method does not need all of the components to generate results of high precision to reduce the size of the spectral index. We deal with this in the following section via different coding schemes.

4.4 Compressed Spectral Inverted Index

We have stated earlier that the benefit of using the CDS method is that it provides high precision while using a compact spectral index. When storing a spectral index, we are faced with the problem of how to store it due to its excessive size. To reduce the size to an acceptable level we will use quantisation when storing elements and a component cropping technique when storing vectors.

4.4.1 Quantisation

To successfully implement a spectral inverted index, we must consider quantisation. If we choose not to quantise the numerical data we will be left with an index of excessive size. For example, a floating point number can be stored in 4 bytes. If we were to store a document set which contained 80,000 document and 120,000 unique words,

Query term	Number of Relevant documents in top 20									
	D1	D2	D3	D4	D5	D6	D7	D8	FDS	VS
Airbus Subsidies	10	11	12	12	12	13	14	14	14	10
Satellite Launch Contracts	13	14	14	14	14	14	14	13	12	7
Rail Strikes	16	17	17	18	18	18	19	18	17	8
Weather Related Fatalities	10	10	10	9	9	8	9	10	11	6
Information Retrieval Systems	6	6	7	6	7	7	7	6	7	3
Attempts to Revive the SALT II Treaty	4	5	7	9	10	12	12	14	12	8
Bank Failures	19	19	17	19	19	17	18	18	18	16
U.S. Army Acquisition of Advanced Weapons Systems	1	3	4	4	4	4	4	3	4	2
International Military Equipment Sales	8	9	8	9	9	11	11	10	11	6
Fiber Optics Equipment Manufacturers	5	8	8	8	8	8	8	8	8	5
Total	92	102	104	108	110	112	116	114	114	71

Table 4.2: Short queries applied to the AP2 document set using the reduced dimension cds-5-4-1 method. The column Dx refers to the cds-5-4-1 method using the first x components and VS refers to the BD-ACI-BCA vector space method.

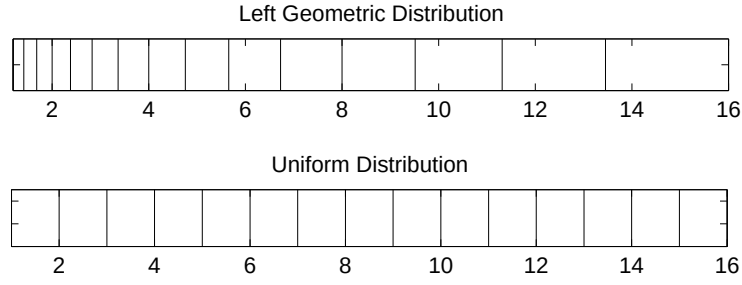


Figure 4.4: Distributions of the two quantisation methods. We can see that the left geometric distribution gives finer quantisation at the lower end of the scale.

and if each word appeared in 1% of the documents, then we would need an index of size $4 \times 80,000 \times 120,000 \times 0.01 = 384$ megabytes per bin (≈ 6 gigabytes for eight bins). Larger data structures also imply slower processing due to time being spent swapping data in and out of memory.

Quantisation is the process of mapping a continuous or a discrete domain to a discrete domain of less elements. By doing so, we are rounding values to a close approximate of the value. Since this new quantised domain has less elements, we do not need as many bits to encode each element. For example, if we were to quantise to a domain which had 16 elements, we would only need 4 bits to encode each element using binary encoding. Therefore quantisation reduces the size of the data, but since we are taking approximations of the original data, we may experience a loss in quality.

Anh *et al.* [5] used three separate quantisation methods to encode the weights of a vector space index, two of which gave fairly precise results. These two methods were labeled *Uniform* and *Left Geometric*. The uniform distribution spreads the quantised values evenly over the entire range. The uniform quantised values are calculated using the following:

$$q_u(x) = \left\lfloor 2^b \frac{x - L}{U - L + \epsilon} \right\rfloor \quad (4.10)$$

where b is the number of bits allotted to store the quantised value, L is the smallest value in the set, U is the greatest value in the set, and ϵ is a small value. The term ϵ is introduced to limit the quantised range to $\{0, 1, \dots, 2^b - 1\}$. The left geometric distribution places more quantised levels at the lower end of the value range. This way we obtain more precise quantised values (due to small quantisation error) at the lower end of the scale and less precise quantised values at the upper end of the scale. This can be seen in figure

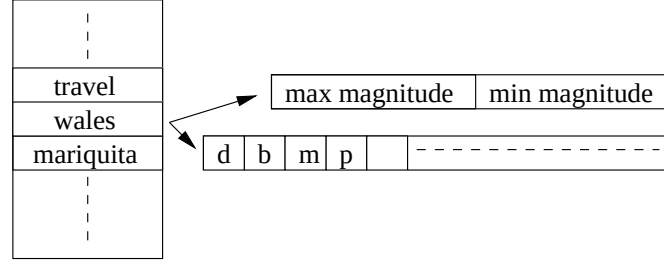


Figure 4.5: Once stored in the index, each term from the document set has an inverted list and an associated maximum and minimum magnitude value used to quantise the inverted list.

4.4. The left geometric distribution is given by:

$$q_{lg}(x) = \left\lfloor 2^b \frac{\log x - \log L}{\log U - \log L + \epsilon} \right\rfloor \quad (4.11)$$

While Anh *et al.* used global quantisation for its simplicity, we prefer to use the alternative local version (local to each term, shown in figure 4.5) for the following reasons: 1) Before we can do any quantisation, we must find the upper and lower bound of the values to be quantised. The values to be quantised are the magnitude and phase of each frequency component of each word in each document. Since phase is a radial value, we know that it is bounded by $-\pi$ and π . All we know about the magnitude is that it is not less than zero. We could use zero as the lower bound, but since we do not record any zero values in the index, this would lead to a sub-optimal quantisation. Therefore we must perform preweighting and the transform, only then can we find the upper and lower bound of the magnitude parameter. If we were to do this globally, we would need to process the entire data set performing the preweighting and transform to obtain the bounds, then rescan data set (performing the same preweighting and transform) to obtain the quantised data. We chose to perform local quantisation because the data from each term (an inverted list) is small enough to perform the preweighting and transform and keep the calculated values in memory. Thus, we can find the upper and lower bound for that term and quantise the data immediately. 2) Each term will have a varying range, therefore we will achieve a smaller quantisation error with local quantisation. Local quantisation requires storage of the upper and lower bound for each term, whereas with the global method, we need only store one upper and lower bound. For our case, the lexicon contained 143044 words, therefore this added an extra 550 kilobytes to the index overhead. This is a small price to pay for higher quality quantisation.

The final issue is to choose how we are to quantise the magnitude and phase values.

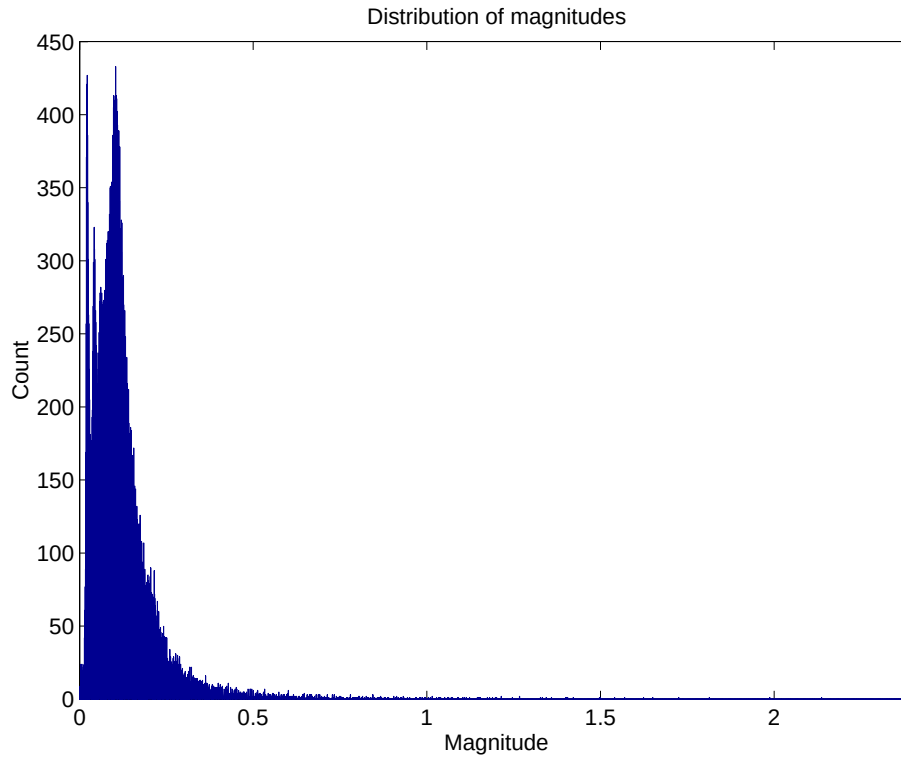


Figure 4.6: Distribution of the magnitude values of the word ‘radio’ from the data set AP2. Most of the magnitude values appear at the lower end of the scale (close to zero).

If we look at the distribution of the magnitude of each word (shown in figure 4.6) we can see that most of the values will appear at the lower end of the scale and only a few at the upper end. This makes sense because it implies that most documents will contain the word only a few times, while only a few documents will contain the larger magnitudes. Therefore, when quantising, we need more quantum levels at the lower end of the scale and less at the upper end. It can be concluded that it is best to use the left geometric quantisation method with the magnitude of each spectral component.

The phase on the other hand is a radial value bound between $-\pi$ and π . It represents the shift in the signal represented by the spectral component. Since this is a relative value (the shift can only be compared relative to another signal), we expect the values to be evenly distributed across the range. In our case (using the cosine transform on a real signal), the only values of phase that will appear are 0 and π , therefore we will use the uniform distribution to represent phase and we will need only one bit to encode it.

Table 4.3 shows the results of an experiment displaying the effects of quantisation.

Bits	Prec.10	Prec.20	Avg. Prec.	R-Prec.
1	0.2034	0.1770	0.1408	0.1737
2	0.3101	0.2747	0.2147	0.2539
3	0.3453	0.3030	0.2519	0.2905
4	0.3514	0.3108	0.2570	0.2960
5	0.3473	0.3132	0.2587	0.2956
6	0.3554	0.3122	0.2599	0.2981
7	0.3554	0.3132	0.2596	0.2975
8	0.3554	0.3128	0.2594	0.2969
9	0.3554	0.3125	0.2597	0.2969
10	0.3554	0.3118	0.2598	0.2970
float	0.3547	0.3108	0.2583	0.2971

Table 4.3: Results from running queries 51-200 on AP2 with varying magnitude quantisation bit size. Prec.10 and Prec.20 are the precision after 10 and 20 documents respectively.

We can see that the precision 10 and precision 20 remain close to the *float*² precision for all values except 1 and 2 bit quantisation. The level of quantisation presents a trade off between index size and precision of results.

We can see from table 4.4a that the spectral indexes size range from three to seven times the size of the spatial index. This was expected due to the large amounts of information stored in the spectral index. In order to show why there is more data in the spectral domain, we can use the simple example of a word signal containing only one non-zero element. This represents the common case of a word appearing in a document once.

$$\begin{bmatrix} 0 & 0 & 1.4 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \xrightarrow{DCT} \begin{bmatrix} 0.49 & 0.38 & -0.26 & -0.68 & -0.49 & 0.13 & 0.64 & 0.58 \end{bmatrix}$$

We can see that the spatial data (before the DCT) can be represented by a single bin position and value, but the spectral data (after the DCT) requires at least eight values to be stored.

Once quantisation has been applied, we can use Golomb coding [47] to compress the data further.

²No quantisation, values are stored as floats.

Inverted index	AP2 %	Inverted index	AP2 %
Spatial	16.4	Spatial	16.4
Spectral		Spectral, $b_{MAG} = 4$	
$b_{MAG} = 1$	43.0	1 component	6.8
$b_{MAG} = 2$	45.8	2 components	14.1
$b_{MAG} = 3$	50.4	3 components	21.1
$b_{MAG} = 4$	56.1	4 components	28.3
$b_{MAG} = 5$	63.1	5 components	34.6
$b_{MAG} = 6$	71.1	6 components	41.9
$b_{MAG} = 7$	80.3	7 components	48.9
$b_{MAG} = 8$	90.9	8 components	56.1
$b_{MAG} = 9$	103.5		
$b_{MAG} = 10$	118.3		

(a)

(b)

Table 4.4: Experiments using document set AP2, AP2% is the size relative to the AP2 document set size. a) Size of inverted indexes relative to the document collection indexed. b) Size of inverted spectral indexes relative to the document collection indexed where the magnitude is quantised to 4 bits.

4.4.2 Component Cropping

To reduce the size of the spectral inverted index further, we can crop the number of components stored. In section 4.3 we showed that we could take advantage of the relationship between the DCT and the Karhunen-Loève transform (KLT) when calculating the document scores. Here we can do the same to reduce the spectral index size.

The KLT is a linear transform which converts a collection of vectors to a space such that the first n dimensions of each vector represents the n principal components of the vector set. By selecting the top n elements of the transformed vector, we are taking the least squares approximation of the vector in n dimensional space. Since the word signal components are independent of each other, the DCT of a word signal is considered to be a close approximation to the KLT for that signal.

We have shown that by using only the first two components we were able to maintain similar precision-recall results when compared to using all components, and we were able to achieve a 40% increase in precision when examining the first 20 documents using the short queries. When using a spatial index, the only advantage in this reduction of dimension was a few less calculations. The index contained spatial data, which is needed

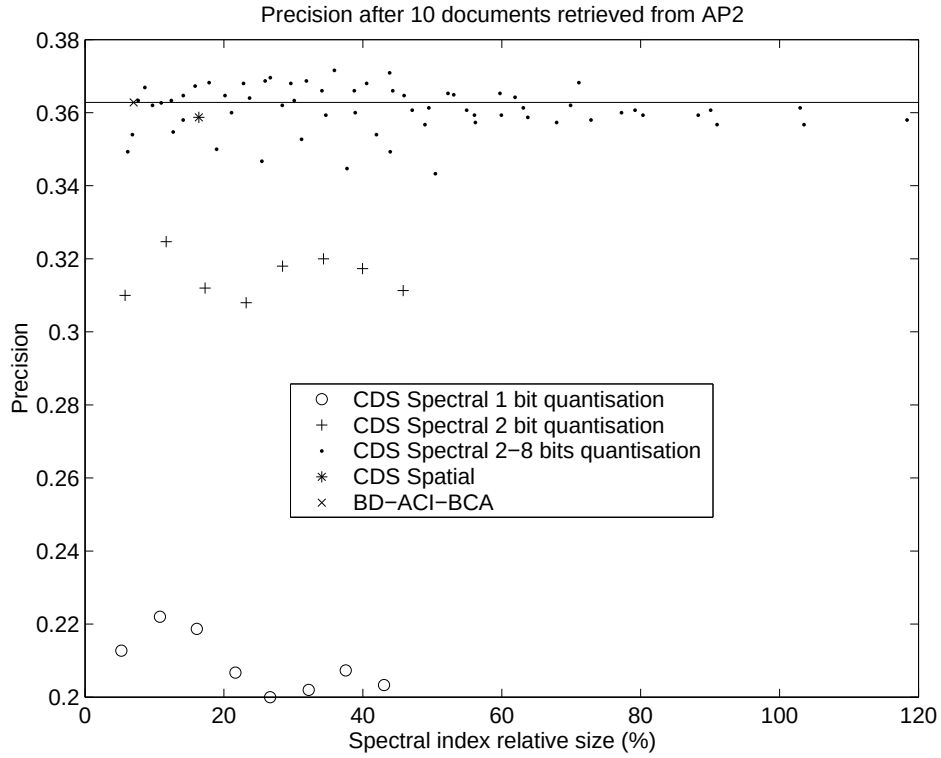


Figure 4.7: A comparison of the CDS method incorporating quantisation and cropping. Each point represents a different magnitude bits, components cropped combination. The cross represents the BD-ACI-BCA method precision and index size.

in its entirety to perform the DCT. Only after we apply the DCT can the least significant components be ignored. Now that we have built a spectral inverted index, we are able to crop the data, which leads to significant reductions in index size.

We can see in table 4.4b that the index size increases linearly with the number of components stored. If we take up to 3 components we have an inverted index of size comparable to that of the spatial inverted index.

4.4.3 Compression achieved

By combining the quantisation and cropping lossy compression methods, we are reducing the spectral index size by a large magnitude. Therefore we must examine the effect of this compression on the precision. Trials were run using quantisation values of 1 to 10 bits and by cropping 1 to 8 components. A plot of the results is shown in figure 4.7.

The plot also contains results for CDS using the slower spatial index and the BD-ACI-BCA vector space method. To aim for the most optimal method from the trials, we want

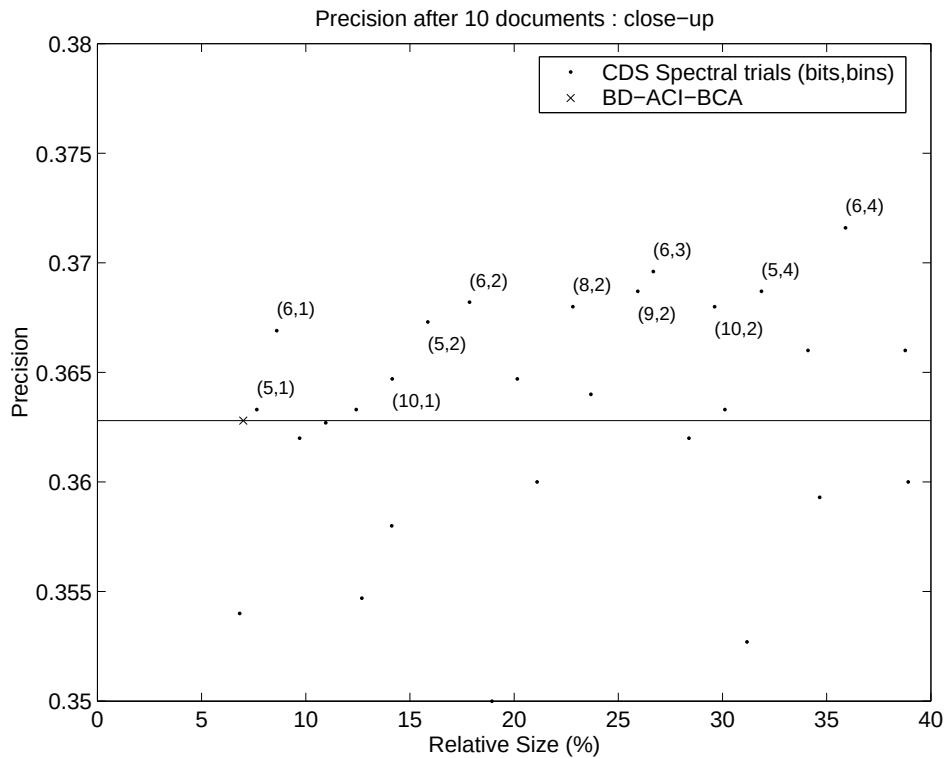


Figure 4.8: A close up of the trials with the greatest precision and smallest spectral index size from figure 4.7. The points are labeled with their (Magnitude bits, components cropped) setting. The cross represents the BD-ACI-BCA method precision and index size.

to examine the trials which have the properties of high precision and small index size. To identify these trials we have taken a close up of this desired region of figure 4.7 in figure 4.8. We can see that the best combination is quantising to 6 bits and cropping to one to four frequency components.

4.5 Fast Document Ranking for Spectral Indices

Once the spectral index has been created, documents can be ranked by extracting and combining certain elements relative to the query. Fast document ranking methods have been considered in vector space methods where the data is ordered in terms of importance. The ranking process will extract the elements and stop when it has obtained *enough* data. We will examine how we can sort the data to give precise rankings and describe a fast document ranking algorithm for spectral inverted indexes.

Vector space index order	Spectral index order
Word	Word
+Magnitude	+Bin
+Document	+Magnitude
	+ Document
	+ Phase

Table 4.5: The order in which vector space and spectral data is encoded after performing quantisation

4.5.1 Sorting the Spectral data

The concept behind fast document ranking is to order the data in an index in such a way that the important information appears first. This way, each score can be quickly approximated and the top d relevant documents can be selected. The scores can also be refined using the additional (less important) data appearing later in the index if desired. Therefore, when building an index, we must decide which information to treat as important and which to treat less important. Important data is information that will have a major effect on a document score. Every inverted index is arranged so that it is ordered into word groups called inverted lists, which allows fast word extraction when a query is issued (this should not be modified). Each word reference in the index contains a list of which documents it appears in, which spectral components it appears in for every document, and the magnitude and phase of every spectral component of every document.

An early paper by Buckley and Lewit [16] shows how the data is sorted according to query weight (w_t). This implies that query terms with high w_t values will contribute to the score, while those with low w_t will probably not. Anh *et al.* [5] reports the effects of sorting each inverted list by weighted frequency, since it is the major influence to the document score. Experiments showed that it had little advantage in terms of precision over the term weight ordering from Buckley and Lewit, but was slightly faster. We know that the cosine transform orders its components by decreasing importance. Therefore it is intuitive to order the spectral components from each word by component number and then magnitude (shown in table 4.5). The document number and phase are stored together in order of appearance.

4.5.2 The Spectral Accumulator

The accumulator set found in the usual vector space searches is the set $\mathcal{A} = \{\mathcal{A}_{i_1}, \mathcal{A}_{i_2}, \dots, \mathcal{A}_{i_K}\}$ where $\mathcal{A}_{i_n}$ is the progressive score (accumulator) of the i_n th document and K is the maximum number of documents wanted from the search. Accumulators are used to keep track of the document scores as the query progresses. Initially, the accumulator set is empty. As each element $w_{d,t}$ is taken from the inverted sorted index, the accumulator set is examined to see if accumulator $\mathcal{A}_d$ exists. If it does, the element is added to the accumulator ($\mathcal{A}_d \leftarrow \mathcal{A}_d + w_{d,t}$). If it does not, an accumulator is created and added to the set, so the weighted value can be assigned to it. Once we reach a predefined number of accumulators (K), we are left with the decision of what to do with the rest of the inverted index elements.

Since we are dealing with a spectral index, we need to make some adjustments to the accumulator set in order to gather the data. We do not combine the phase and magnitude information until we have gathered all of the documents, so we will need separate magnitude and phase accumulators. The magnitude and phase are combined according to the component value, so we will also need separate entries for each component. The accumulators used are of the form $\mathcal{A}_{d,b} = (H_{d,b}, \phi_{d,b})$. Once the score accumulation is over, the corresponding magnitude and phase values are multiplied and the results are added to obtain the document score.

4.5.3 Fast Query Processing

Fast query processing follows the ideology that if we want K documents, why should we have to calculate the scores of more than K . There have been a few strategies for fast query processing, we will briefly mention two of them. *Quit* and *Continue*, by Moffat and Zobel [82], both require that the query terms are ordered from highest to lowest weight (w_t). This ordering is done because it is assumed that the terms with the greatest weight (w_t) will have the most influence on the document score. The first strategy called *quit* fills the accumulators until K accumulators have been created. When this event occurs, the values in the accumulators are used to rank the documents. For example, if we take the inverted index shown in table 4.6 and we wanted to quit as soon as three accumulators were non zero, the process would proceed as follows. First, we set all accumulators to zero ($A_d = 0 \forall n$). Since the term *mariquita* has the greatest value of w_t , we will begin with it. The first (and only) non zero value we come to is for d_2 , therefore we create A_2 and assign $A_2 \leftarrow A_2 + w_{d,t} \cdot w_{q,t}$. Since there are no more non-zero values for the term *mariquita*, we move on to the term with the next greatest term weight (*wales*). The

Query terms	w_t	d_1	d_2	d_3	d_4	d_5
mariquita	0.602	0	2	0	0	0
travel	0.301	0	1	0	0	2
wales	0.398	1	1	0	1	0

Table 4.6: Example inverted index sorted by w_t .

first value to appear comes from document 1 (d_1), therefore we create A_1 and assign $A_1 \leftarrow A_1 + w_{d,t} \cdot w_{q,t}$. The next nonzero value comes from document 2, then document 4. Once accumulator 4 is created, we find that we have three accumulators, which means we stop and return the results.

The second strategy called *continue* fills the accumulators until n are non-zero and then continues to fill only those accumulators, ignoring data that would normally be sent to other accumulators. For example, if we followed the continue strategy we would proceed as in the quit example until three accumulators were created. At this stage there are no values left to process in the term *wales* so we would move onto the term with the next greatest term weight (*travel*). The first value that appears is from document 2. Since accumulator 2 exists, we add this weighted value to it ($A_2 \leftarrow A_2 + w_{d,t} \cdot w_{q,t}$). The final value comes from document five and is ignored since accumulator five does not exist.

4.5.4 Fast Query Processing with Spectral Accumulators

If we try to apply the quit method to our spectral index, we will soon find that we will only be using the data from the first spectral component. This is due to the ordering of the index by component first. To make use of the data found in the other components, we have combined the quit and continue algorithms with the spectral accumulators to come up with the *quit-n* algorithm (algorithm 4.2). If we order the spectral data by component then by magnitude, we are able to perform the *Continue* method and quit when we reach a specific component number.

By incorporating *Quit-n* into CDS and running the same trials as found in section 4.4.3, we obtain the plot in figure 4.9. We can clearly see that the time per query of CDS using *Quit-n* is comparable to the query time of BD-ACI-BCA with *Continue*. The average time of a query using the spatial index is of the order of seconds, therefore by building a spectral index and using fast document ranking, we have achieved a speedup by a factor of 100. Figure 4.10 gives a plot of the more precise trials and their average query times.

1. For each inverted list $I_t = \{\langle d, b, H_{d,t,b}, \phi_{d,t,b} \rangle | \forall d, \forall b\}$
 - (a) Order elements by increasing bin value
 - (b) For each bin set $\{\langle d, b, H_{d,t,b}, \phi_{d,t,b} \rangle | \forall d\}$
 - i. Order elements by decreasing magnitude
2. Set $\mathcal{A} \leftarrow \emptyset$
3. For each query term $t \in T$, retrieve I_t
4. While $I_t \neq \emptyset \forall t \in T$
 - (a) Select $\langle \delta, \beta, H_{\delta,t,\beta}, \phi_{\delta,t,\beta} \rangle \in \{I_t | \forall t\}$ where $\beta = \min(b)$ and $H_{\delta,t,\beta} = \max(H_{d,t,\beta})$
 - (b) If $\beta < \hat{\beta}$ (stopping component)
 - i. If $\mathcal{A}_{\delta,\beta} \notin \mathcal{A}$, $\mathcal{A} \leftarrow \mathcal{A} + \{\mathcal{A}_{\delta,\beta}\}$
 - ii. Set $\mathcal{A}_{\delta,\beta} \leftarrow \mathcal{A}_{\delta,\beta} + (H_{\delta,t,\beta}, \phi_{\delta,t,\beta})$
 - iii. Set $I_t \leftarrow I_t - \{\langle \delta, \beta, H_{\delta,t,\beta}, \phi_{\delta,t,\beta} \rangle\}$
 - iv. If $|\mathcal{A}| > K$, goto step 5
5. While $\beta < \hat{\beta}$
 - (a) Select $\langle \delta, \beta, H_{\delta,t,\beta}, \phi_{\delta,t,\beta} \rangle \in \{I_t | \forall t\}$ where $\beta = \min(b)$ and $H_{\delta,t,\beta} = \max(H_{d,t,\beta})$
 - (b) If $\mathcal{A}_{\delta,\beta} \in \mathcal{A}$
 - i. Set $\mathcal{A}_{\delta,\beta} \leftarrow \mathcal{A}_{\delta,\beta} + (H_{\delta,t,\beta}, \phi_{\delta,t,\beta})$
 - ii. Set $I_t \leftarrow I_t - \{\langle \delta, \beta, H_{\delta,t,\beta}, \phi_{\delta,t,\beta} \rangle\}$
6. For each document d such that $\mathcal{A}_{d,b} = (H_{d,b}, \Phi_{d,b}) \in \mathcal{A}$
 - (a) Set $\bar{\Phi}_{d,b} \leftarrow |\Phi_{d,b}|/|T|$
 - (b) calculate $s_d = \sum_{b=0}^{\hat{\beta}} \bar{\Phi}_{d,b} H_{d,b}$
7. Identify the r highest values of s_d

Algorithm 4.2: Quit- n for CDS using K accumulators and $\hat{\beta}$ components.

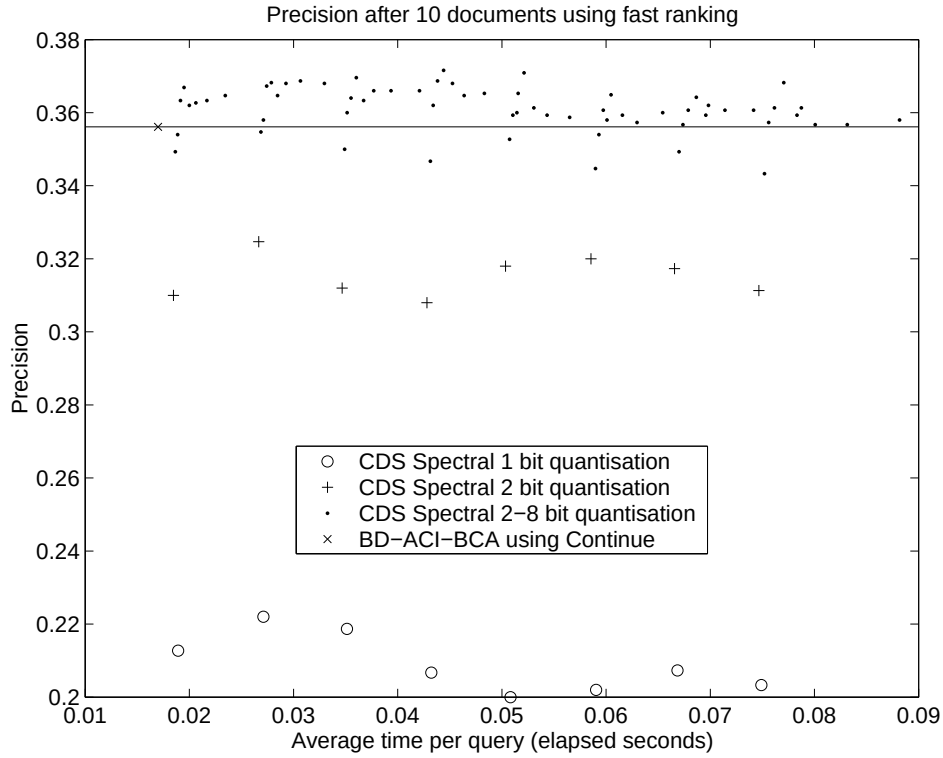


Figure 4.9: A comparison of the CDS method incorporating quantisation, cropping and fast document ranking. Each point represents a different magnitude bits, components cropped combination.

4.5.5 Effect on Precision

If we perform the same trials as found in section 4.4.3, but this time use fast ranking, we will notice that the results are very similar (almost identical). Figure 4.11 shows precision vs. index size plot, the major difference is the lower precision of the BD-ACI-BCA method. This is due to the method incorporating the *Continue* fast document ranking method. The small variance in recall in the CDS trials would be due to the DCT. In these cases, the DCT is performing principal component analysis, which means it is shifting the important information from each word signal into the lower components. Therefore the document set relative to the query can be chosen based on the initial components.

4.6 Discussion and Summary

We have introduced the new method called Cosine Domain Scoring (CDS) which uses the Discrete Cosine Transform (DCT) to perform document ranking.

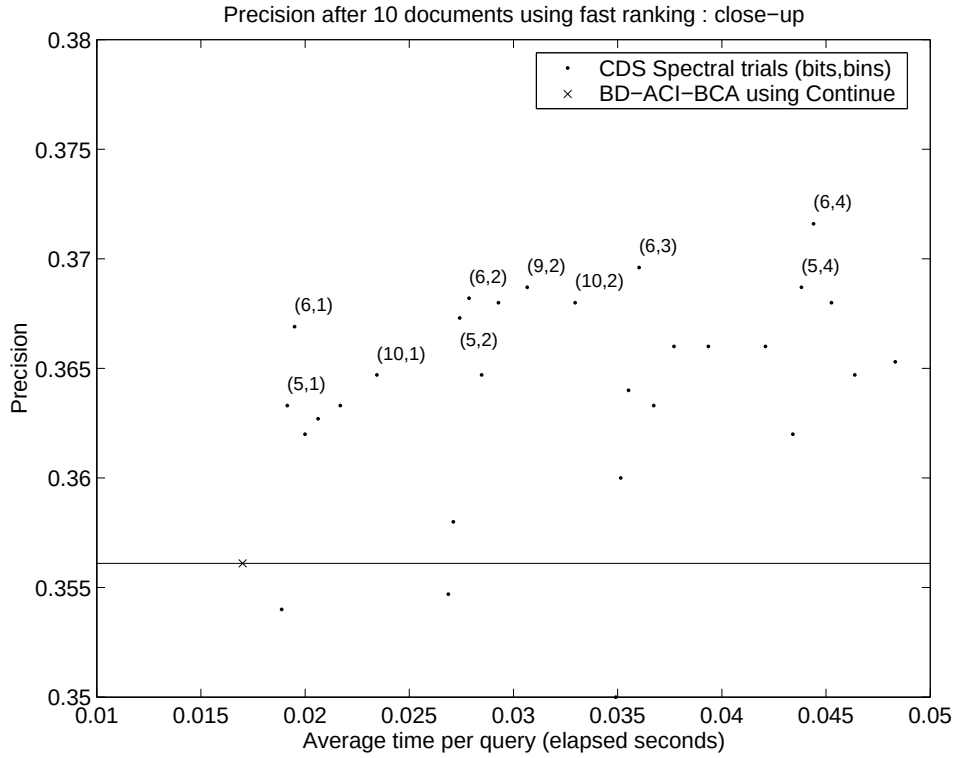


Figure 4.10: A close up of the trials which have the fastest query time from figure 4.9. Each point is accompanied with a (quantised magnitude bits, components cropped) label.

It has been demonstrated that CDS is able to offer results of a similar precision to those of the FDS method and when combined with a spectral index, we are able to produce these results in a competitive time, with a compact index. To build the spectral inverted index we considered the factors of precision, query speed and index size. In this chapter, we have successfully shown how to build a spectral inverted index by employing techniques such as quantisation and cropping so that the index is compact. We have also shown how we can take advantage of the spectral index to perform fast document ranking, giving a query time speed up by a factor of 100 relative to the slower spatial index.

We have obtained a spectral index of comparable size to an index used by vector space methods and shown that the CDS method can retrieve documents at a comparable rate to the BD-ACI-BCA vector space method and still maintain its high precision and recall.

From the experiments, we have concluded that replacing the DFT with the DCT gives

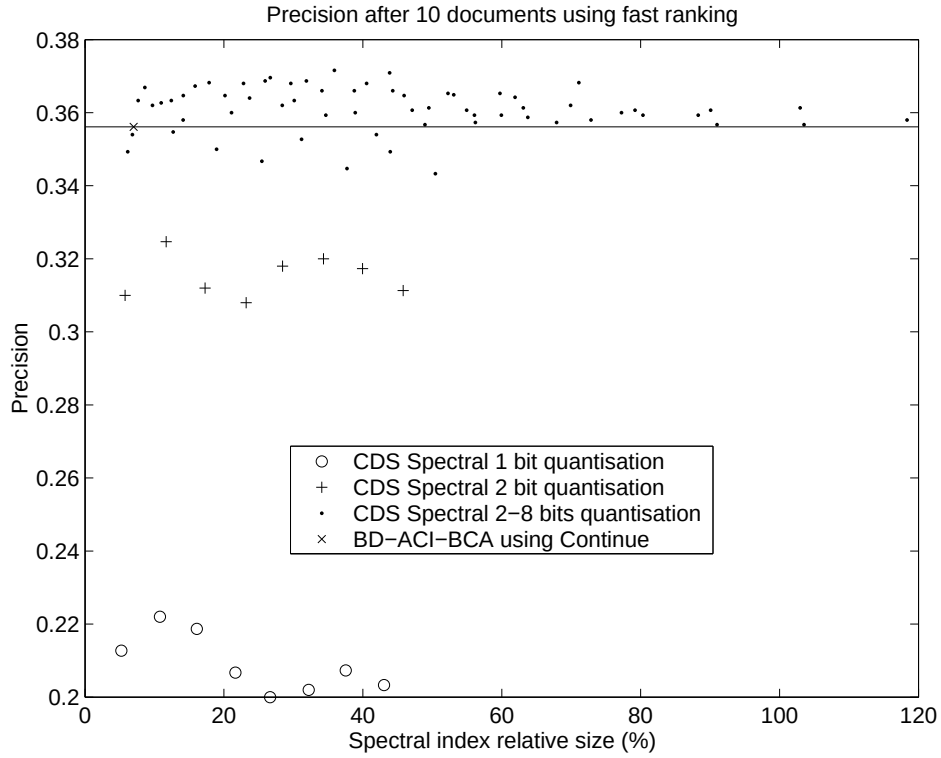


Figure 4.11: A comparison of the CDS method incorporating quantisation, cropping and fast document ranking. We can see the results are almost identical to those with out fast document ranking.

us similar high precision by only using a fraction of the storage. The DCT's relationship to the KLT has allowed us to obtain a deeper understanding of the components produced by the transform. These new results are just as good as our earlier Fourier Domain Scoring (FDS) method, requiring fewer calculations, and allowing us to store the index in a more compact manner and thus compete with vector space methods.

Now that we have shown that we can design a spectral document retrieval system of equivalent query time and storage requirements to the vector space method, we can focus our attention on improving on the high precision which our spectral methods provide. The following chapter will discuss a use for Gaussian functions as a way of limiting the term range of the phase precision. By doing so, we are able to extend the range for rare terms and reduce the range for common terms.

CHAPTER FIVE

GAUSSIAN SPREADING

5.1 Introduction

FOURIER Domain Scoring is an information retrieval method which combines both vector space and proximity searching by not only including the occurrence of the query terms in the score calculation, but also the phase (positional) information of the query terms. If we examine the area of proximity searching, we will notice that most methods have a means of selecting the region in which terms will have some relevance to each other [66, 24, 6, 57, 58], but the proximity measure used in FDS (phase precision) causes each term to have a proximity which covers the whole document.

As shown in chapter 3, the core of the FDS method comes from the land of discrete signal processing. By using the discrete Fourier transform, we are only scratching the surface of this domain of knowledge. One of the main fields is that of the study of linear time-invariant filters (LTI filters). Much of the LTI filter analysis is performed in the frequency domain [85, 94], just as our term spectrum analysis is performed. By using the ideas already explored in signal processing, we will be able to apply a Gaussian function (the filter of our choice) to limit the proximity of query terms according to their rarity. We will also be able to perform this faster since our data has been mapped into the frequency domain.

In this chapter, we will examine how the proximity limiting can be applied in the frequency domain in section 5.2. Section 5.3 will explain how we have chosen the parameters for the Gaussian function. Section 5.4 will discuss the issues behind sampling the Gaussian function. In section 5.5 we will discuss the various methods which we will use. Section 5.6 will analyse the validity of the method we have chosen and Section 5.7 will outline the experiments performed and examine the results.

5.2 Term Signal Dispersion

Proximity searches use the idea that a document is relevant to a certain set of query terms if the query terms appear within a certain number of terms from each other. If the query terms appear closer, the document is considered more relevant than one in which the terms are not as close.

The concept of an influence curve was given by de Kretser [31] in his paper on locality based information retrieval. He suggests that each term in a document has an area of influence represented by some function of finite support, centered on the word in question. If this area of influence is found for every query term, the influence curve can be generated by finding the sum of the curves. The main factors that effected the final influence curve were the shape, height and spread of the function used to determine the influence of each term. An interesting concept used was that the height and spread of the curves were a function of the rarity of the term. Therefore, terms which appeared less often would have a greater and wider influence than terms that appear more often through the document.

If we were to take our binned document in the spatial domain and we were to transform each sample into a curve of influence, we could achieve the same effect as de Kretser, but we would be applying it to a sampled document, rather than the entire document set (the influence will not cross the boundaries of the documents).

5.2.1 Gaussian Function

Instead of choosing one of the curves already investigated, we decided to examine the Gaussian function:

$$\mathcal{G}_\sigma(x) = a_i e^{-(x-x_i)^2/2\sigma_i^2} \quad (5.1)$$

This function is shown in figure 5.1 and is best known as the normal distribution [43]. It has three parameters (a_i , x_i , and σ_i) which are used to control the height, position and width respectively. One of the nice features of the Gaussian function is that its Fourier transform is also a Gaussian function.

$$\mathcal{F}\{\mathcal{G}_\sigma(x)\} = \check{\mathcal{G}}_\sigma(x) = a_i e^{-\pi x^2(2\sigma_i^2)} \quad (5.2)$$

We can think of this transformation from the word positions to the influence of the positions as replacing each of the bins with a curve multiplied by the bin value.

$$\langle f, \mathcal{G}_\sigma(x) \rangle = \sum_{n=0}^{N-1} f[n] a_i e^{-(x-n)^2/2\sigma_i^2} \quad (5.3)$$

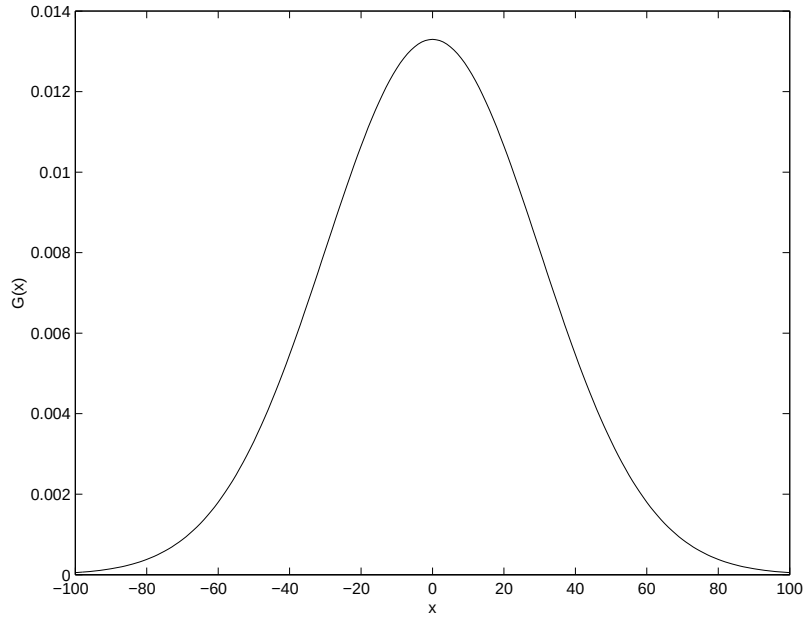
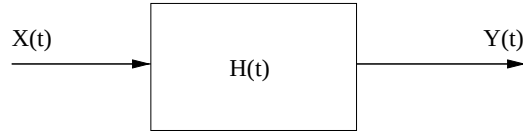


Figure 5.1: The normalised Gaussian function

Figure 5.2: A filter diagram. In our case, $H(x) = \mathcal{G}(x)$, $X(t)$ is the term signal, and $Y(t)$ is the dispersed term signal.

Any curves that cross paths will be added. Alternatively, we could represent this transformation as an LTI filter, which also implies convolution.

5.2.2 Convolution

The application of the Gaussian function to each spatial bin position can be shown as a convolution between the input signal (the binned word) and the filter (the Gaussian influence curve), due to the symmetry of the Gaussian function.

$$(f \star g)(t) \equiv \int_{-\infty}^{\infty} g(\tau) f(t - \tau) d\tau \quad (5.4)$$

where $\star$ is the convolution operator and f and g are arbitrary functions of time t .

The convolution operator has a dual which exists in the frequency domain and the cosine domain [76]. Its dual in the frequency domain being:

Theorem 5.2.1 (Convolution Theorem). *If $f(t)$ and $g(t)$ be arbitrary functions of time t with Fourier transforms, then*

$$\mathcal{F}\{(f \star g)(t)\} = \mathcal{F}\{f(t)\} \mathcal{F}\{g(t)\} \quad (5.5)$$

where $\mathcal{F}$ is the Fourier transform.

We will restrict our attention to using the Fourier transform from now on since it is a more well defined area of signal processing.

If we take theorem 5.2.1 into consideration, we will see that by applying the Gaussian function to the term signal in the form of convolution, we are also multiplying the Fourier transform of the Gaussian function to the resulting term signal spectrum.

$$\mathcal{F}\{(\tilde{\omega}_{d,t} \star \mathcal{G}_\sigma)\} = \tilde{v}_{d,t} \bowtie \check{\mathcal{G}}_\sigma \quad (5.6)$$

where $\tilde{v}_{d,t}$ is the Fourier transform of $\tilde{\omega}_{d,t}$ and $\bowtie$ is the element-wise multiplication operator.

5.2.3 Signal Energy

If we are given a real signal (comprising of values in the real domain only), we are able to find the energy of the signal using the following formula:

$$E = \int_{-\infty}^{\infty} g(t)^2 dt \quad (5.7)$$

We know that the Fourier transform adjusts the basis of this time signal to a set of orthonormal basis vectors in the frequency domain, therefore the energy of the signal is conserved through this transform. To find the energy of a signal spectrum, we will use the relationship found from the convolution theorem

$$\int_{-\infty}^{\infty} G_1(\omega) G_2(f - \omega) d\omega = G_1(f) \star G_2(f) \quad (5.8)$$

$$= \mathcal{F}\{g_1(t)g_2(t)\} \quad (5.9)$$

$$= \int_{-\infty}^{\infty} g_1(t)g_2(t)e^{-i\omega t} dt \quad (5.10)$$

If we set ω to zero we obtain the relationship

$$\int_{-\infty}^{\infty} G_1(\omega) G_2(-\omega) d\omega = \int_{-\infty}^{\infty} g_1(t)g_2(t) dt \quad (5.11)$$

To obtain the energy equation, we let $g_1(t) = g_2(t) = g(t)$ and therefore $G_1(\omega) = G_2(\omega) = G(\omega)$. Since $g(t)$ is a real valued signal, we also know that

$$G(-\omega) = G^*(\omega) \quad (5.12)$$

where $G^*(\omega)$ is the complex conjugate of $G(\omega)$. From this we can see that equation 5.11 collapses to the *Rayleigh Energy Theorem*:

$$\int_{-\infty}^{\infty} g(t)^2 dt = \int_{-\infty}^{\infty} |G(\omega)|^2 d\omega \quad (5.13)$$

where $|G(\omega)|^2$ is known as the energy density spectrum. To obtain the relationship for the discrete case, we will let $g[n] = g(nT)$ and $G[k] = G(kF)$, where $n, k \in \mathbb{N}$ and $T, F \in \mathbb{R}$. Therefore the discrete signal energy is

$$E = \sum_{n=-\infty}^{\infty} T(g[n])^2 = T \sum_{n=0}^{N-1} g[n]^2 \quad (5.14)$$

where the signal is of length N beginning at $n = 0$. The Fourier transform and its inverse are defined as

$$G[k] = T \sum_{n=0}^{N-1} g[n] e^{-j2\pi kn/N} \quad g[n] = F \sum_{k=0}^{N-1} G[k] e^{j2\pi kn/N} \quad (5.15)$$

and discrete convolution is given as

$$G_1[\eta] \star G_2[\eta] = F \sum_{n=0}^{N-1} G_1[n] G_2[\eta - n] \quad (5.16)$$

By following the same procedure as in the continuous case, we arrive at

$$E = F \sum_{n=0}^{N-1} |G[n]|^2 = T \sum_{n=0}^{N-1} g[n]^2 \quad (5.17)$$

Therefore, the energy of the discrete signal is

$$E = T \sum_{n=0}^{N-1} g[n]^2 = \frac{1}{TN} \sum_{n=0}^{N-1} |G[n]|^2 \quad (5.18)$$

where $TF = 1/N$. We can see that this calculation of energy from the spectral density is very similar to our own document score calculation in the way that the magnitudes are added. If we consider the $G[n]$ term to be our weighted frequency magnitude, we could use the energy of the signal to be the document score. Therefore documents which produce term signals with greater energy will give a higher score than documents which produce term signals with less energy.

The FDS score function with Gaussian spreading using the Rayleigh Energy Theorem (using the l_2 norm to combine the frequency bins) is:

$$S_d = \sum_{b=0}^{B-1} \left(\Phi_{d,b} \sum_{t \in T} H_{d,b,t} \check{g}_t \right)^2 \quad (5.19)$$

where $\Phi_{d,b}$ is the phase precision of bin b in document d , and $H_{d,b,t}$ is the magnitude of term t in frequency bin b of document d .

5.3 Choosing the Gaussian parameters

Before we can select parameters for the Gaussian function, we must ask ourselves ‘what do we want from the application of this function?’. The answer is that we want the application of this function to represent the region of the document in which the term the Gaussian represents will take effect.

When using the Fourier method, we use the phase information to find a general proximity of the query terms to each other, but we have no control over the region in which the query term should be compared with other query terms.

The parameters we have to experiment with are the amplitude (a_i), the position (x_i) and the spread (σ_i). We will be applying the Gaussian function after applying a weighting scheme. The weighting scheme will adjust the height of the term signals to normalise it relative to the rest of the document set. Therefore the amplitude of the Gaussian function is given as the resulting term bin value after weighting. Since this is done beforehand, we can ignore the amplitude parameter.

The positional parameter will be determined by the position of the value in the signal which the Gaussian is being applied to. Since we are applying the Gaussian to every position of the term signal through convolution, we do not need to set this and therefore it can be ignored as well.

The final parameter (the spread) is the most important. This will determine the effect of the term through the document. This parameter can be set for each individual term. Therefore, we want a narrow spread for common terms and a wide spread for rare terms. The statistic most suited to this is the term weight (w_t). The term weight is usually inversely related to the number of documents the term has appeared in, and related to the total number of documents. Therefore, the higher the term weight, the more rare the term is through the document set. So the scale parameter must be related to this term weight

The scale parameter must also be related to the length of the document. When we divide the document amongst the predefined number of bins, we must allocate different amounts of document terms to the bins for different documents to make sure all of the terms are allocated evenly. If we were to set the Gaussian scale parameter as the term weight, the width of the function would not be kept constant across documents of different lengths. This effect can be seen in figure 5.3. The document in the top half of figure 5.3 is a longer document than the document shown in the bottom half and hence has more terms per bin. Without document length scaling, we would stretch the Gaussian function over a longer portion of document terms in the top document. This would lead

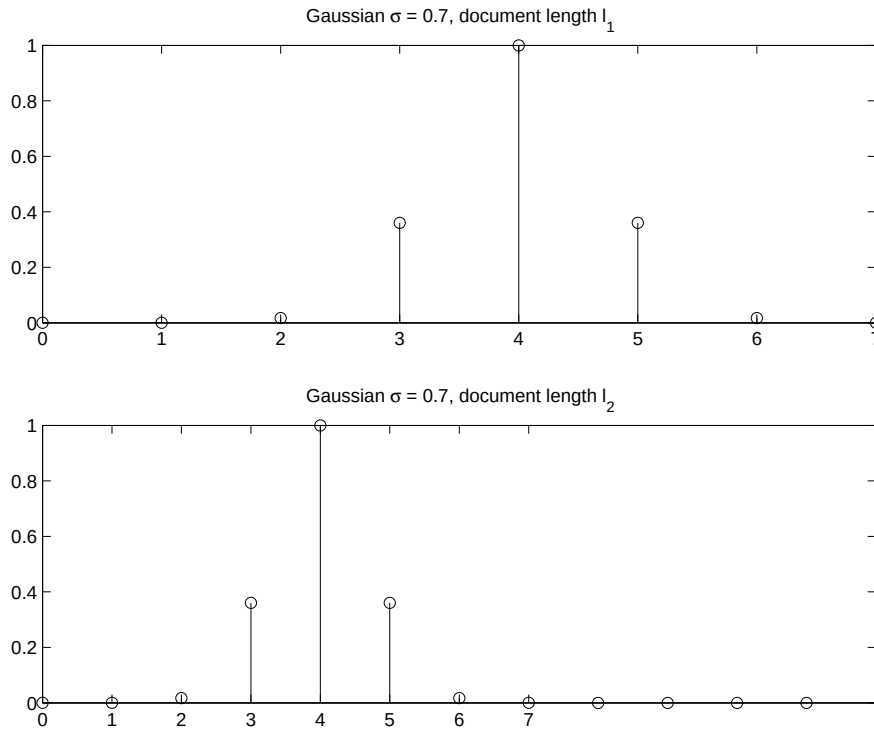


Figure 5.3: Gaussian function applied to two terms from different documents. The longer document of length l_1 has a wider spread than the shorter document l_2 if the scale parameter only depends on the term weight.

to the terms in the longer document to have more significance than the same terms in the shorter document.

To choose a value of σ for a given term, we must know what effect the given value of σ will have on the Gaussian function. If we choose σ to be too small, it would be as if we did not apply the Gaussian at all. If we choose it to be too large, we will lose all of the spatial information we have gathered by only keeping the zeroth (DC) component. Examples of this are shown in figure 5.4.

The form of the Gaussian function we have chosen is that of the normal distribution, therefore σ represents the standard deviation of the distribution from the mean. We also know that 95% of the Gaussian distribution lies within two standard deviations from the mean [43]. This implies that the width of the Gaussian function is approximately four standard deviations or 4σ . If we examine the distribution of the term weights through the document set AP2WSJ2, where w_t is defined as in SMART's Lnu.ltu weighting scheme [17] in figure 5.5, we can see that the distribution is skewed towards the right. The multitude of high term weights implies that the document set contains many terms

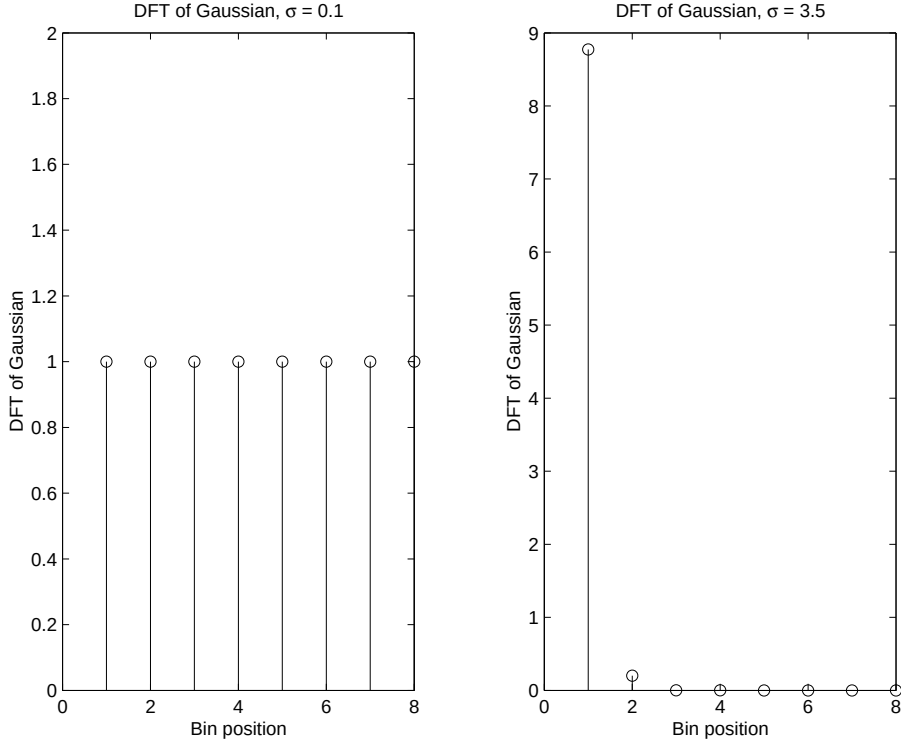


Figure 5.4: Fourier transform of a Gaussian function with $\sigma = 0.1$ and $\sigma = 3.5$

which appear in only a few documents and a few terms which appear in many documents, which would be the case for most document sets. The gaps that can be seen at the high end of the w_t scale are due to the fashion in which the value is calculated. If the histogram groupings were of a finer scale we would see many of these gaps along the w_t scale, with the gap size increasing due to the log function. The gaps are due to the f_t term being used in the calculation, which is an integer. For this document set, w_t ranges between 0 and 12.

Based on the information we have attained, we have chosen the following spread function:

$$\sigma = \frac{w_t B S}{4 f_d} \quad (5.20)$$

where f_d is the number of terms appearing in document d , B is the number of bins chosen, and therefore f_d/B is the number of terms per bin in document d . S is a constant to define the width of the spread. We choose this based on the following points:

1. The spread should be normalised according to the document length
2. It should be related to the term weight

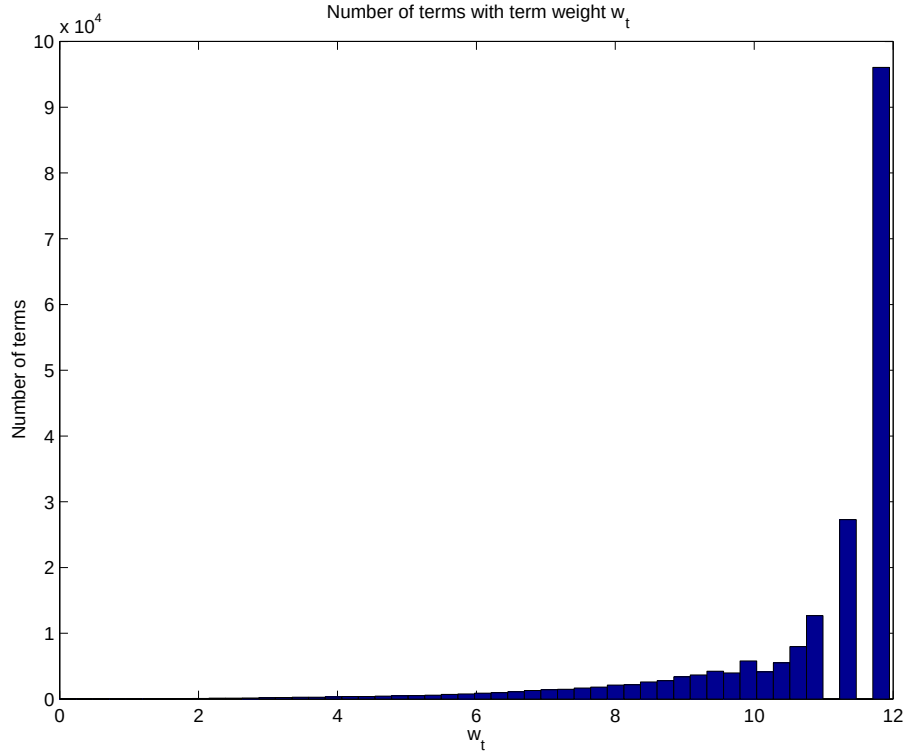


Figure 5.5: Histogram showing the distribution of w_t for all terms in the document set ap2wsj2 using SMART Lnu.ltu weighting.

3. The user should be able to set the number of terms the spread should contain.

The first two points can be easily seen in the equation, the last point is satisfied by noting that the majority of the spread is contained within 4σ . Each unit (bin) in the term signal consists of B terms. If we set $w_t = 1$ we have:

$$S = 4\sigma \frac{f_d}{B} \quad (5.21)$$

This implies that S is the number of terms engulfed by a term with $w_t = 1$.

5.4 Sampling Artifacts

We cannot take the continuous, smooth Gaussian function of infinite support and sample it into a discrete finite domain without obtaining any unwanted effects in both the space and frequency domains. In this section we will describe the aliasing effects found in the frequency domain and the wrap around effects found in the spatial domain, and we will give methods of how to overcome these problems.

5.4.1 Aliasing Effects

We have found that the Fourier transform of a Gaussian is of the same form and we have applied this knowledge to our work. But so far, we have failed to notice that the mathematical forms we were dealing with were continuous functions of infinite domain and that our application requires mapping these continuous functions into our discrete domain. This operation is known as sampling and has many implications involved.

To sample a function, we use the Dirac comb:

$$\delta_T(t) = \sum_{m=-\infty}^{\infty} \delta(t - mT) \quad (5.22)$$

which consists of an infinite sequence of uniformly separated delta functions. The separation constant is given as T and is known as the sampling period. The Fourier transform of this sequence of impulses is:

$$\mathcal{F}\{\delta_T\}(f) = \frac{1}{T} \sum_{n=-\infty}^{\infty} \delta(f - nF) \quad (5.23)$$

which is also a sequence of impulses uniformly separated by the constant $F = 1/T$, which is known as the sampling rate.

If we take the signal we want to sample and convert it into coefficients of the Dirac comb, we obtain the sampled version of our signal. The Fourier transform of our sampled signal can be found as follows [59]:

$$g_\delta(t) = \sum_{m=-\infty}^{\infty} g(mT)\delta(t - mT) \quad (5.24)$$

$$\therefore \mathcal{F}\{g_\delta\}(f) = F \sum_{n=-\infty}^{\infty} G(f - nF) \quad (5.25)$$

using the Fourier transform of Poisson's sum formula [118]:

$$g_p(t) = \sum_{m=-\infty}^{\infty} g(t - mT) = \frac{1}{T} \sum_{n=-\infty}^{\infty} G(nF) \exp\left(\frac{j2\pi n t}{T}\right) \quad (5.26)$$

$$\mathcal{F}\{g_p(t)\} = F \sum_{n=-\infty}^{\infty} G(nF)\delta(f - nF) \quad (5.27)$$

and the duality of the Fourier transform.

This implies that the Fourier transform of a sampled signal is equivalent to the spectrum of the unsampled signal repeated, uniformly spaced across the entire frequency spectrum. An example of this using the Gaussian function is shown in figure 5.6. We can see in this figure that problems are introduced in the frequency domain after sampling. From the above equation, we can see that the sampled spectra are placed F hertz apart.

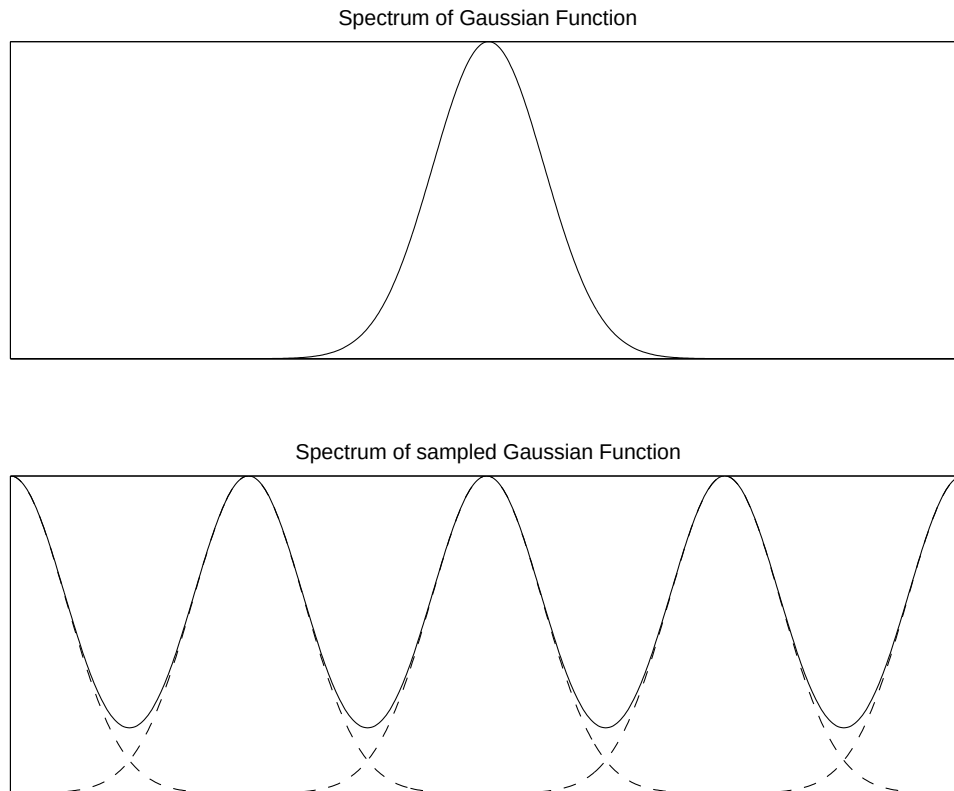


Figure 5.6: The effects of sampling in the frequency domain. The spectrum of the Gaussian function is centered on the positions Nf_s . If the spectrum is too wide, aliasing occurs.

If the width of our signal spectrum (from the centre of the signal spectrum to the edge) is not less than half of the sampling rate, there will be some additive overlap of the sampled repetitive spectrum (as in figure 5.6). This effect is called aliasing and was noticed by Nyquist [59], who proposed the following:

1. A band-limited signal of finite energy, which has no frequency components higher than W hertz, is completely described by specifying the values of the signal at instants of time separated by $1/2W$ seconds.
2. A band-limited signal of finite energy, which has no frequency components higher than W hertz, may be completely recovered from a knowledge of its samples taken at the rate of $2W$ samples per second.

Therefore, we can either sample the data at a higher rate, to make sure that aliasing never occurs (or has little effect in the case of the infinite width Gaussian function), or we could apply an anti-aliasing filter to limit the width of the repeating spectra, or we could simply

calculate the spectrum of the Gaussian by including the aliasing effects.

The first choice is not an option because the width of our Gaussian function is dependent on the term weight of the term the signal is representing. The term weight (w_t) is a real value which has a lower bound of zero and upper bound depending on the choice of weighting. This lower bound of zero implies the corresponding Gaussian function will be an impulse, which has a spectrum of infinite width. To capture this spectrum, an infinite sampling rate is required, which is not feasible.

The second choice, which is using an anti-aliasing filter, is not an option either. By limiting the bandwidth of the Gaussian, we are producing artifacts in the time domain. We are convolving the Gaussian function with the term signal to develop regions of influence in the term signal. By adding an anti-aliasing filter, we would be changing the shape of the Gaussian region of influence. We want to conserve this Gaussian shape, therefore we will not use a filter.

The last option is to take the effects of aliasing in to account when calculating the Fourier transform of the Gaussian function. To do so we have to consider the infinite sum:

$$\mathcal{F}\{\mathcal{G}\}[n] = \sum_{m=-\infty}^{\infty} \sqrt{2\sigma^2\pi} e^{-(n+mN)^2 2\sigma^2\pi^2/N^2} \quad (5.28)$$

After a brief examination of the Gaussian function form, we can see that most of the components of the infinite sum are very close to zero if σ is not close to zero. Therefore, we need only calculate a few components of the infinite sum to get a close approximation to the Gaussian function spectrum. We have chosen to select the four closest components of the infinite sum to obtain an estimate of the actual transform.

$$\hat{\mathcal{F}}\{\mathcal{G}\}[n] = \sum_{m=-2}^1 \sqrt{2\sigma^2\pi} e^{-(n+mN)^2 2\sigma^2\pi^2/N^2} \quad (5.29)$$

By doing this, we have provided a close estimate of the Fourier transform of the sampled Gaussian function for all σ not close to zero. We can see in figure 5.7 the size of the error for various values of σ . The error is very close to zero for all values of $\sigma > 0.26$. As the value of σ decreases the Gaussian function converges to an impulse function and the width of the spectrum increases to infinity, therefore many more of the repeated spectra will overlap our region of interest and cause significant error.

If we examine the actual Fourier transform of the Gaussian for small σ , we will notice that all of the frequency components are very close to 1. This is due to the Gaussian having only one significant component in the time domain. Therefore, to reduce this error for small σ , we will introduce a threshold and assume the transformed sampled

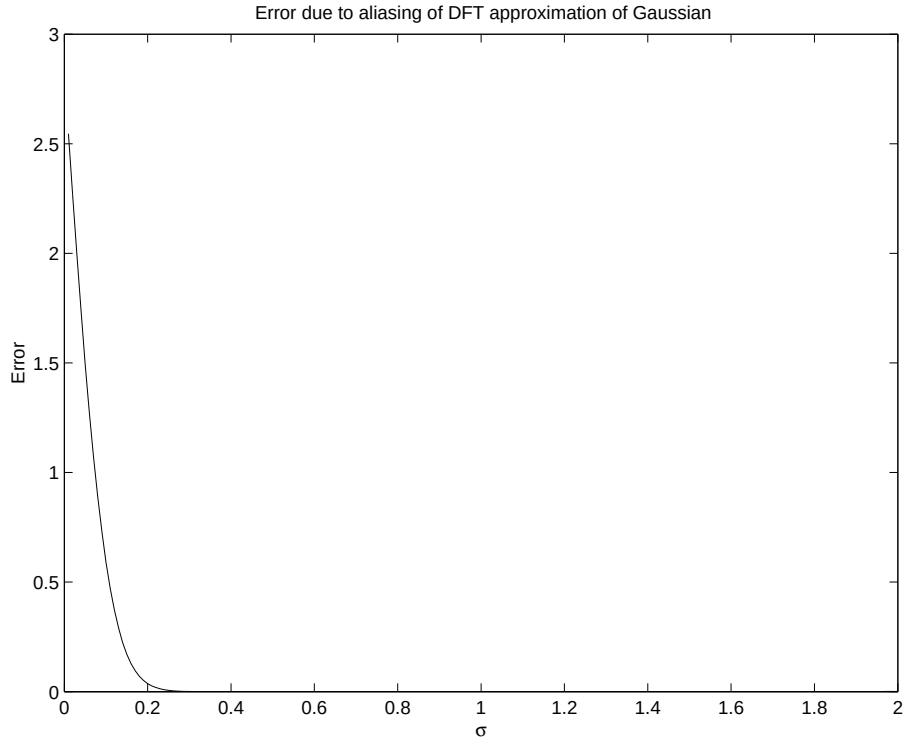


Figure 5.7: The error induced by including only the four closest Gaussian functions in the frequency domain. We can see that the error increases for small σ due to the wide spread of each Gaussian in the frequency domain.

Gaussian function components are all 1 for $\sigma < 0.26$. This leaves us with the equation:

$$\hat{\mathcal{F}}\{\mathcal{G}\}[n] = \begin{cases} \sum_{m=-2}^1 \sqrt{2\sigma^2\pi} e^{-(n+mN)^2 2\sigma^2\pi^2 / N^2} & \sigma > 0.26 \\ 1 & \sigma \leq 0.26 \end{cases} \quad (5.30)$$

This estimate of the Fourier transform of the sampled Gaussian gives an error as shown in figure 5.8. We can clearly see that this is a better estimate. A close observation of the region about the point 0.26 reveals a small amount of error of magnitude 10^{-3} . Table 5.1 displays the results from summing the values from figures 5.7 and 5.8. This clearly shows that the threshold is a significant improvement for the estimate of the sampled Gaussian function.

5.4.2 Wrap around effects

We have shown that when data is sampled, we experience problems with aliasing due to the frequency information wrapping around the edges of the data samples. If we analyse

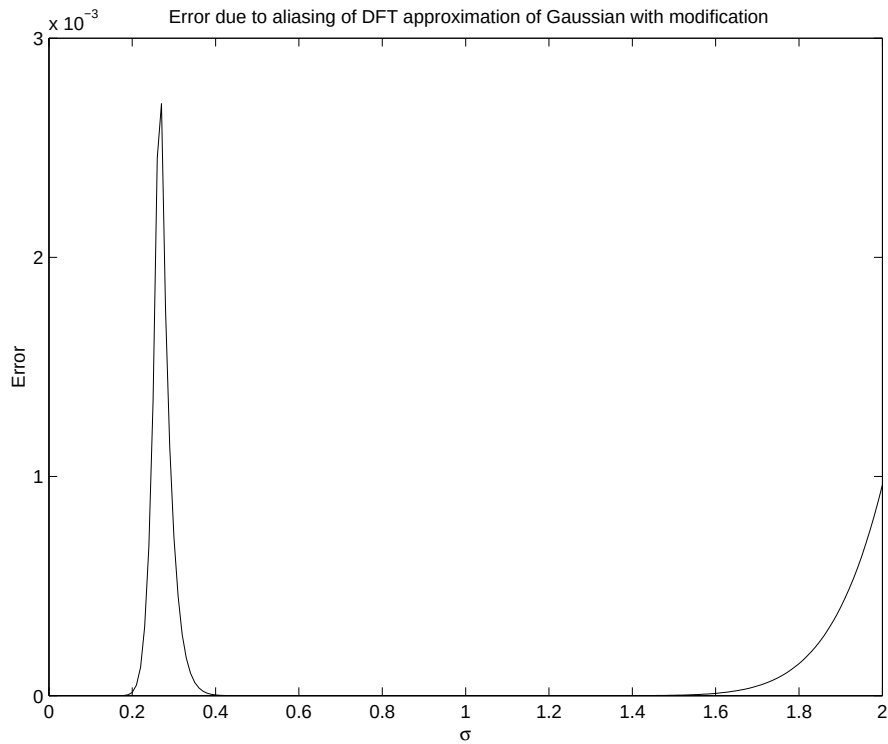


Figure 5.8: The error induced by including only the four closest Gaussian functions in the frequency domain with a cutoff value of $\sigma = 0.26$. We can see that the error for small σ has been reduced significantly due to the cutoff being added.

Method	Sum of errors
sampled Gaussian estimate	16.6213
sampled Gaussian estimate with threshold	0.0232

Table 5.1: Sum of the error between the Gaussian estimate and the actual Gaussian function for σ values of 0 to 2.

the frequency information of the sampled data, we must also expect this wrap around effect to occur in the spatial domain. Therefore if we are comparing positions of terms by analysing the frequency domain, the wrap around implies that the beginning of the document is next to the end of the document. This would cause unwanted effects, for example if a document contained the query terms a and b at either end of the document:

$$\begin{aligned} \text{term a : } & [1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0] \\ \text{term b : } & [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1] \end{aligned}$$

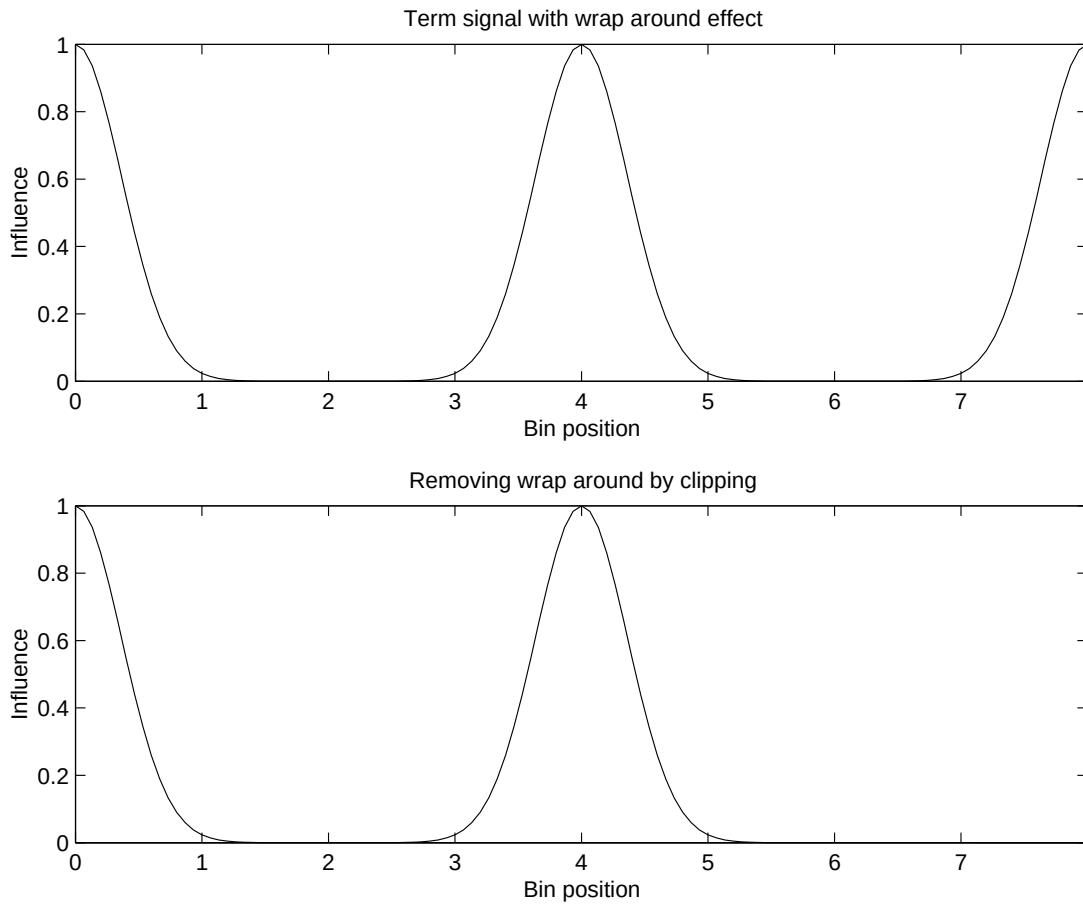


Figure 5.9: If the wrapped Gaussian is clipped, the total signal energy is reduced.

it would be as relevant as a document containing the query terms together

$$\begin{aligned} \text{term a : } & [1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0] \\ \text{term b : } & [0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0] \end{aligned}$$

To remove this problem, we could alter the Gaussian transformation so that there is no wrap around at the edges (clip the edges), but that would lead to the terms appearing at the edges having less influence than the other terms in the document. This effect can be seen in figure 5.9.

Another method which would help reduce the effect of the wrap around is padding. If we introduce n more empty bins at the end of the B bin term signal sample, the Gaussian functions will still wrap around, but they will be more distant.

The number of bins to be added as padding should be dependent on the initial number of bins chosen. We want the end bins to be $B - 1$ bins apart from each other (since this would be the case if there were no wrap around), therefore we must choose the padding

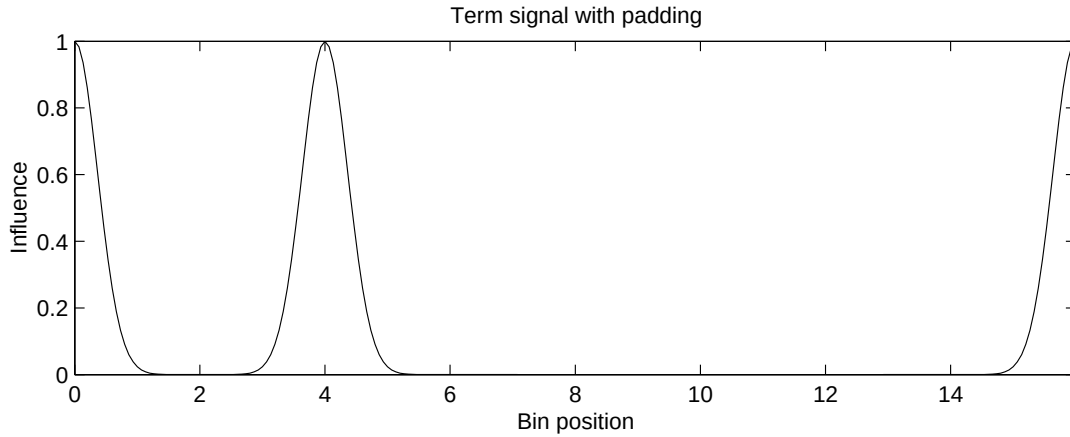


Figure 5.10: By adding padding, we increase the distance between the first and last bins.

to be $B - 1$ or greater. We have chosen B empty bins to use as padding because this will keep the bin size to a power of two (which allows us to perform a faster Fourier transform).

If we use padding on the example we gave before we obtain the following:

```
term a : [ 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ]
term b : [ 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 ]
```

We can see that the two terms are now distant from each other, even if we include the wrap around effect.

5.5 Methods

In the following pages, we will experiment with two different methods. The first will be an application of Gaussian spreading to the FDS method, the second will be a Gaussian Convolution method (no Fourier transform involved). The former of the two methods involves the combination of the phase precision and the Gaussian spreading to retrieve documents with related query terms (shown in algorithm 5.1). The latter utilises only the Gaussian spreading to determine the proximity of the terms (shown in algorithm 5.2), but requires less effort to produce the retrieval results.

5.6 Analysis

Before we begin experimenting, we would like to see that the FDS method with Gaussian spreading behaves as expected. It should provide a higher score if the magnitude of the

1. For each query term $t \in T$
 - (a) Create query term signals $\{\tilde{f}_{0,t}, \tilde{f}_{1,t}, \dots, \tilde{f}_{D,t}\}$
 - (b) Weight signals $\tilde{\omega}_{d,t} = \Omega_d(\tilde{f}_{d,t})$
 - (c) Transform signals into spectrums $\tilde{v}_{d,t} = \text{DFT}(\tilde{\omega}_{d,t})$
 - (d) Apply Gaussian spreading $\tilde{\gamma}_{d,t} = \tilde{v}_{d,t} \bowtie \check{\mathcal{G}}_\sigma$
2. For each document d in collection
 - (a) For each spectral component $\gamma_{d,t,b} \in \tilde{\gamma}_{d,t}$
 - i. Calculate component magnitudes $H_{d,b}^*$
 - ii. Calculate component phase precision $\Phi_{d,b}^*$
 - iii. Obtain component score $s_{d,b} = \Phi_{d,b}^* H_{d,b}^*$
 - (b) Combine component scores $s_{d,b}$ to obtain document score s_d

Algorithm 5.1: Fourier Domain Scoring (FDS) algorithm with Gaussian term spreading. Note that $\bowtie$ is an element-wise multiplication.

1. For each query term $t \in T$
 - (a) Create query term signals $\{\tilde{f}_{0,t}, \tilde{f}_{1,t}, \dots, \tilde{f}_{D,t}\}$
 - (b) Weight signals $\tilde{\omega}_{d,t} = \Omega_d(\tilde{f}_{d,t})$
 - (c) Convolve signals with Gaussian $\tilde{\gamma}_{d,t} = \tilde{\omega}_{d,t} \star \check{\mathcal{G}}_\sigma$
2. For each document d in collection
 - (a) For each spectral component $\gamma_{d,t,b} \in \tilde{\gamma}_{d,t}$
 - i. Obtain component score $s_{d,b} = \sum_{t \in T} \gamma_{d,t,b}$
 - (b) Combine component scores $s_{d,b}$ to obtain document score s_d

Algorithm 5.2: Gaussian convolution method.

query term signals increase and it should also produce a higher score if the displacement between two different query terms decreases. We will examine this in the following sections.

5.6.1 Occurrence Analysis

Since we are applying the Gaussian spreading to the FDS method, the occurrence analysis can be performed by simply using the FDS occurrence analysis equation and adding the Gaussian spreading as a weight to the equation.

Theorem 5.6.1. *The magnitude component of the document score for term t monotonically increases with respect to the count of term t in document, if the l_2 method is used to sum score components.*

Proof. We know from the Rayleigh Energy Theorem (equation 5.18) that:

$$E = \sum_{n=0}^{N-1} g[n]^2 = \frac{1}{N} \sum_{n=0}^{N-1} |G[n]|^2 \quad (5.31)$$

where $T = 1$. If the number of count of a term in a document increases, then so does the sum of the bins of that term. Therefore the count of a term in a document is monotonically related to the energy of the term signal. Since we are using this term signal energy as our magnitude component of the score, this implies that the count of a term in a document is monotonically related to the magnitude component of the score of that document. $\square$

We can see that this also applies to the Gaussian convolution method.

5.6.2 Proximity Analysis

Just as we used the FDS analysis to gather results for the occurrence analysis, so too can we use a modified version of the FDS proximity analysis (by adding the Gaussian spreading) to perform the proximity analysis.

We can recall that if the query given contains two terms, the document score of a document containing each of those terms once is:

$$S_{FDS} = \sum_{b=0}^{B/2} \left| 2 \cos \frac{bd\pi}{B} \right| \quad (5.32)$$

where each term has a weight of 1, B is the number of components and d is the bin displacement of the two terms (the number of bins the second terms is apart from the

first). By letting the score be the sum of squares rather than just a sum, and by including the Gaussian spreading weighting we arrive at the following equation:

$$S_{FDS} = \sum_{b=0}^{B/2} \left(\check{g}_{\sigma}(b) \cdot 2 \cos \frac{bd\pi}{B} \right)^2 \quad (5.33)$$

We want to examine the effect of the Gaussian on the score derived on the proximity of the query terms, therefore we need to compute proximity plots for multiple values of σ . We have provided plots for values of $\sigma = 0, 0.5, 1.4$ and 2.4 in figure 5.11 and the results have behaved as we have expected them to. For $\sigma = 0$, the width of the Gaussian function is zero, so it is just as if the Gaussian was never applied. We can see in figure 5.11 that the position where the displacement is zero is much greater than the other displacements. Every other displacement value provides a similar score. For the case where $\sigma = 0.5$, we can see that the desired curvature is starting to take place. As the displacement grows, the score gradually reduces. For $\sigma = 1.4$, we can see the effect of the Gaussian function taking place. As the displacement grows, the score function decreases very smoothly. As we get to the upper end of the spread scale we see that the displacement effect reduces. We can see that for $\sigma = 2.4$ the Gaussian is so wide that it nearly covers the whole document. This causes the score to reduce very slowly as the term displacement grows.

For the case of the Gaussian convolution (where we exclude the Fourier transform) we obtain the plots shown in figure 5.12. We can see that the score changes very smoothly as the displacement varies. We can also see that the curves evolve smoothly and always keep the desired shape as we adjust the σ parameter.

5.7 Experiments

We performed experiments to analyse our new method and to compare our new method against the existing spectral method it is built upon. The experiments involved several methods of Gaussian spreading which we have mentioned and it involves adjusting the S parameter to try to find an optimal value for this data set.

Our experiments were performed on the TREC document set AP2WSJ2 using queries 51 to 200. As the base FDS method, we chose SMART's Lnu.Ltu weighting (represented as fds-9-y-z) combined with Zero phase precision (represented as fds-x-4-z) and we summed all score components (represented as fds-x-y-1) which is given as fds-9-4-1.

Two different methods of Gaussian spreading were used. The first one used a spread parameter of $\sigma = \frac{w_t BS}{4f_d}$ (shown in equation 5.20) which depends on the rarity of the term.

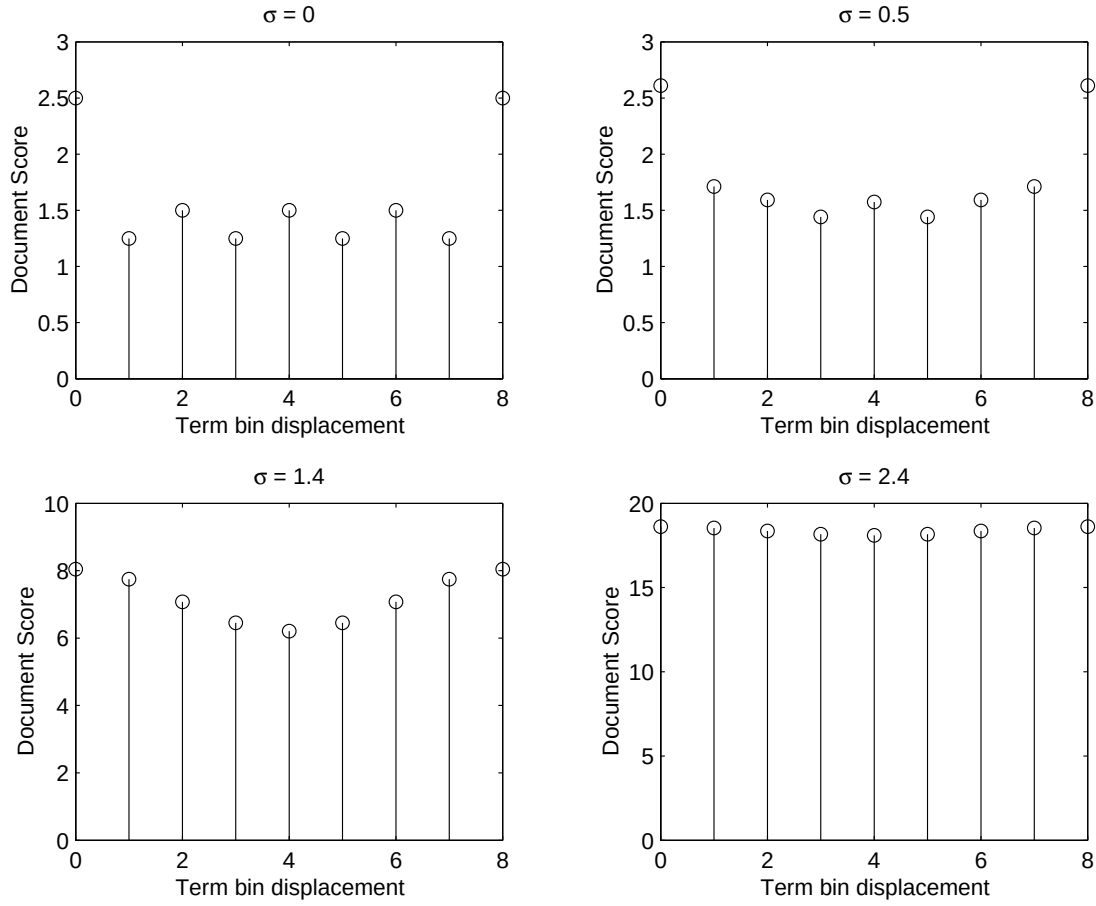


Figure 5.11: The FDS + Gaussian method scores derived from a document containing two query terms separated by term bin displacement. Different plots are given for different values of σ to show how the proximity is affected.

The second spread function used was:

$$\sigma = S \frac{w_{d,t}}{W_d} \quad (5.34)$$

which is based on the weighted term count in the document. If the term is rare, this factor should be large. If the term is common it will be small. The values of S chosen for the experiment were 0.1, 0.5, 1, 5, 10 and 20. We expect the best value of S to be close to 1 due to our selection of the spread function.

The methods performed were run with and without padding. Each method uses 8 bins, therefore the padding extended this to 16 bins. Using padding implies more calculations are to be done due to more bins, but as explained in section 5.4.2, the results should be more precise by removing wrap around.

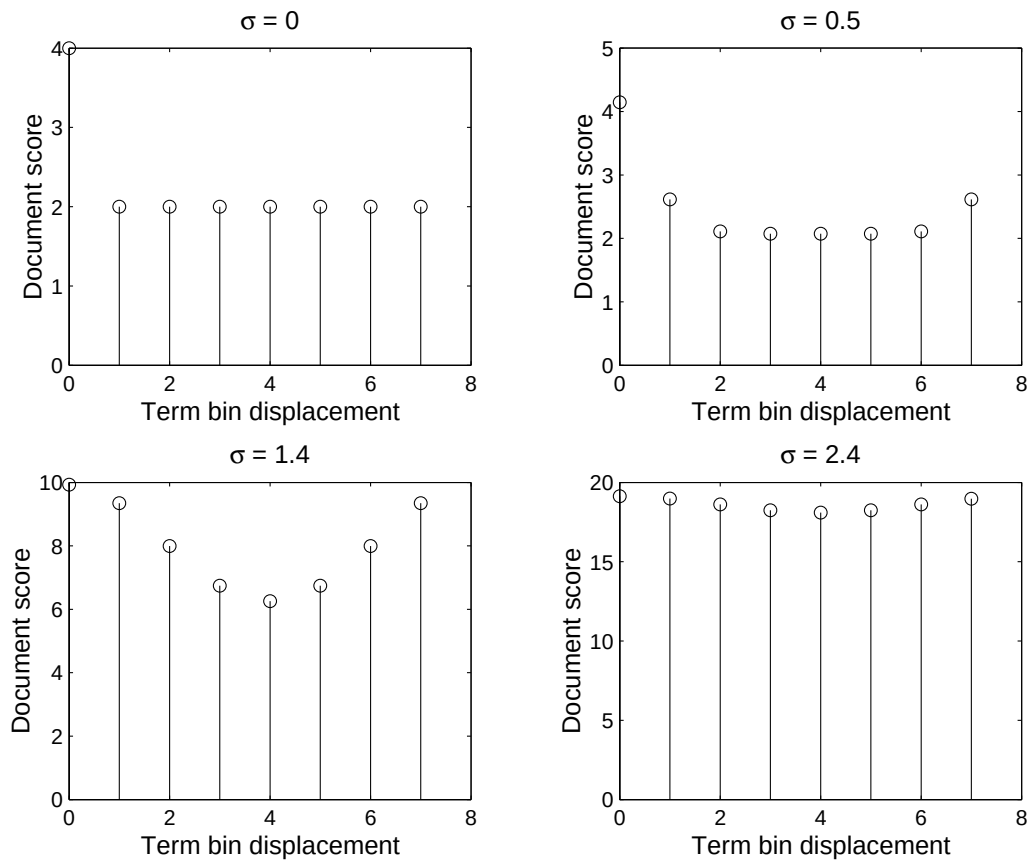


Figure 5.12: The Gaussian Convolution Method scores derived from a document containing two query terms separated by term bin displacement. Different plots are given for different values of σ to show how the proximity is affected.

5.7.1 Application to Fourier Spectral Domain

The first set of experiments were performed using the fds-9-4-6 method. The Gaussian spreading was applied as the weight to each of the frequency components of each query term in each document. Common terms had little change after the weighting, therefore their position in the document was still important. The weighting for rare terms boosted the lower frequency components and reduced the higher frequency components, to make its location less specific.

The experimental results can be seen in tables 5.2 and 5.3 and figures 5.13 and 5.14. Tables 5.2 and 5.3 show the results of the top twenty methods based on precision after 5, 10, 15 and 20 documents. These tables also contain the precision for the same methods without the Gaussian spreading. These two methods (one without and one with

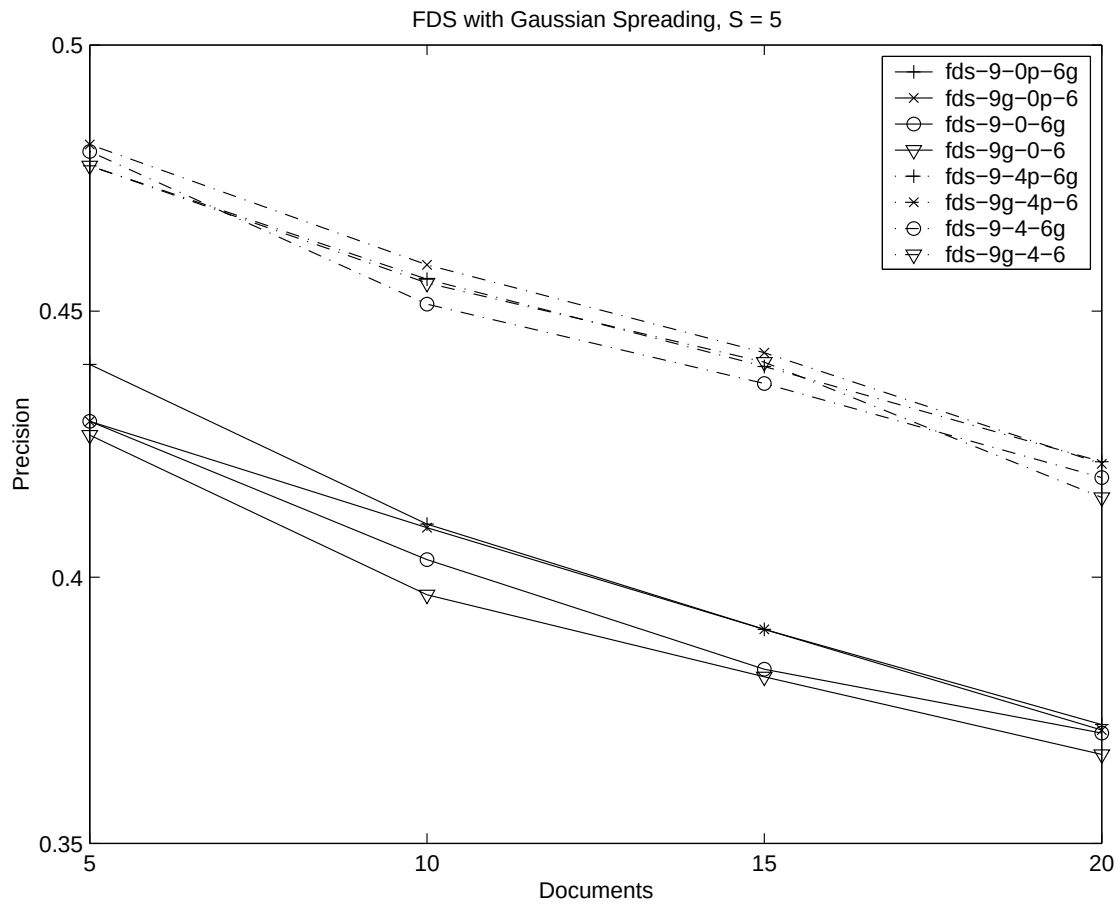


Figure 5.13: Plot of precision after 5,10,15 and 20 documents for $S = 5$. The dashed lines represent phase precision included in the calculation, and cross markers indicate padding was added, the closed markers (circle and triangle) imply no padding. Note how the trials with phase precision included provide much better results for $S = 5$.

padding) are found at the top of each table and are separated from the rest of the results by a horizontal bar. The graphs in figures 5.13 and 5.14 are there to give us a visual way of seeing how changing the parameters of the experiments can change the precision of the method.

We can see from tables 5.2 and 5.3 that the only method that improves upon the two methods without Gaussian spreading is `fds-9g-4p-6.s5.b9` and that is only at precision after 10 documents. We can also see that most of the top methods have the same precision as itself without Gaussian spreading and that they have a lower value of S . We can assume that these methods have a S value that is too small, so the Gaussian spread is becoming a spike and therefore acting as if the Gaussian spreading was not added.

Method	Precision 5	Method	Precision 10
fds-9-4-6.b5	0.4840	fds-9-4-6.b5	0.4560
fds-9-4p-6.b9	0.4827	fds-9-4p-6.b9	0.4580
fds-9g-4-6.s01.b5	0.4840	fds-9g-4p-6.s5.b9	0.4587
fds-9g-4-6.s1.b5	0.4840	fds-9-4p-6g.s1.b9	0.4580
fds-9g-4-6.s05.b5	0.4840	fds-9g-4p-6.s1.b9	0.4580
fds-9-4-6g.s05.b5	0.4840	fds-9-4p-6g.s01.b9	0.4580
fds-9-4-6g.s01.b5	0.4840	fds-9-4p-6g.s05.b9	0.4580
fds-9-4-6g.s1.b5	0.4840	fds-9g-4p-6.s05.b9	0.4580
fds-9g-4p-6.s05.b9	0.4827	fds-9g-4p-6.s01.b9	0.4580
fds-9-4p-6g.s01.b9	0.4827	fds-9g-4-6.s01.b5	0.4560
fds-9g-4p-6.s01.b9	0.4827	fds-9g-4-6.s05.b5	0.4560
fds-9-4p-6g.s1.b9	0.4827	fds-9g-4-6.s1.b5	0.4560
fds-9g-4p-6.s1.b9	0.4827	fds-9-4-6g.s05.b5	0.4560
fds-9-4p-6g.s05.b9	0.4827	fds-9-4-6g.s01.b5	0.4560
fds-9g-4p-6.s5.b9	0.4813	fds-9-4-6g.s1.b5	0.4560
fds-9-4-6g.s5.b5	0.4800	fds-9-4p-6g.s5.b9	0.4560
fds-9g-4-6.s5.b5	0.4773	fds-9g-4-6.s5.b5	0.4553
fds-9-4p-6g.s5.b9	0.4773	fds-9-4-6g.s5.b5	0.4513
fds-9-4p-6g.s10.b9	0.4680	fds-9g-4p-6.s10.b9	0.4453
fds-9-4-6g.s10.b5	0.4680	fds-9-4p-6g.s10.b9	0.4407
fds-9g-4-6.s10.b5	0.4573	fds-9g-4p-6.s20.b9	0.4327
fds-9g-4p-6.s10.b9	0.4547	fds-9g-4-6.s10.b5	0.4313

(a)
(b)

Table 5.2: Results for the method fds-9-4-6 with and without padding and the top 20 results of the same methods involving different forms of Gaussian spreading a) Precision after the first 5 documents. b) Precision after the first 10 documents.

Method	Precision 15	Method	Precision 20
fds-9-4-6.b5	0.4422	fds-9-4-6.b5	0.4193
fds-9-4p-6.b9	0.4431	fds-9-4p-6.b9	0.4233
fds-9-4p-6g.s01.b9	0.4431	fds-9g-4p-6.s1.b9	0.4233
fds-9-4p-6g.s1.b9	0.4431	fds-9-4p-6g.s01.b9	0.4233
fds-9g-4p-6.s05.b9	0.4431	fds-9-4p-6g.s1.b9	0.4233
fds-9g-4p-6.s01.b9	0.4431	fds-9-4p-6g.s05.b9	0.4233
fds-9-4p-6g.s05.b9	0.4431	fds-9g-4p-6.s05.b9	0.4233
fds-9g-4p-6.s1.b9	0.4431	fds-9g-4p-6.s01.b9	0.4233
fds-9g-4-6.s01.b5	0.4422	fds-9-4p-6g.s5.b9	0.4217
fds-9g-4p-6.s5.b9	0.4422	fds-9g-4p-6.s5.b9	0.4213
fds-9g-4-6.s05.b5	0.4422	fds-9g-4-6.s01.b5	0.4193
fds-9g-4-6.s1.b5	0.4422	fds-9g-4-6.s05.b5	0.4193
fds-9-4-6g.s01.b5	0.4422	fds-9g-4-6.s1.b5	0.4193
fds-9-4-6g.s05.b5	0.4422	fds-9-4-6g.s05.b5	0.4193
fds-9-4-6g.s1.b5	0.4422	fds-9-4-6g.s01.b5	0.4193
fds-9g-4-6.s5.b5	0.4404	fds-9-4-6g.s1.b5	0.4193
fds-9-4p-6g.s5.b9	0.4396	fds-9-4-6g.s5.b5	0.4187
fds-9-4-6g.s5.b5	0.4364	fds-9g-4-6.s5.b5	0.4150
fds-9-4p-6g.s10.b9	0.4267	fds-9-4p-6g.s10.b9	0.4100
fds-9-4-6g.s10.b5	0.4262	fds-9g-4p-6.s10.b9	0.4007
fds-9g-4p-6.s10.b9	0.4178	fds-9-4-6g.s10.b5	0.4003
fds-9g-4-6.s10.b5	0.4124	fds-9g-4-6.s10.b5	0.3937

(a)
(b)

Table 5.3: Results for the methods fds-9-4-6 with and without padding and the top 20 results of the same methods involving different forms of Gaussian spreading a) Precision after the first 15 documents. b) Precision after the first 20 documents.

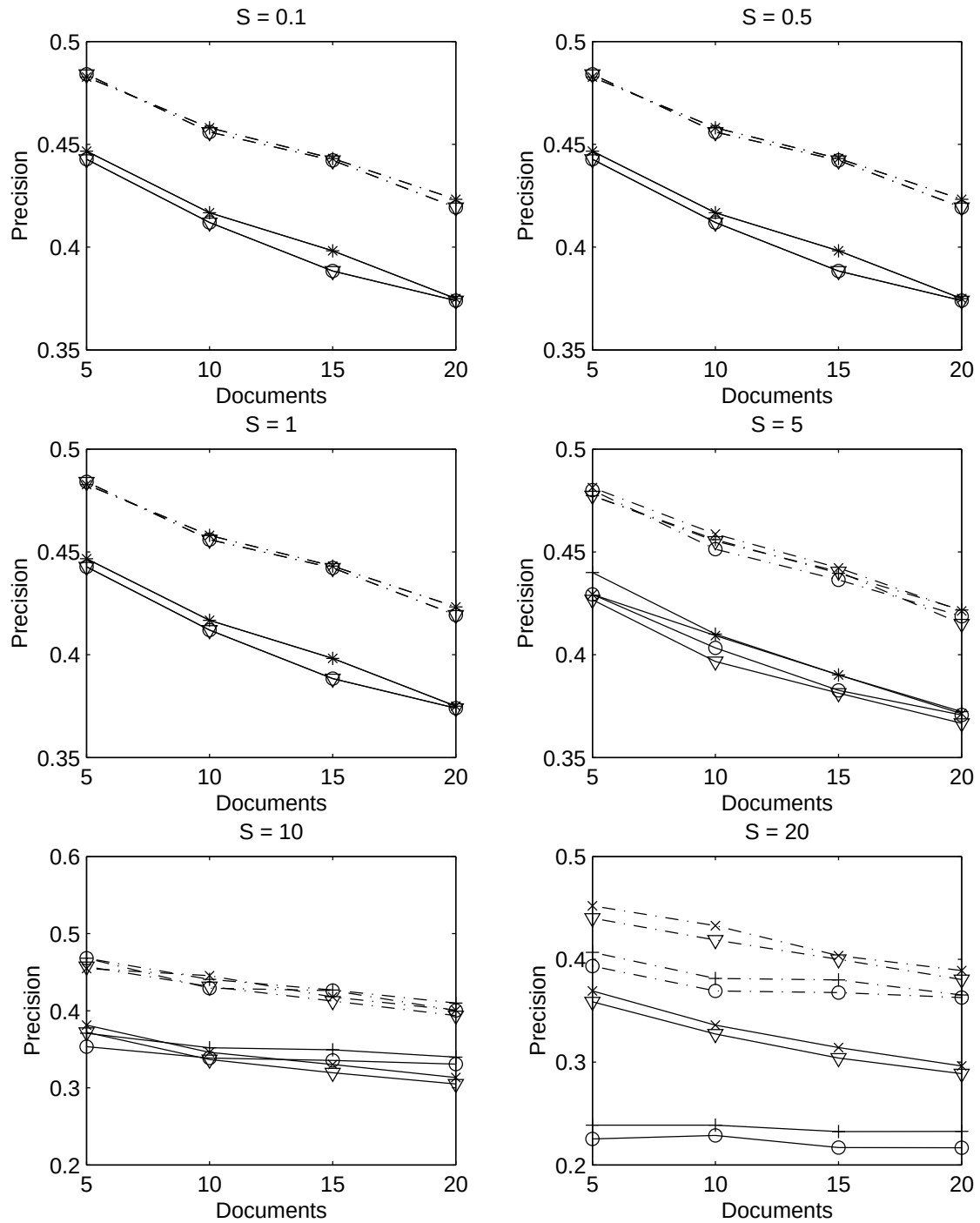


Figure 5.14: Plot of precision after 5,10,15 and 20 documents for $S = 0.1, 0.5, 1, 5, 10$ and 20. The legend is the same as in figure 5.13. Note that the clustering of trials with phase precision and without still exists.

If we take a close look at the plot for $S = 20$ in figure 5.14, we can see that the points marked with crosses are shifted above the points marked with triangles. Also the points marked with plus signs are shifted above the points marked with circles. If we examine the difference between these shifted pairs, we will see that the trial which is more precise has added padding. We can also see this in many of the other plots in figure 5.14 and figure 5.13. If we observe the results in tables 5.2 and 5.3, we can see that the trials containing padding appear at top of the results for precision at 10, 15 and 20 documents, but the methods without padding appear at the top of the results for precision at 5 documents.

The results overall were poorer than expected. The idea of applying Gaussian spreading to improve the precision of an FDS method seems valid, but the results say otherwise. There was one result where the Gaussian spreading improved the precision of the FDS method, so maybe with more fine tuning or a better spreading function we would be able to obtain the more precise results that we wanted.

5.7.2 Gaussian Convolution

Rather than apply the Gaussian function after the Fourier transform, we can leave the Fourier transform out all together by performing a Gaussian convolution to the query term signals. By doing this we lose the use of the phase precision calculation, but we are left with a simpler algorithm (shown in algorithm 5.2) which uses equation 5.20 to perform the spreading.

The Gaussian ranking method will provide a higher score to documents which have a greater number of clustered query terms. For example, if the query term 'wales' appeared many times in a region of a document, the Gaussian method would give this document a high score even though it contained only one query term. This problem is removed when the Gaussian method is used in conjunction with the document and term weighting schemes. This is due to the scaling down in weight of multiple occurrences of the same term. Therefore a greater number of unique terms would provide a higher document score.

We can see in tables 5.4 and 5.5 that the Gaussian convolution retrieval method provides precision results similar to those found in the previous FDS with Gaussian spreading experiments (tables 5.2 and 5.3). That is except for the increase in precision found after 10 documents, which surpassed the FDS method by a fair margin. The main difference between this method (gauss-9-4-6) and the fds-9-0-6g is the way the term signals are combined. The former method adds the query term signals before the magnitude is taken while the latter takes the magnitude and then adds the signals. We know from the

Method	Precision 5	Method	Precision 10
fds-9-4-6.b5	0.4840	fds-9-4-6.b5	0.4560
fds-9-4p-6.b9	0.4827	fds-9-4p-6.b9	0.4580
gauss-9-4-6.s3.b8	0.4813	gauss-9-4p-6.s2.b8	0.4707
gauss-9-4-6.s2.b8	0.4813	gauss-9-4-6.s2.b8	0.4700
gauss-9-4-6.s1.b8	0.4813	gauss-9-4-6.s3.b8	0.4700
gauss-9-4-6.s05.b8	0.4813	gauss-9-4-6.s1.b8	0.4700
gauss-9-4-6.s01.b8	0.4813	gauss-9-4-6.s05.b8	0.4700
gauss-9-4p-6.s2.b8	0.4760	gauss-9-4-6.s01.b8	0.4700
gauss-9-4-6.s4.b8	0.4760	gauss-9-4-6.s4.b8	0.4693
gauss-9-4-6.s5.b8	0.4720	gauss-9-4p-6.s3.b8	0.4673
gauss-9-4p-6.s3.b8	0.4667	gauss-9-4-6.s5.b8	0.4667
gauss-9-4-6.s6.b8	0.4667	gauss-9-4-6.s6.b8	0.4660
gauss-9-4p-6.s4.b8	0.4640	gauss-9-4p-6.s4.b8	0.4553
gauss-9-4p-6.s5.b8	0.4587	gauss-9-4p-6.s5.b8	0.4440
gauss-9-4-6.s10.b8	0.4480	gauss-9-4-6.s10.b8	0.4353
gauss-9-4p-6.s6.b8	0.4427	gauss-9-4p-6.s6.b8	0.4280

(a)
(b)

Table 5.4: Results for the methods fds-9-4-6 with and without padding and the results of gauss-9-4-6. a) Precision after the first 5 documents. b) Precision after the first 10 documents.

triangle inequality that these are only equal if the signal is real and positive:

$$\left| \sum_{k=1}^n a_k \right| \leq \sum_{k=1}^n |a_k| \quad (5.35)$$

5.8 Discussion and Summary

We have introduced Gaussian spreading. This is a technique that can be applied to the FDS method (to obtain extra information about the locality of the query terms), or it can simply be applied as a convolution. We have addressed such issues as how we can deal with the application of the Gaussian function in the frequency and spatial domains using the convolution theorem, conserving the energy of the term signals after we convert them into the frequency domain using the Rayleigh Energy Theorem, choosing the right parameters for the Gaussian function, dealing with the effects of aliasing by approxim-

Method	Precision 15	Method	Precision 20
fds-9-4-6.b5	0.4422	fds-9-4-6.b5	0.4193
fds-9-4p-6.b9	0.4431	fds-9-4p-6.b9	0.4233
gauss-9-4-6.s3.b8	0.4440	gauss-9-4-6.s2.b8	0.4177
gauss-9-4-6.s2.b8	0.4436	gauss-9-4-6.s1.b8	0.4177
gauss-9-4-6.s1.b8	0.4436	gauss-9-4-6.s05.b8	0.4177
gauss-9-4-6.s05.b8	0.4436	gauss-9-4-6.s01.b8	0.4177
gauss-9-4-6.s01.b8	0.4436	gauss-9-4-6.s3.b8	0.4173
gauss-9-4p-6.s2.b8	0.4413	gauss-9-4-6.s4.b8	0.4170
gauss-9-4p-6.s3.b8	0.4413	gauss-9-4p-6.s2.b8	0.4167
gauss-9-4-6.s4.b8	0.4413	gauss-9-4p-6.s3.b8	0.4160
gauss-9-4-6.s6.b8	0.4409	gauss-9-4-6.s5.b8	0.4147
gauss-9-4-6.s5.b8	0.4404	gauss-9-4-6.s6.b8	0.4147
gauss-9-4p-6.s4.b8	0.4316	gauss-9-4p-6.s4.b8	0.4087
gauss-9-4p-6.s5.b8	0.4280	gauss-9-4p-6.s5.b8	0.4010
gauss-9-4-6.s10.b8	0.4262	gauss-9-4-6.s10.b8	0.4007
gauss-9-4p-6.s6.b8	0.4111	gauss-9-4p-6.s6.b8	0.3953

(a)
(b)

Table 5.5: Results for the methods fds-9-4-6 with and without padding and the results of gauss-9-4-6 a) Precision after the first 15 documents. b) Precision after the first 20 documents.

ing the wrap around in the frequency domain and dealing with wrap around effects in the spatial domain by adding padding. We also observed the effects of the score of a document by changing the proximity of terms and changing the magnitude of the terms.

Experiments involving the application of Gaussian spreading to the FDS method, had a degrading effect on the precision, except for one case where the Gaussian spreading improved the precision after 10 documents were retrieved. It was also found that adding padding to the FDS with Gaussian spreading method increased the results in certain FDS methods. The application of Gaussian convolution to the query term signal brought on an impressive increase in precision at the 10 documents mark, without utilising any phase information. The use of padding with the Gaussian convolution method seemed to hinder the precision results.

CHAPTER SIX

MULTIRESOLUTION DOCUMENT RETRIEVAL

6.1 Introduction

ONE of the amazing features found in our environment is the self similarity of natural objects at different resolution levels. The question of ‘how long is the coast of Britain?’ was raised by Mandelbrot [74] to help us appreciate the way which we are able to see detail at different levels of resolution. He states that the coast could be measured by using a stick of unit length and finding how many sticks would fit around the perimeter of the island. But by using a smaller stick, we would obtain a longer answer. As our stick approaches zero, the answer approaches infinity. At a low resolution, we are able to see the main bays and peninsulas, but at finer resolutions we are able to see the bays and peninsulas within the main ones. This aspect of self similarity is also found in snow flakes, trees, mountains, clouds, waves, and many more parts of nature. All of these objects have slowly evolved over time to reach their current state, just like written language.

The idea of self similarity can be used in the field of information retrieval. If we want to examine a document and label it with the topic it suggests, it would be like using the stick to find the length of the coast of Britain. We could also look deeper into the document to find sub-topics, and even deeper to find sub-sub-topics. The current vector space methods [18, 96, 125, 4, 126] use ‘big sticks’ to calculate document scores, while the proximity methods [24, 58, 49, 66] use ‘small sticks’ to calculate the score. If we were to take many resolutions into account, we would obtain the best of both worlds. To do so we have to perform multi-resolution analysis and the best tool for doing so is the wavelet transform.

The wavelet transform has had a long journey to get to its current standing and has been used in many areas of engineering and science [29] because of its ability to decompose natural signals into scaled and shifted versions of itself.

Earlier we described methods of using the Fourier transform (chapter 3) or the cosine transform (chapter 4) to analyse the documents frequency spectrum. The experiments have shown that these methods achieve high precision, but require substantial computing time to perform the transformations.

Our focus in this chapter is to introduce a new method of text document ranking using the discrete wavelet transform so that we can analyse the documents at various resolutions. We will also compare this new method with existing methods of document ranking, including Fourier Domain Scoring. This chapter is organised as follows: section 6.2 introduces the wavelet transform and its self similarity properties, section 6.3 discusses the desired properties for using wavelets in information retrieval. Section 6.4 explains the method in which we will use the wavelets to extract term position and magnitude information. Section 6.5 discusses time complexity and storage issues. Section 6.6 looks at the basic requirements that a document ranking method must comply with, and Section 6.7 gives details of the experiments performed with results displayed in different forms. Section 6.8 discusses alternative methods which would increase the speed of the query process.

6.2 Multiresolution Analysis

The Fourier transform (FT) and it's relatives are excellent tools for analysing information as a whole. When given a signal, the Fourier transform of the signal will provide information about every frequency component that exists in the signal. This type of analysis is sufficient for stationary signals, but what if we want to analyse transient signals? For example, we might want to find the point in time where a signal contains certain frequency components, or for a spatial signal, we might want to find a position in space where certain frequency components exist.

Our previous work in the field of text retrieval gave us insight as to how we can use the Fourier and Cosine transforms to easily combine the term magnitude and phase information into the document score and obtain high precision results. But, as stated above, the Fourier transform provides frequency information for the whole document, hence we were not able to focus on important portions of the document. In this section, we will explain the limitations of the Fourier transform by observing the Heisenberg uncertainty principle, and show how we can utilise both the frequency and positional

information using the wavelet transform.

6.2.1 Time-Frequency Analysis

To be able to select a certain time interval and analyse the frequency components within that interval, we must first observe the Heisenberg Uncertainty principle.

Theorem 6.2.1 (Heisenberg Uncertainty). *The temporal variance (σ_t) and the frequency variance (σ_w) of $f \in \mathbf{L}^2(\mathbb{R})$ satisfy*

$$\sigma_t^2 \sigma_w^2 \geq \frac{1}{4} \quad (6.1)$$

This theorem shows that it is impossible to select a specific position of time in the signal and calculate the frequency components which exist. The same goes for selecting a specific frequency and finding the points in time where it exists. There must be a range (or variance) in time for any frequency values to exist. How specific these components will be depends on the variance in time we take. For example, the Fourier transform operates on the whole signal. In this case, we can identify certain frequencies and we know they exist in the whole signal, but we do not know if certain frequencies exist in specific time intervals. On the other hand, if we choose only part of the signal, we must expect to receive frequency values with a certain variance.

6.2.2 Short Time Fourier Transform

The initial solution to solve the problem of analysing transient signals was to use the Short Time Fourier Transform (STFT also known as the Windowed Fourier Transform). The STFT was similar to the FT in the way that it converted the signal to the frequency domain, but it windowed the time signal before doing so. By selecting many different intervals of the signal, the STFT allowed the analyst to observe frequency components local to a certain time or position in the signal. The STFT comes in the form:

$$Sf(u, \xi) = \int_{-\infty}^{+\infty} f(t)g(t-u) \exp(-i\xi t) dt \quad (6.2)$$

where $g(t-u)$ is the window function. According to the Heisenberg Uncertainty, this set window size (or variance in time) implies that the frequency window size is also a set size. This means that the time and frequency resolution will be the same for all frequencies once the transform has been performed.

A cyclic transient signal will contain frequency components which exist for all times. In this case we do not care about the time resolution, but we would like high frequency resolution. Therefore a STFT with a large window size or the Fourier transform would

suffice in obtaining the frequency information. If we wanted to find where certain impulses existed in a transient signal, the Fourier transform would fail. All it could tell would be that the impulses are in the signal, but it could not specify at which times they occur. In this case we would use a STFT with small window size. The small window size implies high time resolution but low frequency resolution (due to the small variance in time selected). By using the STFT we could tell where in time the impulses occurred, but we would not be able to tell which frequency values were involved.

6.2.3 Wavelet Transform

We have seen that the STFT can be used to find time and frequency information at a set resolution and we have also seen that the resolution must be set in order to extract certain time-frequency information from a signal. If we want to examine a signal across different time-frequency resolution scales we must look into multi-resolution analysis (MRA). MRA is intended to allow for high time resolution with poor frequency resolution at high frequency levels and poor time resolution with high frequency resolution at low frequency levels. To perform this task a wavelet is used instead of a windowing function. The name wavelet means a small wave or a wave which is not of infinite length (as in a sinusoidal wave).

Before we examine the mathematical properties of the wavelet transform, we will provide a simple example of how we can use the transform to provide us with the different levels of resolution of a signal. The Haar wavelet is equivalent to one cycle of a square wave. To perform the transform, we take every possible scaled and shifted version of the wavelet and find how much of this wavelet is within our signal (by finding the dot product). For example, if our signal is $\tilde{f}_{d,t} = [2 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0]$ and we use the Haar wavelet transform:

$$W = \begin{bmatrix} \sqrt{1/8} & \sqrt{1/8} & \sqrt{1/8} & \sqrt{1/8} & \sqrt{1/8} & \sqrt{1/8} & \sqrt{1/8} & \sqrt{1/8} \\ \sqrt{1/8} & \sqrt{1/8} & \sqrt{1/8} & \sqrt{1/8} & -\sqrt{1/8} & -\sqrt{1/8} & -\sqrt{1/8} & -\sqrt{1/8} \\ \sqrt{1/4} & \sqrt{1/4} & -\sqrt{1/4} & -\sqrt{1/4} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \sqrt{1/4} & \sqrt{1/4} & -\sqrt{1/4} & -\sqrt{1/4} \\ \sqrt{1/2} & -\sqrt{1/2} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \sqrt{1/2} & -\sqrt{1/2} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \sqrt{1/2} & -\sqrt{1/2} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \sqrt{1/2} & -\sqrt{1/2} \end{bmatrix}$$

the wavelet components will be:

$$W\tilde{f}'_{d,t} = \begin{bmatrix} 5/\sqrt{8} & 1/\sqrt{8} & 1/\sqrt{4} & 2/\sqrt{4} & 2/\sqrt{2} & -1/\sqrt{2} & 0 & 0 \end{bmatrix}' \quad (6.3)$$

Resolution	Whole	Halves	Quarters		Eighths			
Sample Signal	$5\sqrt{8}$	$1\sqrt{8}$	$1\sqrt{4}$	$2\sqrt{4}$	$2\sqrt{2}$	$-1\sqrt{2}$	0	0

Table 6.1: The many resolutions of decomposition after the wavelet transform of $\tilde{f}_{d,t}$ is applied in equation 6.3.

where x' is x transposed. This transformed signal shows us the positions of the terms at many resolutions. The first component (5) shows that there are 5 occurrences of the term. The second component (1) shows that there is one more occurrence of the term at the first half of the signal than at the end. The third component shows that there is one more occurrence of the term in the first quarter compared to the second quarter. The fourth component compares the third and fourth quarters. The next four components compare the eighths of the signal. Therefore we can observe the signal at different levels of resolution by splitting the transformed signal as shown in table 6.1. Therefore, if each component of the original signal represented a portion of a document, we could use the wavelet transform to analyse the query term positions at multiple document resolutions. If we chose only the first wavelet component, we would treat the document without spatial information (as in the vector space methods). The matrix multiplication causes this transformation to be of order $O(N^2)$. To speed up this process, we must use the wavelets scaling function as well as the wavelet function. Applying the scaling function to our signal allows us to extract all of the information orthogonal to the wavelet of the current resolution and also adjusts the signal to the next level of resolution. Therefore we can obtain the wavelet transform by following a simple recursive process:

- While $\tilde{x} \neq \emptyset$:
 1. Apply wavelet function to signal $\tilde{x} \Rightarrow$ keep values
 2. Apply scaling function to signal $\tilde{x} \Rightarrow$ result becomes new $\tilde{x}$

We will show an example of this using the same $\tilde{f}_{d,t}$ as before and the Haar wavelet which has wavelet coefficients of $[1/\sqrt{2} \ -1/\sqrt{2}]$ and its scaling function $[1/\sqrt{2} \ 1/\sqrt{2}]$. The first iteration of the scaling function application is a convolution between the scaling function and the signal, this can also be thought of the dot product of the many shifted versions of the scaling function at the first resolution with the signal. Therefore we have:

$$f_{d,t} \cdot \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}' = 2/\sqrt{2}$$

$$f_{d,t} \cdot \begin{bmatrix} 0 & 0 & 1/\sqrt{2} & 1/\sqrt{2} & 0 & 0 & 0 & 0 \end{bmatrix}' = 1/\sqrt{2}$$

$$f_{d,t} \cdot \begin{bmatrix} 0 & 0 & 0 & 0 & 1/\sqrt{2} & 1/\sqrt{2} & 0 & 0 \end{bmatrix}' = 2/\sqrt{2}$$

$$f_{d,t} \cdot \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix}' = 0/\sqrt{2}$$

Therefore, the convolution of the Haar scaling function and the signal $\tilde{f}_{d,t}$ produces $\begin{bmatrix} 2/\sqrt{2} & 1/\sqrt{2} & 2/\sqrt{2} & 0 \end{bmatrix}$. By performing the same operation with the wavelet function, we receive $\tilde{f}_{d,t}$ produces $\begin{bmatrix} 2/\sqrt{2} & -1/\sqrt{2} & 0 & 0 \end{bmatrix}$. These results are concatenated to produce the first iteration of the wavelet transform (shown in equation 6.4). The scaling function result ($\tilde{f}_{d,t}$ produces $\begin{bmatrix} 2/\sqrt{2} & 1/\sqrt{2} & 2/\sqrt{2} & 0 \end{bmatrix}$) is passed onto the second iteration and the wavelet result is kept as part of the answer.

The complete process is:

$$\begin{bmatrix} 2 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} \Rightarrow \begin{bmatrix} 2/\sqrt{2} \\ 1/\sqrt{2} \\ 2/\sqrt{2} \\ 0 \\ 2/\sqrt{2} \\ -1/\sqrt{2} \\ 0 \\ 0 \end{bmatrix} \Rightarrow \begin{bmatrix} 3/\sqrt{4} \\ 2/\sqrt{4} \\ 1/\sqrt{4} \\ 2/\sqrt{4} \\ 2/\sqrt{2} \\ -1/\sqrt{2} \\ 0 \\ 0 \end{bmatrix} \Rightarrow \begin{bmatrix} 5/\sqrt{8} \\ 1/\sqrt{8} \\ 1/\sqrt{4} \\ 2/\sqrt{4} \\ 2/\sqrt{2} \\ -1/\sqrt{2} \\ 0 \\ 0 \end{bmatrix} \quad (6.4)$$

In this example the values produced by the shifted wavelet function appear below the bar and are kept. The values produced by the scaling function appear above the bar and are fed back in to the recursive splitting process. We can see that this process is a divide and conquer method of the wavelet transform. If we examine this process, we will see that we can collapse the recursive process in to a single pass over the data to produce the transformed data and hence reduce the wavelet transform to order $O(N)$. By using the wavelet and scaling functions together, we do not need to scale the wavelet function (only shifts are performed), the data is inversely scaled by the scaling function to allow for this. For a more rigorous proof we refer to Mallat [73]. Now that we have studied this example, we can move onto a more mathematical analysis of the transform.

A wavelet is described by a function $\psi \in \mathbf{L}^2(\mathbb{R})$ with a zero average and norm of one. A wavelet can be scaled and translated by adjusting the parameters s and u respectively.

$$\psi_{u,s}(t) = \frac{1}{\sqrt{s}} \psi\left(\frac{t-u}{s}\right) \quad (6.5)$$

The scaling factor keeps the norm equal to one for all s and u . The wavelet transform of $f \in \mathbf{L}^2(\mathbb{R})$ at time u and scale s is:

$$Wf(u,s) = \langle f, \psi_{u,s} \rangle = \int_{-\infty}^{+\infty} f(t) \frac{1}{\sqrt{s}} \psi^*\left(\frac{t-u}{s}\right) dt \quad (6.6)$$

where ψ^* is the complex conjugate of ψ .

The appeal of the wavelet transform is its ability to focus on regions of the signal. If we were to compare two or more signals for similarity, we could examine the signals top down. When the signals differ at a certain level, we know that we do not have to delve any deeper. This is an exciting property which can be used in information retrieval. If we consider a word signal, the wavelet transform of this signal will give us the location of the words at any desired resolution. For example, the first wavelet component will tell us if the word exists in the document. The second will tell us where the main cluster of the word is in the document. The third and fourth will identify the general area which the word appears in the first and second half of the document respectively and so on until we get to the exact location of the word.

To construct a wavelet function, $\psi_{u,s}(t)$, we must first obtain a scaling function, $\phi_{u,s}(t) \in V_n$. One of the properties that the scaling function must satisfy is that:

$$\dots \subset V_{n+1} \subset V_n \subset V_{n-1} \dots$$

where the set of $\phi_{u,s}(t)$ for all u is a basis of V_n ($s = 2^n$ for dyadic scaling), and $\bigcup_{n \in \mathbb{Z}} V_n = \mathbf{L}^2(\mathbb{R})$. This implies that we can show each set of scaling functions in terms of its subset of scaled scaling functions:

$$V_{n-1} = V_n \cup W_{n-1}, \quad V_n \perp W_{n-1} \quad (6.7)$$

where $\perp$ implies orthogonality. An example of the relationship between V_n and W_n can be seen in figure 6.1. If we observe the set of functions W_n , we can see that it satisfies the following properties:

$$\bigcup_{n \in \mathbb{Z}} W_n = \mathbf{L}^2(\mathbb{R}) \quad \bigcap_{n \in \mathbb{Z}} W_n = \emptyset \quad (6.8)$$

Therefore the set of functions W_n for all n is a basis for $\mathbf{L}^2(\mathbb{R})$. This set W_n is the set of shifted wavelet functions at resolution n .

6.2.4 Discrete Wavelet Transform

We have seen that the wavelet transform divides $\mathbf{L}^2(\mathbb{R})$ into the sets $\dots \subset V_{n+1} \subset V_n \subset V_{n-1} \dots$, where $W_n = V_n \cap \overline{V_{n+1}}$ or $V_n = W_n \cup V_{n+1}$, $W_n \cap V_{n+1} = \emptyset$. This is a recursive filtering process, where each resolution of scaling functions ($\phi_{u,s}(t) \in V_n$) is split into the next resolution of wavelet functions ($\psi_{u,s}(t) \in W_n$) and the next resolution of scaling functions ($\phi_{u,2s}(t) \in V_{n+1}$).

When given in its discrete form, the dyadic wavelet transform can be shown as a sequence of high pass and low pass filters, where the filter coefficients of the high pass

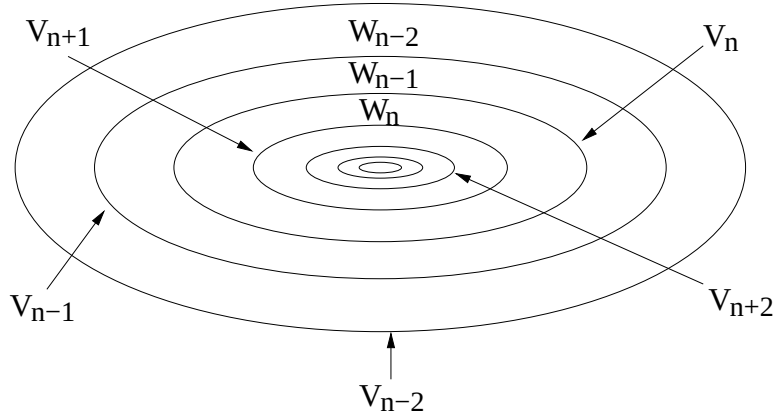


Figure 6.1: An example of the enclosing scaling function ($\phi_{u,s}(t) \in V_n$) spaces as ovals and the wavelet function spaces ($\psi_{u,s}(t) \in W_n$) as annuli.

filter describe the wavelet transform and the low pass filter coefficients describe the scaling function which is taking place (shown in figure 6.2). We can see that the output of the high pass filter (the wavelet components) are part of the resulting transform coefficients and the output of the low pass filter is fed back into another high and low pass filter to be split again (the coefficients of the scaling function decomposition). Until the mid 1980's there was no such filter that could provide perfect reconstruction of the decomposed signal, therefore there was no way this idea could be applied in practice. The Conjugate Mirror Filter (CMF) [113, 114] provided a means to build these wavelet filter banks and provided the spark to the wavelet community. By performing this recursive filtering process, we are able to complete the transform in linear time, which is faster than the Fourier transform.

6.3 Choosing a wavelet

Before performing the wavelet transform on the data we have gathered, we must first choose a wavelet from the many varieties that exist. We often hear of the common wavelets with names such as Daubechies, Shannon, Battle-Lemariè, Meyer, Symmlets, Spline, Biorthogonal and be puzzled as to which wavelet is best for the task at hand. Before we can choose one, it is necessary to understand the properties that define each wavelet.

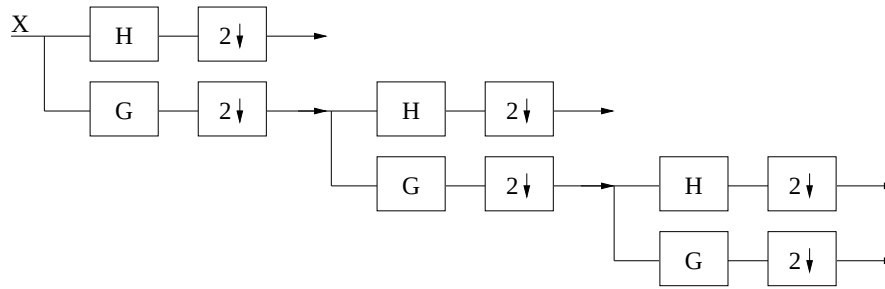


Figure 6.2: Recursive filtering process. H is a high pass filter (using the wavelet coefficients) and G is a low pass filter (using the scaling function coefficients). The low pass data gets split and decimated repeatedly to obtain the wavelet transform. An example of this is shown in equation 6.4.

6.3.1 Wavelet Properties

The two main factors which will influence our choice of wavelet are the number of vanishing moments and the size of support. Both of these must be considered to extract the most information in the least space from the data set which we are using.

Vanishing moments

When storing or transmitting a signal, it is best if we can do so by using the least amount of storage or the least amount of bandwidth. In the document retrieval domain, we want to store our data in the smallest space possible (which is also simple to access and retrieve). When compressing a signal, whether it be for storage or transmission, we want to fit as much information into the smallest possible space. Mapping data into a transformed space is a simple method of compression because the transform algorithm can be used at the compression and decompression sides without any statistical knowledge of the data. The perfect compression transform would involve a component which is proportional to the function to be compressed. If this were to be used, we would only need to send one component (the rest would be zero if they were orthogonal to each other). Unfortunately, we cannot do this because it would require us to have a component consisting of every type of signal that we would want to compress at the compressor and the decompressor sides. Therefore, we want a set of orthogonal basis functions which would be best for most of the signals we want to compress. The k th moment of a function $f(t)$ is defined as

$$\int_{-\infty}^{\infty} t^k f(t) dt \quad (6.9)$$

Therefore, for the k th moment to vanish, equation 6.9 must equal zero. A wavelet is said to have n vanishing moments if equation 6.9 holds for $0 \leq k < n$. We will show that wavelets with a higher number of vanishing moments are able to represent smooth functions in a more compact manner.

The number of vanishing moments of a wavelet is related to the differentiability of the wavelet. If we consider the signal f which is m times differentiable in the region $[v - h, v + h]$, we can find the Taylor polynomial approximation to this at v :

$$p_v(t) = \sum_{k=0}^{m-1} \frac{f^{(k)}(v)}{k!} (t - v)^k \quad (6.10)$$

which has the error $\epsilon_v(t) = f(t) - p_v(t)$ where:

$$|\epsilon_v(t)| \leq \frac{|t - v|^m}{m!} \sup_{u \in [v-h, v+h]} |f^{(m)}(u)| \quad \forall t \in [v - h, v + h] \quad (6.11)$$

The wavelet transform of f is

$$Wf(u, s) = \int_{-\infty}^{\infty} f(t) \frac{1}{\sqrt{s}} \psi\left(\frac{t - u}{s}\right) dt \quad (6.12)$$

$$= \frac{1}{\sqrt{s}} \int_{-\infty}^{\infty} f(t) \psi(t') dt \quad (6.13)$$

$$= \frac{1}{\sqrt{s}} \int_{-\infty}^{\infty} p_v(t) \psi(t') dt + \frac{1}{\sqrt{s}} \int_{-\infty}^{\infty} \epsilon_v(t) \psi(t') dt \quad (6.14)$$

where $\psi(t)$ has n vanishing moments. If the polynomial p_v has degree of at most $n - 1$ we can see that the first term vanishes and we are left with the wavelet transform of the error term.

$$Wf(u, s) = W\epsilon_v(u, s) \quad (6.15)$$

Therefore, the more vanishing moments a wavelet has, the smaller the error term will be. This implies that if we use a wavelet with more vanishing moments we will produce transformed data components in which only a few will be significant, requiring little storage space.

Size of Support

Another factor that effects the focusing of the wavelet transform is the support size of the wavelet. The support of a function f is the domain in which the function is non-zero [119]. A function f has compact support if its support is bounded. For example the square function (shown in figure 6.3):

$$f_{sq}(t) = \begin{cases} 1 & 0 \leq t \leq 1 \\ 0 & \text{otherwise} \end{cases} \quad (6.16)$$

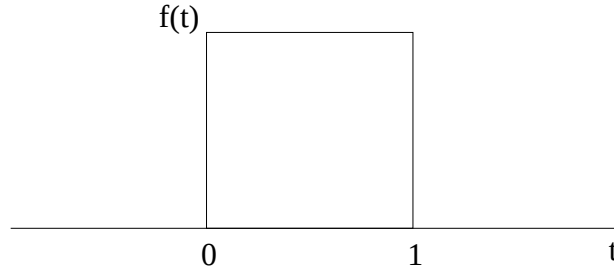


Figure 6.3: Functions exhibit compact support if they have no infinite interval of non-zero values.

has support $t \in [0, 1]$ which is compact, therefore f_{sq} has compact support. If we observe the function $f_{exp}(t) = e^t$, we can see that it has support $t \in \mathbb{R}$ which is not compact. Therefore f_{exp} does not have compact support.

When choosing a wavelet to use for a specific data set, it is essential that we examine the wavelets size of support. If we examine the wavelet:

$$\psi_{j,n}(t) = 2^{-j/2} \psi(2^{-j}t - n) \quad (6.17)$$

we can see that if there exists a large singularity at point t_0 in function f , then $\langle f, \psi_{j,n} \rangle$ may have a large magnitude. If there are K wavelets whose support includes the point t_0 for each level of scale 2^j , then the wavelet function has support of size K . Therefore, the greater the size of support of the wavelet, the more wavelet components will include the singularity t_0 , and therefore the more likely many high magnitude wavelet coefficients will exist. If we reduce the size of the support of the wavelet, we will have less high magnitude components, and therefore less significant components to consider (for storage or transmission) when performing later calculations.

Choosing a small size of support is essential in our application of the wavelet transform. If we examine our term signals, we will see that they consist of a few singularities (most will contain one). Therefore, the larger the support of the wavelet, the more non-zero components will exist in the transformed data. Therefore our index size will be more compact if we choose a wavelet with a small size of support.

6.3.2 Selected Wavelets

To analyse the positions of the terms and their relationship with other terms in the document, we must be able to analyse their relationship document-wise (as the low resolution set of wavelets should do by including every bin position in their calculations) and we must also be able to analyse the terms position-wise (as the high resolution set of

wavelets should do, by including only single bin positions). Every wavelet has a lowest form of resolution which includes every element in the set to be transformed, but not every wavelet can focus tightly on the elements wanted. We mentioned before that if we want to find singularities in a signal and obtain transformed data with the least coefficient, we must choose a wavelet which has a small size of support. The wavelets with the smallest support size are the Haar wavelet and the Daubechies-4 wavelet. This can be seen by the number of filter coefficients needed to describe each of them (two for Haar and four for Daubechies-4).

The Haar Wavelet

According to Graps [50], the Haar wavelet was designed by Alfréd Haar and appeared in an appendix of his thesis entitled *Zur Theorie der orthogonalen Funktionensysteme*. The Haar wavelet has compact support of size 1 but is not continuously differentiable. The compact support of the wavelet implies that the transformed signal will require less storage space than one which does not have such a compact support. For example, if we take the Fourier transform of an impulse, we get a significant value (not close to zero) for each of the frequency components (therefore we have to store B values for B bins). If we do the same with the Haar wavelet, we obtain $B/2$ significant values. This example is a very common case for word signals, therefore the size of the index should reduce by more than 50%.

Due to the shape of the Haar wavelet, if we apply the Haar wavelet to a signal, we can see that we are calculating the difference between the left and right components of the signal. If the resulting inner product to a positive real signal is positive we can deduce that the signal is biased to the left. If the inner product is negative, then the signal is biased to the right. This also goes for the positions of the signal when examining the focused components

It is useful to recall that when using the Fourier and Cosine Transforms, we obtained the word signal and mapped it to the corresponding domain using the transform. By obtaining the spectral information we were able to compare signals and find their relative position to each other using the phase of the spectrum, and we were able to identify the frequency of the terms by examining the magnitude of the signal. The Fourier transform was initially chosen because of its ability to map shifts in a signal to a phase change. The Cosine transform was chosen for its partial ability to map shift to phase changes and because it produced less information to store (hence a smaller faster spectral index). If we examine the Haar transform we will notice that it deals with shifts in its own unique way. For example, if we were to examine an impulse at bin one, we would obtain the

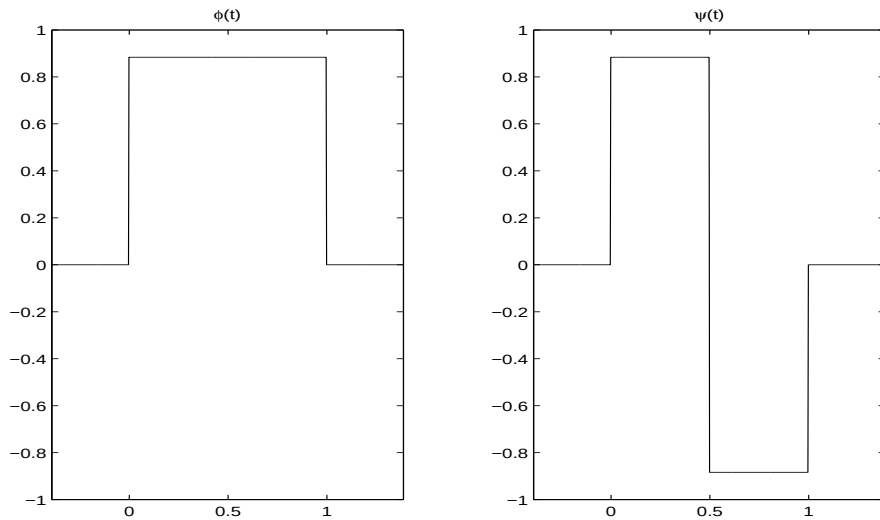
Figure 6.4: The Haar scaling function ($\phi(t)$) and wavelet ($\psi(t)$)

Table 6.2: Haar wavelet decomposition of signal with impulse at term bin position.

Term bin position	Haar coefficients							
1	0.35	0.35	0.50	0	0.71	0	0	0
2	0.35	0.35	0.50	0	-0.71	0	0	0
3	0.35	0.35	-0.50	0	0	0.71	0	0
4	0.35	0.35	-0.50	0	0	-0.71	0	0
5	0.35	-0.35	0	0.50	0	0	0.71	0
6	0.35	-0.35	0	0.50	0	0	-0.71	0
7	0.35	-0.35	0	-0.50	0	0	0	0.71
8	0.35	-0.35	0	-0.50	0	0	0	-0.71

mapping shown in table 6.2. The same signal shifted one position can be seen in table 6.2 in term bin position 2.

We can see from table 6.2 that the same goes for impulses at bins three and four, five and six, and seven and eight. If we examine the shift across a factor of two boundary ($\text{bin } 2x \Rightarrow 2x + 1$), we will see more than one coefficient change. This is due to the decomposition of the discrete dyadic wavelet transform and also because the Haar wavelet contains only two filter coefficients. We can see that if we take only the first 1, 2 or 4 coefficients, we will be observing the function at a different resolution. Wavelets give us a new perspective by extracting the positions at different resolutions.

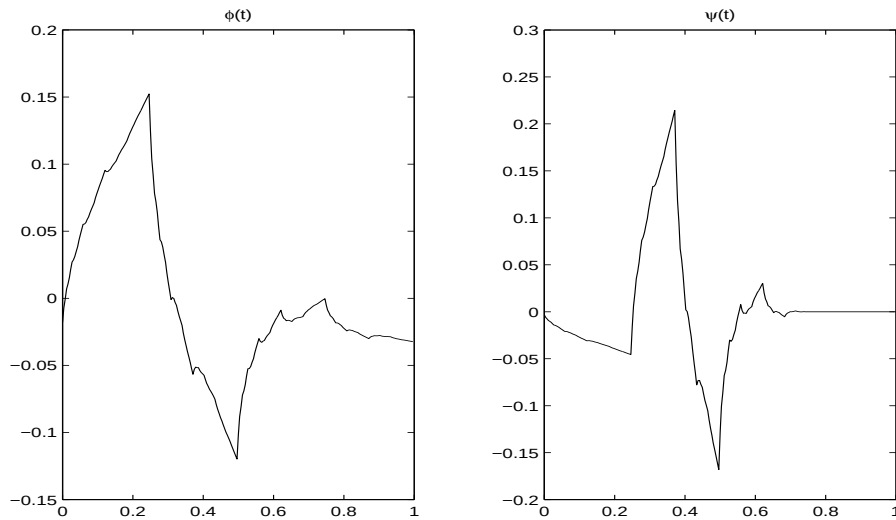


Figure 6.5: The Daubechies-4 scaling function ($\phi(t)$) and wavelet ($\psi(t)$)

Daubechies Wavelets

A short time after the introduction of conjugate mirror filters, the conditions of the filters were found to be identical to those of orthogonal wavelets. Ingrid Daubechies [28] took advantage of this relationship and found that for a wavelet (filter) to have p vanishing moments, it must have a support of at least $2p$. This theorem set a lower limit on the number of filter coefficients needed to perform a wavelet decomposition of a certain smoothness. From this knowledge, she was also able to derive a set of wavelets which have this minimum support for a given number of vanishing moments, these are called Daubechies wavelets.

It is interesting to note that the wavelet derived with minimum support when given only one vanishing moment, is in fact the Haar wavelet.

The wavelet we will be examining is the Daubechies-4 wavelet (so called because it has four filter coefficients in each high pass and low pass filter, and 2 vanishing moments) shown in figure 6.5. Going back to the self similarity concept of Mandelbrot, we can see in the Daubechies-4 wavelet and scaling function that this self similarity property exists in many portions of the wavelet. This is due to the construction of the wavelet by using recursive high pass and low pass filtering.

6.4 Document Retrieval

As stated in the introduction, there have been many different ideas on how we should retrieve text documents. Many people believe that if a document contains a set of terms appearing many times throughout the document, then that set of terms is the topic of the document. Others believe that the distribution of the terms throughout the document governs whether the terms belong to the topic. We believe that a set of query terms belongs to the topic of a document if they appear frequently and together throughout the document.

The majority of document retrieval systems use an algorithm called the vector space method. In this method, we treat every document as if it is a vector. Each element of the document vector is the frequency of a word in the document. Therefore, the document vectors are m dimensional (where m is the number of unique words in the document set) and each document vector will contain many zero elements. To normalise the vectors due to document lengths, term rarity and term occurrence, different weighting schemes have been developed. Some weighting schemes which have performed well are BD-ACI-BCA, BI-ACI-BCA, AB-AFD-BAA [126] and the Lnu.ltu weighting from SMART [17].

Proximity methods have been combined with vector space methods in order to rank documents based on the query term positions in a document. This leads to a slower query time since the query terms must be extracted and compared to the positions of the other query terms during the query time. By using a wavelet transform on a document, we are able to prepare the data in such a way so that the positions of the query terms can easily and quickly be compared during the query time. The wavelet document scoring method shown in algorithm 6.1 is further explained in the following subsections.

6.4.1 Create term signals

To perform any form of spectral information retrieval we must first produce term signals. A *term signal* ($\tilde{f}_{d,t}$) represents the flow of a specific term (t) throughout a specific document (d). To create such a signal, we first establish a set bin size for every signal (B). We then proceed to divide the selected document into B equal sequences of words, each element of the signal will represent the count of the selected term in the selected document. So if we chose $B = 8$ (eight bins), the first element of the term signal will be equal to the number of times the selected term appeared in the first eighth of the selected document. The second element represents the count of the term in the second eighth of the document, and so on. We create term signals for every term in every document and store them in an index file. Term signal creation is also performed when using the

Fourier and cosine methods of retrieval (section 3.2.1).

6.4.2 Apply preweighting

Apply the term weighting and document weighting ($\tilde{w}_{d,t} = W_P(\tilde{f}_{d,t})$). When dealing with statistical information about text, we must introduce normalisation factors to emphasise or dampen certain properties of the data. Examples of this are removing the impact of common terms and eliminating the effect of document length. The weighting scheme chosen ($W_P()$) is up to the user and can be any one of the numerous vector space method weighting schemes currently used in document retrieval. For the experiments we performed, we chose to use BD-ACI-BCA, BI-ACI-BCA, AB-AFD-BAA [126] and the Lnu.ltu weighting from SMART [17]. The AB-AFD-BAA method was chosen because it proved itself (in [126]) to be the overall best vector space method for short queries and the BI-ACI-BCA produced the best results for long queries. The BD-ACI-BCA method produced the best results overall when providing long and short queries. The Lnu.ltu method from SMART was also chosen because it is a well known method which has produced excellent results at TREC.

6.4.3 Apply Wavelet transform

The next step is to apply the wavelet transform to each of the query term signals from every document ($\tilde{\zeta}_{d,t} = \text{DWT}(\tilde{w}_{d,t})$). We have chosen to use the Haar and Daubechies-4 wavelets because they both have a small size of support, which is crucial when looking at the proximity of terms. The small support implies a low number of vanishing moments for the chosen wavelets, which should not have a big impact on the results.

6.4.4 Combine query term spectra

Once we have the wavelet components for each query term in the document, we must combine them into one signal. We know that the magnitude of the term signal components is related to the occurrence of the term and the phase of the term signal components is related to the position of the term in the document, therefore we deal with these separately. The combined word spectrum ($\tilde{s}_d$) is defined as:

$$\tilde{s}_d = \begin{bmatrix} s_{d,0} & s_{d,1} & \dots & s_{d,B-1} \end{bmatrix} \quad s_{d,b} = \bar{\Phi}_{d,b} \sum_{t \in T} H_{d,t,b}$$

where B is the number of spatial bins chosen, $H_{d,t,b}$ is the magnitude of the b th wavelet component of the t th query term in the d th document and $\bar{\Phi}_{d,b}$ is the Zero phase precision

1. For each query term $t \in T$
 - A** Retrieve inverted list I_t containing word signals $\{\tilde{f}_{0,t}, \tilde{f}_{1,t}, \dots, \tilde{f}_{D,t}\}$
 - B** Weight signals $\tilde{w}_{d,t} = W_P(\tilde{f}_{d,t})$
 - C** Transform signals $\tilde{\zeta}_{d,t} = \text{DWT}(\tilde{w}_{d,t})$
2. For each document d in collection
 - D** For each spectral component $\zeta_{d,t,b} \in \tilde{\zeta}_{d,t}$
 - (a) Calculate signal component magnitudes $H_{d,t,b} = W_Q(|\zeta_{d,t,b}|)$
 - (b) Calculate signal component phase $\phi_{d,t,b} = \text{sgn}(\zeta_{d,t,b})$
 - (c) For each component b in word signal spectrums
 - i. Calculate the selective phase precision $\bar{\Phi}_{d,b} = \left| \frac{\sum_{t \in T} \phi_{d,t,b}}{\#(T)} \right|$
 - ii. Obtain component score $s_{d,b} = \bar{\Phi}_{d,b} \sum_{t \in T} H_{d,t,b}$
 - E** Combine component scores to obtain document score $S_d = \|\tilde{s}_d\|_p$

Algorithm 6.1: Wavelet Domain Scoring

of the b th wavelet component in the d th document. The magnitude and Zero phase precision values are extracted from the wavelet components in the following way:

$$\zeta_{d,t,b} = H_{d,t,b} e^{(i\theta_{d,t,b})} \quad (6.18)$$

where $\zeta_{d,t,b}$ is the b th wavelet component of the t th query term in the d th document. The phase vector is defined as follows:

$$\phi_{d,t,b} = \frac{\zeta_{d,t,b}}{|\zeta_{d,t,b}|} = e^{(i\theta_{d,t,b})}$$

The Haar and Daubechies-4 wavelets transform do not produce complex values when applied to a real signal, hence, we treat the sign of the component as the phase. Therefore $\zeta_{d,t,b}$ is real and so $\theta_{d,t,b}$ must be of the form πn , where n is an integer. This implies that we will have only $\phi_{d,t,b} \in \{-1, 1\}$. The Zero phase precision equation can be simplified to:

$$\begin{aligned} \text{Zero phase precision} := \bar{\Phi}_{d,b} &= \left| \frac{\sum_{t \in T: H_{d,t,b} \neq 0} \phi_{d,t,b}}{\#(T)} \right| \\ &= \left| \frac{\sum_{t \in T} \text{sgn}(\zeta_{d,t,b})}{\#(T)} \right| \end{aligned} \quad (6.19)$$

where T is the set of query terms, $\#(T)$ is the cardinality of the set T , and

$$\text{sgn}(y) = \begin{cases} 1 & \text{if } y \geq 0 \\ 0 & \text{if } y = 0 \\ -1 & \text{if } y < 0 \end{cases}$$

6.4.5 Combine score components

Finally we combine the score components using the l_p norm measure. For our experiments, we have chosen p to be 1 and 2 (where l_2 is the Euclidean norm). Since we are decomposing the term signals into levels of different resolution, we also tried only combining the score components of up to the resolution which we desired. For example, if we did not care about the proximity of query terms, we would only need to choose the first component (which is proportional to the count of the term in the document). By choosing the first two components, we add information about which half of the document the query terms appear. If we choose the first four, we include information about which quarter of the document the query terms appear, and so on.

6.5 Computational Complexity and Storage

To thoroughly examine a retrieval method, we must not only examine precision of results, but also the calculation times and the storage involved.

6.5.1 Computational Complexity

If we take note of how that discrete wavelet transform can be performed using a recursive filter decomposition (shown in section 6.2.4), we can easily see that the time taken to perform the discrete dyadic wavelet transform is linear with respect to the number of discrete elements chosen to transform [73]. Therefore if we choose B bins for our term signal vectors, the wavelet transform will be of order $O(B)$, and the FFT is of order $O(B \log B)$ [94]. This is performed for each query term. In the following lists, we will represent the number of documents in the document set as N , the number of query terms as τ and the number of bins as B .

Each vector space method of document retrieval at query time involves:

1. Apply the specific weighting scheme to each query term in each document ($O(N\tau)$).
2. Summing each weighted query term in each document to obtain the document scores for each document ($O(N\tau)$).

Therefore, the overall computational complexity of a vector space method is $O(N\tau)$. The spectral methods require a bit more computation.

1. Apply the specific weighting scheme to each bin of each query term in each document ($O(N\tau B)$).
2. Perform the selected transform on each weighted word signal in each document ($O(N\tau B \log B)$ for the FFT or $O(N\tau B)$ for the DWT).
3. Calculate the phase precision for each spectral bin across each query term in each document ($O(N\tau B)$).
4. Calculate the magnitude for each spectral bin across each query term in each document ($O(N\tau B)$).
5. Calculate the score components by multiplying the magnitude and phase precision of each bin in each document ($O(NB)$).
6. Find the document score by summing the score components from each document ($O(NB)$).

Therefore, the overall time complexity of FDS is $O(N\tau B \log B)$ and the time complexity of the wavelet method is $O(N\tau B)$ if the transform is performed during the query time.

At this time, we should take note that the vector space method of information retrieval is a specialised case of the wavelet information retrieval method where $B = 1$. Therefore, by using the wavelet method, we are able to have more freedom as to how we perform our searches by choosing alternative B values to tradeoff between speed and accuracy.

6.5.2 Storage Size

To examine the impact our wavelet method has on storage, we will look at the common case of a document containing one occurrence of a term. In this case, previous methods using the Fourier and cosine transforms would transform a term signal with one non-zero element, to a term signal with all non-zero elements. This leaves us with B times as many elements to store when compared to the vector space methods. Using compression techniques such as quantisation and cropping (sections 4.3.3 and 4.4.1) led us to the conclusions that we were able to retain the high precision results of the Fourier and cosine retrieval methods with an index of only twice the size of that of the vector space methods.

By using the wavelet transform, we obtain term signals with $B/2$ non-zero elements when transforming a term signal with one element. Therefore, by using the same compression techniques found in the Fourier and cosine methods, we would expect to reduce the index size even further to about 150% of the vector space methods index size while still retaining the high precision of the wavelet retrieval method.

6.6 Analysis

Before we present the experimental results of our new method, we first demonstrate that our scoring function satisfies two important basic properties:

- The score must monotonically increase with the increase of occurrences of a query term.
- The score must monotonically increase as the displacement between two query terms decreases.

We will examine these properties in the following sections with both the Haar and Daubechies wavelets.

6.6.1 Occurrence Analysis

Both of the Haar and the Daubechies-4 wavelets produce orthonormal components when performing the wavelet transform. Therefore, the total energy of the set of components is conserved, which means that if there are a large number of occurrences of a term in a document, the transformed document signal will also produce a large count.

$$\|\mathbf{w}_{d,t}\|_2 = \|\mathbf{1}_{d,t}\|_2 \quad (6.20)$$

where $\zeta_{d,t,u,s} \in \mathbf{1}_{d,t}$ is the wavelet transform of $w_{d,t,b} \in \mathbf{w}_{d,t}$ of term t in document d , u is the shift and s is the scale. Therefore for the l_2 case, the score monotonically increases with the increase of occurrences of a query term.

The wavelet transform is a linear transformation, therefore it satisfies the following properties:

1. $Wf(\mathbf{x}_1 + \mathbf{x}_2) = Wf(\mathbf{x}_1) + Wf(\mathbf{x}_2)$
2. $Wf(\alpha\mathbf{x}) = \alpha Wf(\mathbf{x})$

The second property shows that if the original signal is scaled by a factor, then the wavelet transform of the signal is also scaled by that factor. If any individual bin is

scaled up, we can see that the wavelet transform of this bin is also scaled up. This implies that the sum of the components will be greater than those containing the unscaled bin.

$$\left\| Wf\left(\sum_{n \neq m} \mathbf{x}_n + \alpha \mathbf{x}_m\right) \right\|_1 = \left\| \sum_{n \neq m} Wf(\mathbf{x}_n) + Wf(\alpha \mathbf{x}_m) \right\|_1 \quad (6.21)$$

$$= \left\| \sum_{n \neq m} Wf(\mathbf{x}_n) + \alpha Wf(\mathbf{x}_m) \right\|_1 \quad (6.22)$$

$$\geq \left\| \sum_{n \neq m} Wf(\mathbf{x}_n) + Wf(\mathbf{x}_m) \right\|_1 \quad \text{if } \alpha \geq 1 \quad (6.23)$$

only if the sign of each component of $Wf(\mathbf{x}_m)$ is the same sign as each component of $\sum_{n \neq m} Wf(\mathbf{x}_n)$. This cannot be guaranteed because the wavelet transform maps the positive real signal to a real signal. If the sign of the coefficients of $Wf(\mathbf{x}_m)$ do not match the sign of the sum of the other coefficients the magnitude of that coefficient will be smaller and may result in a lower document score. Therefore for the l_1 case, the score does not monotonically increase with the increase of occurrences of a query term.

6.6.2 Proximity Analysis

The spectral document retrieval methods achieve high precision because they combine the ideas from the vector space methods which only analyse the term counts in the documents, and the proximity methods which take into account the displacement of the query terms from each other. In this section we will examine how each method reacts to the position of two different terms. It is ideal that each method gives a score which is monotonically decreasing when the displacement of the terms becomes larger, but we will see that that is not always the case.

To conduct the experiment, we have chosen two query terms (q_m, q_n) which exist in a document once each at a displacement of b bins apart. Each query term has the same term weight and since they appear in the same document, the same document normalisation.

Due to the nature of the Haar wavelet decomposition, each wavelet coefficient of q_m will have the same sign as the wavelet coefficients of q_n if the word appears in the same position. If we normalise our weighted term signals for q_m and q_n so that only ones and zeros occur, we are able to show the effect of each position on the Haar wavelet transform.

Analysing the proximity of terms is not as simple as in the Fourier transform case. We cannot use displacement as a parameter because the wavelets has compact support.

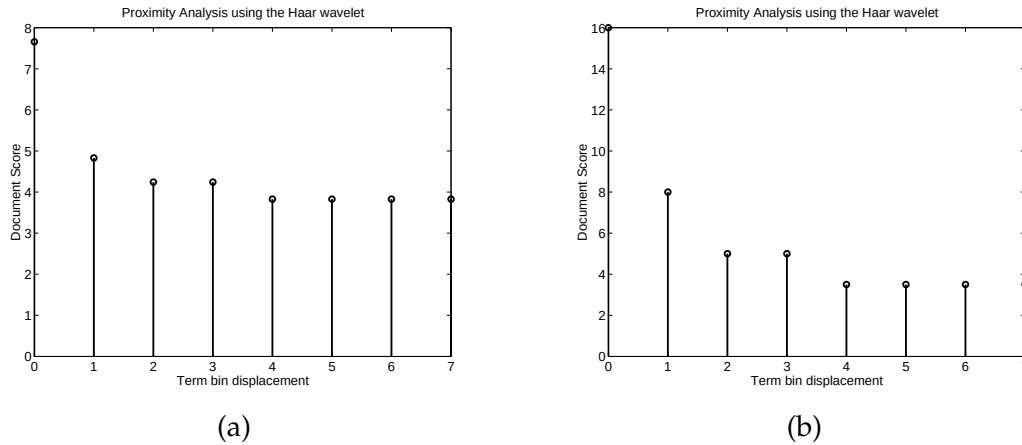


Figure 6.6: a) Score derived from the Haar wavelet transform using a) sum, b) sum of squares, of a document containing two query terms of equal weight. The first is found in bin zero, the second is found in bin 'term bin displacement'.

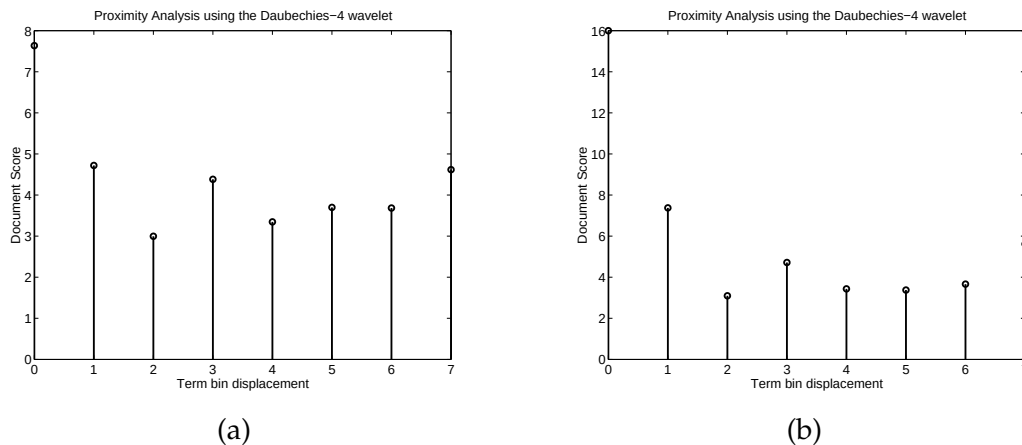


Figure 6.7: a) Score derived from the Daubechies-4 wavelet transform using a) sum, b) sum of squares, of a document containing two query terms of equal weight. The first is found in bin zero, the second is found in bin 'term bin displacement'.

To analyse the change of the score when the proximity of the terms change we experimented with the Haar and Daubechies-4 methods. Each experiment compared a single term appearing at bin position zero and another term whose position was adjusted. The score was then plotted against the displacement of the two terms. We would hope that the score decreased as the displacement between the two terms grew. The results are shown in figures 6.6a, 6.6b, 6.7a and 6.7b. The results show that both of the Haar methods have a nice monotonic decrease as the displacement increases, but both of the Daubechies-4 methods do not. We expected problems with the Daubechies-4 wavelet method because the wavelet itself does not decrease as we move away from its centre.

terms	document 1	document 2
laugh	$\begin{bmatrix} 2 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \end{bmatrix}$
diary	$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$
smile	$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$

terms	document 3
laugh	$\begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$
diary	$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$
smile	$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$

Table 6.3: A sample set of terms with their term signals within three documents.

The wavelet does decrease, but then increases above zero and then back to zero. This is a common behaviour amongst wavelets because of the property that they must integrate to zero. For this reason we will assume that most other wavelets will behave in the same fashion as the Daubechies-4 wavelet.

6.7 Experiments and Results

We have performed many extensive experiments to empirically show that the wavelet information retrieval method is superior to the vector space and proximity methods. But before we discuss our in depth results, we will go through a simple example of how this wavelet document retrieval method proceeds.

6.7.1 Sample document set

We will use the sample data found in table 6.3 and their wavelet transforms found in table 6.4. The table contains three selected terms from three different documents. Each term in each document has a corresponding term signal containing eight elements. Each element represents the occurrence of that term in that portion of the document. For example, the first element of the term signal “laugh” in document 1 is 2, this implies that the term “laugh” occurs twice in the first eighth of document 1. The fourth element contains a 1, which implies that the term “laugh” occurs once in the fourth eighth of document 1. We will ignore the initial preweighting stage of the process and process the data as if the weights were 1. Prewighting is an important part of the retrieval process, it is only left out in this example to focus on the wavelet retrieval process.

If we query our database with the terms “smile diary”, the system will extract the

terms	document 1	document 2
laugh	$\begin{bmatrix} \frac{5}{\sqrt{8}} & \frac{1}{\sqrt{8}} & \frac{1}{\sqrt{4}} & \frac{2}{\sqrt{4}} & \frac{2}{\sqrt{2}} & \frac{-1}{\sqrt{2}} & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} \frac{2}{\sqrt{8}} & 0 & \frac{-1}{\sqrt{4}} & \frac{1}{\sqrt{4}} & 0 & \frac{-1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} & 0 \end{bmatrix}$
diary	$\begin{bmatrix} \frac{2}{\sqrt{8}} & 0 & \frac{-1}{\sqrt{4}} & \frac{1}{\sqrt{4}} & 0 & \frac{-1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} & 0 \end{bmatrix}$	$\begin{bmatrix} \frac{2}{\sqrt{8}} & 0 & \frac{1}{\sqrt{4}} & \frac{1}{\sqrt{4}} & \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} & 0 \end{bmatrix}$
smile	$\begin{bmatrix} \frac{1}{\sqrt{8}} & \frac{-1}{\sqrt{8}} & 0 & \frac{-1}{\sqrt{4}} & 0 & 0 & 0 & \frac{1}{\sqrt{2}} \end{bmatrix}$	$\begin{bmatrix} \frac{1}{\sqrt{8}} & \frac{-1}{\sqrt{8}} & 0 & \frac{1}{\sqrt{4}} & 0 & 0 & \frac{1}{\sqrt{2}} & 0 \end{bmatrix}$
terms	document 3	
laugh	$\begin{bmatrix} \frac{3}{\sqrt{8}} & \frac{-1}{\sqrt{8}} & \frac{1}{\sqrt{4}} & 0 & \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix}$	
diary	$\begin{bmatrix} \frac{2}{\sqrt{8}} & \frac{2}{\sqrt{8}} & 0 & 0 & \frac{-1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} & 0 & 0 \end{bmatrix}$	
smile	$\begin{bmatrix} \frac{1}{\sqrt{8}} & \frac{-1}{\sqrt{8}} & 0 & \frac{1}{\sqrt{4}} & 0 & 0 & \frac{-1}{\sqrt{2}} & 0 \end{bmatrix}$	

Table 6.4: The wavelet transforms of the term signals found in table 6.3.

data for those terms only (therefore we will ignore the data for the term “laugh” from now on). The system will calculate the score for each document, so we will examine the score calculation for the first document in detail. Our wavelet components are:

Terms	Document 1
diary	$\begin{bmatrix} \frac{2}{\sqrt{8}} & 0 & \frac{-1}{\sqrt{4}} & \frac{1}{\sqrt{4}} & 0 & \frac{-1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} & 0 \end{bmatrix}$
smile	$\begin{bmatrix} \frac{1}{\sqrt{8}} & \frac{-1}{\sqrt{8}} & 0 & \frac{-1}{\sqrt{4}} & 0 & 0 & 0 & \frac{1}{\sqrt{2}} \end{bmatrix}$

Take the magnitudes and sum them:

Terms	Document 1 Magnitude
diary	$\begin{bmatrix} \frac{2}{\sqrt{8}} & \frac{0}{\sqrt{8}} & \frac{1}{\sqrt{4}} & \frac{1}{\sqrt{4}} & \frac{0}{\sqrt{2}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & \frac{0}{\sqrt{2}} \end{bmatrix}$
smile	$\begin{bmatrix} \frac{1}{\sqrt{8}} & \frac{1}{\sqrt{8}} & \frac{0}{\sqrt{4}} & \frac{1}{\sqrt{4}} & \frac{0}{\sqrt{2}} & \frac{0}{\sqrt{2}} & \frac{0}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix}$
Total	$\begin{bmatrix} \frac{3}{\sqrt{8}} & \frac{1}{\sqrt{8}} & \frac{1}{\sqrt{4}} & \frac{2}{\sqrt{4}} & \frac{0}{\sqrt{2}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix}$

Take the phase and find the Zero phase precision:

Terms	Document 1 Phase
diary	$\begin{bmatrix} 1 & 0 & -1 & 1 & 0 & -1 & -1 & 0 \end{bmatrix}$
smile	$\begin{bmatrix} 1 & -1 & 0 & -1 & 0 & 0 & 0 & 1 \end{bmatrix}$
Zero phase precision	$\begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{2} & 0 & 0 & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \end{bmatrix}$

Combine the magnitude and phase:

Document 1 magnitude	$\begin{bmatrix} \frac{3}{\sqrt{8}} & \frac{1}{\sqrt{8}} & \frac{1}{\sqrt{4}} & \frac{2}{\sqrt{4}} & \frac{0}{\sqrt{2}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix}$
Document 1 Zero phase precision	$\begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{2} & 0 & 0 & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \end{bmatrix}$
Document 1 score vector	$\begin{bmatrix} \frac{3}{\sqrt{8}} & \frac{1}{2\sqrt{8}} & \frac{1}{2\sqrt{4}} & \frac{0}{\sqrt{4}} & \frac{0}{\sqrt{2}} & \frac{1}{2\sqrt{2}} & \frac{1}{2\sqrt{2}} & \frac{1}{2\sqrt{2}} \end{bmatrix}$

We can choose either the sum or the squared sum of the score vector as our document score. If we choose the squared sum we obtain:

$$\text{Document 1 score} = \frac{51}{32} = 1.5938$$

If we follow the same process for the other two documents we obtain:

Document	Score
1	1.5938
2	4.3437
3	1.5625

If we examine the positions of the query terms (in the term signals) we can see that the document scores reflect the proximity and occurrence of the query terms in the documents. Document two scored the highest and has the query terms closest together within the document, we can see in document 2 that both query terms appear in the same bin. Document 1 ranked second and we can see that the query terms appear in neighbouring bins. Finally document 3 ranked third, with its query terms more distant than the other two documents.

6.7.2 Application to TREC document set

To judge how accurate our wavelet document retrieval methods are, we will compare them to the current methods which have shown to be of high precision. The experiments we have performed involved a variety of methods shown in table 6.5 and we compared these against the vector space methods AB-AFD-BAA, BD-ACI-BCA, BI-ACI-BCA and Lnu.ltu weighting from SMART. We also compared our method against two successful term proximity measures called shortest-substring retrieval [24] (labeled Prox.SSS) and Pairs distance & Collection Frequency [66] (labeled Prox.Keen). Included in the results are the precision values of our fds-7-4-1 method [91]. This is a similar method to our wavelet method which uses a Fourier transform in place of the wavelet transform. The fds-7-4-1 method uses AB-AFD-BAA preweighting. To perform any experiment in information retrieval, we need a substantial database of documents and a well defined set of queries and documents which are relevant to these queries. The TREC collection¹ is just this. The collection contains hundreds of thousands of documents, hundreds of queries and relevance judgments which have been accumulated over the past ten years. These experiments were performed using the TREC ap2wsj2 document collection using the titles of queries 51 to 200 (examples can be seen in figure 6.8). We are interested in

¹<http://trec.nist.gov/>

- Airbus Subsidies
- South African Sanctions
- Leveraged Buyouts
- Satellite Launch Contracts
- Insider Trading
- Prime (Lending) Rate Moves, Predictions
- MCI
- Rail Strikes
- Weather Related Fatalities

Figure 6.8: Examples of queries taken from TREC queries 51-200 titles.

an algorithm that would be effective for text retrieval on the Web. We have observed that most Web search engine users will only observe the first 20 documents retrieved. If they do not obtain the results they wanted within this selection, the query is usually reformulated and the search is tried again. Therefore we will only observe the results of Precision after the first 5, 10, 15 and 20 documents (this represents the impatience of the typical Web user). The results sorted by precision after 5, 10, 15 and 20 documents are given in tables 6.6a, 6.6b, 6.7a and 6.7b respectively. Each table shows the results of the top 20 methods and the results of any of the comparative methods below the bar if they did not appear in the top 20.

We can see from the tables that the methods that appear at the top of each list use all eight wavelet components to calculate the score and that most use the l_2 method to combine the score. Each of the methods uses either the AB-AFD-BAA, BD-ACI-BCA or Lnu.ltu weighting with Zero phase precision. We can also see that the Shortest-substring method performs poorly compared with the other methods. This is because our queries are the titles of queries 51 to 200. The titles are a set of one to six terms. Since these terms do not include any boolean algebra, we have assumed that the user would like all of the terms to appear in the document which implies an AND between the terms. For a document to be relevant for this type of query relative to the shortest-substring method, all of the terms must appear in the document. If one or more does not, the document is not scored.

It is interesting to see that even though the Daubechies-4 proximity analysis was not

Table 6.5: Method names are of the form wavelet-x-y-z.ln where the values of x,y,z,n associate to the description in this table

Label	Value	Description
x	5	BD-ACI-BCA preweighting
	7	AB-AFD-BAA preweighting
	8	BI-ACI-BCA preweighting
	9	Lnu.ltu preweighting
y	1	Sum vectors with No phase precision
	3	Sum magnitude with Non-zero phase precision
	4	Sum magnitude with Zero phase precision
z	1	Combine using l_1 norm
	6	Combine using l_2 norm
n	{1,2,4,8}	Number of score components added

monotonic, the results were just as good as the Haar method for the l_2 combination case.

A plot showing the precision after 5,10,15 and 20 documents (figure 6.9) displays that the wavelet methods are preferred for this level of retrieval (which is the level required for Web searching).

6.8 Extension

To improve the wavelet document retrieval method we could easily look into storing the wavelet coefficients into the inverted index rather than the term position values. We investigated this with the discrete cosine transform and showed that the retrieval time reduced by a factor of 100 (which is comparable to the vector space methods retrieval times).

Another interesting aspect to examine would be wavelet packet decompositions as discussed by Coifman and Wickerhauser [25]. By doing this we would be able to decompose each of the term signals into the smallest amount on non-zero terms and store them into the inverted index. This would lead to a smaller inverted index due to the wavelet packet compression qualities.

Method	Precision 5	Method	Precision 10
haar-5-4-6.b8	0.5000	daub4-7-4-6.b8	0.4687
daub4-9-7q-1.b8	0.4973	daub4-5-4-6.b8	0.4653
haar-7-4-6.b8	0.4960	haar-5-4-6.b8	0.4633
daub4-5-4-6.b8	0.4960	daub4-9-4-6.b8	0.4620
daub4-7-4-6.b8	0.4947	haar-9-7q-1.b8	0.4593
daub4-9-4-6.b8	0.4907	haar-7-4-6.b8	0.4593
haar-9-4-6.b8	0.4893	haar-5-4-6.b4	0.4593
AB-AFD-BAA	0.4880	haar-9-4-6.b8	0.4593
daub4-9-7q-1.b1	0.4853	daub4-9-7q-1.b8	0.4587
haar-9-7q-1.b1	0.4840	haar-7-4-6.b2	0.4573
daub4-7-4-1.b1	0.4827	haar-9-4-1.b8	0.4560
haar-7-4-6.b1	0.4827	haar-7-4-6.b4	0.4560
haar-7-4-1.b1	0.4827	daub4-5-4-6.b2	0.4553
daub4-7-4-6.b1	0.4827	haar-7-4-1.b2	0.4547
haar-5-4-6.b1	0.4813	daub4-7-4-1.b8	0.4547
haar-5-4-6.b2	0.4813	daub4-7-4-6.b4	0.4547
haar-7-4-6.b2	0.4813	daub4-9-4-6.b4	0.4540
daub4-7-4-6.b4	0.4813	daub4-7-4-6.b2	0.4540
daub4-5-4-6.b1	0.4813	daub4-5-4-6.b4	0.4540
daub4-9-4-1.b8	0.4813	haar-5-4-6.b2	0.4533
SMART	0.4693	AB-AFD-BAA	0.4493
BD-ACI-BCA	0.4440	SMART	0.4493
Prox.Keen	0.4360	BD-ACI-BCA	0.4247
BI-ACI-BCA	0.4347	Prox.Keen	0.4207
Prox.SSS	0.3718	BI-ACI-BCA	0.4100
		Prox.SSS	0.3362

(a)

(b)

Table 6.6: Top 20 methods using data set AP2WSJ2 with queries 51 to 200 sorted by precision after a) 5 and b) 10 documents. The methods under the lower bar are the vector space and proximity methods which did not make the top 20.

Method	Precision 15	Method	Precision 20
haar-7-4-6.b8	0.4449	daub4-7-4-6.b8	0.4257
daub4-7-4-6.b8	0.4449	daub4-9-4-6.b8	0.4223
daub4-9-4-6.b8	0.4431	haar-7-4-6.b8	0.4223
haar-9-4-6.b8	0.4413	haar-9-4-6.b8	0.4217
haar-5-4-6.b8	0.4404	AB-AFD-BAA	0.4217
AB-AFD-BAA	0.4404	haar-5-4-6.b8	0.4213
haar-9-4-1.b8	0.4396	haar-9-4-1.b8	0.4190
daub4-5-4-6.b8	0.4391	haar-7-4-1.b8	0.4183
haar-7-4-6.b4	0.4382	SMART	0.4180
haar-7-4-1.b8	0.4382	daub4-5-4-6.b8	0.4177
haar-9-7q-1.b8	0.4378	haar-9-7q-1.b8	0.4160
daub4-7-4-6.b4	0.4373	daub4-7-4-6.b4	0.4157
haar-7-4-6.b2	0.4373	haar-7-4-6.b4	0.4143
SMART	0.4356	daub4-9-7q-1.b8	0.4140
daub4-9-4-1.b8	0.4351	haar-7-4-6.b2	0.4137
daub4-7-4-6.b2	0.4347	haar-5-4-1.b8	0.4123
daub4-9-7q-1.b8	0.4338	daub4-5-4-6.b4	0.4117
haar-9-4-6.b4	0.4333	haar-9-4-6.b4	0.4107
daub4-9-4-6.b4	0.4329	daub4-9-4-1.b8	0.4103
haar-7-4-6.b1	0.4324	daub4-9-4-6.b4	0.4090
BD-ACI-BCA	0.4142	BD-ACI-BCA	0.3953
Prox.Keen	0.4116	Prox.Keen	0.3920
BI-ACI-BCA	0.3862	BI-ACI-BCA	0.3657
Prox.SSS	0.3078	Prox.SSS	0.2856

(a)

(b)

Table 6.7: Top 20 methods using data set AP2WSJ2 with queries 51 to 200 sorted by precision after a) 15 and b) 20 documents. The methods under the lower bar are the vector space and proximity methods which did not make the top 20.

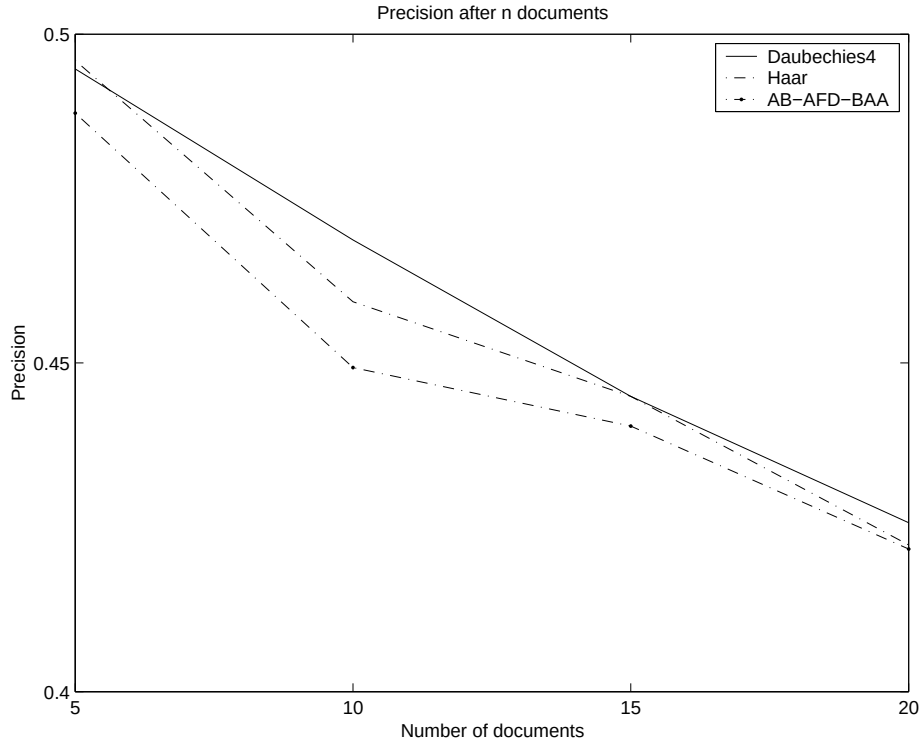


Figure 6.9: Precision-recall plot for recall of 5, 10, 15 and 20 documents for the Haar and Daubechies-4 wavelet methods, and the AB-AFD-BAA vector space method.

6.9 Discussion and Summary

The wavelet transform is a tool which has been used in many areas of science and engineering to extract information about the self similarity of signals. Using this information we are able to encode the signals in a more compact manner and easily extract desired content to perform other tasks (eg. compare signals for similarity).

We have shown that we can successfully extract the self-similarity components of the occurrence of terms through documents. This knowledge can be used to compare term signal activity and obtain better estimates of whether a document is relative given a set of query terms.

The experiments performed using the Haar and Daubechies-4 wavelets generated precision results which were significantly better than the vector space methods and proximity method shown in this chapter when using the l_2 component combination on all components and Zero phase precision.

The query time was shown to be $O(N\tau B)$ for the wavelet method and $O(N\tau B \log B)$ for the Fourier transform method which is slower by an order of $\log B$. Both of these give

similar precision results, therefore the wavelet document retrieval method is preferred when performing the transform during query time.

CHAPTER SEVEN

ANAPHORA RESOLUTION AND RELEVANCE FEEDBACK

7.1 Introduction

THE land of information retrieval includes the query term analysis which we have shown in the previous chapters, and it also includes extensions to these methods which are used to try to increase the precision of the document retrieval system. The query term analysis methods we have observed take a set of query terms and use the spatial or spectral information about these query terms in each documents to derive a document score. But there is only so much information that can be obtained by examining only the query term information.

In this chapter, we will explore two methods of extracting additional information from the document sets used in order to achieve a better document retrieval system. The first method, called Pronoun Resolution, attempts to find all of the pronouns located in the document set and replace them with the nouns they represent. By doing this we hope to have a better representation on the query terms throughout each document, and therefore have document score which truly represent the document.

The second method, called Relevance Feedback, exploits the documents returned as relevant by selecting additional query terms from them and re-running the document retrieval process. Documents considered relevant can be either chosen by the user or assumed by the machine. By feeding back the relevant documents into the querying process we hope to expand on the few query terms the user has initially given to focus on the topic and provide a more relevant documents based on these additional query terms.

7.2 Anaphora Resolution

When performing text document searches using a set of query terms, the only information that we can use in the document set is the appearance of the query terms in each document, that is unless we use some form of natural language processing.

Natural language processing (NLP) comes in many flavours, it could be as simple as using a thesaurus or as complex as finding equivalent sentences. But the goal of NLP is to enable a machine to interact with a human by interpreting our language.

- Automatic text summarisation
- Language translation
- Building lexicons
- Natural language query processing
- Discourse processing

In this chapter we will focus on a subsection of discourse processing called anaphora resolution in English. We will use this NLP technique on our existing document set to try to obtain a document retrieval method with increased precision.

7.2.1 Anaphoric Resolution in English

The process of resolving anaphora is one that we perform when reading or listening all of the time. The fact that the human brain can perform this resolution so easily has intrigued linguists and psychologists for decades. To understand how we as humans can deduce this complex mapping has been considered one of the steps in unlocking the secret to understanding the human brain.

The term *anaphora* comes from the ancient Greek term *αναφορα*. This term consists of the words *ανα* (ana) meaning back or upstream and the word *φορα* (phora) relating to the act of carrying. Therefore the whole term is related to the act of carrying back upstream [80, 33]. An anaphor is a term which reflects the meaning of a previous term earlier in the sentence, or in a previous sentence (the term which is upstream, also called the antecedent).

The most common occurrence of anaphor is a pronoun. Pronouns are terms which are used in the place of nouns so that the noun does not have to be repeated. For example:

Michael bought a CD, it cost \$20.

Australian Huntsman spiders belong to the Family Sparassidae (formerly Heteropodidae) and are famed as being the hairy so-called 'tarantulas' on house walls that terrify people by scuttling out from behind curtains. In fact, they are a diverse and relatively harmless group of spiders, with 13 genera and 94 described species.

Figure 7.1: http://www.austmus.gov.au/factsheets/huntsman_spiders.htm

The pronoun *it* refers to the CD, but even in this simple example there are two nouns, namely Michael and the CD. We know that *it* refers to the gender-less noun, and therefore we associate *it* to the CD.

7.2.2 Speech Tagging

Before we can begin performing any analysis on a text document, we must first understand the structure of the documents and how each term relates to each other. A *Part-of-speech* (POS) tagger does this job by tagging each term in the document with its syntactical identifier. The two most well known POS taggers are Brill's tagger [13] and LTPOS [77]. We chose LTPOS for its availability and documentation. The LTPOS tagger is split into three components, they are a tokeniser, a morphological classifier and a morphological disambiguator. The tokeniser separates the string into words, the morphological classifier assigns tags to each word. Each word can be assigned more than one tag. The morphological disambiguator then proceeds to analyse the text and select one tag for each word from the set of assigned tags. The tag set used is from the Penn Treebank Project [103]. A sample of the tags is given in table 7.1. If we were to use the text in figure 7.1 as the input to LTPOS, we would receive the tagged text appearing in figure 7.2. We can see that the output text is not only tagged but grouped by tags and brackets. Each sentence is enclosed within <S> and <S/> tags (same as HTML tags), and noun groups and verb groups are found within [[]] and (()) respectively.

7.2.3 Types of Anaphora

An informative introductory paper by Denber [33] categorises seven cases of anaphora.

Pronominal This is the most common case of anaphora. It involves replacing pronouns (such as 'he', 'his', or 'himself') with their antecedent. For example:

Oscar brought *his* pencils so that *he* could draw.

POS Tag	Description	Example
JJ	adjective	green
JJR	adjective, comparative	greener
JJS	adjective, superlative	greenest
RB	adverb	however, usually
RBR	adverb, comparative	better
RBS	adverb, superlative	best
NN	common noun	table
NNS	noun plural	tables
NNP	proper noun	John
NNPS	plural proper noun	Vikings
VB	verb base form	take
VBD	verb past	took
VBG	gerund	taking
VCN	past participle	taken
VBP	verb, present, non-3d	take
VBZ	verb present, 3d person	takes
FW	foreign word	d'hoevre
CD	cardinal number	1, third
CC	coordinating conjunction	and
DT	determiner	the
EX	existential there	there is
IN	preposition/subordinating conjunction	in, of, like
LS	list marker	1)
MD	modal	could, will
PDT	predeterminer	both the boys
POS	possessive ending	friend's
PRP	personal pronoun	I, he, it
PRP\$	possessive pronoun	my, his
RP	particle	give up

Table 7.1: A non-exhaustive list of tags inserted after using a part-of-speech (POS) tagger. Taken from the Penn Treebank Tag-Set and used with LTPOS [51].

```

<S>[[ Australian_NNP Huntsman_NNP spiders_NNS ]] (( belong_VBP )) to_TO
[[ the_DT Family_NNP Sparassidae_NNP ]] (.(formerly_RB [[ Heteropodi-
dae_NNP ]]) and_CC (( are_VBP )) famed_JJ as_IN (( being_VBG )) [[ the_DT
hairy_JJ so-called_JJ 'tarantulas_NNS ]]'-' on_IN [[ house_NN walls_NNS ]] [[
that_WDT ]] (( terrify_VBP )) [[ people_NNS ]] by_IN scuttling_VBG out_RP
from_IN behind_IN [[ curtains_NNS ]],-> </S> <S>In_IN [[ fact_NN ]],-> [[
they_PRP ]] (( are_VBP )) [[ a_DT diverse_JJ and_CC relatively_RB harmless_JJ
group_NN ]] of_IN [[ spiders_NNS ]],-> with_IN [[ 13_CD genera_NNS ]] and_CC
[[ 94_CD described_JJ species_NNS ]],-> </S>

```

Figure 7.2: The output of LTPOS when given figure 7.1. Each term is tagged with a Penn Treebank tag.

In this case *his* and *he* refer to Oscar.

Quantifier This form of anaphora describes the antecedent by describing an amount (such as ‘one’, ‘some’, ‘every’, ‘a few’). For example:

There were no black pencils, so he choose a blue *one*.

Here *one* refers to a pencil.

Ordinal This form of anaphora describes the antecedent by the position it was given in (such as ‘first’, ‘former’). These are not very common. For example:

He drew two pictures, the *first* was of a dragon.

In this example *first* refers to the first picture.

Question This includes the anaphoric use of the words ‘who’, ‘what’, ‘which’, ‘where’, ‘when’. For example:

Oscar drew the mouth open, because that is *where* the fire will be.

In this example *where* refers to the mouth.

Event Anaphora In some cases, an anaphor will refer to an event rather than just a noun phrase. For example:

Oscar drew the fire bursting onto a tree, *it* took him a long time.

Here the *it* refers to Oscar drawing the fire bursting onto a tree.

Pleonastic it This is a case of a pronoun which does not refer to anything. For example:

It happened to be dinner time, so he stopped drawing.

Here the *it* does not refer to anything and so is not considered an anaphor. This case is important and should be found in texts so that we do not confuse them with other cases.

Special cases This case is left for the idioms of the English language which are not covered in the above cases.

To make our journey though anaphora resolution easier, we will choose to focus on the most general case mentioned which is pronoun resolution.

7.2.4 Pronoun Resolution

If we were to integrate a new system into our existing document retrieval system, we would need it to be fast and error free. Therefore, to become part of a document retrieval system, we need the pronoun resolution component to be as fast as possible and as error free as possible. In other words, we need a system that will only resolve the pronouns to their true antecedent if the system is certain that the resolution is correct.

In the late 1970's Hobbs algorithm [46, 80, 110] came to the attention of the natural language processing community. It was one of the first pronominal resolution algorithms which used the syntactical knowledge of text to perform its task.

There are many papers covering the topic of pronoun resolution [68, 46, 110, 41, 83, 19], but they all require a large amount of time to perform, and therefore would not suit the needs of an information retrieval system. A pronoun resolution system by Baldwin [7] called CogNIAC breaks away from the trend of natural language processing by focusing on the set of pronouns which it is very sure that it can resolve, rather than trying to resolve all of the pronouns. Baldwin states that in conversation, the listener will find it hard to understand which antecedents are referred to by pronouns unless questions are asked, so how do we expect to implement a solution to pronoun resolution. This method has a high accuracy when resolving pronouns, but only a few would be resolved out of the many that occurred in the text.

The approach taken by Mitkov [79] (which he calls *Knowledge poor*) does not need to analyse the entire document before any resolution can be done. He makes use of a part-of-speech tagger and some noun phrase rules. When a pronoun is encountered, all of the noun phrases which precede the pronoun, which match in terms of gender and number, are candidates for substitution. Each candidate is graded a score of -1,0,1 or 2 based on

a set of rules. The scores for each rule are summed to give the final score for the noun phrase. The candidate noun phrase with the highest score is chosen as the representative of the pronoun. This approach proved successful 89.7% of the time, out of every pronoun it tried to resolve. The pronouns that were not changed were not examined.

7.2.5 Application to Information retrieval

In the field of Information Retrieval, documents are classified relevant or irrelevant based on the appearance of the query terms throughout the document. Our problem is that when a document is focused on a certain topic, there will be many appearances of pronouns which are used to represent the topic. For example if the pronouns in the text of figure 7.1 were resolved properly, the terms “Australian Huntsman spiders” would be spread evenly across the paragraph. This means that the terms would appear more often and hence the document would be considered more relevant.

The FDS method we have described [91], not only examines the document set for term appearances, but also for query term proximity. Since the terms “Australian Huntsman spiders” would appear together through the text, the FDS method would produce a higher document score if given it as a query.

If we were to develop a few basic syntactical rules to replace the pronouns in the text only when we were sure, then we would be able to use the pronominal resolved text to perform our document retrieval on.

A problem that arises from using pronoun resolution is that our document retrieval system is no longer multi-lingual. The system was designed to treat terms in documents as objects, if a documents has more query terms and the query terms are in the same proximity, then the document is given a high score. Now that we have added semantical rules from the English language to the document retrieval system, we are only able to retrieve documents written in English. To remove this problem, we must devise rules for each of the languages which we will search in and we must state which documents are written in which language.

7.2.6 Experiments

Before we could begin any experimentation, we had to design a basic pronoun resolution algorithm which was fast enough to use with a document retrieval system. Experiments by Tetreault [110] were compared to a basic method that he had written. It consisted of keeping a list of all of the recent nouns encountered. When a pronoun is found, the most recent noun which matches in terms of gender, number and binding constraints is

POS Tag	Description	Example
NN	common noun	table
NNS	noun plural	tables
NNP	proper noun	John
NNPS	plural proper noun	Vikings

Table 7.2: The list of noun types and their POS tags.

POS Tag	Description	Example
PRP	personal pronoun	I, he, it
PRP\$	possessive pronoun	my, his

Table 7.3: The list of pronoun types and their POS tags.

selected as its antecedent. The number matching can easily be performed, because it is shown in the POS tags assigned to each term. To perform the gender matching, we must have a list of all terms and their matching gender. Because we do not have a gender list and it would take too much time to compile one for every document set, we decided not to include gender as one of the matching criteria.

After we have passed the document through the POS tagger, we are able to extract the nouns and pronouns. The nouns come in the four flavours shown in table 7.2 and the pronouns have two categories listed in table 7.3. To begin, we obtained the set of personal and possessive pronouns and split them into groups with their matching noun type (as shown in table 7.4). Note that we tried to resolve only personal and possessive pronouns because these were the most obvious after the POS tagging had taken place.

Now that we had each of the pronouns divided into at least one of the noun categories, we had to choose an algorithm for pronoun resolution in the case that there were more than one nouns available (which is a very common case). For example, if the resolution program came across an *it* and there was a previous proper noun and a common noun, which should be chosen for the replacement? We devised three simple methods of replacement:

- Replace the pronoun with the first matching noun of the previous sentence
- Replace the pronoun with the first matching noun of the current sentence
- Replace the noun with the most recent matching noun.

NNP	NNPS	NNS	NN
i	we	they	it
you	they	them	its
he	you	their	his
she	us	theirs	hers
it	them		
me	our		
him	ours		
her	yours		
my	their		
mine	theirs		
yours			
his			
hers			
its			

Table 7.4: List of noun types and the possible personal and possessive pronouns that can reference them

To test these methods, we extracted a single random document from the TREC AP2 document set and performed the pronoun resolution using each of the three methods. We found by observation that the first two methods had close to 0% accuracy and the last about 50%. Therefore we choose the third method as the replacement scheme in our experiment.

To begin our experiment, we chose the AP2 document set from the TREC collection and processed it with our simple pronoun resolver. Once this was done, we used the processed AP2 to build an index for our FDS method query. We choose to use queries 51-200 (from TREC 1,2 and 3) to evaluate our method. The experiment was performed using the fds-9-4-1 method and the results are shown in table 7.5.

The results show fds-9-4-1 without pronoun resolution at the top of the list as a comparison as to how the pronoun resolution has affected the precision. The method which has used the pronominal resolved text is shown as fds.pr-9-4-1.bb, where b is the number of frequency bins that were added to obtain the score (eg. if $b = 2$, the 0th and 1st frequency bins were added to obtain the document score for every document and every query). We can see in table 7.5 that the method involving the pronoun resolution has a lower precision at 5,10,15 and 20 documents, except for the case of precision at 20

Method	Precision 5	Method	Precision 10
fds-9-4-1	0.4240	fds-9-4-1	0.3780
fds.pr-9-4-1.b5	0.4200	fds.pr-9-4-1.b4	0.3707
fds.pr-9-4-1.b4	0.4080	fds.pr-9-4-1.b5	0.3653
fds.pr-9-4-1.b3	0.4053	fds.pr-9-4-1.b2	0.3627
fds.pr-9-4-1.b1	0.3920	fds.pr-9-4-1.b3	0.3613
fds.pr-9-4-1.b2	0.3907	fds.pr-9-4-1.b1	0.3600
(a)		(b)	
Method	Precision 15	Method	Precision 20
fds-9-4-1	0.3467	fds-9-4-1	0.3267
fds.pr-9-4-1.b3	0.3449	fds.pr-9-4-1.b3	0.3337
fds.pr-9-4-1.b2	0.3413	fds.pr-9-4-1.b4	0.3270
fds.pr-9-4-1.b4	0.3413	fds.pr-9-4-1.b2	0.3247
fds.pr-9-4-1.b5	0.3404	fds.pr-9-4-1.b1	0.3217
fds.pr-9-4-1.b1	0.3400	fds.pr-9-4-1.b5	0.3147
(c)		(d)	

Table 7.5: Results for the method fds-9-4-1 using AP2 and the same method involving our simple pronoun resolution algorithm. We can see that the added pronoun resolution increases precision only at the 20 document point (d) when summing 3 and 4 bins.

documents where only the first 4 and 3 frequency bins are added.

These results were disappointing because the concept seemed to be an exciting new direction. We still believe that pronoun resolution can help an information retrieval system perform at a higher level, but we need to implement a more sophisticated pronominal resolver.

7.3 Relevance Feedback

The job of the interactive system designer is to understand the needs of the user and to be able to design an interface such that everything is easily accessible and operates as the user would expect it to. For example, if a user was faced with a counter joined to up arrow and a down arrow buttons, the user would expect clicking on the up arrow makes the counter increase and clicking on the down arrow makes the counter decrease. The system designer would create the system so that the user has a limited number of choices, implying that the user has little freedom. This is a problem when designing a

search engine because the user is usually faced with an empty text box, which awaits the query terms of the user. The user has total freedom as to which query terms to supply to the engine. If we were to design the perfect search engine, which returned the most relevant documents of the document set every time the correct query terms were given, we would still not have a perfect machine in the eyes of the user. The first step in giving a query is to choose the right query terms for the job. This is not always obvious to the user and hence, poor query terms lead to poor results.

It is not the results, but the query that is poor

If we make the assumption that our search engine is perfect, it implies that for a given set of query terms we will receive the most relevant documents of the document set at the top of the results. By doing this we have removed the problem from finding the best search algorithm to finding the best query terms for the desired query.

7.3.1 Rocchio Relevance Feedback

A method of addressing the problem at hand is to present the user with a set of documents to which he will then mark as relevant or irrelevant to the query he has in mind. Based on these documents, the search engine would then proceed to extract from these documents the terms which would describe them best and which also set them apart from the irrelevant documents. These terms are then used for the query. This method was introduced in the paper by Rocchio [98] using the SMART vector space document retrieval system. Rocchio believed that since each document and query is treated as a vector, the documents that are relevant to the query that the user desires would be clustered amongst this desired query vector. His method involves the following procedure:

1. Perform the vector space method with a query supplied by the user.
2. Present the user with the top ranked documents.
3. The user marks a selection of the presented documents with either a relevant or irrelevant tag.
4. The vectors representing the marked relevant and irrelevant documents are then linearly combined with the query vector to produce a new query vector.
5. The vector space method is then performed with the new query vector which produces a refined ranking of documents.

The equation to produce the refined query vector was defined by Rocchio as:

$$q_{\text{new}} = A \times q_{\text{old}} + B \times \text{relevant document vectors} - C \times \text{irrelevant document vectors} \quad (7.1)$$

where A , B and C are constant weights applied to the vector sets. We can see from this equation that the refined query vector is pushed towards the selected relevant documents and pushed away from the selected irrelevant documents.

This feedback operation was fine for small document sets. To calculate the document score was a simple dot product between the refined query vector and desired document. To rank them, this would have been performed for each document in the set.

For large document sets, the feedback operation was more difficult. For large document sets we need an efficient retrieval method. This usually involves inverting the index, so that we are able to extract words from the data collection rather than documents. By adding document vectors to the query vector, we introduce many new terms into the query which must be extracted from the inverted index and stored in memory during the query time. This slows down the query process and requires a lot of memory to store the query information.

To reduce this heavy load, Buckley [17, 15] choose to select only the terms in the relevant and irrelevant documents which are the most effective. In other words only the terms which produced the highest weights in the document vectors were selected to be added to the refined query vector.

7.3.2 Automatic Feedback

Employing feedback techniques does not guarantee better results. Feedback guarantees a more refined set of the items of which the feed back was used. Therefore, if the document which were used for feedback were relevant to the query in question, we should obtain a more precise ranking overall, but if the documents were not relevant to the query, then the feedback will cause a poorer ranking of documents. This means that automatic feedback will only improve the results if used with a retrieval method which will obtain an initial set of relevant document and interactive feedback will provide a more accurate ranking of documents if the user is able to classify the documents as relevant and irrelevant. There have been many attempts to remove this problem.

Lu *et al.* [70] observed this problem and set out a strategy based on the following rules. We know that both the relevant and irrelevant documents retrieved all have the

query in common, but the relevant document will have a higher tendency to group than the irrelevant documents. Therefore if a large cluster appears after some clustering algorithm is applied to the selected documents, this means that most of the documents will be of the topic given by the query and feedback with these documents is recommended. If there are many small clusters after the clustering, then it implies that the documents are all of different topic and feedback with these documents will only degrade the ranking. Clustering methods suggested by Lundquist *et al.* [72] are:

Highest Pair The two documents with the highest similarity in the retrieved set are selected. Any other documents with a similarity above a certain threshold are also used for the feedback.

Largest Group The document which is similar to the most documents is selected with the documents in its similarity group.

Largest Group, Expanded Largest group, but any documents which are similar to any documents in the selected set are also included.

7.3.3 Interactive Feedback

The basic Rocchio feedback method was extended by Ide [63] in an attempt to achieve higher precision. He experimented with the Q_0+ and Dec Hi methods (shown in equations 7.2 and 7.3 respectively).

$$\tilde{Q}_{i+1} = 4\tilde{Q}_0 + \tilde{Q}_i + \sum_{i=1}^{n'_r} \tilde{r}_i \quad (7.2)$$

$$\tilde{Q}_{i+1} = \tilde{Q}_i + \sum_{i=1}^{n'_r} \tilde{r}_i - \tilde{s}_1 \quad (7.3)$$

where $\tilde{Q}_{i+1}$ is the new query vector, $\tilde{Q}_0$ is the original query vector, $\tilde{Q}_i$ is the current query vector, $\tilde{r}_i$ is a relevant document, $\tilde{s}_1$ is the first irrelevant document, and n'_r is the number of relevant document retrieved for feedback.

His experiments compared the Rocchio, Q_0+ and Dec Hi methods and the results showed that each method was just as good as the other. There were some queries which worked well with one of the methods and some queries which did not.

The topic defined by a set of query terms is always user specific. For example the query “evaluation of retrieval systems” could mean we want to find methods of evaluating retrieval systems or that we want actual evaluations of retrieval systems. A method developed by Borodin *et al.* [12, 36] tries to create query sets based on the number of

significant clusters of documents resulting from the feedback process. Once these query sets are created, they are feed back into the retrieval system and the results are examined independently. The query sets are meant to describe all of the possible meanings which we could perceive the original query terms as.

7.3.4 Query Expansion

Sorting Techniques

Once we have received the documents which are considered relevant and we want to perform a query expansion, we must first decide how to extract the terms from the documents to choose as our new query terms. Harman [52] lists four effective sorting methods which have been used in the literature:

$$\text{score}_{i,j} = \log_2 \frac{p_{i,j}(1 - q_{i,j})}{(1 - p_{i,j})q_{i,j}} \quad (7.4)$$

$$\text{score}_{i,j} = \text{IDF}_i \times (p_{i,j} - q_{i,j}) \quad (7.5)$$

$$\text{score}_{i,j} = \log_2 \frac{p_{i,j}(1 - q_{i,j})}{(1 - p_{i,j})q_{i,j}} \times (p_{i,j} - q_{i,j}) \quad (7.6)$$

$$\text{score}_{i,j} = \text{IDF}_i \times f_{D,t} \quad (7.7)$$

where IDF_i is the inverse document frequency for term i , $f_{D,t}$ is the frequency of term t in the set of relevant or irrelevant documents, $\text{score}_{i,j}$ is the score assigned to the i th term in the j th query, $p_{i,j}$ and $q_{i,j}$ are the probabilities that the i th term in j th query is assigned within the set of relevant and irrelevant documents respectively. These are given as:

$$p_{i,j} = \frac{r + 0.5}{R + 1.0} \quad q_{i,j} = \frac{n - r + 0.5}{N - R + 1.0} \quad (7.8)$$

where r is the number of relevant documents term i appears in, R is the total number of relevant documents, n is the number of documents term i appears in, and N is the number of documents in the set.

Once we have obtained a sorted list of terms which represent the relevant documents (and a list representing the irrelevant documents if we are also including negative feedback), we choose the top T terms to use in our next iteration of the query feedback process. Harman experimented with multiple techniques on the Cranfield 1400 document collection. She found that query expansion should be used and that queries should be expanded by about $T = 20$ terms.

Massive Query Expansion

A method used by Buckley *et al.* [15] called “Massive Query Expansion” proposed that if we were to expand the initial query so that we were to take every significant term occurring in the selected relevant document set, then the terms which were common amongst the selected document set would be dominant. The terms which were not common amongst the selected document set would be insignificant and therefore be treated as noise. The query vector is created using all of the terms found and therefore the terms which appear in most documents will contribute to most of the document scores.

7.3.5 Term Re-weighting

If we are expanding the query by choosing terms representing the chosen relevant and irrelevant documents, or if we are just applying the knowledge about the relevant and irrelevant documents to the original query, we must use query term re-weighting so that the query terms reflect the chosen relevant and irrelevant documents.

Harman [52] suggests that the probabilistic model should be used. This is added to the query term weighting by replacing the inverse document frequency factor with equation 7.4. Buckley *et al.* [15] use the Rocchio query term re-weighting shown in equation 7.1

7.3.6 Iterations

When performing relevance feedback on large data sets, it is usual to perform only one iteration [23, 2, 15, 17, 3]. This is usually due to the time involved in performing each iteration. But experiments have been performed where the feedback process has gone through many iterations. Harman [52] presented results on multiple iterative feedback. The experiment was performed by initially ranking the document using the query. Each iteration consisted of the user viewing the top 10 documents and classifying which were relevant, the documents were re-ranked using this information. It was shown that each iteration brought a higher precision, but the change in precision decreased. Therefore the number of iterations must be based on the time we want to spend and the precision of the results.

Since we classify the top ranked documents, we expect them to also appear at the top of the re-ranked document set after feedback. This implies that multiple iteration will mainly affect the tail of the list of relevant documents. In a Web environment, where the user only views the first page of results, multiple iterations of feedback will not be effective.

7.3.7 Local Context Analysis

Xu and Croft [123, 124] experimented with a feedback method based on the INQUERY retrieval system called Local Context Analysis (LCA). Their method is similar to the feedback methods mentioned above, but they use passages instead of documents to extract feedback terms, and they select noun phrases instead of single terms (a noun phrase is a set of one to three adjacent nouns found in a passage of a document). The process ranks the document set into passages which are most similar to the query. Noun phrases are extracted from the top n passages and ranked according to a comparison formula which is related to the occurrence and appearance of the noun phrases. The most relevant m noun phrases are added to the query (they choose about 70 noun phrases) and the passages are re-ranked with the new query set.

7.3.8 Feedback in the Spectral Domain

If we apply relevance feedback to a spectral document ranking system, we introduce a few more factors that must be considered. The procedure we took was a modification of the method used by Buckley *et al.* [17]. They use the Rocchio relevance feedback method, but they include only the most relevant 50 terms to the query. They experimented with both automatic and user feedback processes. Their automatic retrieval system ranked the documents using the users query. The top 20 documents were considered relevant and the most frequent terms were extracted from these to perform the feedback process. The Rocchio parameters used were (8,8,0), where the 0 implies that no negative feedback was used. The user feedback process ranked the documents using the users query. The user would then select out of the top ranked documents which were relevant and which were irrelevant to the query. The most frequent terms were extracted from these documents and used in the feedback process. The Rocchio parameters used in this process were (64,64,2). The difference in magnitude between the relevant and irrelevant document Rocchio feedback parameters shows that they wanted the negative feedback to have a very small influence on the result.

If we perform the same feedback using FDS, the initial search will be the same, but the feedback stage will require some modification. If we extract the top terms from the relevant and irrelevant documents and feed them back into the FDS system, magnitude and phase precision will be calculated for both of the positive and negative feedback terms. Documents containing positive feedback terms will have an increase in magnitude and documents containing negative feedback terms will have a decrease in magnitude. This is all right because the magnitude is proportional to the document score. When the phase

precision is calculated we find that documents which have all of the terms together will have a higher phase precision than those that do not. This includes positive and negative feedback terms. This should not be the case, we do not care where the negative feedback terms are, we only want the positive feedback terms together throughout the document. Therefore, we have devised two methods of dealing with the phase precision from negative feedback terms.

- Do not include negative feedback terms in the phase precision calculations
- Calculate the phase precision separately for the negative feedback terms and combine it using Rocchio's formula.

Therefore, we will combine the results of the magnitude from the original, positive and negative feedback query terms using Rocchio's method. We will also combine the results of the phase precision using Rocchio's method but we will experiment by removing the negative feedback phase precision.

7.3.9 Experiments

We experimented using the AP2WSJ2 document set from the TREC collection, using queries 51-200. Our experiment consisted of running the new fds-9-7-1 method. The fds-9-7-1 method is the same as fds-9-4-1, with the difference of when calculating the phase precision, we do not use unit vectors. The phase vectors have magnitude $w_{q,t}$. This was to give more emphasis to the rarer terms. The same method was run with the application of the feedback processes using several different settings. These results were then compared to the same method without feedback so that we could observe any improvements.

The experiments were split into two separate groups. The first set involved automatic feedback (there is no user interaction, the top 20 ranked documents are assumed relevant). In this group no negative feedback was used and the Rocchio parameters were set to (8,8,0) as suggested in Buckley [17]

The second set involved user feedback. To simulate the user feedback, we extracted the top 20 documents from the initial query run and compared them to the relevance list provided by TREC. Each of the queries were marked either relevant or irrelevant and passed back into the retrieval system. The Rocchio parameters we used were (8,8,5) and (64,64,2) as suggested in Buckley [17], and we experimented with having negative phase precision and no phase precision for the negative feedback query terms.

Method	Rocc	Prec. 5	Method	Rocc	Prec. 10
fds-9-7-1		0.4947	fds-9-7-1		0.4560
fds-9-7-1fn.25t	(64,64,2)	0.5039	fds-9-7-1fnp.25t	(8,8,5)	0.4630
fds-9-7-1fn.50t	(8,8,5)	0.5008	fds-9-7-1fnp.50t	(64,64,2)	0.4614
fds-9-7-1fn.25t	(8,8,5)	0.5008	fds-9-7-1fnp.50t	(8,8,5)	0.4606
fds-9-7-1fn.50t	(64,64,2)	0.5008	fds-9-7-1fnp.25t	(64,64,2)	0.4606
fds-9-7-1fnp.50t	(8,8,5)	0.4976	fds-9-7-1fn.25t	(8,8,5)	0.4598
fds-9-7-1fnp.25t	(64,64,2)	0.4961	fds-9-7-1fn.25t	(64,64,2)	0.4598
fds-9-7-1fnp.50t	(64,64,2)	0.4961	fds-9-7-1fn.50t	(8,8,5)	0.4567
fds-9-7-1fnp.25t	(8,8,5)	0.4929	fds-9-7-1fn.50t	(64,64,2)	0.4567
fds-9-7-1f.50t	(8,8,0)	0.4707	fds-9-7-1f.25t	(8,8,0)	0.4387
fds-9-7-1f.25t	(8,8,0)	0.4693	fds-9-7-1f.50t	(8,8,0)	0.4380
(a)			(b)		
Method	Rocc	Prec. 15	Method	Rocc	Prec. 20
fds-9-7-1		0.4360	fds-9-7-1		0.4170
fds-9-7-1fn.25t	(8,8,5)	0.4420	fds-9-7-1fnp.50t	(64,64,2)	0.4189
fds-9-7-1fn.25t	(64,64,2)	0.4415	fds-9-7-1fn.50t	(8,8,5)	0.4181
fds-9-7-1fnp.25t	(64,64,2)	0.4415	fds-9-7-1fn.50t	(64,64,2)	0.4181
fds-9-7-1fnp.25t	(8,8,5)	0.4409	fds-9-7-1fnp.50t	(8,8,5)	0.4177
fds-9-7-1fnp.50t	(64,64,2)	0.4399	fds-9-7-1fn.25t	(8,8,5)	0.4169
fds-9-7-1fn.50t	(8,8,5)	0.4383	fds-9-7-1fn.25t	(64,64,2)	0.4169
fds-9-7-1fnp.50t	(8,8,5)	0.4378	fds-9-7-1fnp.25t	(64,64,2)	0.4154
fds-9-7-1fn.50t	(64,64,2)	0.4378	fds-9-7-1fnp.25t	(8,8,5)	0.4126
fds-9-7-1f.50t	(8,8,0)	0.4244	fds-9-7-1f.50t	(8,8,0)	0.4017
fds-9-7-1f.25t	(8,8,0)	0.4244	fds-9-7-1f.25t	(8,8,0)	0.4017
(c)			(d)		

Table 7.6: Results for the method fds-9-7-1 using AP2WSJ2 and the same method involving various methods of relevance feedback.

For each method, we also experimented with the number of query terms to retrieve from the feedback documents. We choose to investigate the effects of retrieving 25 and 50 terms.

The feedback terms were chosen by selecting the terms with the greatest term weight in the top 20 documents ranked by the initial classification.

We can see that the results for the automatic feedback were poor. In each of the

results, the precision is at least 1% lower than the FDS method without feedback. This might be due to the way which we choose our terms for feedback.

We can also see that after 5 documents, the manual feedback methods which ignore the phase for negative feedback provide a better precision than the methods which include negative feedback phase precision. If we examine the precision after 10 documents, the order has switched. The methods which include negative feedback phase appear to provide a greater precision than the methods which ignore it.

After the first 15 documents, the precision of the methods which include the top 25 positive and negative feedback terms are greater than the ones which use 50 positive and negative feedback terms. We can see that after 20 documents the methods have switched again in terms of precision. The top four methods all use 50 positive and negative feedback terms, while the next four use 25 positive and negative feedback terms.

Most of the manual feedback methods have a greater precision when compared to the FDS methods without feedback (fds-9-7-1). After 20 documents, the precision for FDS when 25 positive and negative feedback terms are used is lower than fds-9-7-1.

After analysing these results, we can deduce that there is no one correct configuration of the feedback parameters, it all depends on how many documents we are willing to examine. For example, if we were impatient, we would choose to use the best method in the list of precision after 5 documents (which is fds-9-7-1fn.25t using Rocchio parameters (64,64,2)).

7.4 Discussion and Summary

We have investigated two different areas of information retrieval used with vector space methods to try to increase the precision of our FDS method.

The first area was that of Anaphora resolution. We believed that replacing any anaphora found in a passage of text with its antecedent would assist an information retrieval system in finding relevant documents. We focused on one type of anaphora, namely pronouns, and implemented a simple and naïve pronoun resolution algorithm. The results showed that the document retrieval system which implemented the pronoun resolution produced results of lower precision than those without the pronoun resolution algorithm. We suggest that the poor results are due to the pronoun resolution algorithm we choose, and that better results could be achieved if a smarter method was used.

The second area used relevance feedback with our FDS method. Relevance feedback allows retrieval methods which produce high precision results to produce higher precision results. This is done by extracting key terms from the top ranked documents

and using them with the original query terms to perform a more refined document retrieval. We applied a form of Rocchio relevance feedback to our document retrieval system. Through experimentation, we found that the method of feedback we used degraded the system if automatic feedback was used and improved the system if manual feedback was used.

For any additions to be effective, we want the improvement in precision to be significant. This is because each addition to the basic system introduces more time in either the index creation or the query process. If we were to add to the complexity of the mentioned extensions, we would also add to the time. Therefore we must ask ourselves if it is worth building a more complex system to gain a small change in precision, but pay by waiting a longer time.

CHAPTER EIGHT

CONCLUSIONS

8.1 Summary of Work

THE main contribution of the thesis is the application of signal processing techniques to the field of information retrieval in order to capture both frequency and proximity information of query terms in documents. We have shown that these techniques are very effective in improving precision of the document retrieval process.

This thesis is an investigation of information retrieval issues in the design and development of spectral based document retrieval systems. Experiments were performed based on well founded signal processing techniques to construct guidelines on how to successfully implement such spectral based systems.

We have examined the following areas of spectral based information retrieval in detail:

- The effects on precision by performing document score calculations based on the Fourier spectral information obtained from the term signals.
- Query time and inverted index size improvements by calculating document scores from cosine spectra.
- Comparison of phase precision to Gaussian term spreading.
- Multiresolution analysis and its suitability to the document retrieval process.
- Combination of spectral methods with pronoun resolution and relevance feedback.

The major results obtained in each part of the dissertation are indicated in the following sections.

8.1.1 Spectral Document Retrieval

By analysing the spectrum of the query terms through each document, we are able to obtain a better understanding of the occurrence and location of each term relative to each other. Our experiments in spectral document retrieval using the Fourier transform, showed that an information retrieval system which utilises the spectral term information, is able to provide superior results when compared to the same system using the vector space measure with the same weighting scheme. The only drawback to the spectral method is the costs involved in either storage or time.

8.1.2 Efficient Implementation

To reduce the cost involved in terms of time or storage of the spectral document retrieval system, we took advantage of the relationship between the cosine transform and the Karhunen-Loève transform to conserve storage space, and we applied some fast querying ideas from the vector space field of information retrieval.

From the experiments, we concluded that by replacing the Fourier transform with the cosine transform, we were able to generate similar results while using an index of size comparable to the vector space methods, and generate query results in times similar to those of the vector space method.

8.1.3 Gaussian Spreading

To present precise query results, we should be able to control the region about the query terms which is considered their proximity. By applying Gaussian spreading to each of the query term positions in a document, we are able to control the terms proximity based on that rarity of the term. This Gaussian spreading can be applied by itself as a convolution in the spatial domain, or after the Fourier transform by multiplying it term-wise with the terms spectrum.

Our experiments involved trying to tune the spreading of the Gaussian to obtain the best results for that data set. We found that convolving the Gaussian with the query term signal increased the precision after 10 documents by a fair margin and that applying the Gaussian in the spectral domain only hindered the results.

8.1.4 Multiresolution Document Retrieval

We observed that if we choose our term signals to have the size of only one bin, our spectral information retrieval system collapses to the vector space method. By taking

advantage of the properties of the wavelet transform, we are able to observe the query term spectra at different levels of resolution, and we are able to do so in a more compact manner than the cosine transform.

We performed many experiments using two different wavelets and found that we were able to generate results of the same quality as the Fourier transform, even when using less information to derive the results.

8.1.5 Anaphora Resolution and Relevance Feedback

There are many information retrieval systems that do not rely solely on the calculations from the stored text. They use other methods with their text processing to assist in presenting the user with a more refined set of results. One of these is the use of pronoun resolution and the other is the more well known relevance feedback. Pronoun resolution attempts to convert all of the pronouns in the text to its referring noun. If that noun is a query term, then the results should be a more true representation of the documents in the set. The method of relevance feedback gathers a larger query term set based on the results of the query and performs the query again with the new terms included. This is thought to be a more refining process.

Unfortunately, the pronoun resolution experiments performed were less precise than those without it. This could be due to the very simple pronominal resolution algorithm used. On the other hand, the relevance feedback method boosted the precision performance of the spectral document retrieval system that it was applied to. This was expected due to the additional information which the feedback process provides.

8.2 Summary of Precision Results

To bring all of the work done in this thesis together, a table of the best 20 methods based on the BD-ACI-BCA weighting scheme and the non-zero phase precision is presented in tables 8.1 and 8.2. Only one weighting scheme is presented to show the reader how the spectral methods build upon the vector space method they are based on.

We can see that the top methods consist of the Haar, Daubechies and Fourier methods, with the Cosine method appearing shortly after. The Gaussian method appears in the Precision after 10 documents list (table 8.1b), which matches the findings obtained in chapter 5. If we examine each list in full, we can see that the Fourier and Cosine methods seem to cluster, which reinforces their relationship to the Karhunen-Loève Transform (as stated in chapter 4). To choose the best methods, we must also consider query time and

Method	Prec. 5	Method	Prec. 10
haar-5-4-6.b8	0.5000	daub4-5-4-6.b8	0.4653
daub4-5-4-6.b8	0.4960	fds-5-4-1.b5	0.4647
cds-5-4-6.b8	0.4907	haar-5-4-6.b8	0.4633
fds-5-4-6.b5	0.4907	fds-5-4-6.b5	0.4600
fds-5-4-6.b4	0.4907	haar-5-4-6.b4	0.4593
cds-5-4-6.b7	0.4893	fds-5-4-6.b4	0.4587
fds-5-4-6.b3	0.4880	fds-5-4-6.b3	0.4580
fds-5-4-1.b5	0.4867	cds-5-4-6.b5	0.4567
cds-5-4-6.b6	0.4840	cds-5-4-6.b7	0.4567
cds-5-4-6.b1	0.4813	gauss-5-4-6.s2.b8	0.4560
daub4-5-4-1.b1	0.4813	fds-5-4-1.b3	0.4560
cds-5-4-1.b1	0.4813	daub4-5-4-6.b2	0.4553
fds-5-4-1.b1	0.4813	cds-5-4-6.b8	0.4547
haar-5-4-1.b1	0.4813	fds-5-4-1.b4	0.4547
gauss-5-4-1.s2.b8	0.4813	daub4-5-4-6.b4	0.4540
daub4-5-4-6.b1	0.4813	fds-5-4-6.b2	0.4540
daub4-5-4-6.b2	0.4813	cds-5-4-6.b2	0.4540
haar-5-4-6.b1	0.4813	cds-5-4-6.b6	0.4540
haar-5-4-6.b2	0.4813	haar-5-4-6.b2	0.4533
fds-5-4-6.b1	0.4813	cds-5-4-6.b3	0.4533
BD-ACI-BCA	0.4440	BD-ACI-BCA	0.4247
(a)		(b)	

Table 8.1: Top 20 spectral document retrieval methods developed using BD-ACI-BCA weighting in terms of precision after a) 5 documents and b) 10 documents. The BD-ACI-BCA vector space method precision is shown below the bar.

storage requirements. Therefore, for precise and fast queries that require average storage, either of the two wavelet methods are suggested, using all eight components. To reduce the storage requirements and hence also slightly reduce the precision, the cosine transform method is suggested.

Method	Prec. 15	Method	Prec. 20
fds-5-4-1.b5	0.4440	haar-5-4-6.b8	0.4213
fds-5-4-1.b4	0.4413	fds-5-4-1.b4	0.4197
haar-5-4-6.b8	0.4404	fds-5-4-1.b5	0.4193
daub4-5-4-6.b8	0.4391	fds-5-4-6.b5	0.4193
fds-5-4-6.b5	0.4382	daub4-5-4-6.b8	0.4177
cds-5-4-6.b8	0.4369	fds-5-4-1.b3	0.4167
fds-5-4-6.b4	0.4338	fds-5-4-6.b4	0.4147
cds-5-4-1.b7	0.4333	cds-5-4-6.b8	0.4147
cds-5-4-6.b7	0.4329	cds-5-4-6.b6	0.4143
cds-5-4-1.b8	0.4324	cds-5-4-6.b7	0.4133
fds-5-4-1.b3	0.4320	haar-5-4-1.b8	0.4123
cds-5-4-6.b4	0.4311	daub4-5-4-6.b4	0.4117
daub4-5-4-6.b4	0.4298	fds-5-4-6.b3	0.4113
fds-5-4-6.b3	0.4293	cds-5-4-1.b8	0.4113
cds-5-4-6.b5	0.4293	cds-5-4-6.b5	0.4107
cds-5-4-6.b6	0.4293	cds-5-4-1.b2	0.4087
haar-5-4-6.b4	0.4293	cds-5-4-6.b4	0.4087
haar-5-4-1.b8	0.4284	cds-5-4-1.b7	0.4083
cds-5-4-6.b3	0.4276	cds-5-4-6.b3	0.4083
fds-5-4-6.b2	0.4276	daub4-5-4-1.b8	0.4080
BD-ACI-BCA	0.4142	BD-ACI-BCA	0.3953
(a)		(b)	

Table 8.2: Top 20 spectral document retrieval methods developed using BD-ACI-BCA weighting in terms of precision after a) 15 documents and b) 20 documents. The BD-ACI-BCA vector space method precision is shown below the bar.

8.3 Future Directions

This thesis marks the beginning of the application of signal processing to information retrieval, namely spectral information retrieval. Since this is such a new concept, there are many new avenues to explore before we can really develop a basis of understanding of how this can improve on the current document retrieval methods. Some future directions which could be investigated are outlined below:

- Analyse the effects of different sampling methods to obtain term signals. In our

approach, we used a simple square positional filter to capture the occurrences of terms within a certain positional span of the documents. By using different positional filter shapes, we would be able to capture a better approximate of the true term positions, and hence a better approximation of the true term signal in the document.

- Experiments could be performed to observe the effects on precision of weighting the phase based on the term rarity when calculating the phase precision.
- Since this work is based on signal processing techniques, we could easily extend it to cover areas other than text. For example sound and images could easily be represented in an index as signals (sound as a one dimensional and images as 2 dimensional signals). Retrieving them would be as easy as giving portions of the sound or picture that we want.
- Experiments could be conducted to analyse the effect of variable bin size for different term signals. For example, we could base the number of bins on the length of the document.
- Functions other than the Gaussian could be convolved with the term signals to evaluate different regions of influence.
- The relevance feedback we performed feed back the terms into the next iteration. An investigation could be done on feeding back term signals rather than just the terms. This would give the next iteration a better estimate of how the terms relate to each other.

We hope that this thesis will become the basis for much further analysis of this area of spectral document retrieval.

BIBLIOGRAPHY

- [1] N. Ahmed, T. Natarajan, and K. R. Rao. Discrete cosine transform. *IEEE Transactions on Computers*, 23:90–93, January 1974.
- [2] N. Alexander, C. Brown, J. Jose, I. Ruthven, and A. Tombros. Question answering, relevance feedback and summarisation: TREC-9 interactive track report. In Voorhees and Harman [117], pages 523–532.
- [3] J. Allan, M.E. Connell, W.B. Croft, F-F Feng, D. Fisher, and X. Li. INQUERY and TREC-9. In Voorhees and Harman [117], pages 551–562.
- [4] James Allan, Jamie Callan, Fang-Fang Feng, and Daniella Malin. Inquiry and trec-8. In Voorhees and Harman [115], pages 637–644.
- [5] Vo Ngoc Anh, Owen de Kretser, and Alistair Moffat. Vector-space ranking with effective early termination. In *Proceedings of the 24th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 35–42. ACM Press, 2001.
- [6] Walid G. Aref, Daniel Barbara, Stephen Johnson, and Sharad Mehrotra. Efficient processing of proximity queries for large databases. In *Proceedings of the Eleventh International Conference on Data Engineering*, pages 147–154, 1995.
- [7] Breck Baldwin. CogNIAC: high precision coreference with limited knowledge and linguistic resources. In *Proceedings of the ACL’97/EACL’97 workshop on Operational factors in practical, robust anaphora resolution*, pages 38–45, Madrid, Spain, 1997.
- [8] N. J. Belkin, R. N. Oddy, and H. M. Brooks. ASK for information retrieval. part 1. background and theory. *Journal of Documentation*, 38:61–71, 1982.
- [9] M. W. Berry, S. T. Dumais, and G. W. O’Brien. Using linear algebra for intelligent information retrieval. Technical report, Computer Science Department, The University of Tennessee, Knoxville, TN, December 1994.

- [10] Michael W. Berry. *Large scale sparse singular value computations*. Department of Computer Science, University of Tennessee, 107 Ayres Hall, Knoxville, 1993.
- [11] Michael W. Berry and Ricardo D. Fierro. Low-rank orthogonal decompositions for information retrieval applications. *Numerical Linear Algebra with Applications*, 3(4):301–328, 1996.
- [12] A. Borodin, L. Kerr, and F. Lewis. *Query splitting in relevance feedback systems*, chapter 19. In Salton [100], 1971.
- [13] Eric Brill. Some advances in rule-based part of speech tagging. In *Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI'94)*, pages 722–727, Seattle, Washington, 1994. MIT Press.
- [14] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems*, 30(1-7):107–117, April 1998.
- [15] C. Buckley, G. Salton, J. Allan, and A. Singhal. Automatic query expansion using SMART : TREC 3. In Harman [54], pages 69–80.
- [16] Chris Buckley and Alan F. Lewit. Optimisation of inverted vector searches. In *Proceedings of the ACM-SIGIR International Conference on Research and Development in Information Retrieval*, pages 97–110. ACM Press, New York, June 1985.
- [17] Chris Buckley, Amit Singhal, Mandar Mitra, and Gerard Salton. New retrieval approaches using smart : TREC 4. In Harman [55], pages 25–48.
- [18] Chris Buckley and Janet Walz. SMART in TREC 8. In Voorhees and Harman [115], pages 577–582.
- [19] Donna K. Byron and James F. Allen. Resolving demonstrative anaphora in the TRAINS93 corpus. In *Proceedings of DAARRC2 - Discourse, Anaphora and Reference Resolution Colloquium*, Lancaster University, August 1998.
- [20] James P. Callan, W. Bruce Croft, and Stephen M. Harding. The inquiry retrieval system. In A. Min Tjoa and Isidro Ramos, editors, *Proceedings of the International Conference on Database and Expert Systems Applications, Spain*, pages 78–83. Springer-Verlag, 1992.
- [21] S. Jeromy Carrière and Rick Kazman. Webquery: searching and visualising the web through connectivity. *Computer Networks and ISDN Systems*, 29:1257–1267, 1997.

- [22] S. Chandrasekaran, B. S. Manjunath, Y. F. Wang, J. Winkeler, and H. Zhang. An eigenspace update algorithm for image analysis. *Graphical Models and Image Processing*, 59(5):321–332, 1997.
- [23] A. Chowdhury, S. Beitzel, E. Jensen, M. Sai-lee, D. Grossman, O. Frieder, M.C. McCabe, and D. Holmes. IIT TREC-9 - entity based feedback with fusion. In Voorhees and Harman [117], pages 241–248.
- [24] Charles L. A. Clarke and Gordon V. Cormack. Shortest-substring retrieval and ranking. *ACM Transactions on Information Systems*, 18(1):44–78, January 2000.
- [25] Ronald R. Coifman and Mladen Victor Wickerhauser. Entropy-based algorithms for best basis selection. *IEEE Transactions on Information Theory*, 38(2):713–718, 1992.
- [26] William S. Cooper. Getting beyond boole. *Information Processing and Management*, 24:243–248, 1988.
- [27] W. B. Croft and D. J. Harper. Using probabilistic models of document retrieval without relevance information. *Journal of Documentation*, 35:285–295, 1979.
- [28] Ingrid Daubechies. Othonormal bases of compactly supported wavelets. *Communications on Pure and Applied Mathematics*, 41:909–996, 1988.
- [29] Ingrid Daubechies. Where do wavelets come from? - a personal point of view. *Proceedings of the IEEE*, 84(4):510–513, April 1996.
- [30] Owen de Kretser. *Locality-Based Information Retrieval*. PhD thesis, Department of Computer Science and Software Engineering, The University of Melbourne, November 2000.
- [31] Owen de Kretser and Alistair Moffat. Locality-based information retrieval. In *Proceedings of the 10th Australasian Database Conference, Auckland, New Zealand*, pages 177–188, January 1999.
- [32] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer, and R. A. Harshman. Indexing by latent semantic analysis. *Journal of the the American Society for Information Science*, 41:391–407, 1990.
- [33] Michael Denber. Automatic resolution of anaphora in english. Technical report, Eastman Kodak Co. Imaging Science Division, June 1998.
- [34] A biography of Melvil Dewey. <http://www.oclc.org/fp/about/biography.htm>.

- [35] L. B. Doyle. Indexing and abstracting by association : Part 1. Technical Report SP-718/001/00, System Development Corporation, Santa Monica, CA, 1962.
- [36] Susan T. Dumais. Enhancing performance in latent semantic indexing (LSI) retrieval. Technical Report TM-ARH-017527, Bellcore, 1990.
- [37] Susan T. Dumais. Improving the retrieval of information from external sources. *Behaviour Research Methods, Instruments & Computers*, 23(2):229–236, 1991.
- [38] David S. Ebert, Amen Zwa, and Ethan L. Miller. Two-handed volumetric document corpus mangement. *IEEE Computer Graphics and Applications*, 17(4):60–62, July/August 1997.
- [39] Peter Elias. Universal codeword sets and representations of the integers. *IEEE Transactions on Information Theory*, 21(2):194–203, March 1975.
- [40] ENIAC computer. <http://www.ideafinder.com/history/inventions/-story072.htm>, May 2002.
- [41] Ebru Ersan and Varol Akman. Focusing for pronoun resolution in english discourse: An implementation. Technical Report BU-CEIS-94-29, Department of Computer Engineering and Information Science, Bilkent University, September 1994.
- [42] Christos Faloutsos. *Signature Files*, chapter 4. In Frakes and Baeza-Yates [45], 1992.
- [43] William Feller. *An Introduction to Probability Theory and Its Applications*. Wiley publications in statistics. John Wiley and Sons, Inc., New York, 2nd edition, 1975.
- [44] Christopher Fox. *Lexical Analysis and Stoplists*, chapter 7. In Frakes and Baeza-Yates [45], 1992.
- [45] William B. Frakes and Ricardo Baeza-Yates, editors. *Information Retrieval: Data structures and Algorithms*. Prentice Hall, Englewood Cliffs, New Jersey, 1992.
- [46] Niye Ge, John Hale, and Eugene Charniak. A statistical approach to anaphora resolution. In *Proceedings of the Sixth Workshop on Very Large Corpora*, pages 161–171, 1998.
- [47] Solomon W. Golomb. Run-length encodings. *IEEE Transactions on Information Theory*, 12:399–401, July 1966.
- [48] G. Golub and C. Reinsch. *Handbook for automatic computation II, Linear Algebra*. Springer-Verlag, New York, 1971.

- [49] Gaston H. Gonnet, Ricardo A. Baeza-Yates, and Tim Snider. *New Indices for Text: PAT trees and PAT arrays*, chapter 5. In Frakes and Baeza-Yates [45], 1992.
- [50] Amara L. Graps. An introduction to wavelets. *IEEE Computational Science and Engineering*, 2(2), Summer 1995.
- [51] Edinburgh Language Technology Group. Penn trebank tag-set, 2002.
- [52] Donna Harman. Relevance feedback revisited. In *Proceedings of the 15th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 1–10. ACM Press, 1992.
- [53] Donna Harman. Overview of the third text retrieval conference (TREC-3). In *The Third Text REtrieval Conference (TREC-3)* [54], pages 1–20.
- [54] Donna Harman, editor. *The Third Text REtrieval Conference (TREC-3)*, Gaithersburg, Md. 20899, March 1994. National Institute of Standards and Technology Special Publication 500-226.
- [55] Donna Harman, editor. *The Fourth Text REtrieval Conference (TREC-4)*, Gaithersburg, Md. 20899, November 1995. National Institute of Standards and Technology Special Publication 500-236.
- [56] Donna Harman. Panel: building and using test collections. In *Proceedings of the 19th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 335–337. ACM Press, 1996.
- [57] D. Hawking and P. Thistlewaite. Proximity operators - so near and yet so far. In Harman [55], pages 131–144.
- [58] David Hawking and Paul Thistlewaite. Relevance weighting using distance between term occurrences. Technical Report TR-CS-96-08, The Australian National University, August 1996.
- [59] Simon Haykin. *An introduction to analog and digital communications*. John Wiley and Sons, Inc., 1989.
- [60] Simon Haykin. *Neural Networks : A comprehensive foundation*. Prentice Hall, London, 2nd edition edition, 1999.
- [61] Adele E. Howe and Daniel Dreilinger. Savvysearch. *AI Magazine*, pages 19–25, Summer 1997.

- [62] David A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the Institute of Radio Engineers*, 40(9):1098–1101, September 1952.
- [63] E. Ide. New experiments in relevance feedback. In Salton [100], chapter 16.
- [64] Ask jeeves. <http://www.ask.com>, January 2003.
- [65] T. Joyce and R. M. Needham. The thesaurus approach to information retrieval. *American Documentation*, 9:192–197, 1958.
- [66] E. Michael Keen. Term position ranking: some new test results. In *Proceedings of the 15th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 66–76. ACM Press, 1992.
- [67] Tamara G. Kolda and Dianne P. O’leary. A semidiscrete matrix decomposition for latent semantic indexing in information retrieval. *ACM Transactions on Information Systems*, 16(4):322–346, October 1998.
- [68] Shalom Lappin and Herbert J. Leass. An algorithm for pronomial anaphora resolution. *Computational Linguistics*, 20(4):535–561, 1994.
- [69] Julie Beth Lovins. Development of a stemming algorithm. *Mechanical Translation and Computational Linguistics*, 11:22–31, 1968.
- [70] Allan Lu, Maen Ayoub, and Jianhua Dong. Ad hoc experiments using eureka. In Ellen M. Voorhees and Donna Harman, editors, *The Fifth Text REtrieval Conference (TREC-5)*, pages 229–240. Gaithersburg, Md. 20899, November 1996.
- [71] Hans P. Luhn. The automatic derivation of information retrieval encodements from machine-readable texts. In A. Kent, editor, *Information Retrieval and Machine Translation*, volume 3, pages 1021–1028. Interscience Publications, New York, 1961.
- [72] Carol Lundquist, David A. Grossman, and Ophir Frieder. Improving relevance feedback in the vector space model. In *Proceedings of the sixth international conference on Information and knowledge management*, pages 16–23. ACM Press, 1997.
- [73] Stéphane Mallat. *A wavelet tour of signal processing*. Academic Press, San Diego, 2nd edition, 2001.
- [74] Benoit B. Mandelbrot. *The fractal geometry of nature*. W. H. Freeman and Company, New York, 3rd edition, 1983.

- [75] M. E. Maron and J. L. Kuhns. On relevance, probabilistic indexing and information retrieval. *Journal of the Association for Computing Machinery*, 7:216–244, 1960.
- [76] Stephen A. Martucci. Symmetric convolution and the discrete sine and cosine transforms. *IEEE Transactions on Signal Processing*, 42(5):1038–1051, May 1994.
- [77] Andrei Mikheev. Part-of-speech guessing rules: Learning and evaluation. *Computational Linguistics*, 23(3):405–423, June 1997.
- [78] Sheila Mincey. Ashurbanipal. <http://web.utk.edu/~smincey/>.
- [79] Ruslan Mitkov. Robust pronoun resolution with limited knowledge. In *Proceedings of the 18th International Conference on Computational Linguistics (COLING'98)/ACL'98 Conference*, pages 869–875, Montreal, Canada, 1998.
- [80] Ruslan Mitkov. Anaphora resolution: the state of the art. Working paper, (Based on the COLING'98/ACL'98 tutorial on anaphora resolution), University of Wolverhampton, Wolverhampton., 1999.
- [81] Dunja Mladenič. Personal webwatcher: design and implementation. Technical report, Dept. for Intelligent Systems, J. Stefan Institute, Jamova 39, 11000 Ljubljana, Slovenia, 1996.
- [82] Alistair Moffat and Justin Zobel. Self-indexing inverted files for fast text retrieval. *ACM Transactions on Information Systems (TOIS)*, 14(4):349–379, 1996.
- [83] Christof Monz and Maarten de Rijke. A tableau calculus for pronoun resolution. In N. V. Murry, editor, *Automated Reasoning with Analytic Tableaux and Related Methods (TABLEAUX'99)*, Lecture Notes in Artificial Intelligence 1617, pages 247–262. Springer, 1999.
- [84] Daniel Siaw Weng Ngu and Xindong Wu. Site helper: a localised agent that helps incremental exploration of the world wide web. *Computer Networks and ISDN Systems*, 29:1249–1255, 1997.
- [85] Alan V. Oppenheim and Ronald W. Schaffer. *Discrete-time signal processing*. Prentice-Hall, Inc., Englewood Cliffs, New Jersey 07632, 1989.
- [86] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. Technical report, Stanford Digital Library Technologies Project, 1998.

- [87] Laurence A. F. Park, Marimuthu Palaniswami, and Ramamohanarao Kotagiri. Internet document filtering using fourier domain scoring. In Luc de Raedt and Arno Siebes, editors, *Principles of Data Mining and Knowledge Discovery*, number 2168 in Lecture Notes in Artificial Intelligence, pages 362–373. Springer-Verlag, September 2001.
- [88] Laurence A. F. Park, Marimuthu Palaniswami, and Kotagiri Ramamohanarao. A novel web text mining method using the discrete cosine transform. In T. Elomaa, H. Mannila, and H. Toivonen, editors, *6th European Conference on Principles of Data Mining and Knowledge Discovery*, number 2431 in Lecture Notes in Artificial Intelligence, pages 385–396, Berlin, August 2002. Springer-Verlag.
- [89] Laurence A. F. Park, Marimuthu Palaniswami, and Kotagiri Ramamohanarao. A novel document ranking method using the discrete cosine transform. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Submitted April 2003.
- [90] Laurence A. F. Park, Kotagiri Ramamohanarao, and Marimuthu Palaniswami. A new implementation technique for fast spectral based document retrieval systems. In Vipin Kumar and Shusaku Tsumoto, editors, *2002 IEEE International Conference on Data Mining*, pages 346–353. IEEE Computer Society, December 2002.
- [91] Laurence A. F. Park, Kotagiri Ramamohanarao, and Marimuthu Palaniswami. Fourier domain scoring : A novel document ranking method. *IEEE Transactions on Knowledge and Data Engineering*, Accepted May 2003.
- [92] Laurence A. F. Park, Kotagiri Ramamohanarao, and Marimuthu Palaniswami. A novel document retrieval method using the discrete wavelet transform. *ACM Transactions on Information Systems*, Submitted June 2003.
- [93] Martin F. Porter. An algorithm for suffix stripping. *Program*, 14(3):130–137, 1980.
- [94] John G. Proakis and Dimitris G. Manolakis. *Digital Signal Processing principles, algorithms and applications*. Prentice-Hall, Inc, Englewood Cliffs, New Jersey 07632, 3rd edition, 1996.
- [95] S. E. Robertson and S. Walker. Some simple effective approximations to the 2-poisson model for probabilistic weighted retrieval. In W. B. Croft and C. J. van Rijsbergen, editors, *Proceedings of the 17th International Conference on Research and Development in Information Retrieval*, pages 232–241, London, 1994. Association of Computing Machinery, Inc., Springer-Verlag.

- [96] S. E. Robertson and S. Walker. Okapi/keenbow at TREC-8. In Voorhees and Harman [115], pages 151–162.
- [97] Stephen E. Robertson, Steve Walker, S. Jones, Micheline Hancock-Beaulieu, and Mike Gatford. Okapi at TREC-2. In *The Second Text REtrieval Conference. National Institute of Standards and Technology (NIST), Special Publication 500-215*, pages 21–34, 1993. online publication: http://trec.nist.gov/pubs/trec1/t1_proceedings.html.
- [98] J. J. Rocchio. Relevance feedback in information retrieval. In Salton [100], chapter 14.
- [99] G. Salton and M. E. Lesk. Computer evaluation of indexing and text processing. *Journal of the Association for Computing Machinery*, 15:8–36, 1968.
- [100] Gerard Salton, editor. *The SMART retrieval system: Experiments in automatic information retrieval*. Prentice-Hall, Englewood Cliffs, N.J., 1971.
- [101] Gerard Salton and Chris Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing and Management*, 24(5):513–523, 1988.
- [102] Gerard Salton and Michael J. McGill. *Introduction to modern information retrieval*. McGraw-Hill, Inc., New York, 1983.
- [103] Beatrice Santorini. *Part-Of-Speech Tagging Guidelines for the Penn Treebank Project*. Department of Computer Science, The University of Pennsylvania, 3rd edition, February 1995.
- [104] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, July 1948.
- [105] Pen shu Yeh. Data compression properties of the hartly transform. *IEEE Transactions of Acousitcs, Speech, and Signal Processing*, 37(3):450–451, March 1989.
- [106] Amit Singhal, Chris Buckley, and Mandar Mitra. Pivoted document length normalization. In Hans-Peter Frei, Donna Harman, Peter Schäuble, and Ross Wilkinson, editors, *Proceedings of the 19th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR'96, August 18-22, 1996, Zurich, Switzerland (Special Issue of the SIGIR Forum)*, pages 21–29. ACM, 1996.
- [107] Ellen Spertus. Parasite: mining structural information on the web. *Computer Networks and ISDN Systems*, 29:1205–1215, 1997.

- [108] National Institute Of Standards and Technology. Text REtrieval Conference (TREC) <http://trec.nist.gov/>. World Wide Web, 2001.
- [109] Danny Sullivan. Searches per day. <http://www.searchenginewatch.com/-reports/perday.html>, February 2003.
- [110] Joel R. Tetreault. A corpus-based evaluation of centering and pronoun resolution. *Computational Linguistics*, 27(4):507–520, 2001.
- [111] Howard Turtle and W. Bruce Croft. Inference networks for document retrieval. In J. L. Vidick, editor, *Proceedings of the Thirteenth International Conference on Research and Development in Information Retrieval*, pages 1–24, New York, 1990. Association for Computing Machinery.
- [112] Cornell University. Gerard salton: In memorium. In *IRLIST Digest*, volume XII. September 1995. Issue 271.
- [113] Martin Vetterli. Filter banks allowing perfect reconstruction. *Signal Processing*, 10(3):219–244, 1986.
- [114] Martin Vetterli and Cormac Herley. Wavelets and filter banks: Theory and design. *IEEE Transactions on Signal Processing*, 40(9):2207–2232, September 1992.
- [115] Ellen M. Voorhees and Donna K. Harman, editors. *The Eighth Text REtrieval Conference (TREC-8)*, Gaithersburg, Md. 20899, November 1999. National Institute of Standards and Technology Special Publication 500-246, Department of Commerce, National Institute of Standards and Technology.
- [116] Ellen M. Voorhees and Donna K. Harman. Overview of the eighth text retrieval conference (TREC-8). In *The Eighth Text REtrieval Conference (TREC-8)* [115], pages 1–23.
- [117] Ellen M. Voorhees and Donna K. Harman, editors. *The Ninth Text REtrieval Conference (TREC-9)*, Gaithersburg, Md. 20899, December 2000. National Institute of Standards and Technology Special Publication 500-249, Department of Commerce, National Institute of Standards and Technology.
- [118] Eric W. Weisstein. Poisson sum formula. In *Eric Weisstein's World of Mathematics*. CRC Press LLC, <http://mathworld.wolfram.com>, 1999.
- [119] Eric W. Weisstein. Support. In *Eric Weisstein's World of Mathematics*. CRC Press LLC, <http://mathworld.wolfram.com>, 1999.

- [120] Wikipedia. Nyquist-shannon sampling theorem. http://www.wikipedia.org/wiki/Nyquist-Shannon_sampling_theorem, February 2003.
- [121] Ian H. Witten, Alistair Moffat, and Timothy C. Bell. *Managing gigabytes : compressing and indexing documents and images*. Morgan Kaufmann Publishers, 1999.
- [122] Ian H. Witten, Radford M. Neal, and John G. Cleary. Arithmetic coding for data compression. *Communications of the ACM*, 30(6):520–540, 1987.
- [123] Jinxi Xu and W. Bruce Croft. Query expansion using local and global document analysis. In *Proceedings of the 19th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 4–11. ACM Press, 1996.
- [124] Jinxi Xu and W. Bruce Croft. Improving the effectiveness of information retrieval with local context analysis. *ACM Transactions on Information Systems*, 18(1):79–112, January 2000.
- [125] Kiduk Yang and Kelly Maglaughlin. IRIS at TREC-8. In Voorhees and Harman [115], pages 645–656.
- [126] Justin Zobel and Alistair Moffat. Exploring the similarity space. In *ACM SIGIR Forum*, volume 32, pages 18–34, Spring 1998.

APPENDIX ONE

METHOD NAMING SCHEME

To generate the results found in this thesis, many methods of spectral document retrieval were created and used. To help the reader quickly identify how a method was constructed, we have presented in tabular form a list of possible combinations of each of the methods. Each method implemented has a name of the form:

Trans-PreCalc-PostCalc-Comb.E

where:

- *Trans* represents the transformation used (eg. Fourier, Cosine) found in table A.1
- *PreCalc* is a number representing the weighting applied before the transform takes place, found in table A.2.
- *PostCalc* is a number representing the calculations which take place after the transformation, found in table A.3
- *Comb* is a number which represents the method which the components have been combined to generate a document score, found in table A.4
- *E* represents the set of constants or extensions which have been applied to the method, found in table A.5. This term can have multiple or no values, depending on the method. The values are separated by a period (.).

For example, if we choose the method fds-9-4-6.b8.f.t20, we can see that the Fourier transform was used (*Trans* = fds), Lnu.ltu preweighting was used (*PreCalc* = 9), Sum magnitude with Zero phase precision was used to generate the magnitude and phase precision values (*PostCalc* = 4), the document scores were generated by summing the squares of the components (*Comb* = 6), the first 8 bins were used in the score calculation (b8), we used positive feedback (f), and we used the top 20 terms for the feedback process (t20).

Notes: * implies that the method was unsuccessful and not mentioned in this thesis.

Trans Code	Transform
fds	Fourier transform
cds	Cosine transform
gauss	Gaussian transform
haar	Haar wavelet transform
daub4	Daubechies-4 wavelet transform

Table A.1: Transforms performed on term signals.

PreCalc code	Pre-transform calculation
1	No preweighting*
2	TF-IDF postweighting*
3	TF-IDF preweighting*
4	Spread TF-IDF preweighting*
5	BD-ACI-BCA preweighting
6	BB-ACB-BAA preweighting*
7	AB-AFD-BAA preweighting
8	BI-ACI-BCA preweighting
9	Lnu.ltu preweighting

Table A.2: Pre-transform calculations performed on term signals.

PostCalc code	Post-transform calculation
0	Sum magnitude with no phase precision
1	Sum vectors with No phase precision
2	Sum magnitude with phase precision*
3	Sum magnitude with Non-zero phase precision
4	Sum magnitude with Zero phase precision
7	Sum magnitude with Non-zero weighted phase precision

Table A.3: Post-transform calculations performed on term signals.

Comb code	Component combination
1	Sum all components
2	Sum largest phase precision components
3	Sum largest magnitude components
4	Sum largest score components
5	Sum threshold phase precision components
6	Sum all squared components

Table A.4: Component combination used.

E code	Constant value
bx	x number of bins used in score calculation
p	padding was used
g	gaussian spreading was added
sx	$S = x$ used in Gaussian spreading
f	positive feedback was used
fn	positive and negative feedback was used
fnp	fn with negative feedback term phase ignored
tx	the number of terms used in the feedback process

Table A.5: Constants and extensions used in generating results.

APPENDIX TWO

QUERY SET

Throughout the thesis we mentioned using queries 51 to 200 of the TREC query set. The whole queries can be found on the TREC web site:

http://trec.nist.gov/data/topics_eng/index.html

as the TREC-1,2,3 ad hoc query sets. The short queries we used were the titles of the above queries and are presented below:

Airbus Subsidies
South African Sanctions
Leveraged Buyouts
Satellite Launch Contracts
Insider Trading
Prime (Lending) Rate Moves, Predictions
MCI
Rail Strikes
Weather Related Fatalities
Merit-Pay vs. Seniority
Israeli Role in Iran-Contra Affair
Military Coups D'etat
Machine Translation
Hostage-Taking
Information Retrieval Systems
Natural Language Processing
Politically Motivated Civil Disturbances
Health Hazards from Fine-Diameter Fibers
Attempts to Revive the SALT II Treaty

Surrogate Motherhood
Border Incursions
Demographic Shifts in the U.S.
Demographic Shifts across National Boundaries
Conflicting Policy
Automation
U.S. Constitution - Original Intent
Poaching
Greenpeace
FRG Political Party Positions
1988 Presidential Candidates Platforms
Financial crunch for televangelists in the wake of the PTL
Genetic Engineering
Measures to Protect the Atmosphere
Alternative/renewable Energy Plant & Equipment Installation
Official Corruption
Bank Failures
Criminal Actions Against Officers of Failed
Crude Oil Price Trends
Downstream Investments by OPEC Member States
Data on Proven Reserves of Oil & Natural Gas
U.S. Army Acquisition of Advanced Weapons Systems
International Military Equipment Sales
What Backing Does the National Rifle Association
Computer-aided Crime
Computer-aided Crime Detection
Computer-Aided Medical Diagnosis
Fiber Optics Applications
Fiber Optics Equipment Manufacturers
Iran-Contra Affair
Controlling the Transfer of High Technology
Design of the Star Wars Anti-missile Defense System
Laser Research Applicable to the U.S.s Strategic Defense
Welfare Reform
Catastrophic Health Insurance
Black Monday

U.S. Control of Insider Trading
Japanese Regulation of Insider Trading
Japanese Protectionist Measures
Find Innovative Companies
Black Resistance Against the South African Government
Nuclear Proliferation
Funding Biotechnology
New Space Satellite Applications
Non-commercial Satellite Launches
Impact of the 1986 Immigration Law
Generic Drug Substitutions
Capacity of the U.S. Cellular Telephone Network
International Terrorists
Actions Against International Terrorists
Economic Impact of International Terrorism
Death from Cancer
RDT&E of New Cancer Fighting Drugs
Research into & Control of Carcinogens
Alternatives to Traditional Cancer Therapies
Anti-smoking Actions by Government
Medical Ethics and Modern Technology
U.S.-U.S.S.R. Arms Control Agreements
Privatization of State Assets
Soviet Spying on the U.S.
Jewish Emigration and U.S.-USSR Relations
McDonnell Douglas Contracts for Military Aircraft
Stealth Aircraft
Hubble Space Telescope
The Human Genome Project
Possible Contributions of Gene Mapping to Medicine
Diversification by Pacific Telesis
Expansion in the U.S. Theme Park Industry
Iranian Support for Lebanese Hostage-takers
Iran's Islamic Revolution - Domestic and Foreign Social
Political Impact of Islamic Fundamentalism
Japan's Handling of its Trade Surplus with the U.S.

Impact of Government Regulated Grain Farming on International
Why Protect U.S. Farmers?
Management Problems at the United Nations
Influence of the Pro-Israel Lobby
Negotiating an End to the Nicaraguan Civil War
Productivity Trends in the U.S. Economy
Conflict in the Horn of Africa
Industrial Espionage
U.S. Political Campaign Financing
Coping with overcrowded prisons
Accusations of Cheating by Contractors on U.S. Defense Projects
Insurance Coverage which pays for Long Term Care
Oil Spills
Right Wing Christian Fundamentalism in U.S.
Efforts to enact Gun Control Legislation
Causes and treatments of multiple sclerosis (MS)
Term limitations for members of the U.S. Congress
Electric Car Development
Vitamins - The Cure for or Cause of
Acid Rain
Automobile Recalls
Vietnam Veterans and Agent Orange
Generic Drugs - Illegal Activities by Manufacturers
Tobacco company advertising and the young
Standardized testing and cultural bias
Regulation of the showing of violence and explicit
Financing AMTRAK
Cost of Garbage/Trash Removal
The Consequences of Implantation of Silicone Gel
Use of Mutual Funds in an Individual's
The Effectiveness of Medical Products and
Smoking Bans
Hazardous Waste Cleanup
NRA Prevention of Gun Control Legislation
Real-life private investigators
English as the Official Language in U.S.

Dog Maulings
U. S. Restaurants in Foreign Lands
Ineffectiveness of U.S. Embargoes/Sanctions
Abuse of the Elderly by Family Members, and Medical and
Commercial Overfishing Creates Food Fish Deficit
Asbestos Related Lawsuits
Corporate Pension Plans/Funds
Reform of the U.S. Welfare System
Difference of Learning Levels Among Inner
Signs of the Demise of Independent Publishing
Beachfront Erosion
Real Motives for Murder
Instances of Fraud Involving the Use of a Computer
Efforts to Improve U.S. Schooling
Oil Spill Cleanup
Toys R Dangerous
The Amount of Money Earned by Writers
Stock Market Perturbations Attributable to
School Choice Voucher System and its effects
Reform of the jurisprudence system
Gene Therapy and Its Benefits to
Legality of Medically Assisted Suicides
Impact of foreign textile imports on U.S.